



Creo.JS User's Guide

12.4.2.0

Copyright © 2025 PTC Inc. and/or Its Subsidiary Companies. All Rights Reserved.

Copyright for PTC software products is with PTC Inc. and its subsidiary companies (collectively “PTC”), and their respective licensors. This software is provided under written license or other agreement, contains valuable trade secrets and proprietary information, and is protected by the copyright laws of the United States and other countries. It may not be copied or distributed in any form or medium, disclosed to third parties, or used in any manner not provided for in the applicable agreement except with written prior approval from PTC. More information regarding third party copyrights and trademarks and a list of PTC’s registered copyrights, trademarks, and patents can be viewed here: <https://www.ptc.com/support/go/copyright-and-trademarks>

User and training guides and related documentation from PTC are also subject to the copyright laws of the United States and other countries and are provided under a license agreement that restricts copying, disclosure, and use of such documentation. PTC hereby grants to the licensed software user the right to make copies of product documentation and guides in printed form, but only for internal/personal use and in accordance with the license agreement under which the applicable software is licensed. Any copy made shall include the PTC copyright notice and any other proprietary notice provided by PTC. Note that training materials may not be copied without the express written consent of PTC. This documentation may not be disclosed, transferred, modified, or reduced to any form, including electronic media, or transmitted or made publicly available by any means without the prior written consent of PTC and no authorization is granted to make copies for such purposes.

UNITED STATES GOVERNMENT RIGHTS

PTC software products and software documentation are “commercial items” as that term is defined at 48 C.F.R. 2.101. Pursuant to Federal Acquisition Regulation (FAR) 12.212 (a)-(b) (Computer Software) (MAY 2014) for civilian agencies or the Defense Federal Acquisition Regulation Supplement (DFARS) at 227.7202-1(a) (Policy) and 227.7202-3 (a) (Rights in commercial computer software or commercial computer software documentation) (FEB 2014) for the Department of Defense, PTC software products and software documentation are provided to the U.S. Government under the PTC commercial license agreement. Use, duplication or disclosure by the U.S. Government is subject solely to the terms and conditions set forth in the applicable PTC software license agreement.

PTC Inc., 121 Seaport Blvd, Boston, MA 02210 USA

Contents

Release Notes	4
Technical Summary of Changes for Creo 12.4.0.0	5
User's Guide	6
Overview of Creo.JS	7
Working with Creo.JS	8
Migrating Existing Web.Link Applications to Creo.JS	15
Sample Applications	29
Creo.JS Testing Web Tool	30
Creo.JS Debugger	33
Session Objects	40
Selection	53
Models	59
Drawings	68
Solid	116
Solid Bodies	134
Windows and Views	135
ModelItem	145
Features	149
Datum Features	164
Geometry Evaluation	173
Dimensions and Parameters	185
Relations	196
Assemblies and Components	197
Family Tables	209
Action Listeners	212
Interface	220
Simplified Representations	257
Task Based Application Libraries	264
Graphics	269
External Data	272
Windchill Connectivity APIs	277
Geometry Traversal	295
Geometry Representations	297
Index	310



1

Release Notes

Technical Summary of Changes for Creo 12.4.0.0.....	5
---	---

Technical Summary of Changes for Creo 12.4.0.0

This section describes the critical and miscellaneous technical changes in Creo 12.4.0.0 and Creo.JS. It also lists the new and superseded functions for this release.

Critical Technical Changes

This section describes the changes in Creo Parametric 12.4.0.0 and Web.Link that might require alteration of existing apCreo.JS applications.

Support For Microsoft Edge Browser

In Creo Parametric 12.4.0.0, the configuration option `windows_browser_type` has a new value `edge_browser` which indicates that Microsoft Edge can be used as the embedded browser in Creo. Currently, Microsoft Edge browser will be available in preview mode only and will be fully supported in a future release of Creo.

New Functions

This section describes new functions for Creo.JS for Creo 12.4.0.0.

Features

New Function	Description
<code>pfcFeature.GetPatternByType()</code>	Returns the pattern of the feature.
<code>pfcFeature.PatternStatus</code>	Returns the pattern status of the feature.
<code>pfcFeature.GroupStatus</code>	Returns the group status of the feature.
<code>pfcFeature.GroupPatternStatus</code>	Returns the group pattern status of the feature.

Evaluation of Surfaces

New Function	Description
<code>pfcSurfXYZData.FirstDerivative</code>	Returns the first partial derivatives of X, Y, and Z, with respect to the <i>u</i> and <i>v</i> parameters.
<code>pfcSurfXYZData.SecondDerivative</code>	Returns the second partial derivatives of X, Y, and Z, with respect to the <i>u</i> and <i>v</i> parameters.

2

User's Guide

Overview of Creo.JS	7
Working with Creo.JS	8
Migrating Existing Web.Link Applications to Creo.JS	15
Sample Applications	29
Creo.JS Testing Web Tool	30
Creo.JS Debugger	33
Session Objects	40
Selection	53
Models	59
Drawings	68
Solid	116
Solid Bodies	134
Windows and Views	135
ModelItem	145
Features	149
Datum Features	164
Geometry Evaluation	173
Dimensions and Parameters	185
Relations	196
Assemblies and Components	197
Family Tables	209
Action Listeners	212
Interface	220
Simplified Representations	257
Task Based Application Libraries	264
Graphics	269
External Data	272
Windchill Connectivity APIs	277
Geometry Traversal	295
Geometry Representations	297

Overview of Creo.JS

Overview of Creo.JS

In a previous release of Creo, the embedded browser, which was used to work with Web.Link applications was updated to run out of the Creo process. The inter-communication overheads caused some performance issues.

In Creo 4.0 M080 release and later, a new toolkit Creo.JS is available. Creo.JS engine helps to improve the performance of the existing Web.Link applications. To leverage the advantages of Creo.JS, you will have to update the existing Web.Link applications. The toolkit uses a Chrome V8 JavaScript engine, which runs inside the Creo process. This removes the inter-communication overheads. The engine supports the ECMAScript 2018 specification. Creo.JS framework runs Creo applications that are written in JavaScript.

Creo.JS Application

Creo.JS application is written in JavaScript. It is deployed on a single web page. It is a distributed application which consists of two parts:

- JavaScript sections that control dynamic HTML user interface and user interaction. These scripts can contain combination of HTML browser APIs for DOM manipulation and Web.Link APIs. This part of the application runs in the browser.
- JavaScript sections that interact with the Creo toolkit APIs. These scripts are loaded in the Creo process and are executed by the Creo.JS engine, which is a part of the Creo process.

The parts of the application that interact with the Creo session can be combination of Web.Link code that is executed in the browser and Creo.JS code that is executed in Creo. This enables you to gradually move your applications from Web.Link to Creo.JS. Since both parts of the application run in different processes they cannot exchange direct references to their runtime objects. They interact by passing JSON objects. All calls between environments are asynchronous. You must use programming patterns, such as, AJAX subscription style, or reactive development in Node.js, for distributed application development in JavaScript.

Use JavaScript promises in Creo.JS to work with callback functions.

Creo.JS supports all the Web.Link APIs. Similar to the Web.Link application, the Creo.JS application is sent to the embedded browser of Creo and does not require any additional components.

Installing Creo.JS

Creo.JS is available on the same CD as Creo Parametric. When Creo Parametric is installed using PTC.SetUp, one of the optional components is API Toolkits. This includes Creo.JS toolkit.

Creo.JS is installed when you install Web.Link. The following folders are installed for Creo.JS:

- A directory called `creojs` is created at `C:\Program Files\PTC\Creo 4.0\apps`. This directory contains all the libraries and example applications specific to Creo.JS.
- A directory called `creojs` is also created at `C:\Program Files\PTC\Creo 4.0\Common Files\creojs`. This directory contains documentation specific to Creo.JS. It contains an online browser called the Creo.JS API Wizard, which contains definitions for modules, classes, and methods. The *Creo.JS User's Guide* in PDF format is also installed in this folder.

MIME Type for Webserver

The administrator can add all common scripts to the server instead of copying them to each user's computer. In such a case, the folder `creojsweb` is located on your local server. If administrator is using IIS Webserver for adding common scripts, a custom defined MIME type `*.creojs` needs to be added to the Webserver IIS (Windows 7/10). You need to configure the server configuration file `web.config` as follows:

```
<configuration>
<system.webServer>
<staticContent>
<mimeMap fileExtension=".creojs" mimeType="text/creojs" />
</staticContent>
</system.webServer>
</configuration>
```

Working with Creo.JS

Working with Creo.JS

This section explains how to work with Creo.JS.

Initialize Creo.JS Framework

Include the `creojs.js` script in every web page of the application to work with Creo.JS. The script enables interaction between the web browser environment and Creo.JS engine. The web page uses the JavaScript object `CreoJS`, which is initialized by this script, to call the Creo.JS toolkit.

`creojs.js` and other scripts that are available with Creo.JS are available in `creojsweb` at `<creojs_loadpoint>`. To use these scripts in the Creo.JS web applications, they should be located in the same folder as the application web content. These scripts are served when the application accesses them. You can copy, modify, and distribute these scripts along with your applications.

To initialize Creo.JS, include the following code in the `<head>` element of the web application:

```
<script src="creojs.js"></script>
```

If you want the Creo.JS script to call the web browser page, include the following code in the `<head>` element of the web application:

```
<script type="text/creojs" src="browser.creojs"></script>
```

Special characters received by the browser are stored in the following two ways:

- Using the extended ASCII table for the selected national alphabet where each special character has its own code from the range 128 to 255.
- Using the Uniform Transformation Format(UTF).



Note

PTC recommends that you use text that contains English characters, digits, all characters from the first half of ASCII table, and any UTF encoded text. Text containing special characters from national alphabets which is not encoded in UTF is not supported in Creo.

Creo.JS Scripts

The JavaScript code executed by Creo.JS must be included in the `<script>` tag, which has the type set as `text/creojs`. The `src` attribute is supported for scripts included in the web page. The scripts must be located relative to the HTML page location.

Absolute scripts references, for example, `src=http://some-server/foobar.creojs` are not supported.

After a web page loads, Creo.JS framework scans the DOM of the web page. It loads all the Creo.JS scripts in the connected Creo session. The scripts are executed by Creo.JS engine.

Creo.JS scripts are loaded in Creo and executed in the order of their appearance on the web page. The Creo.JS scripts are loaded and executed asynchronously when the web page loads.

When the web page fully loads, Creo.JS scripts may not have been fully loaded, even though all page elements `onload` event listeners are executed. If you want the web page to wait for Creo.JS initialization and termination events, set the event listeners `CreoJS.$ADD_ON_LOAD` and `CreoJS.$ADD_ON_UNLOAD` respectively.

The `CreoJS.$ADD_ON_LOAD` adds an event handler that executes the initialization script only after all the Creo.JS scripts are loaded. To ensure that the initialization code which interacts with Creo.JS code is executed after all page scripts are initialized, set Creo.JS `onload` event handler inside the `onload` event handler of the page in the `<body>` tag. See the following sample code:

```
<html>
  <head>
    <script src="creojs.js"></script>
    <script type="text/creojs" src="browser.creojs"></script>
    <script type="text/creojs">
      function getActiveModelName () {
        return pfcGetCurrentSession ().GetActiveModel
        ().FileName;
      }
    </script>
  </head>
  <body onload="CreoJS.$ADD_ON_LOAD (initialize)">
    <b>Active model:</b> <span id="MODELNAME"></span>
    <script>
      function initialize () {
        CreoJS.getActiveModelName ()
          .then (function (name) {
            MODELNAME.innerHTML = name;
          })
      }
    </script>
  </body>
</html>
```

Use the `CreoJS.$ADD_ON_UNLOAD` event handler to execute the deinitialization code when the browser unloads the page. These handlers execute the script before Creo.JS destroys context associated with the page being unloaded. You can also release the Creo resources allocated by the page.

Each web page has its own isolated context inside the Creo.JS engine. The context is created when the web page is loading and destroyed when the web page is unloading. The pages can communicate with each other using the Creo.JS session storage, which maintains a global context of Creo.JS engine. This is similar to the browser property `Window.sessionStorage`.

Embedded Scripts

It is recommended to use the `id` attribute for embedded Creo.JS scripts. The `id` are returned in errors which further helps to identify location of the problem.

The following example shows an embedded script.

```
<script type="text/creojs" id="example1.creojs">
    function getActiveModelName () {
        let session = pfcGetCurrentSession ();
        let model = session.GetActiveModel ();
        return model ? model.FileName : null
    }
</script>
```

External Scripts

When you use external Creo.JS scripts it is recommended to use `.creojs` extension. Do not use `.js` extensions for external scripts. The `.js` scripts may not load properly in the web page due to the file type association of the system.

If scripts do not load in Creo with `.creojs` extension, check your system for `.creojs` file type association. If there is a default program assigned to handle `.creojs` files, remove it to resolve this problem.

This following example shows an external script.

```
<script type="text/creojs" src="test/example3.creojs"></script>
```

Calling Creo.JS Code from Browser Code

Browser code can call any globally declared Creo.JS function using the syntax `CreoJS.functionName(parameters)`. Such calls are executed asynchronously and return a Promise object, which is used to access the result when it is ready. The input arguments are serialized as JSON, which are sent to Creo.JS environment along with the function name. When Creo.JS environment is ready to run the call, it is asynchronously executed by the engine and a JSON serialized result is sent back to the browser page. To process the result, the browser code subscribes to it by providing a processing callback function to the method `then()`. The callback function is available in the Promise object returned by Creo.JS function call.

In the example below, the browser code calls `getModelType` function, which is declared in `<script type="text/creojs">` section. A callback function is registered to consume the result and use the result value to render the HTML text in the current document.

```

<script type="text/creojs" id="curmodelname.creojs">
    let session = pfcGetCurrentSession ()
    function getModelType (filename) {
        try {
            let model = session.GetModelFromFileName (filename)
            return model ? model.Type.string () : null
        }
        catch (ex) {
            return null
        }
    }
}
</script>

<script>
    var filename = document.getElementById ('filenameinput').value
    CreoJS.getModelType (filename).then (function (modelType) {
        var type = modelType || 'not found'
        var text = filename + ' has type "' + type + '"'
        document.appendChild (document.createTextNode (text))
    })
</script>

```

JavaScript which runs in the browser uses the `CreoJS.functionName (parameters)` syntax to call a function, which is declared in the `Creo.JS`. This call returns a `Promise`. A `Promise` represents a return value, which is available when the calling function exits. The return value is delivered asynchronously from `Creo.JS` to the browser environment. To consume the return value, the code that calls it must have a callback function. When `Creo.JS` sends the result to the browser environment, the callback function is called with the return value. Use `Promise.then (valueConsumingCallback)` syntax to register the callback.

Creo.JS Exception Handling

You can handle `Creo.JS` exceptions in its code. This ensures that exceptions are not thrown between the browser and `Creo` environment. If the exceptions are not caught on the `Creo` side, they can be processed by the browser callbacks. There are two types of interceptors for exception handling:

- Errors are handled by `onError()` interceptor. `onError()` is used to process JavaScript error objects or exceptions derived from JavaScript error objects.

Such exceptions contain traceback information and can be used for more detailed problem reporting.

- Application exceptions are handled by `onException()` interceptor. `onException()` is used to process all other thrown objects.

You can also use `catch()` to setup a callback that handles any type of throw objects.

The following example shows how to handle exceptions.

```
<script type="text/creojs" id="exceptions.creojs">
    function testException (isError) {
        if (isError) {
            throw new Error ('this is error')
        }
        else {
            throw {test: 'this is application exception'}
        }
    }
</script>

<script>
    var isError = document.getElementById ('iserrorcheckbox').
checked
    CreoJS.testException (isError)
        .onError (function (err) {
            // catches new Error ('this is error')
        })
        .onException (function (ex) {
            // catches {test: 'this is application exception'}
        })
</script>
```

Use `Promise` to call `finally()`. The function is called after the execution of the `CreoJS` function either after `then()` or `catch()` callback.

Chaining Promises are used to simplify the sequences of asynchronous calls.

Note

The `Creo` embedded browser supports Internet Explorer, which does not have embedded support for Promises. Due to this limitation of Internet Explorer, a small subset of the Promise features, `then()`, `catch()`, and `finally()`, are supported. There is no support for `async` and `await`, which are keywords of ECMAScript specification. However, full Promise interface is supported in `CreoJS`.

Calling Browser from Creo.JS

Creo.JS code can call the global functions declared in the browser page. These calls are asynchronous and also return Promise. Use the browser object to access the browser functions. You can call all the global functions, which are available in the browser environment. For example, `alert()`, `prompt()`, and `confirm()`, and so on.

```
<script type="text/creojs" id="callbrowser.creojs">
    function callBrowser (message) {
        Browser.alert ('this is alert: ' + message)
        Browser.addTextNode (message)
    }
</script>

<script>
    function addTextNode (message) {
        document.appendChild (document.createTextNode (message))
    }
    CreoJS.callBrowser ('Just text')
</script>
```

ECMAScript 8 keywords `async` and `await` can be used to synchronously call browser code from Creo.JS. In the following example, Creo.JS script uses the browser method `prompt()` to make two browser calls to enquire about the first and last name of the user. It uses browser method `alert()` to display the full name.

In the example below, the function `askFullName` is declared as `async` and is executed asynchronously by Creo.JS environment. However, internally it synchronizes with two browser `prompt()` calls using `await` construct. All the functions that are declared `async` return a Promise. The result is consumed with a callback lambda-function, which is set using `Promise.then()`.

```
<script type="text/creojs" id="askfullname.creojs">
    async function asyncCall (message, response = '') {
        return Browser.prompt (message, response)
    }

    async function askFullName (fn, ln) {
        const first = await asyncCall ('What is your first name?',
fn)
        const last = await asyncCall ('What is your last name?',
ln)
        return {first, last}
    }

    const user = pfcGetCurrentSession ().GetEnvironmentVariable
('USERNAME')
    askFullName (user.slice (0,1).toUpperCase (),
        user.slice (1,2).toUpperCase () + user.slice (2))
        .then (name =>
```

```
        Browser.alert (`Your name is ${name.first} ${name.last}
    `))
</script>
```

Recommended Best Practices

It is recommended to use the following best practices while creating Creo.JS applications:

- Do not have frequent calls between environments. There are inter-process communication overheads of passing Creo.JS call parameters and its result between the browser and Creo processes.
- Collect all the data required by a transaction and exchange it in a single call.

Enabling Security Settings

Every time the Internet Explorer or Chromium browser receives HTTP requests from unknown sites, which try to access Creo.JS or Web.Link methods and objects, a warning message appears. You are prompted to confirm if you want to allow the request from the site.

The warning message provides you with the ability to save the decision.

- If the request from the site is allowed, in future all the requests from the site will be executed without any warning message.
- If the request from the site is not allowed, in future all the requests from the site will not be executed, and the warning message is not displayed.

Migrating Existing Web.Link Applications to Creo.JS

Migrating Existing Web.Link Applications to Creo.JS

This section describes how to migrate existing Web.Link to Creo.JS.

Overview

You can migrate your existing Web.Link applications to Creo.JS. You must split the existing code in two groups of script sections, one section that runs in the browser and another that runs in Creo.

Web.Link code in the sections that continue running in the browser, does not have to be migrated as all the Web.Link features are supported. You can begin by migrating the critical sections of your Web.Link application, whose performance

has been affected by the browser architecture change. Such sections usually contain many calls to Creo APIs in a single user interaction with web application. For example, querying and modifying items, features or parameters of Creo models, traversing assembly structures, and so on.

Guidelines to Migrate the Applications

Use the following guidelines to move existing Web.Link applications to Creo.JS.

While Creo.JS supports the same set of APIs as Web.Link, it implements some data representation changes to work with the most recent ECMAScript extensions. The subsequent sections of the guide explain the API changes.

This new approach improves the development productivity. The `weblink.legacy.creojs` script reduces the effort of migrating the existing Web.Link applications. `weblink.legacy.creojs` wraps most of the Creo.JS-style APIs into old Web.Link-style APIs. This enables developers to preserve the old style during the migration. See the subsequent sections for information on where Web.Link style is supported by including the `weblink.legacy.creojs` script.

```
<script type="text/creojs" src="weblink.legacy.creojs"></script>
```

Convert Collections in Web.Link to Arrays in Creo.JS

Creo.JS changed Web.Link sequences representation to JavaScript arrays. This enables use of JavaScript array construction and processing APIs.

The Web.Link functions that accept and return collections can be converted to use arrays in Creo.JS.

Consider the following Web.Link code with collections:

```
<script>
    var features=SubComp.ListFeaturesByType
        (true, pfcCreate ("pfcFeatureType").FEATTYPE_
COMPONENT);
    for (var mi=0; mi < features.Count; mi++)
    {
        var feature=features.Item(mi);
        . . .
    }
</script>
```

You can rewrite the Web.Link code to Creo.JS as below:

```
<script type="text/creojs" id="iteratefeatures.creojs">
    const features = SubComp.ListFeaturesByType
        (true, pfcFeatureType.FEATTYPE_COMPONENT);
    // classical loop
    for (var mi=0; mi < features.length; mi++) {
        const feature=features [mi];
        // . . .
    }
</script>
```



```

// range loop
for (const feature of features) { // . . .
}
// functional style with arrow function
features.forEach (feature => { /* ... */ })
</script>

```

The existing Web.Link style can be retained by including the `weblink.legacy.creojs` script.

Use the Method `pfcGetCurrentSession ()` Instead of `MpfcCOMGlobal.GetProESession ()`

Web.Link method `pfcCOMGlobal.GetProESession ()` must be replaced with the method `pfcGetCurrentSession ()` in Creo.JS.

Consider a Web.Link code that uses the method

`pfcCOMGlobal.GetProESession ()`.

```

<script>
  var session = null;
  function init()
  {
    var ComObj = pfcCreate ("MpfcCOMGlobal");
    if (ComObj) {
      session = ComObj.GetProESession ();
      alert ("Successfully connected to Creo!");
    }
    else {
      alert ("Cannot connect to Creo!");
    }
  }
  init ()
</script>

```

You can rewrite the Web.Link code to Creo.JS as below:

```

<script type="text/creojs" id="initsession.creojs">
  const session = pfcGetCurrentSession ()
</script>

```

The existing Web.Link style can be retained by including the `weblink.legacy.creojs` script.

Call Global Functions Directly Without Calling `pfcCreate`

Global functions that are declared on model levels are explicitly predeclared in the global context of Creo.JS. These functions can be directly referenced with module prefix and function name in the applications.

Consider the following Web.Link code:

```

<script>
  var compPath=pfcCreate ("MpfcAssembly").CreateComponentPath
    (asmObj, intPath);

```

```
</script>
```

You can rewrite the Web.Link code to Creo.JS as below:

```
<script type="text/creojs" id="testglobalfunc.creojs">
    const compPath=pfcCreateComponentPath(asmObj, intPath);
</script>
```

The existing Web.Link style can be retained by including the `weblink.legacy.creojs` script.

Use Arrays for Basic Types Collections

In Web.Link an array of integers is created as `intseq`.

```
<script>
    var intPath=pfcCreate ("intseq"); // Create sequence
    intPath.Append (1); // Append item
    // . . .
    intPath.Append (4); // Append item
    // Remove last item
    intPath.Remove (intPath.Count - 1, intPath.Count);
</script>
```

In Creo.JS an array of integers is created as `intPath`.

```
<script type="text/creojs" id="arrayvsseq.creojs">
    const intPath=[]; // Create array
    intPath.push (1); // Push item 1
    intPath.push (2,3,4); // Push a group of items 2,3,4
    // Remove last item
    intPath.pop ();
</script>
```

In Creo.JS you can also use an array initializer.

```
<script type="text/creojs" id="arrayvsseq.creojs">
    const intPath=[1,2,3,4]; // Create and initialize array
    // Remove last item
    intPath.pop ();
</script>
```

If you want to pass a sequence as an argument to a Web.Link method, you can pass an array of JavaScript objects. The conversion of the array to sequence is done internally.

The existing Web.Link style can be retained by including the `weblink.legacy.creojs` script.

Enumerations are Represented as Objects Without Calling `pfcCreate`

In Creo.JS you can reference enumerations directly without calling `pfcCreate`. This is because all types definitions are predeclared in the global context of Creo.JS.

Consider following Web.Link code, which calls `pfcCreate` to reference enumerations.

```
<script>
  var componentType=pfcCreate ("pfcFeatureType").FEATTYPE_COMPONENT
  var featureStatus=pfcCreate ("pfcFeatureStatus").FEAT_ACTIVE
  var partType=pfcCreate ("pfcModelType").MDL_PART
</script>
```

You can rewrite the Web.Link code to Creo.JS as below:

```
<script type="text/creojs" id="enumerations.creojs">
  const componentType=pfcFeatureType.FEATTYPE_COMPONENT
  const featureStatus= pfcFeatureStatus.FEAT_ACTIVE
  const partType= pfcModelType.MDL_PART
</script>
```

In Web.Link enumeration value is an integer, while in Creo.JS it is an object. In Creo.JS the method `value()` returns the integer value and `string()` returns the string representation. The method `values()` returns an array of objects representing all enumeration values.

For example, the following code defines a Creo.JS method, which returns a JavaScript object, which maps enumeration string representations to their integer values. The HTML rendering part of the application calls the Creo.JS function and appends unordered list to the current document with enumeration names and their respective integer values.

Note

If you are combining Web.Link and Creo.JS code in the same application, it is recommended to convert enumerations to integers.

```
<script type="text/creojs" id="enumitemtypes.creojs">
  function getModelItemTypes () {
    const types = {}
    for (const v of pfcModelItemType.values()) {
      types [v.string()] = v.value()
    }
    return types
  }
</script>

<script>
  CreoJS.getModelItemTypes ().then (function (types) {
    if (types) {
      var listElement = document.createElement ('ul')
      document.appendChild (listElement)
      for (var itemType in types) {
        var listItem = document.createElement ('ul')
```

```

        listItem.innerHTML = '<b>' + itemType + '</b>: ' +
                               types [itemType]
        listElement.appendChild (listItem)
    }
}
}))
</script>

```

`pfcFeatureType.FEATTYPE_COMPONENT` returns the enumeration value object in Creo.JS. It generally does not return an integer value. The method `toPrimitive(mode)` is defined on all enumeration objects. This method must not to be called explicitly. JavaScript executes this method implicitly depending on context. In Creo.JS, this method is defined to return enumeration value name when it is being used in string context and to return its integer value when used in numeric context. In the following example, the method prints numeric value of `pfcFeatureType.FEATTYPE_COMPONENT`.

```

<script type="text/creojs" id="enumtoprimitive.creojs">
    Browser.alert (+pfcFeatureType.FEATTYPE_COMPONENT);
</script>

```

If it is not clear which context is used, call the method `value()` to get the integer value.

Additional API Features Introduced in Creo.JS

Getting Creo Object Type Information

Creo API object hierarchy design uses multiple inheritance hierarchy to model the API concepts. Use JavaScript `instanceof` construct to check only the most derived type of the object. To check if object is an instance of any intermediate class in the hierarchy use `isInstanceOf()` method as shown in the following example. Note that the API accepts either string type name `pfcAssembly` or type variable `pfcModel.getClassName()` method to return the name of object class as a string.

```

<script type="text/creojs" id="typeinfoexample.creojs">
    let currentModel = pfcGetCurrentSession().CurrentModel
    print (currentModel.isInstanceOf ("pfcAssembly"))
    print (currentModel.isInstanceOf (pfcModel))
    print (currentModel.isInstanceOf (pfcSolid))
    print (currentModel.isInstanceOf (currentModel.getClassName()))
</script>

```

Managing Session Data

While each set of Creo.JS scripts that belong to the same page work in their isolated contexts, they can interact with each other through session storage maintained in the Creo.JS global context. The following functions manipulate this context:

- `saveToSession(key, value)`—Saves `value` to the global session map under the name `key`.
- `getFromSession(key)`—Returns the global session map value saved under the given name.
- `removeFromSession(key)`—Removes value from the global session map specified by the given `key`.

Managing User Application Data

Creo.JS contains a set of functions that allows you to store and access data in your Application Data (AppData) folder (both local and roaming.) The following functions enable you to manipulate application data:

- `saveToAppData(relativeFileName, data, roaming=false)`—Saves the given data string to a file inside your AppData folder for Creo.JS data (CreoJS/data by default). Relative file name may contain directories. If such a directory does not exist in `getCreojsAppDataFolder(roaming)`, it is created.
- `getFromAppData(relativeFileName, roaming=false)`—Reads a specified file from your AppData folder for Creo.JS data. The function throws an exception if the file does not exist. On success returns a string with the content of the specified file.
- `deleteFromAppData(relativeFileName, roaming=false)`—Removes specified file from your AppData folder. The function throws an exception on failure.
- `existsAppData(relativeFileName, roaming=false)`—Returns `true` when a file with the given relative name exists in `getRoamingAppDataFolder(roaming)`, `false` otherwise.

The following functions enable you to retrieve the full name of your AppData folders:

- `getAppDataFolder(roaming=false)`—Returns the full name of the current user's AppData folder—Local (default) or Roaming.
- `getLocalAppDataFolder()`—Returns the full name of the current user's Local subfolder inside the AppData folder. Same as `%LOCALAPPDATA%` or `getAppDataFolder(false)`.
- `getRoamingAppDataFolder()`—Returns the full name of the current user's Roaming subfolder inside the AppData folder. Same as `%APPDATA%` or `getAppDataFolder(true)`.

-
- `getCreojsAppDataFolder(roaming=false)` —Returns the full name of the directory inside the current user's AppData designated to store Creo.JS objects. By default, it is `CreoJS/data`. Can be reconfigured through the setting `appdata_subdir` in the `creo_js_app.conf` file.
 - `getCreojsLocalAppDataFolder()` —Same as `getCreojsAppDataFolder(false)`.
 - `getCreojsRoamingAppDataFolder()` —Same as `getCreojsAppDataFolder(true)`.

Uploading or Downloading Data to and from the Web server

- `uploadFile(URL, filePath, responseHandler=null)` — Uploads the specified file to the specified server. If `responseHandler` is not provided, works in sync mode and returns a URL received from the server on success. The function throws an exception on failure.

If `responseHandler` is specified, the function returns an empty string and executes the `responseHandler` upon completing the upload providing HTTP response object as an argument of the `responseHandler`.

Note

You must have read access to the directory from which you are trying to upload the file.

For example:

```
function respHandler (httpResponse) {  
  print ('Response received: ' +  
    JSON.stringify (httpResponse, null, 4));  
}
```

```
uploadFile ('http://my.server.com/upload.php', 'my_file.bin',  
  respHandler);
```

- `uploadFiles(URL, jsonOptions ..., responseHandler=null)` —Uploads one or more files with a single request. Unlike `uploadFile` method, this function uses multipart mode. For each file to be uploaded you can specify a name by setting the `fieldName` as `jsonOptions` and content type.

In sync mode where no response handler provided, the function returns a string representation of a JSON containing information about each uploaded file on success. If response handler is specified, the returned JSON response will be passed as an argument of the response handler.

For example:

```
let resp = uploadFiles (
  'http://my.server.com/filestorage/upload.cgi',
  {filePath: 'project.gif', fieldName: 'fileToUpload'},
  {filePath: 'config.pro', fieldName: 'anotherFileToUpload'
  contentType: 'plain/text'}
);
```

- `downloadFile(URL, filePath, responseHandler=null)`—Downloads a file from the server and saves it with the specified name. Like `uploadFile` this function also works in sync or async mode. In sync mode it returns the name of the file on success or throws an exception on failure.

 Note

You must have write access to the download directory.

- `uploadString(URL, valueToUpload, responseHandler=null)`—Uploads a string to the server. It works like `uploadFile`, however uploads a simple string and not a file.
- `downloadString(URL, responseHandler=null)`—Downloads a string from the specified URL. In sync mode, it returns the body of the HTTP response as a string on success or throws an exception on failure.

For example:

```
let resp = uploadString (
  'http://my.server.com/uploadMessage.php',
  'Message to send to server'
);
print (`Retrieve though: ${resp}`);
let content = downloadString (resp);
print (`downloaded content:\n<div><i>${content}</i></div>`);
```

- `uploadJSON(URL, jsonObject, responseHandler=null)`—Uploads a JSON object to the Web server. Works like `uploadString` applied to JSON stringified `jsonObject`, however uses `Content-Type: application/json` when sending the HTTP request.
- `downloadJSON(URL, responseHandler=null)`—Downloads a string from the specified server and then parses it to convert to a JSON object. In sync mode, returns this object, and in async mode, converts the body of the HTTP response to the JSON object and then invokes the `responseHandler`.

For example:

```
const respHandler = function (httpResponse) {
  if (httpResponse.statusCode == 200 && httpResponse.body) {
    print (`Address: ${httpResponse.body.street},
    ${httpResponse.body.city},
    ${httpResponse.body.state}
    ${httpResponse.body.zip}`);
  }
};
downloadJSON ('http://my.server.com/download.php&myfile',
respHandler
);
```

Reading and Writing files

Creo.JS provides functions to access the local files.



Note

You must have read and write access rights to the folders in which the files are located.

- `readFileAsString(filename)` — Reads the content of the specified file and returns it as a single string. The parameter `filename` is either a full or a relative file name. When the relative file name is provided, the file is searched for in the working directory. The function throws an exception in case of failure.



Note

This function is defined only if the `config.pro` file contains the option `web_link_file_read` set as YES.

-
- `writeFileAsString (filename, text)`—Writes specified text to the specified file. The parameter `filename` is either a full or a relative file name. When the relative file name is provided, the file is searched for in the working directory. The function, throws an exception in case of failure.

 Note

This function is defined only if the `config.pro` file contains the option `web_link_file_read` set as YES.

Creo.JS Web server

Creo.JS has an internal Web server that provides access to Creo.JS from remote computers and allows you to access Creo.JS via HTTP protocols and through web sockets. It also lets you use Chrome DevTools for remote debugging of the Creo.JS script. The following functions manage the Web server:

- `startWebServer (port=-1)`—Starts the Web server listening on the given port. When `port` is set to -1, the value of the port is taken from the configuration file `creo_js_app.conf`. When `port` is set to 0, server will start on any available port, otherwise the server will try to start on the specified port.
- `isWebServerStarted()`—Returns true if the Web server is successfully started, false otherwise.
- `getWebServerPort()`—Returns the port number on which the Web server is listening. If the Web server is not started, the function returns -1.
- `addWebSocketServerHandler (path, dataHandler, closeHandler=null, errorHandler=null, openHandler=null)` or `addWebSocketServerHandler (path, {dataHandler, closeHandler, errorHandler, openHandler})`—Sets a web socket handler for the specified path. The second parameter is an object that may contain the following handlers:
 - `dataHandler`— function (WebSocketConnection conn, string msg)
 - `closeHandler`— function (WebSocketConnection conn)
 - `errorHandler`— function (WebSocketConnection conn, string msg)
 - `openHandler`— function (WebSocketConnection conn)

For example:

```
addWebSocketServerHandler ('/test', {  
  dataHandler: function (conn, msg) {
```

```

print (`'/test' handler received message:
<div>${msg}</div>`);
conn.send (`'/test' handler successfully received
${msg.length} bytes`);
}
errorHandler: function (conn, err) {
    print (`Error: ${err}`);}
}
);

```

- `removeWebSocketServerHandler(path)` —Removes a web socket handler that corresponds to the specified path.
- `addServerHandler(path, dataHandler, method=RequestMethod.ALL)` —Sets a Web server handler for the specified path. Data handler is the function function (WebServerRequest req, WebServerResponse resp) that processes the requests.

Request `req` contains information about the incoming request, while response `resp` has tools for sending response. The `method` argument specifies the type or types of requests this handler serves. Following are the possible values:

- `RequestMethod.GET`,
- `RequestMethod.POST`,
- `RequestMethod.PUT`,
- `RequestMethod.HEAD`,
- `RequestMethod.PUT`,
- `RequestMethod.DELETE`,
- `RequestMethod.OPTIONS`,
- `RequestMethod.PATCH`,
- `RequestMethod.ALL`
- or any combinations of the above.

For example:

```

addServerHandler ('/api',
    function (req, resp) {
        print (`Executing function ${req.GET['function']}`);
        var res = execute (req.GET['function'],
            req.GET ['arg']);
        resp.writeHead (200, 'OK',
            {'Content-type': 'application/json'})
        .end (JSON.stringify (res));
    }
);

```

```
}, RequestMethod.GET);
```

- object `WebRequest`—Creo.JS type that contains the following methods:
 - `url(string)`
 - `headers(map)`
 - `method(string)`
 - `query(string)`—part of the URL address after '?' (not including)
 - `remoteAddress(string)`—address of the machine that sent the request
 - `remotePort(int)`—port of the remote machine
 - `contentType(string)`—value of the 'Content-Type' header
 - `body(string)`—body of the request
 - `GET(map)`—all arguments from the query string
 - `POST(map)`—all arguments from the body of the POST request
 - `PARAMS(map)`—all arguments from the combined methods
 - `saveToFile(filename)`—saves the body of the request in the given file, provided Creo.JS has write access to the directory. If filename is a relative path, it is considered from the Creo current running directory.
 - `saveToAppDataFile(relFileName, roaming=false)`—saves the body of the request to a file located in the current user's AppData folder Local or Roaming.
- object `WebResponse`—Creo.JS type that has the following methods:
 - `writeHead(statusCode, statusMessage, headers {name:value})`
 - `setHeader(name, value)` — add response header
 - `write(data)`—append the data to the response body
 - `writeFromFile write(filename)`—writes the specified file to the response output stream, provided Creo.JS has read access to the directory, where the file is located.
 - `writeFromAppDataFile(relFileName, roaming=false)`—writes contents of the specified file from the current user's AppData Local or Roaming folder to the response stream
 - `end(data=null)`—complete and send the response
- `removeServerHandler(path)`—Removes the Web server handler for the given path.
- `stopWebServer()`—Stops the currently running Web server.

Including External Scripts

You can split complex scripts into a set of scripts where one main script uses other scripts as libraries in Creo.JS. This is implemented through the `require` method like NodeJS has.

- `require(requiredScriptName)`—Behaves in the same way NodeJS `require` does: embraces the code in the required file with `function (module, exports){and },)` executes the created function and returns `module.exports` as the result. Searches for the `requiredScriptName` in the current script directory, then in the directory one level up, and so on until the file in need is found or directory under consideration is not an accessible directory, then keeps searching in the `getScriptPath()`.

When a file was successfully required (found and executed) in a context, it gets cached for this context and all consecutive calls of the `require` function with this script returns the cached value.

- `forgetRequired(requiredScriptName)`—Removes the required file from cache. That is, consecutive calls of `require` results in reloading and re-executing the required script. This function could be helpful in the developing stage.
- `forgetAllRequired()`—Removes all cached required scripts from the current context.

Accessing Scripts Local Location

Creo.JS has an API that gives you access to locally stored script files:

- `getScriptPath()`—Returns an array of folders where `require` is looking for a required script. The script path is to be set in the `creo_js_app.conf` file by setting `js_path` property.
- `getScriptFolder()`—Returns the full path of the directory JavaScript scripts located in.
- `getScriptsList()`—Returns a string containing a list of all scripts names from `getScriptFolder()` separated by commas. Script names are the relative names to `getScriptFolder()`.
- `getScripts()`—Returns an array of all script names from `getScriptFolder()`.

Other API Features

- `setCIPSequenceHandler(arrayHandler)`—Sets a JavaScript callback that gets executed every time an invoked OTK method returns a CIP sequence. All CIP sequences get converted to JavaScript arrays. This callback allows you to define a function which takes a CIP sequence converted to an

array and define some methods. For example, `Item()`, `Set()`, `Append()`, etc. to make it behave similar to WebLink sequences for backward compatibility. See how `weblink.legacy.creojs` uses this function.

- `setUseCIPSequenceHandler(bool on)`—Turns on/off the invocation of the CIP sequence handler (if set.)

`setUseCIPSequenceHandler(false)` works like `setCIPSequenceHandler(null)`. If CIP sequence handler is not set, this function has no effect.

- `setOnContextClose(handler)`—Sets a JavaScript callback to execute upon closing the current context. For example, when the embedded browser window which is open for the current context, is closed.
- `help(name=)`—Opens a browser window with OTK help for the specified class or class method. The argument `name` can be either a `className` or `className.MethodName`. For example:

```
help ('pfcSession');  
help ('pfcSession.GetAppInfo');
```

- `print(message)`—Prints a given message to the current browser window.

Array-Like Structures

Creo.JS has array-like structures.

- Additional features are introduced on array-like structures, which represent two and three dimensional concepts, such as, `pfcPoint3D`, `pfcPoint2D`, `pfcMatrix3D`, and so on.
- Properties `x`, `y`, `z`, if applicable, representing `get(0)`, `get(1)`, and `get(2)` respectively.
- Method `dims()`—Returns an array of sizes for each dimension.
- Method `dim(dimensionIndex)`—Returns the size of the dimension with specified index.

Sample Applications

Sample Applications

This section lists the sample applications provided with Creo.JS.

Location of Sample Applications

The Creo.JS sample applications are available in `creojsweb` at `<creojs_loadpoint>`. Open the samples in the embedded browser of Creo Parametric from a local directory or from a web server.

The web page `creo_js_tests.html` lists all the sample applications, which can be opened in the embedded browser window.

Interactive Web Tool

The page `script_engine_testing.html` is a web tool, which enables interactive testing of Creo.JS code. The tool has a JavaScript editor and an output console.

The web tool supports loading and saving of test scripts from the run directory.

The web tool is accessible from `creo_js_tests.html` page also. See the section [Creo.JS Testing Web Tool on page 30](#), for more information.

Creo.JS Testing Web Tool

Creo.JS Testing Web Tool

This section provides a brief overview of the testing web tool, which is available in the Creo.JS framework.

Overview

To help you test the Creo.JS scripts with current Creo session, a testing web tool is provided with the Creo.JS installation. The tool provides an interactive user interface to work with Creo.JS APIs. You can test the APIs before using them in web applications. The page is called `script_engine_testing.html` and is available in `creojsweb` folder at `<creojs_loadpoint>`. To start using it, drag and drop the page in the Creo embedded browser. You can add the page location to favorites also.



Figure 1

The page consists of four areas:

- Creo.JS scripts toolbar
- Creo.JS script edit area
- Creo.JS script execution toolbar
- Creo.JS script output area

To run a script, type the script in the editor and click **Run**.

The script is sent to Creo.JS engine and executed. After execution, the output generated by the script is printed in the output area. In the example shown in Figure 1, the following script is executed in the tool:

```
{
  let session = pfcGetCurrentSession ();
  print (session.GetEnvironmentVariable ("PRO_DIRECTORY"));
}
```

The following result is displayed in the output area:

C:\Program Files\PTC\Creo 4.0\M080\Common Files

Note that the script is included in curly braces. This prevents the variable definitions within the script to be populated into the page context, which can prevent the consecutive execution of the same script. If you do not add the curly braces, the script can be executed only once. If you try to run it for a second time you will get an error message.

Every script executed by this page is executed in the very same context. Thus, after the script is executed the first time, the context has a new variable session defined in it. When the script is executed the second time, it tries to define already defined variable and fails. If we add { and } to the scripts, we limit the visibility area for variable session by the block specified by its boundaries { and }. Thus, after the script is executed the variable session is removed from context and can be reused.

Creo.JS Scripts Toolbar

To save a script that you executed in the tool, click **Save As** in the Creo.JS scripts toolbar. Specify a name for the script. You can specify a name with relative path in it. For example, if you specify the name as `test/test_env_var.js`, the script is stored in a subdirectory `test` of the script directory.



Scripts lists all the saved scripts. If you select a script from the list, it is loaded in the script editor. If you modify the script, the **Save** button is enabled.

The Creo.JS installation includes a library of sample Creo.JS scripts, which can you can execute in the web tool.

Creo.JS Scripts Execution Toolbar

To execute the script in the editor, click **Run**. It sends the script to the Creo.JS engine. If the script executes successfully, the output of the script appears in the area at the bottom of the page. **Clear** button enables you to clear the area, and **Clear and Run** first clears the output area and then executes the script.



The tool also has a box field where you can execute single line scripts. This gives the user the ability to execute a single command in the same Creo.JS context. For example, you can get information about an element type by running `help()` method. For example, `help("pfcSession")` opens a new window with information about the `pfcSession` type. Similarly, you can select a text in the script editor and click **Help** on the Creo.JS scripts toolbar. This will also run the function `help(selected_text)`.

Reset and Run set a new Javascript context and then execute the script. For example, the following script can be executed sequentially time after time if you use **Reset and Run**. In this case, the page is reloaded by first terminating the current Creo.JS context and initializing a new one:

```
let session = pfcGetCurrentSession ();
print (session.GetEnvironmentVariable ("PRO_DIRECTORY"));
```

Creo.JS Debugger

Creo.JS Debugger

JavaScript code might contain syntax or logical errors that can be difficult to diagnose. These errors do not have indications and need to be debugged in a runtime environment. In Creo Parametric 7.0.2.0 and later you can debug Creo.JS code by using Chrome DevTools(CDT). The Creo.JS toolkit uses a Chrome V8 JavaScript engine that runs within the Creo process and supports the standard CDT protocol. This process enables you to use the remote debugger through DevTools provided by the Chrome Web browser.

To Debug Using Chrome Remote Debugger

CDT debugger establishes a connection with Creo.JS instance in the V8 inspector protocol through a Web socket within the internal Creo.JS Web server. To make a debugger connection, the internal Creo.JS Web server must be started. To start the internal Web server, perform the following steps:

1. Manually configure Creo to specify the desired Web server port by editing the `creo_js_app.conf` file located in your working directory.
2. You can use a special Web page `start_creojis_debugger.html` to check the status of the Creo.JS Web server.

Note

The `start_creojis_debugger.html` file is used to invoke the special Web page and is included in the Creo installation at `<creojis_loadpoint>\creojis\creojisweb` location.

3. To start the Web server, open the `start_creojis_debugger.html` file in the embedded browser by specifying the full path of the file. After the page opens, you get a link for connecting to the Creo.JS via Chrome remote debugger. For example, if the Creo.JS Web server starts on port 8080, you will get the following message:

```
Creo.JS debugger
Creo.JS web server is started on port 8080.
To debug your CreoJS code please do the following:
1. Open Chrome browser
2. Open the link in a browser window

devtools://devtools/bundled/inspector.html?v8only=true&ws=
127.0.0.1:8080/debug
```

3. In the Creo embedded browser open a page you want to debug

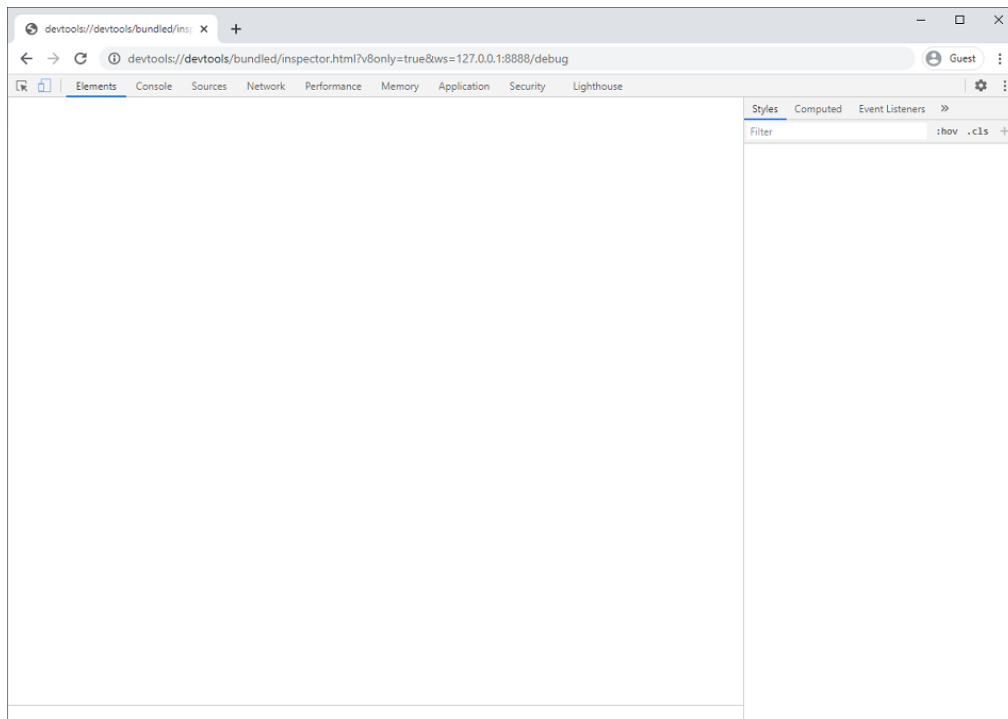
4. In the Console bottom panel select a context (usually title of the page to debug) you plan on debugging

You can also start the Web server using the `startWebServer{port_number}` function which can be invoked using the `script_engine_testing.html` web tool.

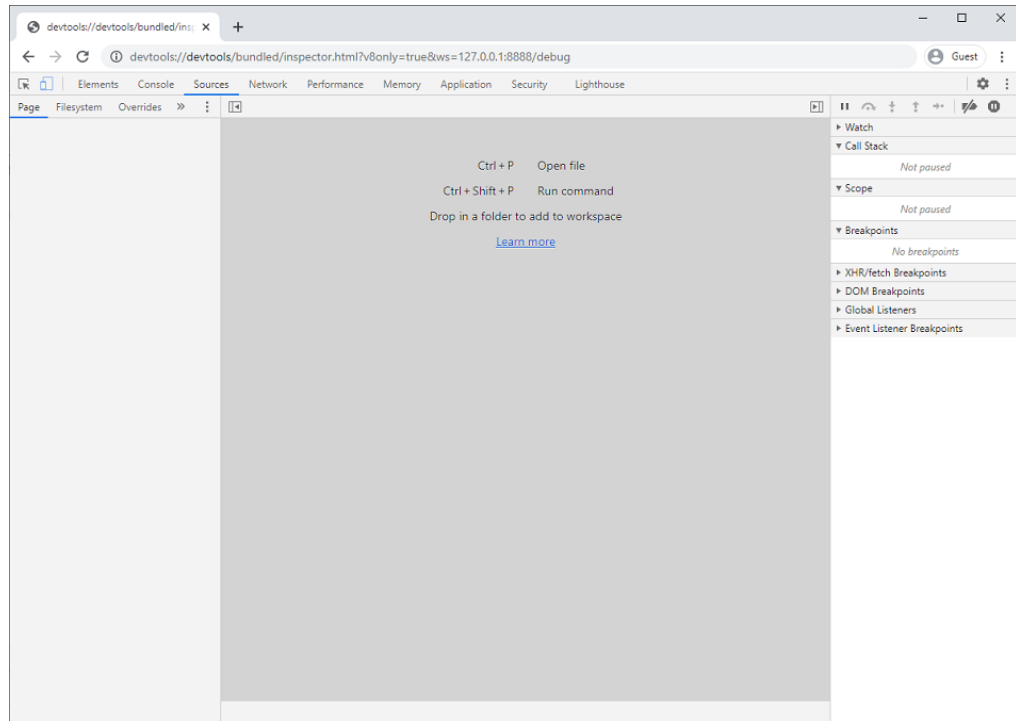
Note

If you want to start the Creo.JS Web server manually, the port number in the above link should be replaced with the configured port number from the `creo_js_app.conf` file located in your working directory.

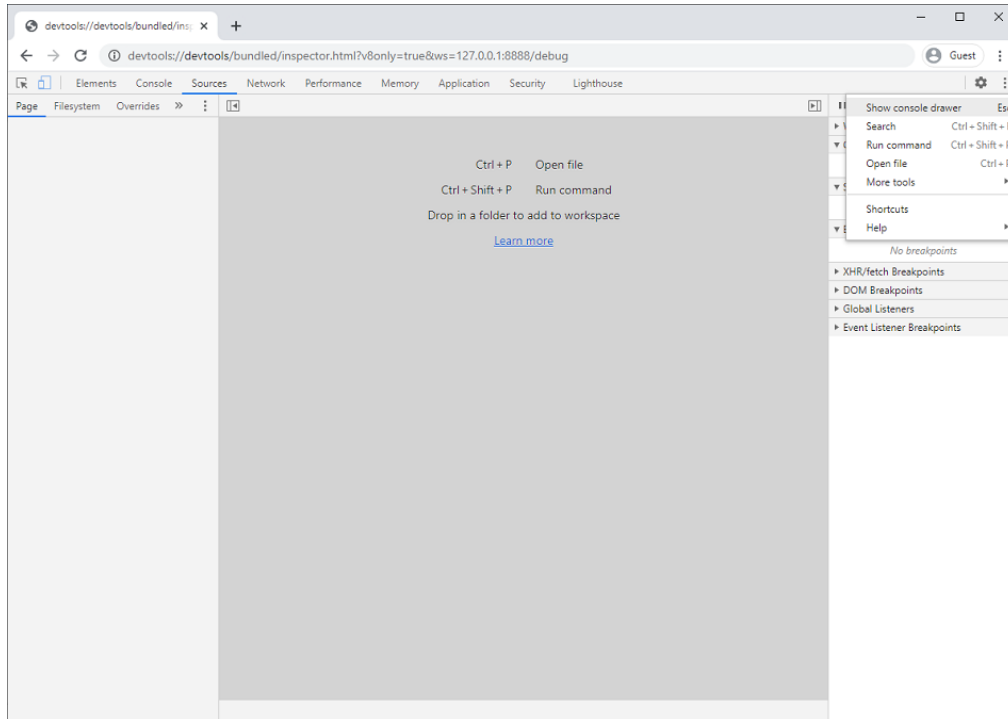
4. After the remote debugger opens, you can perform various operations using the toolset provided in the debugger, such as browsing the source of Creo.JS scripts, setting break points in debugging scripts, viewing local variables of the current context, evaluating expressions within the current context, and switching between different Creo.JS contexts. For more details about the various operations that can be performed in the debugger, refer to the `start_creojs_debugger.html` page.
5. If you are using Chrome DevTools for the first time, you will see a window similar to the following image:



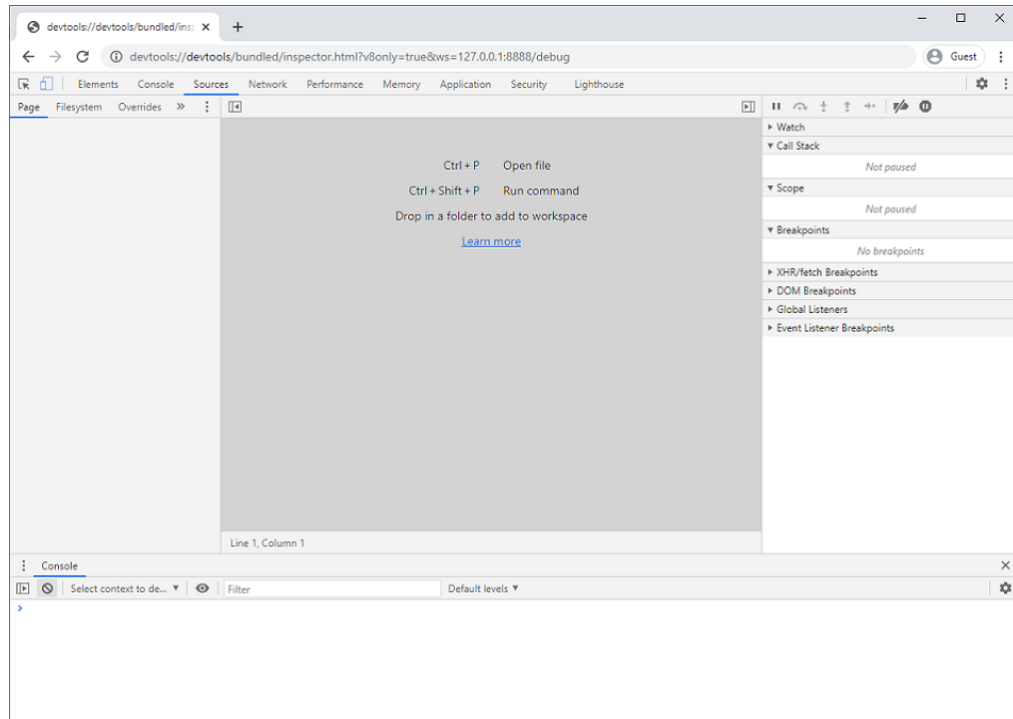
6. You can switch to the **Sources** tab if you want to access the Creo.JS scripts and perform other operations such as call stack view, local and global variables, and break points.



7. Although you can see a separate **Console** tab, it is recommended that you use the **Console** tab next to the **Sources** tab.



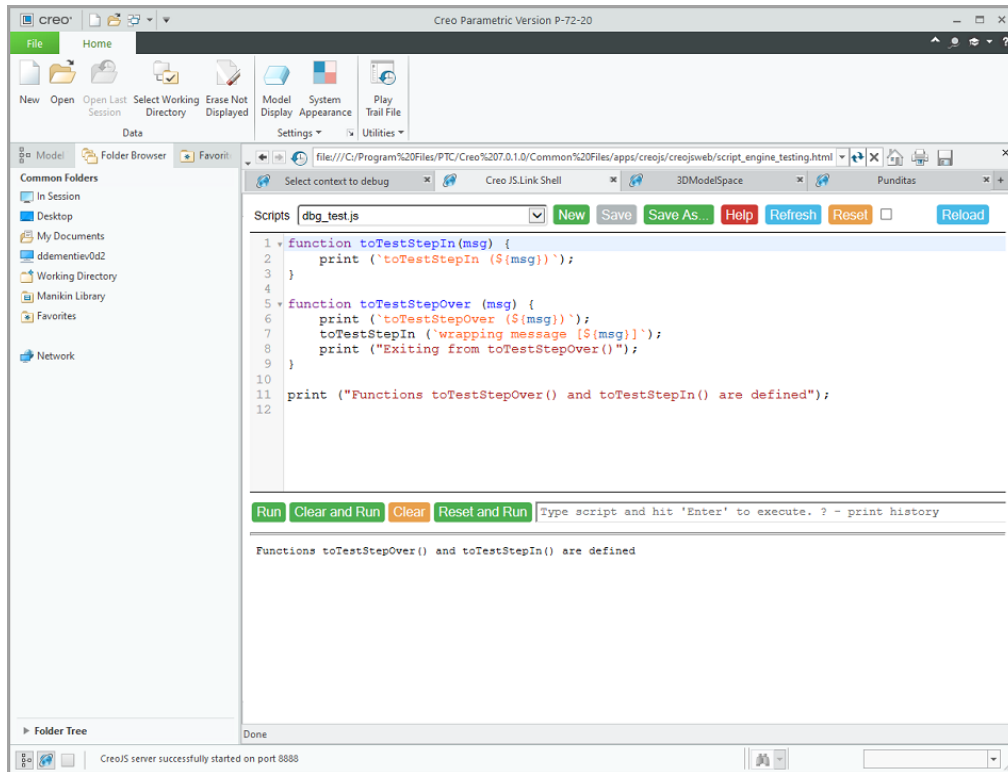
8. In the **Console** tab, you can see the context that you have currently selected and the text area where the console commands can be executed. You can see the behavior of the functions present in the present context.



Example 1: Debugging Creo.JS script

If you have already started the Creo.JS Web server and opened the Chrome remote debugger using the link provided on the `start_creojs_debugger.html` page, you can refer to the below example to see how debugging works. The following example is an illustration of the operations that you can perform in the debugger.

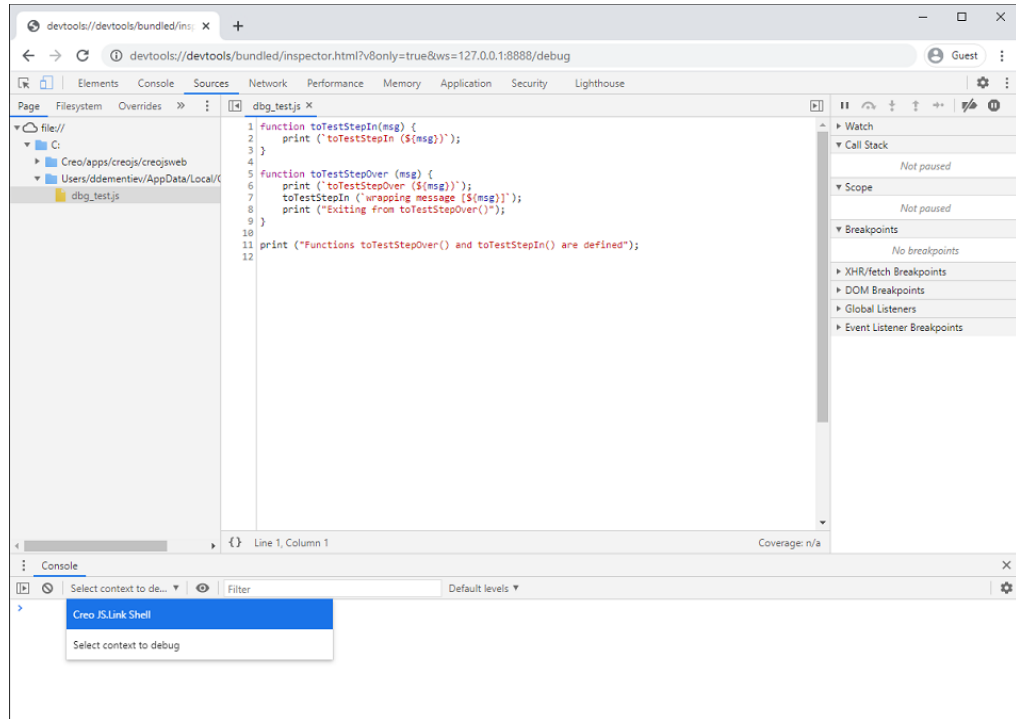
1. Open the `script_engine_testing.html` web tool in the embedded browser by specifying the full path of the file as `<creojs_loadpoint>\creojs\creojsweb` directory.



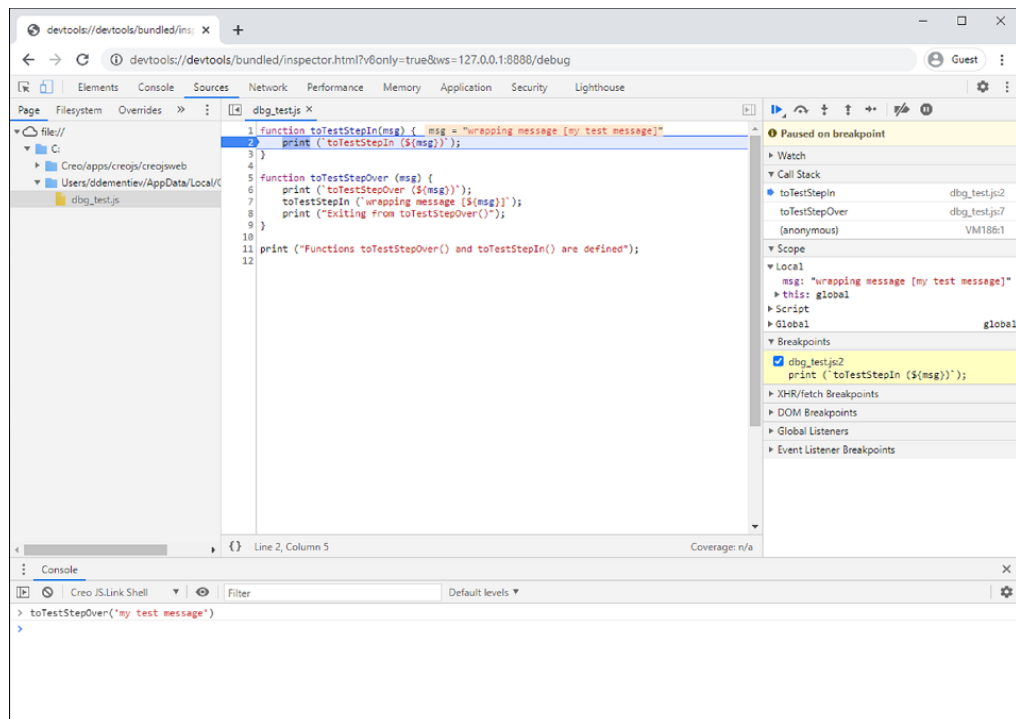
2. Type the script:

```
function toTestStepIn(msg) {
  print (`toTestStepIn (${msg})`);
}
function toTestStepOver (msg) {
  print (`toTestStepOver (${msg})`);
  toTestStepIn (`wrapping message [${msg}]`);
  print ("Exiting from toTestStepOver()");
}
print ("Functions toTestStepOver() and toTestStepIn() are defined");
```
3. Click **Save As** in the embedded browser to save the script as `dbg_test.js` file.
4. Click **Run** to execute the script. The remote debugger page opens in your default browser.
5. In the remote debugger, click **Page** on the **Sources** tab. Navigate to the `dbg_test.js` file to open the file in the center panel.

6. Ensure that you open **Creo JS.Link Shell** as the context in the context list of the **Console** tab as shown in the image:



7. Set up the break point at line 2 of the script by clicking on the left side of the line number as shown in the image:



-
8. To manually execute the function, type `toTestStepOver('my test message')` in the **Console** tab and press ENTER.

Note the following important points:

- Script execution stops at line 2 where the break point is set.
 - In the right side of the panel, **Call Stack** option displays the current function is `toTestStepIn` and the earlier one is `toTestStepOver`.
 - You can see the local and global variables in the **Scope** option.
9. Click  on the right side of the panel to continue with the script execution.

Session Objects

Session Objects

This section describes how to program on the session level using Creo.JS.

Overview of Session Objects

The Creo Parametric `pfcSession` object, contained in the class `pfcSession`, is the highest level object in Creo.JS. Any program that accesses data from Creo Parametric must first get a handle to the `pfcSession` object before accessing more specific data.

The `pfcSession` object contains methods to perform the following operations:

- Accessing models and windows, described in the Models and Windows sections.
- Working with the Creo Parametric user interface.
- Allowing interactive selection of items within the session.
- Accessing global settings such as line styles, colors, and configuration options.

The following sections describe these operations.

Getting the Session Object

Methods Introduced:

- **`pfcGetCurrentSessionWithCompatibility()`**
- **`pfcGetCurrentSession()`**
- **`pfcGetProESession()`**

For every application, Creo assigns a unique session. The session contains license information, the compatibility information as a `pfcCreoCompatibility` enumerated data type, and other additional data of the application. When a session

is assigned to an application, Creo sets the compatibility to `CompatibilityUndefined` in the associated `pfcAppInfo` object. You must set the compatibility of the application before working with sessions. To set the compatibility, call the method `pfcGetCurrentSessionWithCompatibility()`. Use the values defined in the enumerated data type `pfcCreoCompatibility` to set the compatibility of the application.

Use the method `pfcGetCurrentSession()` to get the current `pfcSession` object in synchronous mode. If the compatibility is not set, the method throws the exception `pfcXCompatibilityNotSet`.

The method `pfcGetProESession()` also gets the `pfcSession` object. This method will be deprecated in a future release of Creo. If you call this method without setting the compatibility, the method sets the compatibility to `C3Compatible`. This setting ensures forward compatibility of the Creo applications. If you set a specific compatibility using the method `pfcGetCurrentSessionWithCompatibility()`, and call the method `pfcGetProESession()`, then all calls to `pfcGetProESession()` return the session with the set compatibility.



Note

You can make multiple calls to this method, but each call gives you a handle to the same object.

Getting Session Information

Methods Introduced:

- **`pfcGetProEArguments()`**
- **`pfcGetProEVersion()`**
- **`pfcGetProEBuildCode()`**

The method `pfcGetProEArguments()` returns an array containing the command-line arguments passed to Creo Parametric, if these arguments follow one of two formats:

- Any argument starting with a plus sign (+) followed by a letter character.
- Any argument starting with a minus (-) followed by a capitalized letter.

The first argument passed in the array is the full path to the Creo Parametric executable.

The method `pfcGetProEVersion()` returns a string that represents the Creo Parametric version, for example “Wildfire”.

The method `pfcGetProEBuildCode()` returns a string that represents the build code of the Creo Parametric session.



Note

The preceding methods can only access information in synchronous mode.

Directories

Methods Introduced:

- **`pfcBaseSession.GetCurrentDirectory()`**
- **`pfcBaseSession.ChangeDirectory()`**

The method `pfcBaseSession.GetCurrentDirectory()` returns the absolute path name for the current working directory of Creo Parametric.

The method `pfcBaseSession.ChangeDirectory()` changes Creo Parametric to another working directory.

File Handling

Methods Introduced:

- **`pfcBaseSession.ListFiles()`**
- **`pfcBaseSession.ListSubdirectories()`**

The method `pfcBaseSession.ListFiles()` returns a list of files in a directory, given the directory path. You can filter the list to include only files of a particular type, as specified by the file extension.

The method `pfcBaseSession.ListFiles()` can also list instance objects when accessing Windchill workspaces or folders. A PDM location for workspace or common space, must be passed as the directory path. The following options are added in the `pfcFileListOpt` enumerated type:

- `FILE_LIST_ALL`—Lists all the files. It may also include multiple versions of the same file.
- `FILE_LIST_LATEST`—Lists only the latest version of each file.
- `FILE_LIST_ALL_INST`—Same as the `FILE_LIST_ALL` option. It returns instances only for PDM locations.
- `FILE_LIST_LATEST_INST`—Same as the `FILE_LIST_LATEST` option. It returns instances only for PDM locations.

The method `pfcBaseSession.ListSubdirectories()` returns the subdirectories in a given directory location.

Configuration Options

Methods Introduced:

- **pfcBaseSession.GetConfigOptionValues()**
- **pfcBaseSession.SetConfigOption()**
- **pfcBaseSession.LoadConfigFile()**

You can access configuration options programmatically using the methods described in this section.

Use the method `pfcBaseSession.GetConfigOptionValues()` to retrieve the value of a specified configuration file option. Pass the *Name* of the configuration file option as the input to this method. The method returns an array of values that the configuration file option is set to. It returns a single value if the configuration file option is not a multivalued option. The method returns a null if the specified configuration file option does not exist.

The method `pfcBaseSession.SetConfigOption()` is used to set the value of a specified configuration file option. If the option is a multi-value option, it adds a new value to the array of values that already exist.

The method `pfcBaseSession.LoadConfigFile()` loads an entire configuration file into Creo Parametric.

Macros

Method Introduced:

- **pfcBaseSession.RunMacro()**

The method `pfcBaseSession.RunMacro()` runs a macro string. A Creo.JS macro string is equivalent to a Creo Parametric mapkey minus the key sequence and the mapkey name. To generate a macro string, create a mapkey in Creo Parametric. Refer to the Creo Parametric online help for more information about creating a mapkey.

Copy the Value of the generated mapkey Option from the **Tools ► Options** dialog box. An example Value is as follows:

```
$F2 @MAPKEY_LABELtest;  
~ Activate `main_dlg_cur` `ProCmdModelNew.file`;  
~ Activate `new``OK`;
```

The key sequence is \$F2. The mapkey name is @MAPKEY_LABELtest. The remainder of the string following the first semicolon is the macro string that should be passed to the method `pfcBaseSession.RunMacro()`.

In this case, it is as follows:

```
~ Activate `main_dlg_cur` `ProCmdModelNew.file`;  
~ Activate `new``OK`;
```

Note

Creating or editing the macro string manually is not supported as the mapkeys are not a supported scripting language. The syntax is not defined for users and is not guaranteed to remain constant across different datecodes of Creo Parametric.

Macros are executed from synchronous mode only when control returns to Creo Parametric from the Creo.JS program. Macros are stored in reverse order, last in, first out.

Colors and Line Styles

Methods Introduced:

- **`pfcBaseSession.SetStdColorFromRGB()`**
- **`pfcBaseSession.GetRGBFromStdColor()`**
- **`pfcBaseSession.SetTextColor()`**
- **`pfcBaseSession.SetLineStyle()`**

These methods control the general display of a Creo Parametric session.

Use the method `pfcBaseSession.SetStdColorFromRGB()` to customize any of the Creo Parametric standard colors.

To change the color of any text in the window, use the method `pfcBaseSession.SetTextColor()`.

To change the appearance of nonsolid lines such as datums, use the method `pfcBaseSession.SetLineStyle()`.

Accessing the Interface

The `pfcSession` object has methods that work with the Creo Parametric interface. These methods provide access to the message window.

The Text Message File

A text message file is where you define strings that appear in the Creo Parametric user interface. This includes the strings on the command buttons that you add to the Creo Parametric number, the help string that appears when a cursor is positioned over such a command button, and text strings that appear in the Message Window. You have the option of including a translation for each string in the text message file.

 Note

Remember that Creo.JS applications, as unregistered web pages, do not currently support the setting of the Creo Parametric text directory. All the resource files for messages must be located under `$PRO_DIRECTORY/text` folder. You can force Creo Parametric to load a message file by registering a Creo TOOLKIT or application using the web page, and calling a function or method that requires the message file. After the file is loaded by Creo Parametric, Creo.JS applications may use any of its key strings for displaying messages in the message window.

Restrictions on the Text Message File

When you name your message file, ensure the following:

- Name of the file is 30 characters or less, including the extension.
- Name of the file contains lower-case characters only.
- File extension is three characters.
- Version number is in the range 1 to 9999.
- All message file names and all message key strings are unique across all applications that run with Creo Parametric. Duplicate message file names or message key strings can cause unexpected results in Creo Parametric. To avoid conflicts with the names of Creo Parametric or foreign application message files or message key strings, PTC recommends that you choose a prefix unique to your application. Then, prepend that prefix to the file name for each message and to the key string for each message corresponding to that application.

 Note

Message files are loaded into Creo Parametric only once during a session. If you make a change to the message file while Creo Parametric is running you must exit and restart Creo Parametric before the change takes effect.

Contents of the Message File

The message file consists of groups of four lines, one group for each message you want to write. The four lines are as follows:

-
1. A string that acts as the identifier for the message. This keyword must be unique for all Creo Parametric messages.
 2. The string that will be substituted for the identifier.
This string can include placeholders for run-time information stored in a `stringseq` object which is shown in Writing Messages to the Message Window.
 3. The translation of the message into another language. This can be blank.
 4. An intentionally blank line reserved for future extensions.

Writing a Message Using a Message Pop-up Dialog Box

Method and Properties Introduced:

- **`pfcSession.UIShowMessageDialog()`**
- **`pfcMessageDialogOptions.Buttons`**
- **`pfcMessageDialogOptions.DefaultButton`**
- **`pfcMessageDialogOptions.DialogLabel`**
- **`pfcMessageDialogOptions.MessageDialogType`**

The method `pfcSession.UIShowMessageDialog()` displays the UI message dialog. The input arguments to the method are:

- *Message*—The message text to be displayed in the dialog.
- *Options*—An instance of the `pfcMessageDialogOptions` containing other options for the resulting displayed message. If this is not supplied, the dialog shows a default message dialog with an **Info** classification and an **OK** button. If this is not to be null, create an instance of this options type with `pfcMessageDialogOptions.Create()`. You can set the following options:
 - *Buttons*—Specifies an array of buttons to include in the dialog. If not supplied, the dialog includes only the **OK** button. Use the property `pfcMessageDialogOptions.Buttons` to set this option.
 - *DefaultButton*—Specifies the identifier of the default button for the dialog box. This must match one of the available buttons. Use the property `pfcMessageDialogOptions.DefaultButton` to set this option.
 - *DialogLabel*—The text to display as the title of the dialog box. If not supplied, the label is the English string `Info`. Use the property `pfcMessageDialogOptions.DialogLabel` to set this option.
 - *MessageDialogType*—The type of icon to be displayed with the dialog box (**Info**, **Prompt**, **Warning**, or **Error**). If not supplied, an **Info** icon is used. Use the property

`pfcMessageDialogOptions.MessageDialogType` to set this option.

Accessing the Message Window

The following sections describe how to access the message window using Creo.JS. The topics are as follows:

- [Writing Messages to the Message Window on page 47](#)
- [Writing Messages to an Internal Buffer on page 47](#)

Writing Messages to the Message Window

Methods Introduced:

- **`pfcSession.UIDisplayMessage()`**
- **`pfcSession.UIDisplayLocalizedMessage()`**
- **`pfcSession.UIClearMessage()`**

Use these methods to display program information..

The input arguments to the methods `pfcSession.UIDisplayMessage()` and `pfcSession.UIDisplayLocalizedMessage()` include the names of the message file, a message identifier, and (optionally) a `stringseq` object that contains up to 10 pieces of run-time information. For `pfcSession.UIDisplayMessage()`, the strings in the `stringseq` are identified as `%0s`, `%1s`, ... `%9s` based on their location in the sequence. For `pfcSession.UIDisplayLocalizedMessage()`, the strings in the `stringseq` are identified as `%0w`, `%1w`, ... `%9w` based on their location in the sequence. To include other types of run-time data such as integers or reals, you must first convert the data to strings and store it in the string sequence.

The method `pfcSession.UIClearMessage()` scrolls the text in the message area up one line after the method `pfcSession.UIDisplayMessage()` is called.

Writing Messages to an Internal Buffer

Methods Introduced:

- **`pfcBaseSession.GetMessageContents()`**
- **`pfcBaseSession.GetLocalizedMessageContents()`**

You can use the methods `pfcBaseSession.GetMessageContents()` and `pfcBaseSession.GetLocalizedMessageContents()` to write a message to an internal buffer instead to the Creo Parametric message area.

These methods take the same input arguments and perform exactly the same argument substitution and translation as the `pfcSession.UIDisplayMessage()` and `pfcSession.UIDisplayLocalizedMessage()` methods described in the previous section.

Message Classification

Messages displayed in Creo.JS include a symbol that identifies the message type. Every message type is identified by a classification that begins with the characters `%C`. A message classification requires that the message key line, line one in the message file must be preceded by the classification code.

Note

Any message key string used in the code should not contain the classification.

Creo.JS applications can display any or all of the following message symbols:

- **Prompt**—This Creo.JS message is preceded by a green arrow. The user must respond to this message type. Responding includes, specifying input information, accepting the default value offered, or canceling the application. If no action is taken, the progress of the application is halted. A response may either be textual or a selection. The classification for Prompt messages is `%CP`
- **Info**—This Creo.JS message is preceded by a blue dot. Info message types contain information such as user requests or feedback from Creo.JS or Creo Parametric. The classification for Info messages is `%CI`

Note

Do not classify messages that display information regarding problems with an operation or process as `Info`. These types of messages must be classified as `Warnings`.

- **Warning**—This Creo.JS message is preceded by a triangle containing an exclamation point. Warning message types contain information to alert users to situations that could potentially lead to an error during a later stage of the process. Examples of warnings could be a process restriction or a suspected data problem. A Warning does not prevent or interrupt a process. Also, a Warning should not be used to indicate a failed operation. Warnings must only caution a user that the completed operation may not have been performed in a completely desirable way. The classification for Warning messages is `%CW`

-
- **Error**—This Creo.JS message is preceded by a broken square. An Error message informs the user that a required task was not completed successfully. Depending on the application, a failed task may or may not require intervention or correction before work can continue. Whenever possible redress this situation by providing a path. The classification for Error messages is %CE
 - **Critical**—This Creo.JS message is preceded by a red X. A Critical message type informs the user of an extremely serious situation that is usually proceeded by loss of user data. Options redressing this situation, if available, should be provided within the message. The classification for a Critical message is %CC

Reading Data from the Message Window

Methods Introduced:

- **pfcSession.UIReadIntMessage()**
- **pfcSession.UIReadRealMessage()**
- **pfcSession.UIReadStringMessage()**

These methods enable a program to get data from the user. These methods obtain keyboard input from a text box in the Creo Parametric user interface.



Note

When the user presses ESC or clicks **Cancel** in the Creo Parametric user interface, these methods throw the exception `pfcXToolkitMsgUserQuit`.

The `pfcSession.UIReadIntMessage()` and `pfcSession.UIReadRealMessage()` methods contain optional arguments that limit the value of the data to a certain range.

The method `pfcSession.UIReadStringMessage()` includes an optional Boolean argument that specifies whether to echo characters that are. Use this argument when prompting a user to enter a password.



Note

A default value is displayed in the text box as input. When a user presses ENTER as input in the user interface, the default value is not passed to the Creo.JS method; instead, the method returns a constant string `use_default_string`. When this string is returned, the application must interpret that the user wants to use the default value.

Displaying Feature Parameters

Method Introduced:

- **`pfcSession.UIDisplayFeatureParams()`**

The method `pfcSession.UIDisplayFeatureParams()` forces Creo Parametric to show dimensions or other parameters stored on a specific feature. The displayed dimensions may then be interactively selected by the user.

File Dialogs

Methods and Properties Introduced:

- **`pfcSession.UIOpenFile()`**
- **`pfcFileOpenOptions.Create()`**
- **`pfcFileOpenOptions.FilterString`**
- **`pfcFileOpenOptions.PreselectedItem`**
- **`pfcFileUIOptions.DefaultPath`**
- **`pfcFileUIOptions.DialogLabel`**
- **`pfcFileUIOptions.Shortcuts`**
- **`pfcFileOpenShortcut.Create`**
- **`pfcFileOpenShortcut.ShortcutName`**
- **`pfcFileOpenShortcut.ShortcutPath`**
- **`pfcSession.UISaveFile()`**
- **`pfcFileSaveOptions.Create()`**
- **`pfcSession.UISelectDirectory()`**
- **`pfcDirectorySelectionOptions.Create()`**

The method `pfcSession.UIOpenFile()` opens the relevant dialog box for browsing directories and opening files. The method lets you specify several options through the input arguments `pfcFileOpenOptions` and `pfcFileUIOptions`.

The method `pfcSession.UIOpenFile()` returns the file selected by you. The application must use other properties or techniques to perform the desired action on the file.

Use the method `pfcFileOpenOptions.Create()` to create a new instance of the `pfcFileOpenOptions` object. A list of options for this object with a description about what each option specifies, follows:

- `FilterString`—Filter string for the type of file accepted by the dialog box. Multiple file types should be listed with wildcards and separated by commas, for example, `*.prt, *.asm, *.txt, *.avi`, and so on. Use the property `pfcFileOpenOptions.FilterString` to set this option.
- `PreselectedItem`—Specifies the name of an item to preselect in the dialog box. Use the property `pfcFileOpenOptions.PreselectedItem` to set this option.

The `pfcFileUIOptions` object contains the following options:

- `DefaultPath`—Specifies the name of the path to be opened by default in the dialog box. Use the property `pfcFileUIOptions.DefaultPath` to set this option.
- `DialogLabel`—Specifies the title of the dialog box. Use the property `pfcFileUIOptions.DialogLabel` to set this option.
- `Shortcuts`—Array of file shortcuts of the type `pfcFileOpenShortcut`. Create this object using the property `pfcFileOpenShortcut.Create`. This object contains the following attributes:
 - `ShortcutName`—Specifies the name of shortcut path to be made available in the dialog box.
 - `ShortcutPath`—Specifies the string for the shortcut path.

Use the property `pfcFileOpenShortcut.ShortcutName` to set the string for the shortcut name.

Use the property `pfcFileOpenShortcut.ShortcutPath` to set the string for the shortcut path.

Use the property `pfcFileUIOptions.Shortcuts` to set the array of file shortcuts.

The method `pfcSession.UISaveFile()` opens the dialog box for saving a file. The method accepts options similar to `pfcSession.UIOpenFile()` through the `pfcFileSaveOptions` and `pfcFileUIOptions` objects. Use

the method `pfcFileSaveOptions.Create()` to create a new instance of the `pfcFileSaveOptions` object. When using the **Save** dialog box, you can set the name to a nonexistent file. The method `pfcSession.UISaveFile()` returns the name of the file selected by you; the application must use other methods or techniques to perform the desired action on the file.

The method `pfcSession.UISelectDirectory()` prompts the user to select a directory using the Creo Parametric dialog box for browsing directories. The method accepts options through the `pfcDirectorySelectionOptions` object which is similar to the `pfcFileUIOptions` object (described for the method `pfcSession.UIOpenFile()`). Specify the default directory path, the title of the dialog box, and a set of shortcuts to other directories to start browsing. If the default path is specified as `NULL`, the current directory is used. Use the method `pfcDirectorySelectionOptions.Create()` to create a new instance of the `pfcDirectorySelectionOptions` object. The method `pfcSession.UISelectDirectory()` returns the selected directory path; the application must use other methods or techniques to perform other tasks with this selected path.

Customizing the Navigation Area

The Creo Parametric navigation area includes the Model and Layer Tree pane, Folder browser pane, and Favorites pane. The methods described in this section enable Creo.JS applications to add custom panes that contain Web pages to the Creo Parametric navigation area.

Adding Custom Web Pages

To add custom Web pages to the navigation area, the Creo.JS application must:

1. Add a new pane to the navigation area.
2. Set an icon for this pane.
3. Set the URL of the location that will be displayed in the pane.

Methods Introduced:

- **`pfcSession.NavigatorPaneBrowserAdd()`**
- **`pfcSession.NavigatorPaneBrowserIconSet()`**
- **`pfcSession.NavigatorPaneBrowserURLSet()`**

The method `pfcSession.NavigatorPaneBrowserAdd()` adds a new pane that can display a Web page to the navigation area. The input parameters follow:

- *PaneName*—Unique name for the pane. Use this name in subsequent calls to `pfcSession.NavigatorPaneBrowserIconSet()` and `pfcSession.NavigatorPaneBrowserURLSet()`.

- *IconFileName*—Name of the icon file, including the extension. A valid format for the icon file is the PTC-proprietary format used by Creo Parametric .BIF, .GIF, .JPG, or .PNG. The new pane is displayed with the icon image. If you specify the value as NULL, the default Creo Parametric icon is used.

The default search paths for finding the icons are:

- `<creo_loadpoint>\<datecode>\Common Files\text\resource`
- `<Application text dir>\resource`
- `<Application text dir>\<language>\resource`

The location of the application text directory is specified in the registry file.

- *URL*—URL of the location to be accessed from the pane.

Use the method `pfcSession.NavigatorPaneBrowserIconSet()` to set or change the icon of a specified browser pane in the navigation area.

Use the method `pfcSession.NavigatorPaneBrowserURLSet()` to change the URL of the page that appears in the browser pane in the navigation area.

Selection

Selection

This section describes how to use Interactive Selection in Creo.JS.

Interactive Selection

Methods and Properties Introduced:

- **`pfcBaseSession.Select()`**
- **`pfcSelectionOptions.Create()`**
- **`pfcSelectionOptions.MaxNumSels()`**
- **`pfcSelectionOptions.OptionKeywords`**

The method `pfcBaseSession.Select()` activates the standard Creo Parametric menu structure for selecting objects and returns a `pfcSelections` array that contains the objects the user selected. Using the *Options* argument, you can control the type of object that can be selected and the maximum number of selections.

In addition, you can pass in a `pfcSelections` sequence to the method. The returned `pfcSelections` sequence contains the input array and any new objects.

The method `pfcSelectionOptions.Create()` and the property `pfcSelectionOptions.OptionKeywords` take a *String* argument made up of one or more of the identifiers listed in the table below, separated by commas.

For example, to allow the selection of features and axes, the arguments are *feature, axis*.

Creo Parametric Database Item	String Identifier	ModelItemType
Datum point	<i>point</i>	ITEM_POINT
Datum axis	<i>axis</i>	ITEM_AXIS
Datum plane	<i>datum</i>	ITEM_SURFACE
Coordinate system datum	<i>csys</i>	ITEM_COORD_SYS
Feature	<i>feature</i>	ITEM_FEATURE
Edge (solid or datum surface)	<i>edge</i>	ITEM_EDGE
Edge (solid only)	<i>sldedge</i>	ITEM_EDGE
Edge (datum surface only)	<i>qltedge</i>	ITEM_EDGE
Datum curve	<i>curve</i>	ITEM_CURVE
Composite curve	<i>comp_crv</i>	ITEM_CURVE
Surface (solid or quilt)	<i>surface</i>	ITEM_SURFACE
Surface (solid)	<i>sldface</i>	ITEM_SURFACE
Surface (datum surface)	<i>qltface</i>	ITEM_SURFACE
Quilt	<i>dtmqlt</i>	ITEM_QUILT
Dimension	<i>dimension</i>	ITEM_DIMENSION
Reference dimension	<i>ref_dim</i>	ITEM_REF_DIMENSION
Integer parameter	<i>ipar</i>	ITEM_DIMENSION
Part	<i>part</i>	N/A
Part or subassembly	<i>prt_or_asm</i>	N/A
Assembly component model	<i>component</i>	N/A
Component or feature	<i>membfeat</i>	ITEM_FEATURE
Detail symbol	<i>dtl_symbol</i>	ITEM_DTL_SYM_INSTANCE
Note	<i>any_note</i>	ITEM_NOTE, ITEM_DTL_NOTE
Draft entity	<i>draft_ent</i>	ITEM_DTL_ENTITY
Table	<i>dwg_table</i>	ITEM_TABLE
Table cell	<i>table_cell</i>	ITEM_TABLE
Drawing view	<i>dwg_view</i>	N/A
Solid body	<i>3d_body</i>	ITEM_BODY
Datum Curve End	<i>curve_end</i>	ITEM_CRV_START or ITEM_CRV_END

When you specify the maximum number of selections, the argument to `pfcSelectionOptionsMaxNumSels()` must be an Integer.

The default value assigned when creating a `pfcSelectionOptions` object is -1, which allows any number of selections by the user.

Accessing Selection Data

Properties Introduced:

- **pfcSelection.SelModel**
- **pfcSelection.SelItem**
- **pfcSelection.Path**
- **pfcSelection.Params**
- **pfcSelection.TParam**
- **pfcSelection.Point**
- **pfcSelection.Depth**
- **pfcSelection.SelView2D**
- **pfcSelection.SelTableCell**
- **pfcSelection.SelTableSegment**

These properties return objects and data that make up the selection object. Using the appropriate properties, you can access the following data:

- For a selected model or model item use `pfcSelection.SelModel` or `pfcSelection.SelItem`.
- For an assembly component use `pfcSelection.Path`.
- For UV parameters of the selection point on a surface use `pfcSelection.Params`.
- For the T parameter of the selection point on an edge or curve use `pfcSelection.TParam`.
- For a three-dimensional point object that contains the selected point use `pfcSelection.Point`.
- For selection depth, in screen coordinates use `pfcSelection.Depth`.
- For the selected drawing view, if the selection was from a drawing, use `pfcSelection.SelView2D`.
- For the selected table cell, if the selection was from a table, use `pfcSelection.SelTableCell`.
- For the selected table segment, if the selection was from a table, use `pfcSelection.SelTableSegment`.

Controlling Selection Display

Methods Introduced:

- **pfcSelection.Highlight()**
- **pfcSelection.UnHighlight()**

- **pfcSelection.Display()**

These methods cause a specific selection to be highlighted or dimmed on the screen using the color specified as an argument.

The method `pfcSelection.Highlight()` highlights the selection in the current window. This highlight is the same as the one used by Creo Parametric when selecting an item—it just repaints the wireframe display in the new color. The highlight is removed if you use the **View, Repaint** command or `pfcWindow.Repaint()`; it is not removed if you use `pfcWindow.Refresh()`.

The method `pfcSelection.UnHighlight()` removes the highlight.

The method `pfcSelection.Display()` causes a selected object to be displayed on the screen, even if it is suppressed or hidden.



Note

This is a one-time action and the next repaint erases this display.

Programmatic Selection

Creo.JS provides methods whereby you can make your own Selection objects, without prompting the user. These Selections are required as inputs to some methods and can also be used to highlight certain objects on the screen.

Methods and Properties Introduced:

- **pfcCreateModelItemSelection()**
- **pfcCreateComponentSelection()**
- **pfcCreateModelSelection()**
- **pfcCreateSelectionFromString()**

The method `pfcCreateModelItemSelection()` creates a selection out of any model item object. It takes a `pfcModelItem` and optionally a `pfcComponentPath` object to identify which component in an assembly the Selection Object belongs to.

The method `pfcCreateComponentSelection()` creates a selection out of any component in an assembly. It takes a `pfcComponentPath` object. For more information about `pfcComponentPath` objects, see the section [Solid on page 116](#).

Use the method `pfcCreateModelSelection()` to create a `pfcSelection` object, based on a `pfcModel` object.

The method `pfcCreateSelectionFromString()` creates a new selection object, based on a Creo.JS style selection string specified as the input.

Selection Buffer

Introduction to Selection Buffers

Selection is the process of choosing items on which you want to perform an operation. In Creo Parametric, before a feature tool is started, a user can select items to use with the tool, in the collector for the tool. Collectors are like storage bins of the references of selected items. The location where preselected items are stored is called the selection buffer.

Depending on the situation, different selection buffers may be active at any one time. In Part and Assembly mode, Creo Parametric offers the default selection buffer, the Edit selection buffer, and other more specialized buffers. Other Creo Parametric modes offer different selection buffers.

In the default Part and Assembly buffer there are two levels at which selection is done:

- **First Level Selection**
Provides access to higher-level objects such as features or components. You can make a second level selection only after you select the higher-level object.
- **Second Level Selection**
Provides access to geometric objects such as edges and faces.



Note

First-level and second-level objects are usually incompatible in the selection buffer.

Creo.JS allows access to the contents of the currently active selection buffer. With the available functions, your application can perform the following actions:

- Get the contents of the active selection buffer.
- Remove the contents of the active selection buffer.
- Add to the contents of the active selection buffer.

Reading the Contents of the Selection Buffer

Properties Introduced:

- **`pfcSession.CurrentSelectionBuffer`**

- **pfcSelectionBuffer.Contents**

The property `pfcSession.CurrentSelectionBuffer` returns the selection buffer object for the current active model in session. The selection buffer contains the items preselected by the user to be used by the selection tool and popup menus.

Use the property `pfcSelectionBuffer.Contents` to access the contents of the current selection buffer. The property returns independent copies of the selections in the selection buffer. If the buffer is cleared, this array is still valid.

Removing the Items of the Selection Buffer

Methods Introduced:

- **pfcSelectionBuffer.RemoveSelection()**
- **pfcSelectionBuffer.Clear()**

Use the method `pfcSelectionBuffer.RemoveSelection()` to remove a specific selection from the selection buffer. The input argument is the *IndexToRemove* specifies the index where the item was found in the call to the method `pfcSelectionBuffer.Contents`.

Use the method `pfcSelectionBuffer.Clear()` to clear the currently active selection buffer of all contents. After the buffer is cleared, all contents are lost.

Adding Items to the Selection Buffer

Method Introduced:

- **pfcSelectionBuffer.AddSelection()**

Use the method `pfcSelectionBuffer.AddSelection()` to add an item to the currently active selection buffer.



Note

The selected item must refer to an item that is in the current model such as its owner, component path, or drawing view.

This method may fail due to any of the following reasons:

- There is no current selection buffer active.
- The selection does not refer to the current model.
- The item is not currently displayed and so cannot be added to the buffer.

-
- The selection cannot be added to the buffer in combination with one or more objects that are already in the buffer. For example: geometry and features cannot be selected in the default buffer at the same time.

Models

Models

This section describes how to program on the model level using Creo.JS.

Overview of Model Objects

Models can be any Creo Parametric file type, including parts, assemblies, drawings, sections, and notebook. The classes in the module `pfcModel` provide generic access to models, regardless of their type. The available methods enable you to do the following:

- Access information about a model.
- Open, copy, rename, and save a model.

Getting a Model Object

Methods and Properties Introduced:

- `pfcFamilyTableRow.CreateInstance()`
- `pfcSelection.SelModel`
- `pfcBaseSession.GetModel()`
- `pfcBaseSession.CurrentModel`
- `pfcBaseSession.GetActiveModel`
- `pfcBaseSession.ListModels()`
- `pfcBaseSession.GetByRelationId()`
- `pfcWindow.Model()`

These methods get a model object that is already in session.

The property `pfcSelection.SelModel` returns the model that was interactively selected.

The method `pfcBaseSession.GetModel()` returns a model based on its name and type, whereas `pfcBaseSession.GetByRelationId()` returns a model in an assembly that has the specified integer identifier.

The property `pfcBaseSession.CurrentModel` returns the current active model.

The property `pfcBaseSession.GetActiveModel` returns the active Creo Parametric model.

Use the method `pfcBaseSession.ListModels()` to return an array of all the models in session.

For more methods that return solid models, refer to the section [Solid on page 116](#).

Model Descriptors

Methods and Properties Introduced:

- **`pfcModelDescriptor.Create()`**
- **`pfcModelDescriptor.CreateFromFileName()`**
- **`pfcModelDescriptor.GenericName()`**
- **`pfcModelDescriptor.InstanceName()`**
- **`pfcModelDescriptor.Type()`**
- **`pfcModelDescriptor.Device()`**
- **`pfcModelDescriptor.Path()`**
- **`pfcModelDescriptor.FileVersion()`**
- **`pfcModelDescriptor.GetFullName()`**
- **`pfcModel.FullName`**

Model descriptors are data objects used to describe a model file and its location in the system. Use the methods in the model descriptor to set specific information that enables Creo Parametric to find the specific model you want.

Use the static utility method `pfcModelDescriptor.Create()` to specify as data to be entered a model type, an instance name, and a generic name. The model descriptor constructs the full name of the model as a string, as follows:

```
String FullName = InstanceName+"<"+GenericName+">";  
                // As long as the  
                // generic name is  
                // not an empty  
                // string ("")
```

If you want to load a model that is not a family table instance, pass an empty string as the generic name argument so that the full name of the model is constructed correctly. If the model is a family table interface, you should specify the instance and generic names.

 Note

You can set other fields in the model descriptor object, but they may be ignored by some methods.

Use the static utility method `pfcModelDescriptor.CreateFromFileName()` to create a new model descriptor from a given a file name. The file name is a string in the form `<name>.<extension>`.

Retrieving Models

Methods Introduced:

- **`pfcBaseSession.RetrieveModel()`**
- **`pfcBaseSession.RetrieveModelWithOpts()`**
- **`pfcBaseSession.OpenFile()`**
- **`pfcSolid.HasRetrievalErrors()`**

These methods cause Creo Parametric to retrieve the model that corresponds to the `pfcModelDescriptor` argument.

The method `pfcBaseSession.RetrieveModel()` retrieves the specified model into the Creo Parametric session given its model descriptor from a standard directory. This method ignores the path argument specified in the model descriptor. But this method does not create a window for it, nor does it display the model anywhere.

The method `pfcBaseSession.RetrieveModelWithOpts()` retrieves the specified model into the Creo Parametric session based on the path specified by the model descriptor. The path can be a disk path, a workspace path, or a commonspace path. The *Opts* argument given by the `pfcRetrieveModelOptions` object, provides the user with the option to specify simplified representations.

The method `pfcBaseSession.OpenFile()` brings the model into memory, opens a new window for it or uses the base window, if it is empty, and displays the model.

 Note

`pfcBaseSession.OpenFile()` actually returns a handle to the window it has created.

The file version set by the `pfcModelDescriptor.FileVersion` property is ignored by the `pfcBaseSession.RetrieveModelWithOpts()` and `pfcBaseSession.OpenFile()` methods. Instead, the latest file version of the model is used by these two methods.

To get a handle to the model you need, use the property `pfcWindow.Model`.

The method `pfcSolid.HasRetrievalErrors()` returns a true value if the features in the solid model were suppressed during the `RetrieveModel` or `OpenFile` operations. This method must be called immediately after the `pfcBaseSession.RetrieveModel()` method or an equivalent retrieval method.

Model Information

Methods and Properties Introduced:

- **`pfcModel.FileName`**
- **`pfcModel.CommonName`**
- **`pfcModel.IsCommonNameModifiable()`**
- **`pfcModel.FullName`**
- **`pfcModel.GenericName`**
- **`pfcModel.InstanceName`**
- **`pfcModel.Origin`**
- **`pfcModel.RelationId`**
- **`pfcModel.Descr`**
- **`pfcModel.Type`**
- **`pfcModel.IsModified`**
- **`pfcModel.Version`**
- **`pfcModel.Revision`**
- **`pfcModel.Branch`**
- **`pfcModel.ReleaseLevel`**
- **`pfcModel.VersionStamp`**
- **`pfcModel.ListDependencies`**
- **`pfcModel.CleanupDependencies`**
- **`pfcModel.ListDeclaredModels()`**
- **`pfcModel.CheckIsModifiable()`**
- **`pfcModel.CheckIsSaveAllowed()`**

The property `pfcModel.FileName` retrieves the model file name in the "name"."type" format.

The property `pfcModel.CommonName` retrieves the common name for the model. This name is displayed for the model in Windchill PDMLink.

Use the method `pfcModel.IsCommonNameModifiable()` to identify if the common name of the model can be modified. You can modify the name for models that are not yet owned by Windchill PDMLink, or in certain situations if the configuration option `let_proe_rename_pdm_objects` is set to `yes`.

The property `pfcModel.FullName` retrieves the full name of the model in the instance <generic> format.

The property `pfcModel.GenericName` retrieves the name of the generic model. If the model is not an instance, this name must be `NULL` or an empty string.

The property `pfcModel.InstanceName` retrieves the name of the model. If the model is an instance, this property retrieves the instance name.

The property `pfcModel.Origin` returns the complete path to the file from which the model was opened. This path can be a location on disk from a Windchill workspace, or from a downloaded URL.

The property `pfcModel.RelationId` retrieves the relation identifier of the specified model. It can be `NULL`.

The property `pfcModel.Descrip` returns the descriptor for the specified model. Model descriptors can be used to represent models not currently in session.



Note

The properties `pfcModel.FullName`, `pfcModel.GenericName`, and `pfcModel.Descrip` throw an exception `pfcXtoolkitCantOpen` if called on a model instance whose immediate generic is not in session. Handle this exception and typecast the model as `pfcSolid`, which in turn can be typecast as `pfcFamilyMember`, and use the method `pfcFamily.GetImmediateGenericInfo()` to get the model descriptor of the immediate generic model. The model descriptor can be used to derive the full name or generic name of the model.

The property `pfcModel.Type` returns the type of model in the form of the `pfcModelType` object. The types of models are as follows:

- `MDL_ASSEMBLY`—Assembly.
- `MDL_PART`—Part.
- `MDL_DRAWING`—Drawing.

-
- MDL_2D_SECTION—2D section.
 - MDL_LAYOUT—Notebook.
 - MDL_DWG_FORMAT—Drawing format.
 - MDL_MFG—Manufacturing model.
 - MDL_REPORT—Report.
 - MDL_MARKUP—Drawing markup.
 - MDL_DIAGRAM—Diagram
 - MDL_CE_SOLID—Layout model.

 Note

Creo.JS methods can read models of type Layout, but cannot pass Layout models as input to other methods. PTC recommends that you review all Creo.JS applications that use the object `pfcModelType` and modify the code as appropriate to ensure that the applications work correctly.

The property `pfcModel.IsModified` identifies whether the model has been modified since it was last saved.

The property `pfcModel.Version` returns the version of the specified model from the PDM system. It can be `NULL`, if not set.

The property `pfcModel.Revision` returns the revision number of the specified model from the PDM system. It can be `NULL`, if not set.

The property `pfcModel.Branch` returns the branch of the specified model from the PDM system. It can be `NULL`, if not set.

The property `pfcModel.ReleaseLevel` returns the release level of the specified model from the PDM system. It can be `NULL`, if not set.

The property `pfcModel.VersionStamp` returns the version stamp of the specified model. The version stamp is a Creo Parametric specific identifier that changes with each change made to the model.

The property `pfcModel.ListDependencies` returns a list of the first-level dependencies for the specified model in the Creo Parametric workspace in the form of the `pfcDependencies` object.

Use the property `pfcModel.CleanupDependencies` to clean the dependencies for an object in the Creo Parametric workspace.

 Note

Do not call the method `pfcModel.CleanupDependencies` during operations that alter the dependencies, such as, restructuring components and creating or redefining features.

The method `pfcModel.ListDeclaredModels()` returns a list of all the first-level objects declared for the specified model.

The method `pfcModel.CheckIsModifiable()` identifies if a given model can be modified without checking for any subordinate models. This method takes a Boolean argument *ShowUI* that determines whether the Creo Parametric conflict resolution dialog box should be displayed to resolve conflicts, if detected. If this argument is false, then the conflict resolution dialog box does not appear, and the model can be modified only if there are no conflicts that cannot be overridden, or are resolved by default resolution actions. For a generic model, if *ShowUI* is true, then all instances of the model are also checked.

The method `pfcModel.CheckIsSaveAllowed()` identifies if a given model can be saved along with all of its subordinate models. The subordinate models can be saved based on their modification status and the value of the configuration option `save_objects`. This method also checks the current user interface context to identify if it is currently safe to save the model. Calling this method at different times might return different results. This method takes a Boolean argument *ShowUI*. Refer to the previous method for more information on this argument.

Model Operations

Methods and Property Introduced:

- **`pfcModel.Backup()`**
- **`pfcModel.Copy()`**
- **`pfcModel.CopyAndRetrieve()`**
- **`pfcModel.Rename()`**
- **`pfcModel.Save()`**
- **`pfcModel.Erase()`**
- **`pfcModel.EraseWithDependencies()`**
- **`pfcModel.Delete()`**
- **`pfcModel.Display()`**

- **pfcModel.CommonName**

These model operations duplicate most of the commands available in the Creo Parametric **File** menu.

The method `pfcModel.Backup()` makes a backup of an object in memory to a disk in a specified directory.

The method `pfcModel.Copy()` copies the specified model to another file.

The method `pfcModel.CopyAndRetrieve()` copies the model to another name, and retrieves that new model into session.

The method `pfcModel.Rename()` renames a specified model.

The method `pfcModel.Save()` stores the specified model to a disk.

The method `pfcModel.Erase()` erases the specified model from the session. Models used by other models cannot be erased until the models dependent upon them are erased.

The method `pfcModel.EraseWithDependencies()` erases the specified model from the session and all the models on which the specified model depends from disk, if the dependencies are not needed by other items in session.

 Note

However, while erasing an active model, `pfcModel.Erase()` and `pfcModel.EraseWithDependencies()` only clear the graphic display immediately, they do not clear the data in the memory until the control returns to Creo Parametric from the Creo.JS application. Therefore, after calling them the control must be returned to Creo Parametric before calling any other function. Otherwise the behavior of Creo Parametric may be unpredictable.

The method `pfcModel.Delete()` removes the specified model from memory and disk.

The method `pfcModel.Display()` displays the specified model. You must call this method if you create a new window for a model because the model will not be displayed in the window until you call `pfcDisplay`.

The property `pfcModel.CommonName` modifies the common name of the specified model. You can modify this name for models that are not yet owned by Windchill PDMLink, or in certain situations if the configuration option `let_proe_rename_pdm_objects` is set to yes.

Running Creo ModelCHECK

Creo ModelCHECK is an integrated application that runs transparently within Creo Parametric. Creo ModelCHECK uses a configurable list of company design standards and best modeling practices. You can configure Creo ModelCHECK to run interactively or automatically when you regenerate or save a model.

Methods and Properties Introduced:

- **`pfcBaseSession.ExecuteModelCheck()`**
- **`pfcModelCheckInstructions.Create()`**
- **`pfcModelCheckInstructions.ConfigDir`**
- **`pfcModelCheckInstructions.Mode`**
- **`pfcModelCheckInstructions.OutputDir`**
- **`pfcModelCheckInstructions.ShowInBrowser`**
- **`pfcModelCheckResults.NumberOfErrors`**
- **`pfcModelCheckResults.NumberOfWarnings`**
- **`pfcModelCheckResults.WasModelSaved`**

You can run Creo ModelCHECK from an external application using the method `pfcBaseSession.ExecuteModelCheck()`. This method takes the model *Model* on which you want to run Creo ModelCHECK and instructions in the form of the object `pfcModelCheckInstructions` as its input parameters. A list of parameters for this object, and what each parameter specifies, follows:

- `ConfigDir`—Location of the configuration files. If this parameter is set to `NULL`, the default Creo ModelCHECK configuration files are used.
- `Mode`—Mode in which you want to run Creo ModelCHECK. The modes are:
 - `MODELCHECK_GRAPHICS`—Interactive mode
 - `MODELCHECK_NO_GRAPHICS`—Batch mode
- `OutputDir`—Location for the reports. If you set this parameter to `NULL`, the default Creo ModelCHECK directory, as per `config_init.mc`, will be used.
- `ShowInBrowser`—Results report should be displayed in the Web browser.

The method `pfcModelCheckInstructions.Create()` creates the `pfcModelCheckInstructions` object containing the Creo ModelCHECK instructions described above.

Use the properties `pfcModelCheckInstructions.ConfigDir`, `pfcModelCheckInstructions.Mode`, `pfcModelCheckInstructions.OutputDir`, and `pfcModelCheckInstructions.ShowInBrowser` to modify the Creo ModelCHECK instructions.

The method `pfcBaseSession.ExecuteModelCheck()` returns the results of the Creo ModelCHECK run in the form of the `pfcModelCheckResults` object. This object contains the following parameters:

- `NumberOfErrors`—Specifies the number of errors detected.
- `NumberOfWarnings`—Specifies the number of warnings found.
- `WasModelSaved`—Specifies whether the model is saved with updates.

Use the properties `pfcModelCheckResults.NumberOfErrors`, `pfcModelCheckResults.NumberOfWarning`, and `pfcModelCheckResults.WasModelSaved` to access the results obtained.

Drawings

Drawings

This section describes how to program drawing functions using Creo.JS.

Overview of Drawings in Creo.JS

This section describes the functions that deal with drawings. You can create drawings of all Creo Parametric models using the functions in Creo.JS. You can annotate the drawing, manipulate dimensions, and use layers to manage the display of different items.

Unless otherwise specified, Creo.JS functions that operate on drawings use world units.

Creating Drawings from Templates

Drawing templates simplify the process of creating a drawing using Creo.JS. Creo Parametric can create views, set the view display, create snap lines, and show the model dimensions based on the template. Use templates to:

- Define layout views
- Set view display
- Place notes
- Place symbols
- Define tables
- Show dimensions

Method Introduced:

- **`pfcBaseSession.CreateDrawingFromTemplate()`**

Use the method `pfcBaseSession.CreateDrawingFromTemplate()` to create a drawing from the drawing template and to return the created drawing. The input parameters are:

- *Name*—New drawing name
- *Template*—Name of an existing template
- *DrawingModel*—Name and type of the solid model to use while populating template views
- *Options*—Sequence of options defined by the enumerated type `pfcDrawingCreateOption` to create the drawing. The options are as follows:
 - `DRAWINGCREATE_DISPLAY_DRAWING`—Display the new drawing.
 - `DRAWINGCREATE_SHOW_ERROR_DIALOG`—Display the error dialog box.
 - `DRAWINGCREATE_WRITE_ERROR_FILE`—Write the errors to a file.
 - `DRAWINGCREATE_PROMPT_UNKNOWN_PARAMS`—Prompt the user on encountering unknown parameters

Drawing Creation Errors

The exception `pfcXToolkitDrawingCreateErrors` is thrown if an error is encountered when creating a drawing from a template. This exception contains a list of errors which occurred during drawing creation.

Note

When this exception type is encountered, the drawing is actually created, but some of the contents failed to generate correctly.

The exception message will list the details for each error including its type, sheet number, view name, and (if applicable) item name. The types of errors are as follows:

- `DWGCREATE_ERR_SAVED_VIEW_DOESNT_EXIST`—Saved view does not exist.
- `DWGCREATE_ERR_X_SEC_DOESNT_EXIST`—Specified cross section does not exist.
- `DWGCREATE_ERR_EXPLODE_DOESNT_EXIST`—Exploded state did not exist.
- `DWGCREATE_ERR_MODEL_NOT_EXPLODABLE`—Model cannot be exploded.

-
- `DWGCREATE_ERR_SEC_NOT_PERP`—Cross section view not perpendicular to the given view.
 - `DWGCREATE_ERR_NO_RPT_REGIONS`—Repeat regions not available.
 - `DWGCREATE_ERR_FIRST_REGION_USED`—Repeat region was unable to use the region specified.
 - `DWGCREATE_ERR_NOT_PROCESS_ASSEM`— Model is not a process assembly view.
 - `DWGCREATE_ERR_NO_STEP_NUM`—The process step number does not exist.
 - `DWGCREATE_ERR_TEMPLATE_USED`—The template does not exist.
 - `DWGCREATE_ERR_NO_PARENT_VIEW_FOR_PROJ`—There is no possible parent view for this projected view.
 - `DWGCREATE_ERR_CANT_GET_PROJ_PARENT`—Could not get the projected parent for a drawing view.
 - `DWGCREATE_ERR_SEC_NOT_PARALLEL`—The designated cross section was not parallel to the created view.
 - `DWGCREATE_ERR_SIMP_REP_DOESNT_EXIST`—The designated simplified representation does not exist.

Obtaining Drawing Models

This section describes how to obtain drawing models.

Methods Introduced:

- **`pfcBaseSession.RetrieveModel()`**
- **`pfcBaseSession.GetModel()`**
- **`pfcBaseSession.GetModelFromDescr()`**
- **`pfcBaseSession.ListModels()`**
- **`pfcBaseSession.ListModelsByType()`**

The method `pfcBaseSession.RetrieveModel()` retrieves the drawing specified by the model descriptor. Model descriptors are data objects used to describe a model file and its location in the system. The method returns the retrieved drawing.

The method `pfcBaseSession.GetModel()` returns a drawing based on its name and type, whereas `pfcBaseSession.GetModelFromDescr()` returns a drawing specified by the model descriptor. The model must be in session.

Use the method `pfcBaseSession.ListModels()` to return a list of objects in a session.

Use the method `pfcBaseSession.ListModelsByType()` to return a list of all the drawings of a particular type in session.

Drawing Information

Methods and Properties Introduced:

- **`pfcModel2D.ListModels()`**
- **`pfcModel2D.GetCurrentSolid()`**
- **`pfcModel2D.ListSimplifiedReps()`**
- **`pfcModel2D.TextHeight`**

The method `pfcModel2D.ListModels()` returns a list of all the solid models used in the drawing.

The method `pfcModel2D.GetCurrentSolid()` returns the current solid model of the drawing.

The method `pfcModel2D.ListSimplifiedReps()` returns the simplified representations of a solid model that are assigned to the drawing.

The property `pfcModel2D.TextHeight` returns the text height of the drawing.

Drawing Operations

Methods Introduced:

- **`pfcModel2D.AddModel()`**
- **`pfcModel2D.DeleteModel()`**
- **`pfcModel2D.ReplaceModel()`**
- **`pfcModel2D.SetCurrentSolid()`**
- **`pfcModel2D.AddSimplifiedRep()`**
- **`pfcModel2D.DeleteSimplifiedRep()`**
- **`pfcModel2D.Regenerate()`**
- **`pfcModel2D.CreateDrawingDimension()`**
- **`pfcModel2D.CreateView()`**

The method `pfcModel2D.AddModel()` adds a new solid model to the drawing.

The method `pfcModel2D.DeleteModel()` removes a model from the drawing. The model to be deleted should not appear in any of the drawing views.

The method `pfcModel2D.ReplaceModel()` replaces a model in the drawing with a related model. The relationship should be by family table or interchange assembly. You can replace models that are shown in drawing views and regenerates the view.

The method `pfcModel2D.SetCurrentSolid()` assigns the current solid model for the drawing. Before calling this method, the solid model must be assigned to the drawing using the methods `pfcModel2D.AddModel()`. To see the changes to parameters and fields reflecting the change of the current solid model, regenerate the drawing using the method `pfcSheetOwner.RegenerateSheet()`.

The method `pfcModel2D.AddSimplifiedRep()` associates the drawing with the simplified representation of an assembly

The method `pfcModel2D.DeleteSimplifiedRep()` removes the association of the drawing with an assembly simplified representation. The simplified representation to be deleted should not appear in any of the drawing views.

Use the method `pfcModel2D.Regenerate()` to regenerate the drawing draft entities and appearance.

The method `pfcModel2D.CreateDrawingDimension()` creates a new drawing dimension based on the data object that contains information about the location of the dimension. This method returns the created dimension. Refer to the section [Drawing Dimensions on page 80](#).

The method `pfcModel2D.CreateView()` creates a new drawing view based on the data object that contains information about how to create the view. The method returns the created drawing view. Refer to the section [Creating Drawing Views on page 75](#).

Drawing Sheets

A drawing sheet is represented by its number. Drawing sheets in Creo.JS are identified by the same sheet numbers seen by a Creo Parametric user.

Note

These identifiers may change if the sheets are moved as a consequence of adding, removing or reordering sheets.

Drawing Sheet Information

Methods and Properties Introduced

- `pfcSheetOwner.GetSheetTransform()`

-
- **`pfcSheetOwner.GetSheetInfo()`**
 - **`pfcSheetOwner.GetSheetScale()`**
 - **`pfcSheetOwner.GetSheetFormat()`**
 - **`pfcSheetOwner.GetSheetFormatDescr()`**
 - **`pfcSheetOwner.GetSheetBackgroundView`**
 - **`pfcSheetOwner.NumberOfSheets`**
 - **`pfcSheetOwner.CurrentSheetNumber`**
 - **`pfcSheetOwner.GetSheetUnits()`**

The method `pfcSheetOwner.GetSheetTransform()` returns the transformation matrix for the sheet specified by the sheet number. This transformation matrix includes the scaling needed to convert screen coordinates to drawing coordinates (which use the designated drawing units).

The method `pfcSheetOwner.GetSheetInfo()` returns sheet data including the size, orientation, and units of the sheet specified by the sheet number.

Use the method `pfcSheetOwner.GetSheetInfo()` and the class `pfcSheetInfo`.

The method `pfcSheetOwner.GetSheetScale()` returns the scale of the drawing on a particular sheet based on the drawing model used to measure the scale. If no models are used in the drawing then the default scale value is 1.0.

The method `pfcSheetOwner.GetSheetFormat()` returns the drawing format used for the sheet specified by the sheet number. It returns a null value if no format is assigned to the sheet.

The method `pfcSheetOwner.GetSheetFormatDescr()` returns the model descriptor of the drawing format used for the specified drawing sheet.

The method `pfcSheetOwner.GetSheetBackgroundView` returns the view object representing the background view of the sheet specified by the sheet number.

The property `pfcSheetOwner.NumberOfSheets` returns the number of sheets in the model.

The property `pfcSheetOwner.CurrentSheetNumber` returns the current sheet number in the model.

 Note

The sheet numbers range from 1 to n, where n is the number of sheets.

The method `pfcSheetOwner.GetSheetUnits()` returns the units used by the sheet specified by the sheet number.

Drawing Sheet Operations

Methods Introduced:

- `pfcSheetOwner.AddSheet()`
- `pfcSheetOwner.DeleteSheet()`
- `pfcSheetOwner.ReorderSheet()`
- `pfcSheetOwner.RegenerateSheet()`
- `pfcSheetOwner.SetSheetScale()`
- `pfcSheetOwner.SetSheetFormat()`

The method `pfcSheetOwner.AddSheet()` adds a new sheet to the model and returns the number of the new sheet.

The method `pfcSheetOwner.DeleteSheet()` removes the sheet specified by the sheet number from the model.

Use the method `pfcSheetOwner.ReorderSheet()` to reorder the sheet from a specified sheet number to a new sheet number.

Note

The sheet number of other affected sheets also changes due to reordering or deletion.

The method `pfcSheetOwner.RegenerateSheet()` regenerates the sheet specified by the sheet number.

Note

You can regenerate a sheet only if it is displayed.

Use the method `pfcSheetOwner.SetSheetScale()` to set the scale of a model on the sheet based on the drawing model to scale and the scale to be used. Pass the value of the *DrawingModel* parameter as null to select the current drawing model.

Use the method `pfcSheetOwner.SetSheetFormat()` to apply the specified format to a drawing sheet based on the drawing format, sheet number of the format, and the drawing model.

The sheet number of the format is specified by the *FormatSheetNumber* parameter. This number ranges from 1 to the number of sheets in the format. Pass the value of this parameter as null to use the first format sheet.

The drawing model is specified by the *DrawingModel* parameter. Pass the value of this parameter as null to select the current drawing model.

Drawing Views

A drawing view is represented by the class `pfcView2D`. All model views in the drawing are associative, that is, if you change a dimensional value in one view, other drawing views are updated accordingly. The model automatically reflects any dimensional changes that you make to a drawing. In addition, corresponding drawings also reflect any changes that you make to a model such as the addition or deletion of features and dimensional changes.

Creating Drawing Views

Method Introduced:

- **`pfcModel2D.CreateView()`**

The method `pfcModel2D.CreateView()` creates a new view in the drawing. Before calling this method, the drawing appear in a window.

The class `pfcView2DCreateInstructions` contains details on how to create the view.

General Drawing Views

The class `pfcGeneralViewCreateInstructions` contains details on how to create general drawing views.

Methods and Properties Introduced:

- **`pfcGeneralViewCreateInstructions.Create()`**
- **`pfcGeneralViewCreateInstructions.ViewModel`**
- **`pfcGeneralViewCreateInstructions.Location`**
- **`pfcGeneralViewCreateInstructions.SheetNumber`**
- **`pfcGeneralViewCreateInstructions.Orientation`**
- **`pfcGeneralViewCreateInstructions.Exploded`**
- **`pfcGeneralViewCreateInstructions.Scale`**
- **`pfcGeneralViewCreateInstructions.ViewScale`**

The method `pfcGeneralViewCreateInstructions.Create()` creates the `pfcGeneralViewCreateInstructions` data object used for creating general drawing views.

Use the property `pfcGeneralViewCreateInstructions.ViewModel` to assign the solid model to display in the created general drawing view.

Use the property `pfcGeneralViewCreateInstructions.Location` to assign the location in a drawing sheet to place the created general drawing view.

Use the property `pfcGeneralViewCreateInstructions.SheetNumber` to set the number of the drawing sheet in which the general drawing view is created.

The property `pfcGeneralViewCreateInstructions.Orientation` assigns the orientation of the model in the general drawing view in the form of the `pfcTransform3D` data object. The transformation matrix must only consist of the rotation to be applied to the model. It must not consist of any displacement or scale components. If necessary, set the displacement to `{0, 0, 0}` using the property `pfcTransform3D.SetOrigin`, and remove any scaling factor by normalizing the matrix.

Use the property `pfcGeneralViewCreateInstructions.Exploded` to set the created general drawing view to be an exploded view.

Use the property `pfcGeneralViewCreateInstructions.Scale` to assign a scale to the created general drawing view. This value is optional, if not assigned, the default drawing scale is used.

Use the property `pfcGeneralViewCreateInstructions.ViewScale` to assign a scale to the created general drawing view. This value is optional, if not assigned, the default drawing scale is used.

Projected Drawing Views

The class `pfcProjectionViewCreateInstructions` contains details on how to create general drawing views.

Methods and Properties Introduced:

- **`pfcProjectionViewCreateInstructions.Create()`**
- **`pfcProjectionViewCreateInstructions.ParentView`**
- **`pfcProjectionViewCreateInstructions.Location`**
- **`pfcProjectionViewCreateInstructions.Exploded`**

The method `pfcProjectionViewCreateInstructions.Create()` creates the `pfcProjectionViewCreateInstructions` data object used for creating projected drawing views.

Use the property

`pfcProjectionViewCreateInstructions.ParentView` to assign the parent view for the projected drawing view.

Use the property

`pfcProjectionViewCreateInstructions.Location` to assign the location of the projected drawing view. This location determines how the drawing view is oriented.

Use the property

`pfcProjectionViewCreateInstructions.Exploded` to set the created projected drawing view to be an exploded view.

Obtaining Drawing Views

Methods and Properties Introduced:

- **`pfcSelection.SelView2D`**
- **`pfcModel2D.List2DViews()`**
- **`pfcModel2D.GetViewByName()`**
- **`pfcModel2D.GetViewDisplaying()`**
- **`pfcSheetOwner.GetSheetBackgroundView`**

The property `pfcSelection.SelView2D` returns the selected drawing view, if the user selected an item from a drawing view. It returns a null value if the selection does not contain a drawing view.

The method `pfcModel2D.List2DViews()` lists and returns the drawing views found. This method does not include the drawing sheet background views returned by the property `pfcSheetOwner.GetSheetBackgroundView`.

The method `pfcModel2D.GetViewByName()` returns the drawing view based on the name. This method returns a null value if the specified view does not exist.

The method `pfcModel2D.GetViewDisplaying()` returns the drawing view that displays a dimension. This method returns a null value if the dimension is not displayed in the drawing.



Note

This method works for solid and drawing dimensions.

The method `pfcSheetOwner.GetSheetBackgroundView` returns the drawing sheet background views.

Drawing View Information

Methods and Properties Introduced:

- **pfcChild.DBParent**
- **pfcView2D.GetSheetNumber()**
- **pfcView2D.IsBackground**
- **pfcView2D.GetModel()**
- **pfcView2D.Scale**
- **pfcView2D.GetIsScaleUserdefined()**
- **pfcView2D.Outline**
- **pfcView2D.GetLayerDisplayStatus()**
- **pfcView2D.IsViewdisplayLayerDependent**
- **pfcView2D.Display**
- **pfcViewDisplay.CableStyle**
- **pfcViewDisplay.Create()**
- **pfcViewDisplay.RemoveQuiltHiddenLines**
- **pfcViewDisplay.ShowConceptModel**
- **pfcViewDisplay.ShowWeldXSection**
- **pfcViewDisplay.Style**
- **pfcViewDisplay.TangentStyle**
- **pfcView2D.GetTransform**
- **pfcView2D.Name**
- **pfcView2D.GetSimpRep()**

The inherited property `pfcChild.DBParent`, when called on a `pfcView2D` object, provides the drawing model which owns the specified drawing view. The return value of the method can be cascaded to a `pfcModel2D` object.

Note

The property `pfcChild.OId` is reserved for internal use.

The method `pfcView2D.GetSheetNumber()` returns the sheet number of the sheet that contains the drawing view.

The property `pfcView2D.IsBackground` returns a value that indicates whether the view is a background view or a model view.

The method `pfcView2D.GetModel()` returns the solid model displayed in the drawing view.

The property `pfcView2D.Scale` returns the scale of the drawing view.

The method `pfcView2D.GetIsScaleUserdefined()` specifies if the drawing has a user-defined scale.

The property `pfcView2D.Outline` returns the position of the view in the sheet in world units.

The method `pfcView2D.GetLayerDisplayStatus()` returns the display status of the specified layer in the drawing view.

The property `pfcView2D.IsViewdisplayLayerDependent` returns a value that indicates if the display is dependent on the layer.

The property `pfcView2D.Display` returns an output structure that describes the display settings of the drawing view.

The property `pfcViewDisplay.Style` returns the display style for hidden lines.

The property `pfcViewDisplay.TangentStyle` returns the display style for tangent lines.

The property `pfcViewDisplay.CableStyle` returns the display style for cables.

The property `pfcViewDisplay.RemoveQuiltHiddenLines` returns the display style for quilt hidden lines.

The property `pfcViewDisplay.ShowConceptModel` returns the display style for skeleton model.

The property `pfcViewDisplay.ShowWeldXSection` returns the display style for weld cross sections.

The property `pfcView2D.GetTransform` returns a matrix that describes the transform between 3D solid coordinates and 2D world units for that drawing view. The transformation matrix is a combination of the following factors:

- The location of the view origin with respect to the drawing origin.
- The scale of the view units with respect to the drawing units
- The rotation of the model with respect to the drawing coordinate system.

The property `pfcView2D.Name` returns the name of the specified view in the drawing.

The simplified representations of assembly and part can be used as drawing models to create general views. Use the method `pfcView2D.GetSimpRep()` to retrieve the simplified representation for the specified view in the drawing.

Drawing Views Operations

Methods Introduced:

- **pfcView2D.Translate()**
- **pfcView2D.Delete()**
- **pfcView2D.Regenerate()**
- **pfcView2D.SetLayerDisplayStatus()**

The method `pfcView2D.Translate()` moves the drawing view by the specified transformation vector.

The method `pfcView2D.Delete()` deletes a specified drawing view. Set the *DeleteChildren* parameter to `true` to delete the children of the view. Set this parameter to `false` or `null` to prevent deletion of the view if it has children.

The method `pfcView2D.Regenerate()` erases the displayed view of the current object, regenerates the view from the current drawing, and redisplay the view.

The method `pfcView2D.SetLayerDisplayStatus()` sets the display status for the layer in the drawing view.

Drawing Dimensions

This section describes the Creo.JS methods that give access to the types of dimensions that can be created in the drawing mode. They do not apply to dimensions created in the solid mode, either those created automatically as a result of feature creation, or reference dimension created in a solid. A drawing dimension or a reference dimension shown in a drawing is represented by the class `pfcDimension2D`.

Obtaining Drawing Dimensions

Methods and Properties Introduced:

- **pfcModelItemOwner.ListItems()**
- **pfcModelItemOwner.GetItemById()**
- **pfcModelItemOwner.GetItemByName()**
- **pfcSelection.SellItem**

The method `pfcModelItemOwner.ListItems()` returns a list of drawing dimensions specified by the parameter *Type* or returns `null` if no drawing dimensions of the specified type are found. This method lists only those dimensions created in the drawing.

The input parameter *Type* is defined by the enumerated data type `pfcModelItemType` and it has the following valid values:

-
- ITEM_FEATURE—Feature
 - ITEM_SURFACE—Surface
 - ITEM_EDGE—Edge
 - ITEM_COORD_SYS—Coordinate system
 - ITEM_AXIS—Axis
 - ITEM_POINT—Point
 - ITEM_QUILT—Quilt
 - ITEM_CURVE—Curve
 - ITEM_LAYER—Layer
 - ITEM_NOTE—Note
 - ITEM_SIMPREP—Simplified representation
 - ITEM_SOLID_GEOMETRY—Solid geometry
 - ITEM_TABLE—Table
 - ITEM_DTL_ENTITY—Detail entity
 - ITEM_DTL_NOTE—Detail note
 - ITEM_DTL_GROUP—Detail group
 - ITEM_DTL_SYM_DEFINITION—Detail system definition
 - ITEM_DTL_SYM_INSTANCE—Detail system instance
 - ITEM_DTL_OLE_OBJECT—Detail OLE object
 - ITEM_EXPLODED_STATE—Exploded state
 - ITEM_EDGE_START—Start of edge
 - ITEM_LOG_EDGE
 - ITEM_EDGE_END—End of edge
 - ITEM_XSEC—Cross section
 - ITEM_LAYER_STATE—Layer state
 - ITEM_COMBINED_STATE—Combined state
 - ITEM_STYLE_STATE
 - ITEM_RP_MATERIAL—
 - ITEM_VIEW—View
 - ITEM_SURF_FIN—Surface finish
 - ITEM_ANNOT_PLANE—Annotation plane
 - ITEM_ANNOTATION_ELEM—Annotation Element
 - ITEM_SET_DATUM_TAG—Datum tag

- `ITEM_DIMENSION`—Dimension
- `ITEM_REF_DIMENSION`—Reference dimension

Set the parameter *Type* to the type of drawing dimension to retrieve. If this parameter is set to null, then all the dimensions in the drawing are listed.

The method `pfcModelItemOwner.GetItemById()` returns a drawing dimension based on the type and the integer identifier. The method returns only those dimensions created in the drawing. It returns a null if a drawing dimension with the specified attributes is not found.

The method `pfcModelItemOwner.GetItemByName()` returns a drawing dimension based on the type and string name.

The property `pfcSelection.SelectedItem` returns the value of the selected drawing dimension.

Creating Drawing Dimensions

Methods Introduced:

- `pfcDrawingDimCreateInstructions.Create()`
- `pfcModel2D.CreateDrawingDimension()`
- `pfcEmptyDimSense.Create()`
- `pfcPointDimenSense.Create()`
- `pfcSplinePointDimSense.Create()`
- `pfcTangentIndexDimSense.Create()`
- `pfcLinAOCTangentDimensionSense.Create()`
- `pfcAngularDimSense.Create()`
- `pfcPointToAngleDimSense.Create()`

The method `pfcDrawingDimCreateInstructions.Create()` creates an instructions object that describes how to create a drawing dimension using the method `pfcModel2D.CreateDrawingDimension()`.

The parameters of the instruction object are:

- *Attachments*—The entities that the dimension is attached to. The selections should include the drawing model view.
- *IsRefDimension*—True if the dimension is a reference dimension, otherwise null or false.
- *OrientationHint*—Describes the orientation of the dimensions in cases where this cannot be deduced from the attachments themselves.

-
- *Senses*—Gives more information about how the dimension attaches to the entity, such as to what part of the entity and in what direction the dimension runs. The types of dimension senses are as follows:

- DIMSENSE_NONE
- DIMSENSE_POINT
- DIMSENSE_SPLINE_PT
- DIMSENSE_TANGENT_INDEX
- DIMSENSE_LINEAR_TO_ARC_OR_CIRCLE_TANGENT
- DIMSENSE_ANGLE
- DIMSENSE_POINT_TO_ANGLE

- *TextLocation*—The location of the dimension text, in world units.

The method `pfcModel2D.CreateDrawingDimension()` creates a dimension in the drawing based on the instructions data object that contains information needed to place the dimension. It takes as input an array of `pfcSelection` objects and an array of `pfcDimensionSense` structures that describe the required attachments. The method returns the created drawing dimension.

The method `pfcEmptyDimSense.Create()` creates an empty dimension sense object. In this case no information such as location or direction is needed to describe the attachment points. For example, if there is a single attachment which is a straight line, the dimension is the length of the straight line. If the attachments are two parallel lines, the dimension is the distance between them.

The method `pfcPointDimSense.Create()` creates a new dimension sense object to be used in creating a drawing dimension. The input parameter *PointType* is defined by the enumerated data type `pfcDimensionPointType` and the valid values are:

- DIMPOINT_END1—The first end of the entity
- DIMPOINT_END2—The second end of the entity
- DIMPOINT_CENTER—The center of an arc or circle
- DIMPOINT_NONE—No information such as location or direction of the attachment is specified. This is similar to setting the *PointType* to DIMSENSE_NONE.
- DIMPOINT_MIDPOINT—The mid-point of the entity

The method `pfcSplinePointDimSense.Create()` creates a new spline point dimension sense object. This means that the attachment is to a point on a spline.

Use the property `pfcSplinePointDimSense.SplinePointIndex` specifies the spline point index.

The method `pfcTangentIndexDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_TANGENT_INDEX`. The attachment is to a tangent of the entity, which is an arc or a circle.

The method `pfcLinAOCTangentDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_LINEAR_TO_ARC_OR_CIRCLE_TANGENT`. The dimension is the perpendicular distance between the a line and a tangent to an arc or a circle that is parallel to the line. The `sense` field is set to the value of the parameter *TangentType*.

The values for *TangentType* and a description of the location of the tangent for each value, follow:

- `DIMLINAOCTANGENT_LEFT0`—The tangent is left of the line, and is on the same side, of the center of the arc or circle, as the line.
- `DIMLINAOCTANGENT_RIGHT0`—The tangent is right of the line, and is on the same side, of the center of the arc or circle, as the line.
- `DIMLINAOCTANGENT_LEFT1`—The tangent is left of the line, and is on the opposite side of the line.
- `DIMLINAOCTANGENT_RIGHT1`— The tangent is right of the line, and is on the opposite side of the line.

The method `pfcAngularDimSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_ANGLE`. The dimension is the angle between two straight entities. The `sense` field is set to the value of the parameter *Options*.

Use the property `pfcAngularDimSense.Options` to determine the location of the angle dimension.

The values for *Options* are as follows:

- `IsFirst`—Is set to `TRUE` if the angle dimension starts from the specified entity in a counterclockwise direction. Is set to `FALSE` if the dimension ends at the specified entity. The value is `TRUE` for one entity and `FALSE` for the other entity forming the angle.
- `ShouldFlip`—If the value of `ShouldFlip` is `FALSE`, and the direction of the specified entity is away from the vertex of the angle, then the dimension attaches directly to the entity. If the direction of the entity is away from the vertex of the angle, then the dimension is attached to the witness line. The witness line is in line with the entity but in the direction opposite to the vertex of the angle. If the value of `ShouldFlip` is `TRUE` then the above cases are reversed.

The method `pfcPointToAngleDimSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_POINT_TO_ANGLE`. The dimension is the angle between a line entity and the tangent to a curved entity. The

curve attachment is of the type `DIMSENSE_POINT_TO_ANGLE` and the line attachment is of the type `DIMSENSE_POINT`. In this case both the `angle` and the `angle_sense` fields must be set. The field `sense` shows which end of the curve the dimension is attached to and the field `angle_sense` shows the direction in which the dimension rotates and to which side of the tangent it attaches.

Drawing Dimensions Information

Methods and Properties Introduced:

- **`pfcDrawing.IsDimensionAssociative`**
- **`pfcDimension.IsReference()`**
- **`pfcDimension2D.IsDisplayed()`**
- **`pfcDrawing.GetDimensionAttachPoints()`**
- **`pfcDrawing.GetDimensionSenses()`**
- **`pfcDrawing.GetDimensionOrientHint()`**
- **`pfcDrawing.GetBaselineDimension()`**
- **`pfcDrawing.SetDimensionLocation`**
- **`pfcDrawing.GetDimensionView()`**
- **`pfcDimension.Tolerance()`**
- **`pfcDrawing.IsDimensionToleranceDisplayed()`**

The property `pfcDrawing.IsDimensionAssociative` returns whether the dimension or reference dimension in a drawing is associative.

The method `pfcDimension.IsReference()` determines whether the drawing dimension is a reference dimension.

The method `pfcDimension2D.IsDisplayed()` determines whether the dimension appears in the drawing.

The dimension senses array returned by the method `pfcDrawing.GetDimensionSenses()` gives more information on how these attachments are interpreted.

The method `pfcDrawing.GetDimensionSenses()` returns an array of dimension senses, describing how the dimension is attached to each attachment returned by the method `pfcDrawing.GetDimensionAttachPoints()`.

The method `pfcDrawing.GetDimensionOrientHint()` returns the orientation hint for placing the drawing dimensions. The orientation hint determines how Creo Parametric will orient the dimension with respect to the attachment points.

Note

These methods described above are applicable only for dimensions created in the drawing mode. It does not support dimensions created at intersection points of entities.

The method `pfcDrawing.GetBaselineDimension()` returns an ordinate baseline drawing dimension. It returns a null value if the dimension is not an ordinate dimension.

Note

The method updates the display of the dimension only if it is currently displayed.

The property `pfcDrawing.SetDimensionLocation` returns the placement location of the dimension.

The method `pfcDrawing.GetDimensionView()` returns the drawing view in which the dimension is displayed. This method applies to dimensions stored in the solid or in the drawing.

The method `pfcDimension.Tolerance()` retrieves the upper and lower tolerance limits of the drawing dimension in the form of the `pfcDimTolerance` object. A null value indicates a nominal tolerance.

Use the method `pfcDrawing.IsDimensionToleranceDisplayed()` determines whether or not the dimension's tolerance is displayed in the drawing.

Drawing Dimensions Operations

Methods Introduced:

- `pfcDrawing.ConvertOrdinateDimensionToLinear()`
- `pfcDrawing.ConvertLinearDimensionToOrdinate()`
- `pfcDrawing.ConvertLinearDimensionToBaseline()`
- `pfcDrawing.SwitchDimensionView()`
- `pfcDimension2D.SetTolerance()`
- `pfcDrawing.EraseDimension()`
- `pfcModel2D.SetViewDisplaying()`

The method `pfcDrawing.ConvertOrdinateDimensionToLinear()` converts an ordinate drawing dimension to a linear drawing dimension. The drawing containing the dimension must appear.

The method `pfcDrawing.ConvertLinearDimensionToOrdinate()` converts a linear drawing dimension to an ordinate baseline dimension.

The method `pfcDrawing.ConvertLinearDimensionToBaseline()` converts a location on a linear drawing dimension to an ordinate baseline dimension. The method returns the newly created baseline dimension.

 Note

The method updates the display of the dimension only if it is currently displayed.

The method `pfcDrawing.SwitchDimensionView()` changes the view where a dimension created in the drawing is displayed.

The method `pfcDimension2D.SetTolerance()` assigns the upper and lower tolerance limits of the drawing dimension.

The method `pfcDrawing.EraseDimension()` permanently erases the dimension from the drawing.

The method `pfcModel2D.SetViewDisplaying()` changes the view where a dimension created in a solid model is displayed.

Drawing Tables

A drawing table in Creo.JS is represented by the class `pfcTable`. It is a child of the `pfcModelItem` class.

Some drawing table methods operate on specific rows or columns. The row and column numbers in Creo.JS begin with 1 and range up to the total number of rows or columns in the table. Some drawing table methods operate on specific table cells. The class `pfcTableCell` is used to represent a drawing table cell.

Creating Drawing Cells

Method Introduced:

- **`pfcTableCell.Create()`**

The method `pfcTableCell.Create()` creates the `pfcTableCell` object representing a cell in the drawing table.

Some drawing table methods operate on specific drawing segment. A multisegmented drawing table contains 2 or more areas in the drawing. Inserting or deleting rows in one segment of the table can affect the contents of other segments. Table segments are numbered beginning with 0. If the table has only a single segment, use 0 as the segment id in the relevant methods.

Selecting Drawing Tables and Cells

Methods and Properties Introduced:

- **pfcBaseSession.Select()**
- **pfcSelection.SelItem**
- **pfcSelection.SelTableCell**
- **pfcSelection.SelTableSegment**

Tables may be selected using the method `pfcBaseSession.Select()`. Pass the filter `dwg_table` to select an entire table and the filter `table_cell` to prompt the user to select a particular table cell.

The property `pfcSelection.SelItem` returns the selected table handle. It is a model item that can be cast to a `pfcTable` object.

The property `pfcSelection.SelTableCell` returns the row and column indices of the selected table cell.

The property `pfcSelection.SelTableSegment` returns the table segment identifier for the selected table cell. If the table consists of a single segment, this method returns the identifier 0.

Creating Drawing Tables

Methods Introduced:

- **pfcTableCreateInstructions.Create()**
- **pfcTableOwner.CreateTable()**

The method `pfcTableCreateInstructions.Create()` creates the `pfcTableCreateInstructions` data object that describes how to construct a new table using the method `pfcTableOwner.CreateTable()`.

The parameters of the instructions data object are:

- *Origin*—This parameter stores a three dimensional point specifying the location of the table origin. The origin is the position of the top left corner of the table.
- *RowHeights*—Specifies the height of each row of the table.
- *ColumnData*—Specifies the width of each column of the table and its justification.

-
- *SizeTypes*—Indicates the scale used to measure the column width and row height of the table.

The method `pfcTableOwner.CreateTable()` creates a table in the drawing specified by the `pfcTableCreateInstructions` data object.

Retrieving Drawing Tables

Methods and Properties Introduced

- **`pfcTableRetrieveInstructions.Create()`**
- **`pfcTableRetrieveInstructions.FileName`**
- **`pfcTableRetrieveInstructions.Path`**
- **`pfcTableRetrieveInstructions.Version`**
- **`pfcTableRetrieveInstructions.Position`**
- **`pfcTableRetrieveInstructions.ReferenceSolid`**
- **`pfcTableRetrieveInstructions.ReferenceRep`**
- **`pfcTableOwner.RetrieveTable()`**
- **`pfcTableOwner.RetrieveTableByOrigin()`**

The method `pfcTableOwner.RetrieveTable()` retrieves a table specified by the `pfcTableRetrieveInstructions` data object from a file on the disk. It returns the retrieved table. The data object contains information on the table to retrieve and is returned by the method `pfcTableRetrieveInstructions.Create()`.

The method `pfcTableRetrieveInstructions.Create()` creates the `pfcTableRetrieveInstructions` data object that describes how to retrieve a drawing table using the methods `pfcTableOwner.RetrieveTable()` and `pfcTableOwner.RetrieveTableByOrigin()`. The method returns the created instructions data object.

The parameters of the instruction object `pfcTableRetrieveInstructions` are:

- *FileName*—Name of the file containing the drawing table.
- *Position*—Coordinates of the point on the drawing sheet, where the retrieved table must be placed. You must specify the value in screen coordinates.

You can also set the parameters for `pfcTableRetrieveInstructions` data object using the following properties:

- `pfcTableRetrieveInstructions.FileName`—Sets the name of the drawing table. You must not specify the extension.

- `pfcTableRetrieveInstructions.Path`—Sets the path to the drawing table file. The path must be specified relative to the working directory.
- `pfcTableRetrieveInstructions.Version`—Sets the version of the drawing table that must be retrieved. If you specify `NULL` the latest version of the drawing table is retrieved.
- `pfcTableRetrieveInstructions.Position`—Sets the coordinates of the point on the drawing sheet, where the table must be placed. You must specify the value in screen coordinates.
- `pfcTableRetrieveInstructions.ReferenceSolid`—Sets the model from which data must be copied into the drawing table. If this argument is passed as `NULL`, an empty table is created.
- `pfcTableRetrieveInstructions.ReferenceRep`—Sets the handle to the simplified representation in a solid, from which data must be copied into the drawing table. If this argument is passed as `NULL`, and the argument *solid* is not `NULL`, then data from the solid model is copied into the drawing table.

The method `pfcTableOwner.RetrieveTable()` retrieves a table specified by the `pfcTableRetrieveInstructions` data object from a file on the disk. It returns the retrieved table. The upper-left corner of the table is placed on the drawing sheet at the position specified by the `pfcTableRetrieveInstructions` data object.

The method `pfcTableOwner.RetrieveTableByOrigin()` also retrieves a table specified by the `pfcTableRetrieveInstructions` data object from a file on the disk. The origin of the table is placed on the drawing sheet at the position specified by the `pfcTableRetrieveInstructions` data object. Tables can be created with different origins by specifying the option **Direction**, in the **Insert Table** dialog box.

Drawing Tables Information

Methods Introduced:

- `pfcTableOwner.ListTables()`
- `pfcTableOwner.GetTable()`
- `pfcTable.GetRowCount()`
- `pfcTable.GetColumnCount()`
- `pfcTable.CheckIfIsFromFormat()`
- `pfcTable.GetRowSize()`
- `pfcTable.GetColumnSize()`
- `pfcTable.GetText()`

- **pfcTable.GetCellNote()**

The method `pfcTableOwner.ListTables()` returns an array of tables found in the model.

The method `pfcTableOwner.GetTable()` returns a table specified by the table identifier in the model. It returns a null value if the table is not found.

The method `pfcTable.GetRowCount()` returns the number of rows in the table.

The method `pfcTable.GetColumnCount()` returns the number of columns in the table.

The method `pfcTable.CheckIfIsFromFormat()` checks if the drawing table was created using the format. The method returns a true value if the table was created by applying the drawing format.

The method `pfcTable.GetRowSize()` returns the height of the drawing table row specified by the segment identifier and the row number.

The method `pfcTable.GetColumnSize()` returns the width of the drawing table column specified by the segment identifier and the column number.

The method `pfcTable.GetText()` returns the array of text in a drawing table cell. Set the value of the parameter *Mode* to `DWGTABLE_NORMAL` to get the text as displayed on the screen. Set it to `DWGTABLE_FULL` to get symbolic text, which includes the names of parameter references in the table text.

The method `pfcTable.GetCellNote()` returns the detail note item contained in the table cell.

Drawing Tables Operations

Methods Introduced:

- **pfcTable.Erase()**
- **pfcTable.Display()**
- **pfcTable.RotateClockwise()**
- **pfcTable.InsertRow()**
- **pfcTable.InsertColumn()**
- **pfcTable.MergeRegion()**
- **pfcTable.SubdivideRegion()**
- **pfcTable.DeleteRow()**
- **pfcTable.DeleteColumn()**
- **pfcTable.SetText()**
- **pfcTableOwner.DeleteTable()**

The method `verases` the specified table temporarily from the display. It still exists in the drawing. The erased table can be displayed again using the method `pfcTable.Display()`. The table will also be redisplayed by a window repaint or a regeneration of the drawing. Use these methods to hide a table from the display while you are making multiple changes to the table.

The method `pfcTable.RotateClockwise()` rotates a table clockwise by the specified amount of rotation.

The method `pfcTable.InsertRow()` inserts a new row in the drawing table. Set the value of the parameter *RowHeight* to specify the height of the row. Set the value of the parameter *InsertAfterRow* to specify the row number after which the new row has to be inserted. Specify 0 to insert a new first row.

The method `pfcTable.InsertColumn()` inserts a new column in the drawing table. Set the value of the parameter *ColumnWidth* to specify the width of the column. Set the value of the parameter *InsertAfterColumn* to specify the column number after which the new column has to be inserted. Specify 0 to insert a new first column.

The method `pfcTable.MergeRegion()` merges table cells within a specified range of rows and columns to form a single cell. The range is a rectangular region specified by the table cell on the upper left of the region and the table cell on the lower right of the region.

The method `pfcTable.SubdivideRegion()` removes merges from a region of table cells that were previously merged. The region to remove merges is specified by the table cell on the upper left of the region and the table cell on the lower right of the region.

The methods `pfcTable.DeleteRow()` and `pfcTable.DeleteColumn()` delete any specified row or column from the table. The methods also remove the text from the affected cells.

The method `pfcTable.SetText()` sets text in the table cell.

Use the method `pfcTableOwner.DeleteTable()` to delete a specified drawing table from the model permanently. The deleted table cannot be displayed again.

Note

Many of the above methods provide a parameter *Repaint*. If this is set to true the table is repainted after the change. If set to false or null Creo Parametric delays the repaint, allowing you to perform several operations before showing changes on the screen.

Drawing Table Segments

Drawing tables can be constructed with one or more segments. Each segment can be independently placed. The segments are specified by an integer identifier starting with 0.

Methods and Properties Introduced:

- **`pfcSelection.SelTableSegment`**
- **`pfcTable.GetSegmentCount()`**
- **`pfcTable.GetSegmentSheet()`**
- **`pfcTable.MoveSegment()`**
- **`pfcTable.GetInfo()`**

The property `pfcSelection.SelTableSegment` returns the value of the segment identifier of the selected table segment. It returns a null value if the selection does not contain a segment identifier.

The method `pfcTable.GetSegmentCount()` returns the number of segments in the table.

The method `pfcTable.GetSegmentSheet()` determines the sheet number that contains a specified drawing table segment.

The `pfcTable.MoveSegment()` moves a drawing table segment to a new location. Pass the co-ordinates of the target position in the format x, y, z=0.



Note

Set the value of the parameter *Repaint* to true to repaint the drawing with the changes. Set it to false or null to delay the repaint.

To get information about a drawing table pass the value of the segment identifier as input to the method `pfcTable.GetInfo()`. The method returns the table information including the rotation, row and column information, and the 3D outline.

Repeat Regions

Methods Introduced:

- **`pfcTable.IsCommentCell()`**
- **`pfcTable.GetCellComponentModel()`**
- **`pfcTable.GetCellReferenceModel`**
- **`pfcTable.GetCellTopModel`**
- **`pfcTableOwner.UpdateTables`**

The methods `pfcTable.IsCommentCell()`, `pfcTable.GetCellComponentModel()`, `pfcTable.GetCellReferenceModel`, and `pfcTableOwner.UpdateTables` apply to repeat regions in drawing tables.

The method `pfcTable.IsCommentCell()` indicates whether a cell in a repeat region contains a comment.

The method `pfcTable.GetCellComponentModel()` returns the path to the assembly component model that is being referenced by a cell in a repeat region of a drawing table. It does not return a valid path if the cell attribute is set to `NO DUPLICATE` or `NO DUPLICATE/LEVEL`.

The property `pfcTable.GetCellReferenceModel` returns the reference component that is being referred to by a cell in a repeat region of a drawing table, even if cell attribute is set to `NO DUPLICATE` or `NO DUPLICATE/LEVEL`.

The property `pfcTable.GetCellTopModel` returns the top model that is being referred to by a cell in a repeat region of a drawing table, even if cell attribute is set to `NO DUPLICATE` or `NO DUPLICATE/LEVEL`.

Use the property `pfcTableOwner.UpdateTables` to update the repeat regions in all the tables to account for changes to the model. It is equivalent to the command **Table, Repeat Region, Update**.

Detail Items

The methods described in this section operate on detail items.

In Creo.JS you can create, delete and modify detail items, control their display, and query what detail items are present in the drawing. The types of available detail items are as follows:

- Draft Entities—Contain graphical items created in Creo Parametric. The items are as follows:
 - Arc
 - Ellipse
 - Line
 - Point
 - Polygon
 - Spline
- Notes—Textual annotations
- Symbol Definitions—Contained in the drawing's symbol gallery.
- Symbol Instances—Instances of a symbol placed in a drawing.

-
- Draft Groups—Groups of detail items that contain notes, symbol instances, and draft entities.
 - OLE objects—Object Linking and Embedding (OLE) objects embedded in the Creo Parametric drawing file.

Listing Detail Items

Methods Introduced:

- **pfcModelItemOwner.ListItems()**
- **pfcDetailItemOwner.ListDetailItems()**
- **pfcModelItemOwner.GetItemById()**
- **pfcDetailItemOwner.CreateDetailItem()**

The method `pfcModelItemOwner.ListItems()` returns a list of detail items specified by the parameter *Type* or returns null if no detail items of the specified type are found.

The values of the parameter *Type* for detail items follow:

- `ITEM_DTL_ENTITY`—Detail Entity
- `ITEM_DTL_NOTE`—Detail Note
- `ITEM_DTL_GROUP`—Draft Group
- `ITEM_DTL_SYM_DEFINITION`—Detail Symbol Definition
- `ITEM_DTL_SYM_INSTANCE`—Detail Symbol Instance
- `ITEM_DTL_OLE_OBJECT`—Drawing embedded OLE object

If this parameter is set to null, then all the model items in the drawing are listed.

If the model has multiple bodies, the method `pfcModelItemOwner.ListItems()` returns the exception `pfcXToolkitMultibodyUnsupported`.

The method `pfcDetailItemOwner.ListDetailItems()` also lists the detail items in the model. Pass the type of the detail item and the sheet number that contains the specified detail items.

Set the input parameter *Type* to the type of detail item to be listed. Set it to null to return all the detail items. The input parameter *SheetNumber* determines the sheet that contains the specified detail item. Pass null to search all the sheets. This argument is ignored if the parameter *Type* is set to `DETAIL_SYM_DEFINITION`.

The method returns an array of detail items and returns a null if no items matching the input values are found.

The method `pfcModelItemOwner.GetItemById()` returns a detail item based on the type of the detail item and its integer identifier. The method returns a null if a detail item with the specified attributes is not found.

Creating a Detail Item

Methods Introduced:

- **`pfcDetailItemOwner.CreateDetailItem()`**
- **`pfcDetailGroupInstructions.Create()`**

The method `pfcDetailItemOwner.CreateDetailItem()` creates a new detail item based on the instruction data object that describes the type and content of the new detail item. The instructions data object is returned by the method `pfcDetail.DetailGroupInstructions.Create()`. The method returns the newly created detail item.

Detail Entities

A detail entity in Creo.JS is represented by the class `pfcDetailEntityItem`. It is a child of the `pfcDetailItem` class.

The class `pfcDetailEntityInstructions` contains specific information used to describe a detail entity item.

Instructions

Methods and Properties Introduced:

- **`pfcDetailEntityInstructions.Create()`**
- **`pfcDetailEntityInstructions.Geometry`**
- **`pfcDetailEntityInstructions.IsConstruction`**
- **`pfcDetailEntityInstructions.Color`**
- **`pfcDetailEntityInstructions.FontName`**
- **`pfcDetailEntityInstructions.Width`**
- **`pfcDetailEntityInstructions.View`**

The method `pfcDetailEntityInstructions.Create()` creates an instructions object that describes how to construct a detail entity, for use in the methods `pfcDetailItemOwner.CreateDetailItem()`, `pfcDetailSymbolDefItem.CreateDetailItem()`, and `pfcDetailEntityItem.Modify()`.

The instructions object is created based on the curve geometry and the drawing view associated with the entity. The curve geometry describes the trajectory of the detail entity in world units. The drawing view can be a model view returned by the method `pfcModel2D.List2DViews()` or a drawing sheet background view returned by the method `pfcSheetOwner.GetSheetBackgroundView`. The background view indicates that the entity is not associated with a particular model view.

The method returns the created instructions object.

 Note

Changes to the values of a `pfcDetailEntityInstructions` object do not take effect until that instructions object is used to modify the entity using `pfcDetailEntityItem.Modify()`.

The property `pfcDetailEntityInstructions.Geometry` returns the geometry of the detail entity item.

The property `pfcDetailEntityInstructions.IsConstruction` returns a value that specifies whether the entity is a construction entity.

The property `pfcDetailEntityInstructions.Color` returns the color of the detail entity item.

The property `pfcDetailEntityInstructions.FontName` returns the line style used to draw the entity. The method returns a null value if the default line style is used.

The property `pfcDetailEntityInstructions.Width` returns the value of the width of the entity line. The method returns a null value if the default line width is used.

The property `pfcDetailEntityInstructions.View` returns the drawing view associated with the entity. The view can either be a model view or a drawing sheet background view.

Detail Entities Information

Method and Property Introduced:

- **`pfcDetailEntityItem.GetInstructions()`**
- **`pfcDetailEntityItem.SymbolDef`**

The method `pfcDetailEntityItem.GetInstructions()` returns the instructions data object that is used to construct the detail entity item.

The property `pfcDetailEntityItem.SymbolDef` returns the symbol definition that contains the entity. This property returns a null value if the entity is not a part of a symbol definition.

Detail Entities Operations

Methods Introduced:

- **`pfcDetailEntityItem.Draw()`**
- **`pfcDetailEntityItem.Erase()`**

- **pfcDetailEntityItem.Modify()**

The method `pfcDetailEntityItem.Draw()` temporarily draws a detail entity item, so that it is removed during the next draft regeneration.

The method `pfcDetailEntityItem.Erase()` undraws a detail entity item temporarily, so that it is redrawn during the next draft regeneration.

The method `pfcDetailEntityItem.Modify()` modifies the definition of an entity item using the specified instructions data object.

OLE Objects

An object linking and embedding (OLE) object is an external file, such as a document, graphics file, or video file that is created using an external application and which can be inserted into another application, such as Creo Parametric. You can create and insert supported OLE objects into a two-dimensional Creo Parametric file, such as a drawing, report, format file, notebook, or diagram. The functions described in this section enable you to identify and access OLE objects embedded in drawings.

Methods and Properties Introduced:

- **pfcDetailOLEObject.ApplicationType()**
- **pfcDetailOLEObject.Outline**
- **pfcDetailOLEObject.Path**
- **pfcDetailOLEObject.Sheet**

The method `pfcDetailOLEObject.ApplicationType()` returns the type of the OLE object as a string, for example, `Microsoft Word Document`.

The property `pfcDetailOLEObject.Outline` returns the extent of the OLE object embedded in the drawing.

The property `pfcDetailOLEObject.Path` returns the path to the external file for each OLE object, if it is linked to an external file.

The property `pfcDetailOLEObject.Sheet` returns the sheet number for the OLE object.

Detail Notes

A detail note in Creo.JS is represented by the class `pfcDetailNoteItem`. It is a child of the `pfcDetailItem` class.

The class `pfcDetailNoteInstructions` contains specific information that describes a detail note.

The class `pfcDetailNoteInstructions` and all the methods under this interface are deprecated.

Instructions

Methods Introduced:

- **pfcDetailNoteInstructions.Create**
- **pfcDetailNoteItem.GetTextLines**
- **pfcDetailNoteItem.SetTextLines**
- **pfcDetailNoteItem.IsDisplayed**
- **pfcDetailNoteItem.SetIsDisplayed**
- **pfcDetailNoteItem.IsReadOnly**
- **pfcDetailNoteItem.SetReadOnly**
- **pfcDetailNoteItem.GetNoteTextStyle**
- **pfcDetailNoteItem.SetNoteTextStyle**
- **pfcAnnotationTextStyle.IsTextMirrored**
- **pfcAnnotationTextStyle.MirrorText**
- **pfcAnnotationTextStyle.GetHorizontalJustification**
- **pfcAnnotationTextStyle.SetHorizontalJustification**
- **pfcAnnotationTextStyle.GetVerticalJustification**
- **pfcAnnotationTextStyle.SetVerticalJustification**
- **pfcAnnotationTextStyle.GetColor**
- **pfcAnnotationTextStyle.SetColor**
- **pfcDetailNoteItem.GetElbowLength**
- **pfcDetailNoteItem.SetElbow**
- **pfcDetailNoteItem.SetOnItemAttachment**
- **pfcDetailNoteItem.SetOffsetAttachment**
- **pfcDetailNoteItem.SetFreeAttachment**
- **pfcDetailNoteItem.GetAttachment**
- **pfcAnnotationTextStyle.GetAngle**
- **pfcAnnotationTextStyle.SetAngle**

Superseded Methods:

- **pfcDetailNoteInstructions.GetTextLines**
- **pfcDetailNoteInstructions.SetTextLines**
- **pfcDetailNoteInstructions.GetIsDisplayed**
- **pfcDetailNoteInstructions.SetIsDisplayed**
- **pfcDetailNoteInstructions.GetIsReadOnly**
- **pfcDetailNoteInstructions.SetIsReadOnly**
- **pfcDetailNoteInstructions.GetIsMirrored**

-
- **`pfcDetailNoteInstructions.SetIsMirrored`**
 - **`pfcDetailNoteInstructions.GetHorizontal`**
 - **`pfcDetailNoteInstructions.SetHorizontal`**
 - **`pfcDetailNoteInstructions.GetVertical`**
 - **`pfcDetailNoteInstructions.SetVertical`**
 - **`pfcDetailNoteInstructions.GetColor`**
 - **`pfcDetailNoteInstructions.SetColor`**
 - **`pfcDetailNoteInstructions.GetLeader`**
 - **`pfcDetailNoteInstructions.SetLeader`**
 - **`pfcDetailNoteInstructions.GetTextAngle`**
 - **`pfcDetailNoteInstructions.SetTextAngle`**

The method `pfcDetailNoteInstructions.Create` creates a data object that describes how a detail note item should be constructed when passed to the methods `pfcDetailItemOwner.CreateDetailItem`, `pfcDetailSymbolDefItem.CreateDetailItem`, or `pfcDetailNoteItem.Modify`. The parameter *inTextLines* specifies the sequence of text line data objects that describe the contents of the note.

Note

Changes to the values of a `pfcDetailNoteInstructions` object do not take effect until that instructions object is used to modify the note using `pfcDetailNoteItem.Modify`.

For creating a detail note item, the method `pfcDetailItemOwner.CreateDetailItem` is deprecated, as it uses the deprecated interface `pfcDetailNoteInstructions`. Use the methods `pfcDetailItemOwner.CreateFreeNote`, `pfcDetailItemOwner.CreateOffsetNote`, `pfcDetailItemOwner.CreateOnItemNote`, or `pfcDetailItemOwner.CreateLeaderNote` instead.

The method `pfcDetailNoteItem.GetTextLines` returns the description of text line contents in the note.

The method `pfcDetailNoteItem.SetTextLines` sets the description of the text line contents in the note.

The method `pfcDetailNoteItem.IsDisplayed` checks if note data is displayed. This is useful for notes whose owner is not displayed in session

The method `pfcDetailNoteItem.SetIsDisplayed` sets note data displayed. This is useful for notes whose owner is not displayed in session.

The method `pfcDetailNoteItem.IsReadOnly` determines whether the note can be edited by the user, while the method `pfcDetailNoteItem.SetReadOnly` toggles the read only status of the note.

The method `pfcDetailNoteItem.GetNoteTextStyle` and `pfcDetailNoteItem.SetNoteTextStyle` get and set the textstyle of note.

The methods `pfcAnnotationTextStyle.IsTextMirrored` checks if the text style is mirrored.

The methods `pfcAnnotationTextStyle.MirrorText` sets the mirroring of the text style. specifies the option to mirror the text style. Specify the input argument *Mirror* to true to mirror the text style.

The methods

`pfcAnnotationTextStyle.GetHorizontalJustification` and `pfcAnnotationTextStyle.SetHorizontalJustification` get and set the horizontal justification of the text style using the enumerated data type `pfcHorizontalJustification`. The valid values are:

- `pfcH_JUSTIFY_LEFT`—Aligns the text style object to the left.
- `pfcH_JUSTIFY_CENTER`—Aligns the text style object in the center.
- `pfcH_JUSTIFY_RIGHT`—Aligns the text style object to the right.
- `pfcH_JUSTIFY_DEFAULT`—Aligns the text using the default justification. The justification selected for the first note becomes the default for all successive notes added during the current session.

The methods

`pfcAnnotationTextStyle.GetVerticalJustification` and `pfcAnnotationTextStyle.SetVerticalJustification` get and set the vertical justification of the text style using the enumerated data type `pfcVerticalJustification`. The valid values are:

- `pfcV_JUSTIFY_TOP`—Aligns the text style object to the top.
- `pfcV_JUSTIFY_MIDDLE`—Aligns the text style object to the middle.
- `pfcV_JUSTIFY_BOTTOM`—Aligns the text style object to the bottom.
- `pfcV_JUSTIFY_DEFAULT`—Aligns the text using the default justification. The justification selected for the first note becomes the default for all successive notes added during the current session.

The methods `pfcAnnotationTextStyle.GetColor` and `pfcAnnotationTextStyle.SetColor` retrieve and set the color of the text style as a `pfcColorRGB` object.

The method `pfcDetailNoteItem.GetElbowLength` returns length of note leader elbow.

The method `pfcDetailNoteItem.SetElbow` sets elbow to leader note.

The method `pfcDetailNoteItem.SetOnItemAttachment` sets on item attachment information of note.

The method `pfcDetailNoteItem.SetOffsetAttachment` sets offset attachment information of note.

The method `pfcDetailNoteItem.SetFreeAttachment` sets free attachment information of note.

The method `pfcDetailNoteItem.GetAttachment` returns the attachment information of note.

The methods `pfcAnnotationTextStyle.GetAngle` and `pfcAnnotationTextStyle.SetAngle` get and set the angle of the text style.

Detail Notes Information

Methods and Properties Introduced:

- **`pfcDetailNoteItem.GetInstructions()`**
- **`pfcDetailNoteItem.SymbolDef`**
- **`pfcDetailNoteItem.GetLineEnvelope()`**
- **`pfcDetailNoteItem.GetModelReference()`**

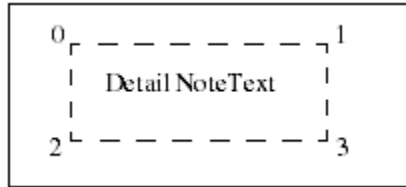
The method `pfcDetailNoteItem.GetInstructions()` returns an instructions data object that describes how to construct the detail note item. This method takes a `ProBoolean` argument, *GiveParametersAsNames*, which determines whether symbolic representations of parameters and drawing properties in the note text should be displayed, or the actual text seen by the user should be displayed.

Note

Creo Parametric does not resolve and replace symbolic callouts for notes which are not displayed. Therefore, if the note is not displayed or is hidden in a layer, the text retrieved may contain symbolic callouts, even when *GiveParametersAsNames* is false.

The property `pfcDetailNoteItem.SymbolDef` returns the symbol definition that contains the note. The property returns a null value if the note is not a part of a symbol definition.

The method `pfcDetailNoteItem.GetLineEnvelope()` determines the screen coordinates of the envelope around the detail note. This envelope is defined by four points. The following figure illustrates how the point order is determined.



The ordering of the points is maintained even if the notes are mirrored or are at an angle.

The method `pfcDetailNoteItem.GetModelReference()` returns the model referenced by the parameterized text in a note. The model is referenced based on the line number and the text index where the parameterized text appears.

Details Notes Operations

Methods Introduced:

- **`pfcDetailNoteItem.Draw()`**
- **`pfcDetailNoteItem.Show()`**
- **`pfcDetailNoteItem.Erase()`**
- **`pfcDetailNoteItem.Remove()`**
- **`pfcDetailNoteItem.KeepArrowTypeAsIs()`**
- **`pfcDetailNoteItem.Modify()`**

The method `pfcDetailNoteItem.Draw()` temporarily draws a detail note item, so that it is removed during the next draft regeneration.

The method `pfcDetailNoteItem.Show()` displays the note item, such that it is repainted during the next draft regeneration.

The method `pfcDetailNoteItem.Erase()` undraws a detail note item temporarily, so that it is redrawn during the next draft regeneration.

The method `pfcDetailNoteItem.Remove()` undraws a detail note item permanently, so that it is not redrawn during the next draft regeneration.

The method `pfcDetailNoteItem.KeepArrowTypeAsIs()` allows you to keep arrow type of the leader note as it is, after a note is modified. You must call this method before the method `pfcDetailNoteItem.Modify()` is called.

The method `pfcDetailNoteItem.Modify()` modifies the definition of an existing detail note item based on the instructions object that describes the new detail note item.

Detail Groups

A detail group in Creo.JS is represented by the class `pfcDetailGroupItem`. It is a child of the `pfcDetailItem` class.

The class `pfcDetailGroupInstructions` contains information used to describe a detail group item.

Instructions

Method and Properties Introduced:

- **`pfcDetailGroupInstructions.Create()`**
- **`pfcDetailGroupInstructions.Name`**
- **`pfcDetailGroupInstructions.Elements`**
- **`pfcDetailGroupInstructions.IsDisplayed`**

The method `pfcDetailGroupInstructions.Create()` creates an instruction data object that describes how to construct a detail group for use in `pfcDetailItemOwner.CreateDetailItem()` and `pfcDetailGroupItem.Modify()`.



Note

Changes to the values of a `pfcDetailGroupInstructions` object do not take effect until that instructions object is used to modify the group using `pfcDetailGroupItem.Modify()`.

The property `pfcDetailGroupInstructions.Name` returns the name of the detail group.

The property `pfcDetailGroupInstructions.Elements` returns the array of the detail items (notes, groups and entities) contained in the group. The property `pfcDetailGroupInstructions.IsDisplayed` returns whether the detail group is displayed in the drawing.

Detail Groups Information

Method Introduced:

- **`pfcDetailGroupItem.GetInstructions()`**

The method `pfcDetailGroupItem.GetInstructions()` gets a data object that describes how to construct a detail group item. The method returns the data object describing the detail group item.

Detail Groups Operations

Methods Introduced:

- **pfcDetailGroupItem.Draw()**
- **pfcDetailGroupItem.Erase()**
- **pfcDetailGroupItem.Modify()**

The method `pfcDetailGroupItem.Draw()` temporarily draws a detail group item, so that it is removed during the next draft generation.

The method `pfcDetailGroupItem.Erase()` temporarily undraws a detail group item, so that it is redrawn during the next draft generation.

The method `pfcDetailGroupItem.Modify()` changes the definition of a detail group item based on the data object that describes how to construct a detail group item.

Detail Symbols

Detail Symbol Definitions

A detail symbol definition in Creo.JS is represented by the class `pfcDetailSymbolDefItem`. It is a child of the `pfcDetailItem` class.

The class `pfcDetailSymbolDefInstructions` contains information that describes a symbol definition. It can be used when creating symbol definition entities or while accessing existing symbol definition entities.

Instructions

Method and Properties Introduced:

- **pfcDetailSymbolDefInstructions.Create()**
- **pfcDetailSymbolDefInstructions.SymbolHeight**
- **pfcDetailSymbolDefInstructions.HasElbow**
- **pfcDetailSymbolDefInstructions.IsTextAngleFixed**
- **pfcDetailSymbolDefInstructions.ScaledHeight**
- **pfcDetailSymbolDefInstructions.Attachments**
- **pfcDetailSymbolDefInstructions.FullPath**
- **pfcDetailSymbolDefInstructions.Reference**

The method `pfcDetailSymbolDefInstructions.Create()` creates an instruction data object that describes how to create a symbol definition based on the path and name of the symbol definition. The instructions object is passed to the methods `pfcDetailItemOwner.CreateDetailItem()` and `pfcDetailSymbolDefItem.Modify()`.

 Note

Changes to the values of a `pfcDetailSymbolDefInstructions` object do not take effect until that instructions object is used to modify the definition using the method `pfcDetailSymbolDefItem.Modify()`.

The property `pfcDetailSymbolDefInstructions.SymbolHeight` returns the value of the height type for the symbol definition. The symbol definition height options are as follows:

- `SYMDEF_FIXED`—Symbol height is fixed.
- `SYMDEF_VARIABLE`—Symbol height is variable.
- `SYMDEF_RELATIVE_TO_TEXT`—Symbol height is determined relative to the text height.

The property `pfcDetailSymbolDefInstructions.HasElbow` determines whether the symbol definition includes an elbow.

The property `pfcDetailSymbolDefInstructions.IsTextAngleFixed` returns whether the text of the angle is fixed.

The property `pfcDetailSymbolDefInstructions.ScaledHeight` returns the height of the symbol definition in inches.

The property `pfcDetailSymbolDefInstructions.Attachments` returns the value of the array of the possible instance attachment points for the symbol definition.

The property `pfcDetailSymbolDefInstructions.FullPath` returns the value of the complete path of the symbol definition file.

The property `pfcDetailSymbolDefInstructions.Reference` returns the text reference information for the symbol definition. It returns a null value if the text reference is not used. The text reference identifies the text item used for a symbol definition which has a height type of `SYMDEF_TEXT_RELATED`.

Detail Symbol Definitions Information

Methods Introduced:

- **`pfcDetailSymbolDefItem.ListDetailItems()`**

- **pfcDetailSymbolDefItem.GetInstructions()**

The method `pfcDetailSymbolDefItem.ListDetailItems()` lists the detail items in the symbol definition based on the type of the detail item.

The method `pfcDetailSymbolDefItem.GetInstructions()` returns an instruction data object that describes how to construct the symbol definition.

Detail Symbol Definitions Operations

Methods Introduced:

- **pfcDetailSymbolDefItem.CreateDetailItem()**
- **pfcDetailSymbolDefItem.Modify()**

The method `pfcDetailSymbolDefItem.CreateDetailItem()` creates a detail item in the symbol definition based on the instructions data object. The method returns the detail item in the symbol definition.

The method `pfcDetailSymbolDefItem.Modify()` modifies a symbol definition based on the instructions data object that contains information about the modifications to be made to the symbol definition.

Retrieving Symbol Definitions

Methods Introduced:

- **pfcDetailItemOwner.RetrieveSymbolDefItem()**

Creo Parametric symbols exist in two different areas: the user-defined area and the system symbols area.

The method `pfcDetailItemOwner.RetrieveSymbolDefItem` retrieves a symbol definition from the user-defined location designated by the configuration option `pro_symbol_dir`. The symbol definition should have been previously saved to a file using Creo Parametric.

The method `pfcDetailItemOwner.RetrieveSymbolDefItem` also retrieves a symbol definition from the system directory. The system area contains symbols provided by Creo Parametric with the Detail module (such as the Welding Symbols Library).

The input parameters of this method follow:

- *FileName*—Name of the symbol definition file.
- *Source*—Source of the symbol definition file. The input parameter *Source* is defined by the enumerated type `pfcDetailSymbolDefItemSource`. The valid values which are supported are listed below:
 - `DTLSYMDEF_SRC_SYSTEM`—Specifies the system symbol definition directory.

-
- `DTLSYMDEF_SRC_PATH`—Specifies the absolute path to a directory containing the symbol definition.
 - *FilePath*—Path to the symbol definition file. It is relative to the path specified by the option `pro_symbol_dir` in the configuration file. A null value indicates that the function should search the current directory.
 - *Version*—Numerical version of the symbol definition file. A null value retrieves the latest version.
 - *UpdateUnconditionally*—True if Creo should update existing instances of this symbol definition, or false to quit the operation if the definition exists in the model.

The method returns the retrieved symbol definition.

Detail Symbol Instances

A detail symbol instance in Creo.JS is represented by the class `pfcDetailSymbolInstItem`. It is a child of the `pfcDetailItem` class.

The class `pfcDetailSymbolInstInstructions` contains information that describes a symbol instance. It can be used when creating symbol instances and while accessing existing groups.

Instructions

Method and Properties Introduced:

- **`pfcDetailSymbolInstInstructions.Create()`**
- **`pfcDetailSymbolInstInstructions.IsDisplayed`**
- **`pfcDetailSymbolInstInstructions.Color`**
- **`pfcDetailSymbolInstInstructions.SymbolDef`**
- **`pfcDetailSymbolInstInstructions.AttachOnDefType`**
- **`pfcDetailSymbolInstInstructions.DefAttachment`**
- **`pfcDetailSymbolInstInstructions.InstAttachment`**
- **`pfcDetailSymbolInstInstructions.Angle`**
- **`pfcDetailSymbolInstInstructions.ScaledHeight`**
- **`pfcDetailSymbolInstInstructions.TextValues`**
- **`pfcDetailSymbolInstInstructions.CurrentTransform`**
- **`pfcDetailSymbolInstInstructions.SetGroups`**

The method `pfcDetailSymbolInstInstructions.Create()` creates a data object that contains information about the placement of a symbol instance.

Note

Changes to the values of a `pfcDetailSymbolInstInstructions` object do not take effect until that instructions object is used to modify the instance using `pfcDetailSymbolInstItem.Modify()`.

The property `pfcDetailSymbolInstInstructions.IsDisplayed` returns a value that specifies whether the instance of the symbol is displayed.

The property `pfcDetailSymbolInstInstructions.Color` returns the color of the detail symbol instance. A null value indicates that the default drawing color is used.

The property `pfcDetailSymbolInstInstructions.SymbolDef` returns the symbol definition used for the instance.

The property `pfcDetailSymbolInstInstructions.AttachOnDefType` returns the attachment type of the instance. The property returns a null value if the attachment represents a free attachment. The attachment options are as follows:

- `SYMDEFATTACH_FREE`—Attachment on a free point.
- `SYMDEFATTACH_LEFT_LEADER`—Attachment via a leader on the left side of the symbol.
- `SYMDEFATTACH_RIGHT_LEADER`—Attachment via a leader on the right side of the symbol.
- `SYMDEFATTACH_RADIAL_LEADER`—Attachment via a leader at a radial location.
- `SYMDEFATTACH_ON_ITEM`—Attachment on an item in the symbol definition.
- `SYMDEFATTACH_NORMAL_TO_ITEM`—Attachment normal to an item in the symbol definition.

The property `pfcDetailSymbolInstInstructions.DefAttachment` returns the value that represents the way in which the instance is attached to the symbol definition.

The property `pfcDetailSymbolInstInstructions.InstAttachment` returns the value of the attachment of the instance that includes location and leader information.

The property `pfcDetailSymbolInstInstructions.Angle` returns the value of the angle at which the instance is placed. The property returns a null value if the value of the angle is 0 degrees.

The property `pfcDetailSymbolInstInstructions.ScaledHeight` returns the height of the symbol instance in the owner drawing or model coordinates. This value is consistent with the height value shown for a symbol instance in the **Properties** dialog box in the Creo Parametric User Interface.

 Note

The scaled height obtained using the above property is partially based on the properties of the symbol definition assigned using the property `pfcDetailSymbolInstInstructions.SymbolDef`. Changing the symbol definition may change the calculated value for the scaled height.

The property `pfcDetailSymbolInstInstructions.TextValues` returns the array of variant text values used while placing the symbol instance.

The property `pfcDetailSymbolInstInstructions.CurrentTransform` returns the coordinate transformation matrix to place the symbol instance.

The property `pfcDetailSymbolInstInstructions.SetGroupspfcDetailSymbolGroupOption`

- `DETAIL_SYMBOL_GROUP_INTERACTIVE`—Symbol groups are interactively selected for display. This is the default value in the GRAPHICS mode.
- `DETAIL_SYMBOL_GROUP_ALL`—All non-exclusive symbol groups are included for display.
- `DETAIL_SYMBOL_GROUP_NONE`—None of the non-exclusive symbol groups are included for display.
- `DETAIL_SYMBOL_GROUP_CUSTOM`—Symbol groups specified by the application are displayed.

Refer to the section [Detail Symbol Groups on page 111](#) for more information on detail symbol groups.

Detail Symbol Instances Information

Method Introduced:

- **`pfcDetailSymbolInstItem.GetInstructions()`**

The method `pfcDetailSymbolInstItem.GetInstructions()` returns an instructions data object that describes how to construct a symbol instance. This method takes a `ProBoolean` argument, *GiveParametersAsNames*, which

determines whether symbolic representations of parameters and drawing properties in the symbol instance should be displayed, or the actual text seen by the user should be displayed.

Detail Symbol Instances Operations

Methods Introduced:

- **`pfcDetailSymbolInstItem.Draw()`**
- **`pfcDetailSymbolInstItem.Erase()`**
- **`pfcDetailSymbolInstItem.Show()`**
- **`pfcDetailSymbolInstItem.Remove()`**
- **`pfcDetailSymbolInstItem.Modify()`**

The method `pfcDetailSymbolInstItem.Draw()` draws a symbol instance temporarily to be removed on the next draft regeneration.

The method `pfcDetailSymbolInstItem.Erase()` undraws a symbol instance temporarily from the display to be redrawn on the next draft generation.

The method `pfcDetailSymbolInstItem.Show()` displays a symbol instance to be repainted on the next draft regeneration.

The method `pfcDetailSymbolInstItem.Remove()` deletes a symbol instance permanently.

The method `pfcDetailSymbolInstItem.Modify()` modifies a symbol instance based on the instructions data object that contains information about the modifications to be made to the symbol instance.

Detail Symbol Groups

A detail symbol group in Creo.JS is represented by the class `pfcDetailSymbolGroup`. It is a child of the `pfcObject` class. A detail symbol group is accessible only as a part of the contents of a detail symbol definition or instance.

The class `pfcDetailSymbolGroupInstructions` contains information that describes a symbol group. It can be used when creating new symbol groups, or while accessing or modifying existing groups.

Instructions

Method and Properties Introduced:

- **`pfcDetailSymbolGroupInstructions.Create()`**
- **`pfcDetailSymbolGroupInstructions.Items`**
- **`pfcDetailSymbolGroupInstructions.Name`**

The method `pfcDetailSymbolGroupInstructions.Create()` creates the `pfcDetailSymbolGroupInstructions` data object that stores the name of the symbol group and the list of detail items to be included in the symbol group.

 Note

Changes to the values of the `pfcDetailSymbolGroupInstructions` data object do not take effect until this object is used to modify the instance using the method `pfcDetailSymbolGroup.Modify()`.

The property `pfcDetailSymbolGroupInstructions.Items` returns the list of detail items included in the symbol group.

The property `pfcDetailSymbolGroupInstructions.Name` returns the name of the symbol group.

Detail Symbol Group Information

Methods Introduced:

- **`pfcDetailSymbolGroup.GetInstructions()`**
- **`pfcDetailSymbolGroup.ParentGroup()`**
- **`pfcDetailSymbolGroup.ParentDefinition()`**
- **`pfcDetailSymbolGroup.ListChildren()`**
- **`pfcDetailSymbolDefItem.ListSubgroups()`**
- **`pfcDetailSymbolDefItem.IsSubgroupLevelExclusive()`**
- **`pfcDetailSymbolInstItem.ListGroups()`**

The method `pfcDetailSymbolGroup.GetInstructions()` returns the `pfcDetailSymbolGroupInstructions` data object that describes how to construct a symbol group.

The method `pfcDetailSymbolGroup.ParentGroup()` returns the parent symbol group to which a given symbol group belongs.

The method `pfcDetailSymbolGroup.ParentDefinition()` returns the symbol definition of a given symbol group.

The method `pfcDetailSymbolGroup.ListChildren()` lists the subgroups of a given symbol group.

The method `pfcDetailSymbolDefItem.ListSubgroups()` lists the subgroups of a given symbol group stored in the symbol definition at the indicated level.

The method

`pfcDetailSymbolDefItem.IsSubgroupLevelExclusive()` identifies if the subgroups of a given symbol group stored in the symbol definition at the indicated level are exclusive or independent. If groups are exclusive, only one of the groups at this level can be active in the model at any time. If groups are independent, any number of groups can be active.

The method `pfcDetailSymbolInstItem.ListGroups()` lists the symbol groups included in a symbol instance. The `pfcSymbolGroupFilter` argument determines the types of symbol groups that can be listed. It takes the following values:

- `DTLSYMINST_ALL_GROUPS`—Retrieves all groups in the definition of the symbol instance.
- `DTLSYMINST_ACTIVE_GROUPS`—Retrieves only those groups that are actively shown in the symbol instance.
- `DTLSYMINST_INACTIVE_GROUPS`—Retrieves only those groups that are not shown in the symbol instance.

Detail Symbol Group Operations

Methods Introduced:

- **`pfcDetailSymbolGroup.Delete()`**
- **`pfcDetailSymbolGroup.Modify()`**
- **`pfcDetailSymbolDefItem.CreateSubgroup()`**
- **`pfcDetailSymbolDefItem.SetSubgroupLevelExclusive`**
- **`pfcDetailSymbolDefItem.SetSubgroupLevelIndependent()`**

The method `pfcDetailSymbolGroup.Delete()` deletes the specified symbol group from the symbol definition. This method does not delete the entities contained in the group.

The method `pfcDetailSymbolGroup.Modify()` modifies the specified symbol group based on the `pfcDetailSymbolGroupInstructions` data object that contains information about the modifications that can be made to the symbol group.

The method `pfcDetailSymbolDefItem.CreateSubgroup()` creates a new subgroup in the symbol definition at the indicated level below the parent group.

The property

`pfcDetailSymbolDefItem.SetSubgroupLevelExclusive` makes the subgroups of a symbol group exclusive at the indicated level in the symbol definition.

Note

After you set the subgroups of a symbol group as exclusive, only one of the groups at the indicated level can be active in the model at any time.

The method

`pfcDetailSymbolDefItem.SetSubgroupLevelIndependent()` makes the subgroups of a symbol group independent at the indicated level in the symbol definition.

Note

After you set the subgroups of a symbol group as independent, any number of groups at the indicated level can be active in the model at any time.

Detail Attachments

A detail attachment in Creo.JS is represented by the class `pfcAttachment`. It is used for the following tasks:

- The way in which a drawing note or a symbol instance is placed in a drawing.
- The way in which a leader on a drawing note or symbol instance is attached.

Method Introduced:

- **`pfcAttachment.GetType()`**

The method `pfcAttachment.GetType()` returns the `pfcAttachmentType` object containing the types of detail attachments. The detail attachment types are as follows:

- `ATTACH_FREE`—The attachment is at a free point possibly with respect to a given drawing view.
- `ATTACH_PARAMETRIC`—The attachment is to a point on a surface or an edge of a solid.
- `ATTACH_OFFSET`—The attachment is offset to another drawing view, to a model item, or to a 3D model annotation.
- `ATTACH_TYPE_UNSUPPORTED`—The attachment is to an item that cannot be represented in PFC at the current time. However, you can still retrieve the location of the attachment.

Free Attachment

The `ATTACH_FREE` detail attachment type is represented by the `pfcFreeAttachment`. It is a child of the `pfcAttachment` class.

Methods and Properties Introduced:

- **`pfcFreeAttachment.AttachmentPoint`**
- **`pfcFreeAttachment.View`**
- **`pfcFreeAttachment.Create()`**

The property `pfcFreeAttachment.AttachmentPoint` returns the attachment point. This location is in screen coordinates for drawing items, symbol instances and surface finishes on flat-to-screen annotation planes, and in model coordinates for symbols and surface finishes on 3D model annotation planes.

The property `pfcFreeAttachment.View` sets the drawing view.

The method `pfcFreeAttachment.Create()` creates a data object used for representing a free attachment. A free attachment is relative to a fixed location, or relative to a fixed location with respect to a drawing view.

Parametric Attachment

The `ATTACH_PARAMETRIC` detail attachment type is represented by the `pfcParametricAttachment`. It is a child of the `pfcAttachment` class.

Method and Property Introduced:

- **`pfcParametricAttachment.AttachedGeometry`**
- **`pfcParametricAttachment.Create()`**

The property `pfcParametricAttachment.AttachedGeometry` returns the `pfcSelection` object representing the item to which the detail attachment is attached. This includes the drawing view in which the attachment is made.

The method `pfcParametricAttachment.Create()` creates a data object, used for attaching a detail item to an item. The input parameter *inAttachedGeometry* specifies the selection representing the item to attach to. This should include the drawing model view. The attachment will occur at the selected parameters

Offset Attachment

The `ATTACH_OFFSET` detail attachment type is represented by the `classpfcOffsetAttachment`. It is a child of the `pfcAttachment` class.

Methods and Properties Introduced:

- **`pfcOffsetAttachment.AttachedGeometry`**
- **`pfcOffsetAttachment.AttachmentPoint`**

- **pfcOffsetAttachment.Create()**

The property `pfcOffsetAttachment.AttachedGeometry` returns the `pfcSelection` object representing the item to which the detail attachment is attached. This includes the drawing view where the attachment is made, if the offset reference is in a model.

The property `pfcOffsetAttachment.AttachmentPoint` returns the attachment point. This location is in screen coordinates for drawing items, symbol instances and surface finishes on flat-to-screen annotation planes, and in model coordinates for symbols and surface finishes on 3D model annotation planes. The distance from the attachment point to the location of the item to which the detail attachment is attached is saved as the offset distance.

The method `pfcOffsetAttachment.Create()` creates a data object, used for attaching a detail item, offset to a reference point or axis. The input arguments are:

- *inAttachedGeometry*— Specifies the selection representing the item to attach to. This should include the drawing model view. The attachment will occur at the selected parameters
- *inAttachmentPoint*— Specifies the attachment point, in screen coordinates.

Unsupported Attachment

The `ATTACH_TYPE_UNSUPPORTED` detail attachment type is represented by the class `pfcUnsupportedAttachment`. It is a child of the `pfcAttachment` class.

Property Introduced:

- **pfcUnsupportedAttachment.AttachmentPoint**

The property `pfcUnsupportedAttachment.AttachmentPoint` returns the attachment point. This location is in screen coordinates for drawing items, symbol instances and surface finishes on flat-to-screen annotation planes, and in model coordinates for symbols and surface finishes on 3D model annotation planes.

Solid

Solid

Most of the objects and methods in `Creo.js` are used with solid models such as parts and assemblies. Because solid objects inherit from the interface `pfcModel`, you can use any of the `pfcModel` methods on any `pfcSolid`, `pfcPart`, or `pfcAssembly` object.

Getting a Solid Object

Methods and Properties Introduced:

- **pfcBaseSession.CreatePart()**
- **pfcBaseSession.CreateAssembly()**
- **pfcComponentPath.Root**
- **pfcComponentPath.Leaf**
- **pfcMFG.GetSolid()**

The methods `pfcBaseSession.CreatePart()` and `pfcBaseSession.CreateAssembly()` create new solid models with the names you specify.

The properties `pfcComponentPath.Root` and `pfcComponentPath.Leaf` specify the solid objects that make up the component path of an assembly component model. You can get a component path object from any component that has been interactively selected.

The method `pfcMFG.GetSolid()` retrieves the storage solid in which the features of the manufacturing model are placed. To create a UDF group in the manufacturing model, call the method `pfcSolid.CreateUDFGroup()` on the storage solid.

Solid Information

Properties Introduced:

- **pfcSolid.RelativeAccuracy**
- **pfcSolid.AbsoluteAccuracy**

You can set the relative and absolute accuracy of any solid model using these methods. Relative accuracy is relative to the size of the solid. For example, a relative accuracy of 0.01 specifies that the solid must be accurate to within 1/100 of its size. Absolute accuracy is measured in absolute units such as inches, centimeters and so on..



Note

For a change in accuracy to take effect, you must regenerate the model.

Solid Operations

Methods and Properties Introduced:

- **pfcSolid.Regenerate()**

-
- **pfcRegenInstructions.Create()**
 - **pfcRegenInstructions.AllowFixUI**
 - **pfcRegenInstructions.ForceRegen**
 - **pfcRegenInstructions.FromFeat**
 - **pfcRegenInstructions.RefreshModelTree**
 - **pfcRegenInstructions.ResumeExcludedComponents**
 - **pfcRegenInstructions.UpdateAssemblyOnly**
 - **pfcRegenInstructions.UpdateInstances**
 - **pfcRegenInstructions.ResolveModeRegen**
 - **pfcSolid.GeomOutline**
 - **pfcSolid.EvalOutline()**
 - **pfcSolid.IsSkeleton**
 - **pfcSolid.ListGroups()**
 - **pfcSolid.GetSurfaceSolidBody()**
 - **pfcSolid.GetEdgeSolidBody()**

The method `pfcSolid.Regenerate()` causes the solid model to regenerate according to the instructions provided in the form of the `pfcRegenInstructions` object. Passing a null value for the instructions argument causes an automatic regeneration.

In No-Resolve mode, if a model and feature regeneration fails, failed features and children of failed features are created and regeneration of other features continues. However, Creo.JS does not support regeneration in this mode. The method `pfcSolid.Regenerate()` throws an exception `pfcXToolkitBadContext`, if Creo Parametric is running in the No-Resolve mode. To continue with this in the Resolve mode, set the configuration option `regen_failure_handling` to `resolve_mode` in the Creo Parametric session.

 Note

Setting the configuration `regen_failure_handling` ensures the old behavior as long as you do not retrieve the models saved under the No-Resolve mode. For consistency, use Resolve mode for your entire Creo Parametric session.

The `pfcRegenInstructions` object contains the following input parameters:

-
- *AllowFixUI*—Determines whether or not to activate the **Fix Model** user interface, if there is an error.
Use the property `pfcRegenInstructions.AllowFixUI` to modify this parameter.
 - *ForceRegen*—Creo Parametric
Use the property `pfcRegenInstructions.ForceRegen` to modify this parameter.
 - *FromFeat*—Not currently used. This parameter is reserved for future use.
Use the property `pfcRegenInstructions.FromFeat` to modify this parameter.
 - *RefreshModelTree*—Creo Parametric **Model Tree**
Use the property `pfcRegenInstructions.RefreshModelTree` to modify this parameter.
 - *ResumeExcludedComponents*—Creo Parametric
Use the property `pfcRegenInstructions.ResumeExcludedComponents` to modify this parameter.
 - *UpdateAssemblyOnly*—Updates the placements of an assembly and all its sub-assemblies, and regenerates the assembly features and intersected parts. If the affected assembly is retrieved as a simplified representation, then the locations of the components are updated. If this attribute is false, the component locations are not updated, even if the simplified representation is retrieved. By default, it is false.
Use the property `pfcRegenInstructions.UpdateAssemblyOnly` to modify this parameter.
 - *UpdateInstances*—Updates the instances of the solid model in memory. This may slow down the regeneration process. By default, this attribute is false.
Use the property `pfcRegenInstructions.UpdateInstances` to modify this parameter.
ResolveModeRegen—Allows regeneration of a solid in resolve mode. The regeneration behavior is controlled by temporarily overriding the default settings.
Use the property `pfcRegenInstructions.ResolveModeRegen` to modify this parameter. By default, it is false. If you want to set your Creo Parametric application in resolve mode, you need to set the value of this parameter to `true`.

 Note

However, resolve mode will be deprecated in a future release of Creo Parametric. Hence, it is recommended to run the application without calling the property `pfcRegenInstructions.ResolveModeRegen`.

The property `pfcSolid.GeomOutline` returns the three-dimensional bounding box for the specified solid.

 Note

Do not use `pfcSolid.GeomOutline` to calculate the outline of a solid as the dimensions of the boundary box could be slightly bigger than the outline dimensions of the geometry. Use `pfcSolid.EvalOutline()` to compute an accurate outline of a solid.

The method `pfcSolid.EvalOutline()` also returns a three-dimensional bounding box, but you can specify the coordinate system used to compute the extents of the solid object.

The property `pfcSolid.IsSkeleton` determines whether the part model is a skeleton or a concept model. It returns a true value if the model is a skeleton, else it returns a false.

The method `pfcSolid.ListGroups()` returns the list of groups (including UDFs) in the solid.

The method `pfcSolid.GetSurfaceSolidBody()` returns the body to which the surface belongs.

The method `pfcSolid.GetEdgeSolidBody()` returns the body to which the edge belongs.

Solid Units

Each model has a basic system of units to ensure all material properties of that model are consistently measured and defined. All models are defined on the basis of the system of units. A part can have only one system of unit.

The following types of quantities govern the definition of units of measurement:

- **Basic Quantities**—The basic units and dimensions of the system of units. For example, consider the Centimeter Gram Second (CGS) system of unit. The basic quantities for this system of units are:
 - Length—cm
 - Mass—g
 - Force—dyne
 - Time—sec
 - Temperature—K
- **Derived Quantities**—The derived units are those that are derived from the basic quantities. For example, consider the Centimeter Gram Second (CGS) system of unit. The derived quantities for this system of unit are as follows:
 - Area—cm²
 - Volume—cm³
 - Velocity—cm/sec

In Creo.JS, individual units in the model are represented by the interface `pfcUnits.Unit`.

Types of Systems of Unit

The types of systems of units follow:

- **Predefined system of units**—This system of unit is provided by default.
- **Custom-defined system of units**—This system of unit is defined by the user only if the model does not contain standard metric or nonmetric units, or if the material file contains units that cannot be derived from the predefined system of units or both.

In Creo Parametric, the system of units are categorized as follows:

- **Mass Length Time (MLT)**—The following systems of units belong to this category:
 - CGS—Centimeter Gram Second
 - MKS—Meter Kilogram Second
 - mmKS—millimeter Kilogram Second
- **Force Length Time (FLT)**—The following systems of units belong to this category:
 - **Creo Parametric Default**—Inch lbm Second. This is the default system followed by Creo Parametric.

- FPS—Foot Pound Second
- IPS—Inch Pound Second
- mmNS—Millimeter Newton Second

In Creo.JS, the system of units followed by the model is represented by the interface `pfcUnits.UnitSystem`.

Accessing Individual Units

Methods and Properties Introduced:

- **`pfcSolid.ListUnits()`**
- **`pfcSolid.GetUnit()`**
- **`pfcUnit.Name`**
- **`pfcUnit.Expression`**
- **`pfcUnit.Type`**
- **`pfcUnit.IsStandard`**
- **`pfcUnit.ReferenceUnit`**
- **`pfcUnit.ConversionFactor`**
- **`pfcUnitConversionFactor.Offset`**
- **`pfcUnitConversionFactor.Scale`**

The method `pfcSolid.ListUnits()` returns the list of units available to the specified model.

The method `pfcSolid.GetUnit()` retrieves the unit, based on its name or expression for the specified model in the form of the `pfcUnit` object.

The property `pfcUnit.Name` returns the name of the unit.

The property `pfcUnit.Expression` returns a user-friendly unit description in the form of the name, for example, `ksi` for ordinary units and the expression, for example, `N/m^3` for system-generated units.

The property `pfcUnit.Type` returns the type of quantity represented by the unit in terms of the `pfcUnitType` object. The types of units are as follows:

- `UNIT_LENGTH`—Length measurement units.
- `UNIT_MASS`—Mass measurement units.
- `UNIT_FORCE`—Force measurement units.
- `UNIT_TIME`—Time measurement units.
- `UNIT_TEMPERATURE`—Temperature measurement units.
- `UNIT_ANGLE`—Angle measurement units.

The property `pfcUnit.IsStandard` identifies whether the unit is system-defined if the value of the property is set to true, or user-defined if the value is set to false.

The property `pfcUnit.ReferenceUnit` returns a reference unit (one of the available system units) in terms of the `pfcUnit` object.

The property `pfcUnit.ConversionFactor` identifies the relation of the unit to its reference unit in terms of the `pfcUnitConversionFactor` object. The unit conversion factors are as follows:

- **Offset**—Offset value applied to the values in the reference unit.
- **Scale**—Scale applied to the values in the reference unit to get the value in the actual unit.

Example - Consider the formula to convert temperature from Centigrade to Fahrenheit

$$F = a + (C * b)$$

where

F is the temperature in Fahrenheit

C is the temperature in Centigrade

a = 32 (constant signifying the offset value)

b = 9/5 (ratio signifying the scale of the unit)

Note

Creo Parametric scales the length dimensions of the model using the factors listed above. If the scale is modified, the model is regenerated. When you scale the model, the model units are not changed. Imported geometry cannot be scaled.

Use the properties `pfcUnitConversionFactor.Offset` and `pfcUnitConversionFactor.Scale` to retrieve the unit conversion factors listed above.

Modifying Individual Units

Methods Introduced:

- **`pfcUnit.Modify()`**
- **`pfcUnit.Delete()`**

The method `pfcUnit.Modify()` modifies the definition of a unit by applying a new conversion factor specified by the `pfcUnitConversionFactor` object and a reference unit.

The method `pfcUnit.Delete()` deletes the unit.



Note

You can delete only custom units and not standard units.

Creating a New Unit

Methods Introduced:

- **pfcSolid.CreateCustomUnit()**
- **pfcUnitConversionFactor.Create()**

The method `pfcSolid.CreateCustomUnit()` creates a custom unit based on the specified name, the conversion factor given by the `pfcUnitConversionFactor` object, and a reference unit.

The method `pfcUnitConversionFactor.Create()` creates the `pfcUnitConversionFactor` object containing the unit conversion factors.

Accessing Systems of Units

Methods and Properties Introduced:

- **pfcSolid.ListUnitSystems()**
- **pfcSolid.GetPrincipalUnits()**
- **pfcUnitSystem.GetUnit()**
- **pfcUnitSystem.Name**
- **pfcUnitSystem.Type**
- **pfcUnitSystem.IsStandard**

The method `pfcSolid.ListUnitSystems()` returns the list of unit systems available to the specified model.

The method `pfcSolid.GetPrincipalUnits()` returns the system of units assigned to the specified model in the form of the `pfcUnitSystem` object.

The method `pfcUnitSystem.GetUnit()` retrieves the unit of a particular type used by the unit system.

The property `pfcUnitSystem.Name` returns the name of the unit system.

The property `pfcUnitSystem.Type` returns the type of the unit system in the form of the `pfcUnitSystemType` object. The types of unit systems are as follows:

- `UNIT_SYSTEM_MASS_LENGTH_TIME`—Mass Length Time (MLT) unit system.

-
- `UNIT_SYSTEM_FORCE_LENGTH_TIME`—Force Length Time (FLT) unit system.

For more information on these unit systems listed above, refer to the section [Types of Systems of Unit on page 121](#).

The property `pfcUnitSystem.IsStandard` identifies whether the unit system is system-defined if the value of the property is set to true, or user-defined if the value is set to false).

Modifying Systems of Units

Method Introduced:

- **`pfcUnitSystem.Delete()`**

The method `pfcUnitSystem.Delete()` deletes a custom-defined system of units.



Note

You can delete only a custom-defined system of units and not a standard system of units.

Creating a New System of Units

Method Introduced:

- **`pfcSolid.CreateUnitSystem()`**

The method `pfcSolid.CreateUnitSystem()` creates a new system of units in the model based on the specified name, the type of unit system given by the `pfcUnitSystemType` object, and the types of units specified by the `pfcUnit` sequence to use for each of the base measurement types such as length, force or mass, and temperature.

Conversion to a New System of Units

Methods and Properties Introduced:

- **`pfcSolid.SetPrincipalUnits()`**
- **`pfcUnitConversionOptions.Create()`**
- **`pfcUnitConversionOptions.DimensionOption`**
- **`pfcUnitConversionOptions.IgnoreParamUnits`**

The method `pfcSolid.SetPrincipalUnits()` changes the principal system of units assigned to the solid model based on the unit conversion options specified by the `pfcUnitConversionOptions` object. The method `pfcUnitConversionOptions.Create()` creates the `pfcUnitConversionOptions` object containing the unit conversion options listed below.

The types of unit conversion options are as follows:

- **DimensionOption**—Use the option while converting the dimensions of the model.

Use the property `pfcUnitConversionOptions.DimensionOption` to modify this option.

This option can be of the following types:

- **UNITCONVERT_SAME_DIMS**—Unit conversion occurs by interpreting the unit value in the new unit system. For example, 1 inch equals to 1 millimeter.
- **UNITCONVERT_SAME_SIZE**—Unit conversion occurs by converting the unit value in the new unit system. For example, 1 inch equals to 25.4 millimeters.
- **IgnoreParamUnits**—This Boolean attribute determines whether or not ignore the parameter units. If it is `null` or `true`, parameter values and units do not change when the unit system is changed. If it is `false`, parameter units are converted according to the rule.

Use the property `pfcUnitConversionOptions.IgnoreParamUnits` to modify this attribute.

Mass Properties

Method Introduced:

- **`pfcSolid.GetMassProperty()`**
- **`pfcSolid.GetMassPropertyWithDensity()`**
- **`pfcAssembly.GetMassPropertyByCompPath()`**

The method `pfcSolid.GetMassProperty()` provides information about the distribution of mass in the part or assembly. It can provide the information relative to a coordinate system datum, which you name, or the default one if you provide `null` as the name. It returns an object containing the following fields:

- Volume
- Surface area
- Density. The density value is 1.0, unless a material has been assigned.

- Mass
- Center of gravity (COG)
- Inertia matrix
- Inertia tensor
- Inertia about the COG
- Principal moments of inertia (the eigenvalues of the COG inertia)
- Principal axes (the eigenvectors of the COG inertia)

The method `pfcSolid.GetMassPropertyWithDensity()` calculates the mass properties of a part or an assembly in the specified coordinate system. The input arguments follow:

- *CoordSysName*—Name of the coordinate system. If this is `Null`, the method uses the default coordinate system.
- *DensityOpt*—Value of the density flag specified using the enumerated data type `pfcMPDensityUse` and the valid values are as follows:
 - `MP_DENSITY_DEFAULT`—Calculate the mass properties using the material density.
 - `MP_DENSITY_USE_ALWAYS`—Calculate the mass properties using the specified density, even if material has a defined density.
 - `MP_DENSITY_USE_IF_MISSING`—Calculate mass properties using specified density, even if material does not have a defined density.
- *density*—Density used while calculating mass properties depending on the value specified for the input argument *DensityOpt*.

The method `pfcAssembly.GetMassPropertyByCompPath()` calculates the mass properties of a solid that is referenced by the specified coordinate system selection, using the respective component paths. The input arguments follow:

- *CompPath*—Component path of the solid. If this is null, the top assembly is referred.
- *CsysItem*—Coordinate system model item. If this is null, default coordinate system is referred.
- *CsysPath*—Component path of the coordinate system. If this is null, default coordinate system is referred.

Annotations

Methods and Properties Introduced:

- **pfcNote.Lines**
- **pfcNote.GetText()**
- **pfcNote.URL**

-
- **pfcNote.Display()**
 - **pfcNote.Delete()**
 - **pfcNote.GetOwner()**

3D model notes are instances of `pfcModelItem` objects. They can be located and accessed using methods that locate model items in solid models, and downcast to the `Note` interface to use the methods in this section.

The property `pfcNote.Lines` returns the text contained in the 3D model note.

The method `pfcNote.GetText()` returns the text of the solid model note. If you set the parameter *GiveParametersAsNames* to `TRUE`, then the text displays the parameter callouts with ampersands (&). If you set the parameter to `FALSE`, then the text displays the parameter values with no callout information.

The property `pfcNote.URL` returns the URL stored in the 3D model note.

The method `pfcNote.Display()` forces the display of the model note.

The method `pfcNote.Delete()` deletes a model note.

The method `pfcNote.GetOwner()` returns the solid model owner of the note.

Cross Sections

Methods Introduced:

- **pfcSolid.ListCrossSections()**
- **pfcSolid.GetCrossSection()**
- **pfcXSection.GetName()**
- **pfcXSection.SetName()**
- **pfcXSection.GetXSecType()**
- **pfcXSection.Delete()**
- **pfcXSection.Display()**
- **pfcXSection.Regenerate()**

The method `pfcSolid.ListCrossSections()` returns an array of cross section objects represented by the `Xsection` interface. The method `pfcSolid.GetCrossSection()` searches for a cross section given its name.

The method `pfcXSection.GetName()` returns the name of the cross section in Creo Parametric. The method `pfcXSection.SetName()` modifies the cross section name.

The method `pfcXSection.GetXSecType()` returns the type of cross section, that is planar or offset, and the type of item intersected by the cross section.

The method `pfcXSection.Delete()` deletes a cross section.

The method `pfcXSection.Display()` forces a display of the cross section in the window.

The method `pfcXSection.Regenerate()` regenerates a cross section.

Materials

Creo.JS enables you to programmatically access the material types and properties of parts. Using the methods and properties described in the following sections, you can perform the following actions:

- Create or delete materials
- Set the current material
- Access and modify the material types and properties

Methods and Properties Introduced:

- **`pfcMaterial.Save()`**
- **`pfcMaterial.Delete()`**
- **`pfcPart.GetMaterial`**
- **`pfcPart.CurrentMaterial`**
- **`pfcPart.ListMaterials()`**
- **`pfcPart.CreateMaterial()`**
- **`pfcPart.RetrieveMaterial()`**

The method `pfcMaterial.Save()` writes to a material file that can be imported into any Creo Parametric part.

The method `pfcMaterial.Delete()` removes material from the part.

The property `pfcPart.GetMaterial` locates a material by name. The material must be stored in the part and not stored in an external file.

Note

By default, while assigning a material to a sheetmetal part, the property `pfcPart.CurrentMaterial` modifies the values of the sheetmetal properties such as Y factor and bend table according to the material file definition. This modification triggers a regeneration and a modification of the developed length calculations of the sheetmetal part. However, you can avoid this by setting the value of the configuration option `material_update_smt_bend_table` to `never_replace`.

The property `pfcPart.CurrentMaterial` may change the model display, if the new material has a default appearance assigned to it.

The property may also change the family table, if the parameter `PTC_MATERIAL_NAME` is a part of the family table.

The method `pfcPart.ListMaterials()` returns a list of the materials available in the part.

The method `pfcPart.CreateMaterial()` creates a new empty material in the specified part.

The method `pfcPart.RetrieveMaterial()` imports a material file into the part. The name of the file read can be as either of the following:

- `<name>.mtl`—New material file format.
- `<name>.mat`—Material file format.

If the material is not already in the part database, `pfcPart.RetrieveMaterial()` adds the material to the database after reading the material file. If the material is already in the database, the function replaces the material properties in the database with those contained in the material file.

Accessing Material Types

Properties Introduced:

- **`pfcMaterial.StructuralMaterialType`**
- **`pfcMaterial.ThermalMaterialType`**
- **`pfcMaterial.SubType`**
- **`pfcMaterial.PermittedSubTypes`**
- **`pfcMaterial.FluidMaterialType`**

The property `pfcMaterial.StructuralMaterialType` sets the material type for the structural properties of the material. The material types are as follows:

- `MTL_ISOTROPIC`—Material with an infinite number of planes of material symmetry, making the properties equal in all directions.
- `MTL_ORTHOTROPIC`—Material with symmetry relative to three mutually perpendicular planes.
- `MTL_TRANSVERSELY_ISOTROPIC`—Material with rotational symmetry about an axis. The properties are equal for all directions in the plane of isotropy.
- `MTL_FLUID`—Specifies a material with fluid properties.

The property `pfcMaterial.ThermalMaterialType` sets the material type for the thermal properties of the material. The material types are as follows:

- `MTL_ISOTROPIC`—Material with an infinite number of planes of material symmetry, making the properties equal in all directions.
- `MTL_ORTHOTROPIC`—Material with symmetry relative to three mutually perpendicular planes.
- `MTL_TRANSVERSELY_ISOTROPIC`—Specifies a material with rotational symmetry about an axis. The properties are equal for all directions in the plane of isotropy.
- `MTL_FLUID`—Specifies a material with fluid properties.

The property `pfcMaterial.SubType` sets the subtype for the `MTL_ISOTROPIC` material type.

Use the property `pfcMaterial.PermittedSubTypes` to retrieve a list of the permitted string values for the material subtype.

The property `pfcMaterial.FluidMaterialType` sets the material type for the fluid properties of the material. The material types are as follows:

- `MTL_ISOTROPIC`—Specifies a material with an infinite number of planes of material symmetry, making the properties equal in all directions.
- `MTL_ORTHOTROPIC`—Specifies a material with symmetry relative to three mutually perpendicular planes.
- `MTL_TRANSVERSELY_ISOTROPIC`—Specifies a material with rotational symmetry about an axis. The properties are equal for all directions in the plane of isotropy.
- `MTL_FLUID`—Specifies a material with fluid properties.

Accessing Material Properties

The methods and properties listed in this section enable you to access material properties.

Methods and Properties Introduced:

-
- **pfcMaterialProperty.Create()**
 - **pfcMaterial.GetPropertyValue()**
 - **pfcMaterial.SetPropertyValue()**
 - **pfcMaterial.SetPropertyUnits()**
 - **pfcMaterial.RemoveProperty()**
 - **pfcMaterial.Description**
 - **pfcMaterial.FatigueType**
 - **pfcMaterial.PermittedFatigueTypes**
 - **pfcMaterial.FatigueMaterialType**
 - **pfcMaterial.PermittedFatigueMaterialTypes**
 - **pfcMaterial.FatigueMaterialFinish**
 - **pfcMaterial.PermittedFatigueMaterialFinishes**
 - **pfcMaterial.FailureCriterion**
 - **pfcMaterial.PermittedFailureCriteria**
 - **pfcMaterial.Hardness**
 - **pfcMaterial.HardnessType**
 - **pfcMaterial.Condition**
 - **pfcMaterial.BendTable**
 - **pfcMaterial.CrossHatchFile**
 - **pfcMaterial.MaterialModel**
 - **pfcMaterial.PermittedMaterialModels**
 - **pfcMaterial.ModelDefByTests**

The method `pfcMaterialProperty.Create()` creates a new instance of a material property object.

All numerical material properties are accessed using the same set of APIs. You must provide a property type to indicate the property you want to read or modify.

The method `pfcMaterial.GetPropertyValue()` returns the value and the units of the material property.

Use the method `pfcMaterial.SetPropertyValue()` to set the value and units of the material property. If the property type does not exist for the material, then this method creates it.

Use the method `pfcMaterial.SetPropertyUnits()` to set the units of the material property.

Use the method `pfcMaterial.RemoveProperty()` to remove the material property.

Material properties that are non-numeric can be accessed using the following properties.

The property `pfcMaterial.Description` sets the description string for the material.

The property `pfcMaterial.FatigueType` sets the valid fatigue type for the material.

Use the property `pfcMaterial.PermittedFatigueTypes` to get a list of the permitted string values for the fatigue type.

The property `pfcMaterial.FatigueMaterialType` sets the class of material when determining the effect of the fatigue.

Use the property `pfcMaterial.PermittedFatigueMaterialTypes` to retrieve a list of the permitted string values for the fatigue material type.

The property `pfcMaterial.FatigueMaterialFinish` sets the type of surface finish for the fatigue material.

Use the property `pfcMaterial.PermittedFatigueMaterialFinishes` to retrieve a list of permitted string values for the fatigue material finish.

The property `pfcMaterial.FailureCriterion` sets the reduction factor for the failure strength of the material. This factor is used to reduce the endurance limit of the material to account for unmodeled stress concentrations, such as those found in welds.

Use the property `pfcMaterial.PermittedFailureCriteria` to retrieve a list of permitted string values for the material failure criterion.

The property `pfcMaterial.Hardness` sets the hardness for the specified material.

The property `pfcMaterial.HardnessType` sets the hardness type for the specified material.

The property `pfcMaterial.Condition` sets the condition for the specified material.

The property `pfcMaterial.BendTable` sets the bend table for the specified material.

The property `pfcMaterial.CrossHatchFile` sets the file containing the crosshatch pattern for the specified material.

The property `pfcMaterial.MaterialModel` sets the type of hyperelastic isotropic material model.

Use the property `pfcMaterial.PermittedMaterialModels` to retrieve a list of the permitted string values for the material model.

The property `pfcMaterial.ModelDefByTests` determines whether the hyperelastic isotropic material model has been defined using experimental data for stress and strain.

Accessing User-Defined Material Properties

Materials permit assignment of user-defined parameters so that you can place non-standard properties on a given material. Therefore `pfcMaterial` is a child of `pfcParameterOwner`, which provides access to user-defined parameters and properties of materials through the methods in that interface.

Solid Bodies

Solid Bodies

This section describes the Creo.JS functions that access the functions of a Creo Parametric part with multiple solid bodies.

Solid Body Information

The state of the body is derived from the features and geometry in which the body is created.

Methods introduced:

- `pfcSolidBody.GetBodyState()`
- `pfcSolidBody.IsConstruction()`
- `pfcSolidBody.ListSurfaces()`
- `pfcSolidBody.GetDefaultBody()`
- `pfcSolidBody.GetFeatures()`
- `pfcSolidBody.GetOutline()`
- `pfcSolidBody.IsSheetmetal()`
- `pfcSolidBody.GetDensity()`
- `pfcSolidBody.GetMassProperty()`

The method `pfcSolidBody.GetBodyState()` specifies the state of the body using the enumerated type `pfcSolidBodyState`. The valid values are:

- `BODY_STATE_MISSING`
- `BODY_STATE_CONSUMED`
- `BODY_STATE_NO_CONTR_FEAT`
- `BODY_STATE_NO_GEOMETRY`
- `BODY_STATE_ACTIVE`

The method `pfcSolidBody.IsConstruction()` checks if the body is a construction body. The method returns a `True` if solid body is a construction body.

The method `pfcSolidBody.ListSurfaces()` lists all the surfaces in the specified body

The method `pfcSolid.GetDefaultBody()` returns the default body in the specified solid.

The method `pfcSolidBody.GetFeatures()` lists the features that are associated with the specified body.

Use the method `pfcSolidBody.GetOutline()` to retrieve the regeneration outline of a solid body, with respect to the base coordinate system orientation. This outline defines the boundary box of the body.

The method `pfcSolidBody.IsSheetmetal()` checks if the specified body is an active sheetmetal body.

The method `pfcSolidBody.GetDensity()` determines the density of the body.

Use the method `pfcSolidBody.GetMassProperty` to retrieve the mass properties of a body in the specified coordinate system.

Windows and Views

Windows and Views

Creo.JS provides access to Creo Parametric windows and saved views. This section describes the methods that provide this access.

Windows

This section describes the Creo.JS methods that access `pfcWindow` objects. The topics are as follows:

- [Getting a Window Object on page 135](#)
- [Window Operations on page 257](#)

Getting a Window Object

Methods and Property Introduced:

- **`pfcBaseSession.CurrentWindow`**
- **`pfcBaseSession.CreateModelWindow()`**
- **`pfcModel.Display()`**

-
- **pfcBaseSession.ListWindows()**
 - **pfcBaseSession.GetWindow()**
 - **pfcBaseSession.OpenFile()**
 - **pfcBaseSession.GetModelWindow()**

The property `pfcBaseSession.CurrentWindow` provides access to the current active window in Creo Parametric.

The method `pfcBaseSession.CurrentWindow` creates a new window that contains the model that was passed as an argument.

 Note

You must call the method `pfcModel.Display()` for the model geometry to appear in the window.

Use the method `pfcBaseSession.ListWindows()` to get a list of all the current windows in session.

The method `pfcBaseSession.GetWindow()` gets the handle to a window given its integer identifier.

The method `pfcBaseSession.OpenFile()` returns the handle to a newly created window that contains the opened model.

 Note

If a model is already open in a window the method returns a handle to the window.

The method `pfcBaseSession.GetModelWindow()` returns the handle to the window that contains the opened model, if it is displayed.

Window Operations

Methods and Properties Introduced:

- **pfcWindow.Height**
- **pfcWindow.Width**
- **pfcWindow.XPos**
- **pfcWindow.YPos**
- **pfcWindow.GraphicsAreaHeight**
- **pfcWindow.GraphicsAreaWidth**

-
- **pfcWindow.Clear()**
 - **pfcWindow.Repaint()**
 - **pfcWindow.Refresh()**
 - **pfcWindow.Close()**
 - **pfcWindow.Activate()**
 - **pfcWindow..GetId()**
 - **pfcBaseSession.FlushCurrentWindow()**

The properties `pfcWindow.Height`, `pfcWindow.Width`, `pfcWindow.XPos`, and `pfcWindow.YPos` retrieve the height, width, x-position, and y-position of the window respectively. The values of these parameters are normalized from 0 to 1.

The properties `pfcWindow.GraphicsAreaHeight` and `pfcWindow.GraphicsAreaWidth` retrieve the height and width of the Creo Parametric graphics area window without the border respectively. The values of these parameters are normalized from 0 to 1. For both the window and graphics area sizes, if the object occupies the whole screen, the window size returned is 1. For example, if the screen is 1024 pixels wide and the graphics area is 512 pixels, then the width of the graphics area window is returned as 0.5.

The method `pfcWindow.Clear()` removes geometry from the window.

Both `pfcWindow.Repaint()` and `pfcWindow.Refresh()` repaint solid geometry. However, the `pfcWindow.Refresh()` method does not remove highlights from the screen and is used primarily to remove temporary geometry entities from the screen.

Use the method `pfcWindow.Close()` to close the window. If the current window is the original window created when Creo Parametric started, this method clears the window. Otherwise, it removes the window from the screen.

The method `pfcWindow.Activate()` activates a window. This function is available only in the asynchronous mode.

The method `pfcWindow.GetId()` retrieves the ID of the Creo Parametric window.

The method `pfcBaseSession.FlushCurrentWindow()` flushes the pending display commands on the current window.

Note

It is recommended to call this method only after completing all the display operations. Excessive use of this method results in a major slow down of systems running on Windows Vista and Windows 7.

Embedded Browser

Methods Introduced:

- **pfcWindow.GetURL()**
- **pfcWindow.SetURL()**
- **pfcWindow.GetBrowserSize()**
- **pfcWindow.SetBrowserSize()**

Use the methods `pfcWindow.GetURL()` and `pfcWindow.SetURL()` to find and change the URL displayed in the embedded browser in the Creo Parametric window.

Use the methods `pfcWindow.GetBrowserSize()` and `pfcWindow.SetBrowserSize()` to find and change the size of the embedded browser in the Creo Parametric window.



Note

The methods `pfcWindow.GetBrowserSize()` and `pfcWindow.SetBrowserSize()` are not supported if the browser is open in a separate window.

Views

This section describes the Creo.JS methods that access `pfcView` objects. The topics are as follows:

- [Getting a View Object on page 138](#)
- [View Operations on page 139](#)

Getting a View Object

Methods Introduced:

- **pfcViewOwner.RetrieveView()**
- **pfcViewOwner.GetView()**
- **pfcViewOwner.ListViews()**

Any solid model inherits from the interface `pfcViewOwner`. You can use these methods on any solid object.

The method `pfcViewOwner.RetrieveView()` sets the current view to the orientation previously saved with a specified name.

Use the method `pfcViewOwner.GetView()` to get a handle to a named view without making any modifications.

The method `pfcViewOwner.ListViews()` returns a list of all the views previously saved in the model.

View Operations

Methods and Properties Introduced:

- **`pfcView.Name`**
- **`pfcView.IsCurrent`**
- **`pfcView.Reset()`**
- **`pfcViewOwner.SaveView()`**

To get the name of a view given its identifier, use the property `pfcView.Name`.

The property `pfcView.IsCurrent` determines if the View object represents the current view.

The `pfcView.Reset()` method restores the current view to the default view.

To store the current view under the specified name, call the method `pfcViewOwner.SaveView()`.

Coordinate Systems and Transformations

This section describes the various coordinate systems used by Creo Parametric and accessible from Creo.JS and how to transform from one coordinate system to another.

Coordinate Systems

Creo Parametric and Creo.JS use the following coordinate systems:

- [Solid Coordinate System on page 140](#)
- [Screen Coordinate System on page 140](#)
- [Window Coordinate System on page 140](#)
- [Drawing Coordinate System on page 141](#)
- [Drawing View Coordinate System on page 141](#)
- [Assembly Coordinate System on page 141](#)
- [Datum Coordinate System on page 141](#)
- [Section Coordinate System on page 141](#)

The following sections describe each of these coordinate systems.

Solid Coordinate System

The solid coordinate system is the three-dimensional, Cartesian coordinate system used to describe the geometry of a Creo Parametric solid model. In a part, the solid coordinate system describes the geometry of the surfaces and edges. In an assembly, the solid coordinate system also describes the locations and orientations of the assembly members.

You can visualize the solid coordinate system in Creo Parametric by creating a coordinate system datum with the option **Default**. Distances measured in solid coordinates correspond to the values of dimensions as seen by the Creo Parametric user.

Solid coordinates are used by Creo.JS for all the methods that look at geometry and most of the methods that draw three-dimensional graphics.

Screen Coordinate System

The screen coordinate system is two-dimensional coordinate system that describes locations in a Creo Parametric window. This is an intermediate coordinate system after which the screen points are transformed to screen pixels. All the models are first mapped to the screen coordinate system. When the user zooms or pans the view, the screen coordinate system follows the display of the solid, so a particular point on the solid always maps to the same screen coordinate. The mapping changes only when the view orientation is changed.

Screen coordinates are used by some of the graphics methods, the mouse input methods, and all methods that draw graphics or manipulate items on a drawing.

Window Coordinate System

The window coordinate system is similar to the screen coordinate system. After mapping the models to the screen coordinate system, they are mapped to the window coordinate before being drawn to screen pixels based on screen resolution. When pan or zoom values are applied to the coordinates in the screen coordinate system, they result in window coordinates. When an object is first displayed in a window, or the option **View ► Refit** is used, the screen and window coordinates are the same.

Window coordinates are needed only if you need to take account of zoom and pan—for example, to find out whether a point on the solid is visible in the window, or to draw two-dimensional text in a particular window location, regardless of pan and zoom.

Drawing Coordinate System

The drawing coordinate system is a two-dimensional system that describes the location on a drawing relative to the bottom, left corner, and measured in drawing units. For example, on a U.S. letter-sized, landscape-format drawing sheet that uses inches, the top, right-corner is (11, 8.5) in drawing coordinates.

The Creo.JS methods and properties that manipulate drawings generally use screen coordinates.

Drawing View Coordinate System

The drawing view coordinate system is used to describe the locations of entities in a drawing view.

Assembly Coordinate System

An assembly has its own coordinate system that describes the positions and orientations of the member parts, subassemblies, and the geometry of datum features created in the assembly.

When an assembly is retrieved into memory each member is also loaded and continues to use its own solid coordinate system to describe its geometry.

This is important when you are analyzing the geometry of a subassembly and want to extract or display the results relative to the coordinate system of the parent assembly.

Datum Coordinate System

A coordinate system datum can be created anywhere in any part or assembly, and represents a user-defined coordinate system. It is often a requirement in a Creo.JS application to describe geometry relative to such a datum.

Section Coordinate System

Every sketch has a coordinate system used to locate entities in that sketch. Sketches used in features will use a coordinate system different from that of the solid model.

Transformations

Methods and Properties Introduced:

- **pfcTransform3D.Invert()**
- **pfcTransform3D.TransformPoint()**
- **pfcTransform3D.TransformVector()**

- **pfcTransform3D.Matrix**
- **pfcTransform3D.GetOrigin()**
- **pfcTransform3D.GetXAxis()**
- **pfcTransform3D.GetYAxis()**
- **pfcTransform3D.GetZAxis()**
- **pfcMakeMatrixOrthonormal()**

All coordinate systems are treated in Creo.JS as if they were three-dimensional. Therefore, a point in any of the coordinate systems is always represented by the `pfcPoint3D` class:

Vectors store the same data but are represented for clarity by the `pfcVector3D` class.

Screen coordinates contain a z-value whose positive direction is outwards from the screen. The value of z is not generally important when specifying a screen location as an input to a method, but it is useful in other situations. For example, if you select a datum plane, you can find the direction of the plane by calculating the normal to the plane, transforming to screen coordinates, then looking at the sign of the z-coordinate.

A transformation between two coordinate systems is represented by the `pfcTransform3D` class. This class contains a 4x4 matrix that combines the conventional 3x3 matrix that describes the relative orientation of the two systems, and the vector that describes the shift between them.

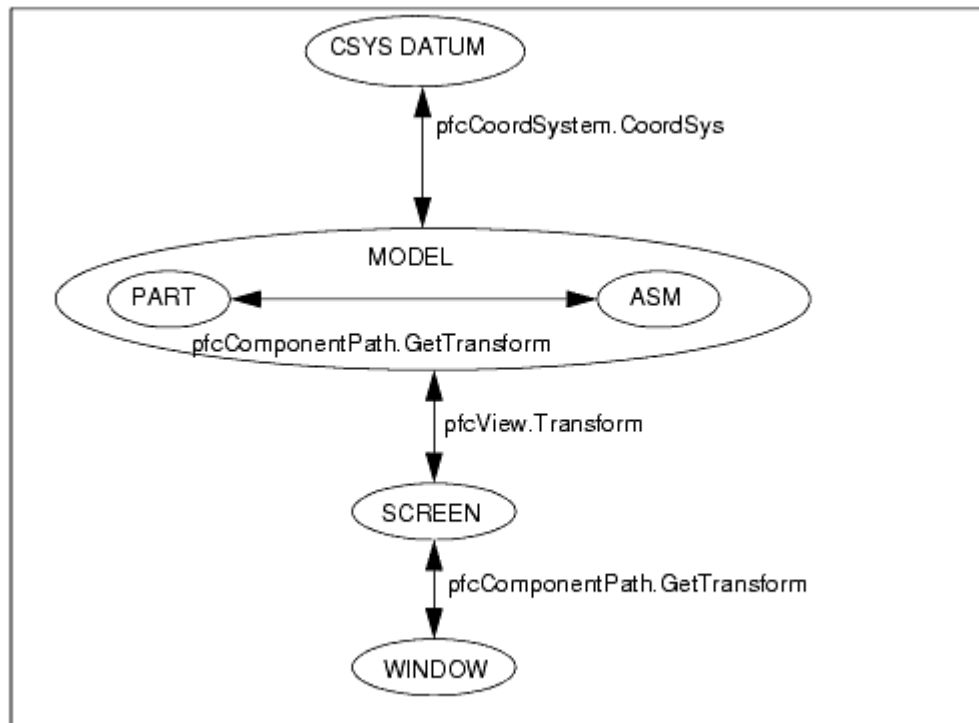
The 4x4 matrix used for transformations is as follows:

$$\begin{bmatrix} X' & Y' & Z' & 1 \end{bmatrix} = \begin{bmatrix} X & Y & Z & 1 \end{bmatrix} \begin{bmatrix} \dots & \dots & \dots & 0 \\ \dots & \dots & \dots & 0 \\ \dots & \dots & \dots & 0 \\ X_s & Y_s & Z_s & 1 \end{bmatrix}$$

The utility method `pfcTransform3D.Invert()` inverts a transformation matrix so that it can be used to transform points in the opposite direction.

Creo.JS provides two utilities for performing coordinate transformations. The method `pfcTransform3D.TransformPoint()` transforms a three-dimensional point and `pfcTransform3D.TransformVector()` transforms a three-dimensional vector.

The following diagram summarizes the coordinate transformations needed when using Creo.JS and specifies the Creo.JS methods that provide the transformation matrix.



The method `pfcMakeMatrixOrthonormal()` converts a non-orthonormal matrix to an orthonormal matrix with the specified scaling factor. The input arguments follow:

- *Matrix*—The matrix to be converted to orthonormal.
- *Scale*—Scale factor to be applied on the matrix.

Transforming to Screen Coordinates

Methods and Properties Introduced:

- **`pfcView.Transform`**
- **`pfcView.Rotate()`**
- **`pfcViewOwner.GetCurrentViewTransform()`**
- **`pfcViewOwner.SetCurrentViewTransform()`**
- **`pfcViewOwner.CurrentViewRotate()`**
- **`pfcTransform3D.Invert()`**

The view matrix describes the transformation from solid to screen coordinates. The property `pfcView.Transform` provides the view matrix for a model in the view.

To get and set the transformation matrix for a model in the current view, use the methods `pfcViewOwner.GetCurrentViewTransform()` and `pfcViewOwner.SetCurrentViewTransform()`.

The method `pfcView.Rotate()` rotates an object, relative to the X, Y, or Z axis for the specified rotation angle.

To rotate an object in the current view, use the method `pfcViewOwner.CurrentViewRotate()`.

To transform from screen to solid coordinates, invert the transformation matrix using the method `pfcTransform3D.Invert()`.

Transforming to Coordinate System Datum Coordinates

Property Introduced:

- **`pfcCoordSystem.CoordSys`**

The property `pfcCoordSystem.CoordSys` provides the location and orientation of the coordinate system datum in the coordinate system of the solid that contains it. The location is in terms of the directions of the three axes and the position of the origin.

Transforming Window Coordinates

Properties Introduced

- **`pfcWindow.ScreenTransform`**
- **`pfcScreenTransform.PanX`**
- **`pfcScreenTransform.PanY`**
- **`pfcScreenTransform.Zoom`**

You can alter the pan and zoom of a window by using a Screen Transform object. This object contains three attributes. `PanX` and `PanY` represent the horizontal and vertical movement. Every increment of 1.0 moves the view point one screen width or height. `Zoom` represents a scaling factor for the view. This number must be greater than zero.

Transforming Coordinates of an Assembly Member

Method Introduced:

- **`pfcComponentPath.GetTransform()`**

The method `pfcComponentPath.GetTransform()` provides the matrix for transforming from the solid coordinate system of the assembly member to the solid coordinates of the parent assembly, or the reverse.

ModelItem

ModelItem

This section describes the Creo.JS methods that enable you to access and manipulate `pfcModelItem`.

Solid Geometry Traversal

Solid models are made up of 11 distinct types of `pfcModelItem`, as follows:

- `pfcFeature`
- `pfcSurface`
- `pfcEdge`
- `pfcCurve` (datum curve)
- `pfcAxis` (datum axis)
- `pfcPoint` (datum point)
- `pfcQuilt` (datum quilt)
- `pfcLayer`
- `pfcNote`
- `pfcDimension`
- `pfcRefDimension`

Each model item is assigned a unique identification number that never changes. In addition, each model item can be assigned a string name. Layers, points, axes, dimensions, and reference dimensions are automatically assigned a name that can be changed.

Getting ModelItem Objects

Methods and Properties Introduced:

- **`pfcModelItemOwner.ListItems()`**
- **`pfcFeature.ListSubItems()`**
- **`pfcLayer.ListItems()`**
- **`pfcModelItemOwner.GetItemById()`**
- **`pfcModelItemOwner.GetItemByName()`**
- **`pfcFamColModelItem.RefItem`**
- **`pfcSelection.SellItem`**

All models inherit from the class `pfcModelItemOwner`. The method `pfcModelItemOwner.ListItems()` returns an array of `pfcModelItems` contained in the model. You can specify which type of `pfcModelItem` to collect by passing in one of the enumerated `pfcModelItemType` values, or you can collect all `pfcModelItems` by passing `null` as the model item type.

If the model has multiple bodies, the method `pfcModelItemOwner.ListItems()` returns the exception `pfcXToolkitMultibodyUnsupported`.

 Note

The part modeling features are excluded from the list of features returned by the method `pfcModelItemOwner.ListItems()`, if the model item type is specified as `ITEM_FEATURE`. For example Edit Round features and other Flexible Modeling features will be excluded from the list.

The methods `pfcFeature.ListSubItems()` and `pfcLayer.ListItems()` produce similar results for specific features and layers. These methods return a list of subitems in the feature or items in the layer.

To access specific model items, call the method `pfcModelItemOwner.GetItemById()`. Use this method to access the model item by identifier.

To access specific model items, call the method `pfcModelItemOwner.GetItemByName()`. Use this method to access the model item by name.

The property `pfcFamColModelItem.RefItem` returns the dimension or feature used as a header for a family table.

The property `pfcSelection.SelectedItem` returns the item selected interactively by the user.

ModelItem Information

Methods and Properties Introduced:

- **`pfcModelItem.GetName()`**
- **`pfcModelItem.SetName()`**
- **`pfcModelItem.Id`**
- **`pfcModelItem.Type`**

Certain `pfcModelItems` also have a string name that can be changed at any time. The methods `pfcModelItem.GetName()` and `pfcModelItem.SetName()` access this name.

The `pfcModelItem.Id` property returns the unique integer identifier for the `pfcModelItem`.

The `pfcModelItem.Type` property returns an enumeration object that indicates the model item type of the specified `pfcModelItem`. See the section [ModelItem on page 145](#) for the list of possible model item types.

Duplicating ModelItems

Method Introduced:

- **`pfcBaseSession.AllowDuplicateModelItems()`**

You can control the creation of `ModelItems` more than twice for the same Creo Parametric item. Use the method `pfcBaseSession.AllowDuplicateModelItems()` to turn ON or OFF the option to duplicate model items. By default, this option is OFF. To turn the option ON, set the Boolean value to `FALSE`.



Note

If this option is not handled correctly on the application side, it can cause memory corruption. Although you can turn ON and OFF this option as many times as you want, PTC recommends turning ON and OFF this option one time, immediately after opening the session.

Layer Objects

In Creo.JS, layers are instances of `pfcModelItem`. The following sections describe how to get layer objects and the operations you can perform on them.

Getting Layer Objects

Method Introduced:

- **`pfcModel.CreateLayer()`**

The method `pfcModel.CreateLayer()` returns a new layer with the name you specify.

See the section [Getting ModelItem Objects on page 145](#) for other methods that can return layer objects.

Layer Operations

Methods and Properties Introduced:

-
- **pfcLayer.Status**
 - **pfcLayer.ListItems()**
 - **pfcLayer.AddItem()**
 - **pfcLayer.RemoveItem()**
 - **pfcLayer.Delete()**
 - **pfcLayer.CountUnsupportedItems()**

Use the property `pfcLayer.Status` to access the display status of a layer. The corresponding enumeration class is `pfcDisplayStatus` and the possible values are `Normal`, `Displayed`, `Blank`, or `Hidden`.

Use the methods `pfcLayer.ListItems()`, `pfcLayer.AddItem()`, and `pfcLayer.RemoveItem()` to control the contents of a layer.

 Note

You cannot add the following items to a layer:

- `ITEM_SURFACE`,
- `ITEM_EDGE`,
- `ITEM_COORD_SYS`,
- `ITEM_AXIS`,
- `ITEM_SIMPREP`,
- `ITEM_DTL_SYM_DEFINITION`,
- `ITEM_DTL_OLE_OBJECT`,
- `ITEM_EXPLODED_STATE`.

For these items the method throws the exception `pfcXToolkitInvalidType`.

The method `pfcLayer.Delete()` removes the layer but not the items it contains from the model.

The method `pfcLayer.CountUnsupportedItems()` returns the number of item types not supported as a `pfcModelItem` object in the specified layer.

Features

Features

All Creo Parametric solid models are made up of features. This section describes how to program on the feature level using Creo.JS.

Access to Features

Methods and Properties Introduced:

- **pfcFeature.ListChildren()**
- **pfcFeature.ListParents()**
- **pfcFeature.GroupDirectHeader**
- **pfcFeatureGroup.GroupLeader**
- **pfcFeaturePattern.PatternLeader()**
- **pfcFeaturePattern.ListMembers()**
- **pfcSolid.ListFailedFeatures()**
- **pfcSolid.ListFeaturesByType()**
- **pfcSolid.GetFeatureById()**
- **pfcBaseSession.QueryFeatureEdit()**

The methods `pfcFeature.ListChildren()` and `pfcFeature.ListParents()` return an array of features that contain all the children or parents of the specified feature.

The property `pfcFeature.GroupDirectHeader` returns the direct header of the group.

To get the first feature in the specified group access the property `pfcFeatureGroup.GroupLeader`.

The method `pfcFeaturePattern.PatternLeader()` and the method `pfcFeaturePattern.ListMembers()` return features that make up the specified feature pattern. See the section [Feature Groups and Patterns on page 153](#) for more information on feature patterns.

The method `pfcSolid.ListFailedFeatures()` returns an array that contains all the features that failed regeneration.

The method `pfcSolid.ListFeaturesByType()` returns an array of features contained in the model. You can specify which type of feature to collect by passing in one of the `pfcFeatureType` enumeration objects, or you can collect all features by passing `void null` as the type. If you list all features, the resulting array includes invisible features that Creo Parametric creates internally. Internal features are invisible features used internally for construction purposes.

Use the 's *VisibleOnly* argument to exclude them. If the argument *VisibleOnly* is `True`, the method lists the public features only. If the argument is `False`, the method lists both public and internal features.

The method `pfcSolid.GetFeatureById()` returns the feature object with the corresponding integer identifier.

A feature can be edited with the **Edit Definition** command in Creo Parametric. The method `pfcBaseSession.QueryFeatureEdit()` returns a list of all the features that are currently being edited by the **Edit Definition** command.

Feature Information

Properties Introduced:

- `pfcFeature.FeatType`
- `pfcFeature.Status`
- `pfcFeature.IsVisible`
- `pfcFeature.IsReadonly`
- `pfcFeature.IsEmbedded`
- `pfcFeature.Number`
- `pfcFeature.FeatTypeName`
- `pfcFeature.FeatSubType`
- `pfcRoundFeat.IsAutoRoundMember`

The enumeration classes `pfcFeatureType` and `pfcFeatureStatus` provide information for a specified feature. The following properties specify this information:

- `pfcFeature.FeatType`—Returns the type of a feature.
- `pfcFeature.Status`—Returns whether the feature is suppressed, active, or failed regeneration.

The other properties that gather feature information include the following:

- `pfcFeature.IsVisible`—Identifies whether the specified feature is visible on the screen. The property distinguishes visible features from internal features. Internal features are invisible features used for construction purposes.
- `pfcFeature.IsReadonly`—Identifies whether the specified feature can be modified.
- `pfcFeature.IsEmbedded`—Specifies whether the specified feature is an embedded datum.
- `pfcFeature.Number`—Returns the feature regeneration number. This property returns `void null` if the feature is suppressed.

The property `pfcFeature.FeatTypeName` returns a string representation of the feature type.

The property `pfcFeature.FeatSubType` returns a string representation of the feature subtype, for example, "Extrude" for a protrusion feature.

The property `pfcRoundFeat.IsAutoRoundMember` determines whether the specified round feature is a member of an Auto Round feature.

Feature Operations

Methods and Properties Introduced:

- **`pfcSolid.ExecuteFeatureOps()`**
- **`pfcFeature.CreateSuppressOp()`**
- **`pfcSuppressOperation.Clip`**
- **`pfcSuppressOperation.AllowGroupMembers`**
- **`pfcSuppressOperation.AllowChildGroupMembers`**
- **`pfcFeature.CreateDeleteOp()`**
- **`pfcDeleteOperation.Clip`**
- **`pfcDeleteOperation.AllowGroupMembers`**
- **`pfcDeleteOperation.AllowChildGroupMembers`**
- **`pfcDeleteOperation.KeepEmbeddedDatums`**
- **`pfcFeature.CreateResumeOp()`**
- **`pfcResumeOperation.WithParents`**
- **`pfcFeature.CreateReorderBeforeOp()`**
- **`pfcReorderBeforeOperation.BeforeFeat`**
- **`pfcFeature.CreateReorderAfterOp()`**
- **`pfcReorderAfterOperation.AfterFeat`**

The method `pfcSolid.ExecuteFeatureOps()` causes a sequence of feature operations to run in order. Feature operations include suppressing, resuming, reordering, and deleting features. The optional `pfcRegenInstructions` argument specifies whether the user can fix the model if a regeneration failure occurs.

Note

Regenerating models in `resolve_mode` is not supported. As a result, the method `pfcSolid.ExecuteFeatureOps()` is not supported anymore.

Some of the operations have specific options that you can modify to control the operation:

- **Clip**—Specifies whether to delete or suppress all features after the selected feature. By default, this option is false.

Use the properties `pfcDeleteOperation.Clip` and `pfcSuppressOperation.Clip` to modify this option.

- **AllowGroupMembers**—If this option is set to `true` and if the feature to be deleted or suppressed is a member of a group, then the feature will be deleted or suppressed out of the group. If this option is set to `false`, then the entire group containing the feature is deleted or suppressed. By default, this option is false. It can be set to `true` only if the option **Clip** is set to `true`.

Use the properties `pfcSuppressOperation.AllowGroupMembers` and `pfcDeleteOperation.AllowGroupMembers` to modify this option.

- **AllowChildGroupMembers**—If this option is set to `true` and if the children of the feature to be deleted or suppressed are members of a group, then the children of the feature are individually deleted or suppressed out of the group. If this option is set to `false`, then the entire group containing the feature and its children are deleted or suppressed. By default, this option is false. It can be set to `true` only if the options **Clip** and **AllowGroupMembers** are set to `true`.

Use the properties `pfcSuppressOperation.AllowChildGroupMembers` and `pfcDeleteOperation.AllowChildGroupMembers` to modify this option.

- **KeepEmbeddedDatums**—Specifies whether to retain the embedded datums stored in a feature while deleting the feature. By default, this option is false.

Use the property `pfcDeleteOperation.KeepEmbeddedDatums` to modify this option.

- **WithParents**—Specifies whether to resume the parents of the selected feature.

Use the property `pfcResumeOperation.WithParents` to modify this option.

- **BeforeFeat**—Specifies the feature before which you want to reorder the features.

Use the property `pfcReorderBeforeOperation.BeforeFeat` to modify this option.

-
- **AfterFeat**—Specifies the feature after which you want to reorder the features.

Use the property `pfcReorderAfterOperation.AfterFeat` to modify this option.

Feature Groups and Patterns

Patterns are treated as features in Creo Parametric. A feature type, `FEATTYPE_PATTERN_HEAD`, is used for the pattern header feature.



Note

The pattern header feature is not treated as a leader or a member of the pattern by the properties described in the following section.

Methods and Properties Introduced:

- **pfcFeature.Group**
- **pfcFeature.Pattern**
- **pfcFeature.GetPatternByType()**
- **pfcFeature.PatternStatus**
- **pfcFeature.GroupStatus**
- **pfcFeature.GroupPatternStatus**
- **pfcSolid.CreateLocalGroup()**
- **pfcFeatureGroup.Pattern**
- **pfcFeatureGroup.GroupLeader**
- **pfcFeaturePattern.PatternLeader**
- **pfcFeaturePattern.ListMembers()**
- **pfcFeaturePattern.Delete()**

The property `pfcFeature.Group` returns a handle to the local group that contains the specified feature.

To get the first feature in the specified group call the property `pfcFeatureGroup.GroupLeader`.

The property `pfcFeaturePattern.PatternLeader` and the method `pfcFeaturePattern.ListMembers()` return features that make up the specified feature pattern.

A pattern is composed of a pattern header feature and a number of member features. You can pattern only a single feature. To pattern several features, create a local group and pattern this group.

You can also create a pattern of pattern. This creates a multiple level pattern. For a pattern of pattern, the method `pfcFeaturePattern.ListMembers()` returns all the pattern header features created at the first level.

For example, consider a model in which there is a pattern such as in the following Model Tree:

	Feat ID
PRT0020.PRT	
RIGHT	1
TOP	3
FRONT	5
PRT_CSYS_DEF	7
Extrude 1	60
Pattern 2 of Pattern 1	176
Pattern 1 of Hole 1	119
Hole 1 [1, 1]	87
Hole 1 [1, 2]	120
Hole 1 [2, 2]	121
Pattern 3 of Hole 2	177
Hole 2 [1, 1]	182
Hole 2 [1, 2]	195
Hole 2 [2, 2]	208
Pattern 4 of Hole 3	221
Hole 3 [1, 1]	226
Hole 3 [1, 2]	239
Hole 3 [2, 2]	252
Pattern 5 of Hole 4	265
Hole 4 [1, 1]	270
Hole 4 [1, 2]	283
Hole 4 [2, 2]	296
Insert Here	

The method `pfcFeaturePattern.ListMembers()` returns the pattern header features with the following IDs for a pattern of pattern:

- 119
- 177
- 221

- 265

The properties `pfcFeature.Pattern` and `pfcFeatureGroup.Pattern` return the `pfcFeaturePattern` object that contains the corresponding `pfcFeature` or `pfcFeatureGroup`. Use the method `pfcSolid.CreateLocalGroup()` to take a sequence of features and create a local group with the specified name. To delete a `pfcFeaturePattern` object, call the method `pfcFeaturePattern.Delete()`.

The method `pfcFeature.GetPatternByType()` returns the pattern of the feature and is specified using the enumerated data type `pfcPatternType`. The valid values are:

- `PATTERN_FEAT`—Feature pattern.
- `PATTERN_GROUP`—Group pattern.

The property `pfcFeature.PatternStatus` returns the pattern status of the feature and is specified using the enumerated data type `pfcPatternStatus`. The valid values are:

- `GRP_PATTERN_NONE`—Feature does not belong to any pattern.
- `GRP_PATTERN_LEADER`—Feature is leader of the pattern.
- `GRP_PATTERN_MEMBER`—Feature is member of the pattern.
- `GRP_PATTERN_HEADER`—Feature is header of the pattern.

The property `pfcFeature.GroupStatus` returns the group status of the feature and is specified using the enumerated data type `pfcGroupStatus`. The valid values are:

- `GROUP_NONE`—Feature does not belong to any group.
- `GROUP_MEMBER`—Feature is member of the group.

The property `pfcFeature.GroupPatternStatus` returns the group pattern status of the feature and is specified using the enumerated data type `GroupPatternStatus`. The valid values are:

- `GRP_PATTERN_NONE`—Feature does not belong to any group pattern.
- `GRP_PATTERN_LEADER`—Feature is leader of the group pattern.
- `GRP_PATTERN_MEMBER`—Feature is member of the group pattern.
- `GRP_PATTERN_HEADER`—Feature is header of the group pattern.

Feature Groups

Feature groups have a group header feature, which shows up in the model information and feature list for the model. This feature will be inserted in the regeneration list to a position just before the first feature in the group.

The results of the header feature are as follows:

-
- Models that contain groups get one extra feature in the regeneration list, of type `FEATTYPE_GROUP_HEAD`. This affects the feature numbers of all subsequent features, including those in the group.
 - Each group automatically contains the header feature in the list of features returned from `pfcFeatureGroup.ListMembers()`.
 - Each group automatically gets the group head feature as the leader. This is returned from `pfcFeatureGroup.GroupLeader`.
 - Each group pattern contains a series of groups, and each group in the pattern is similarly constructed.

User Defined Features

Groups in Creo Parametric represent sets of contiguous features that act as a single feature for specific operations. Individual features are affected by most operations while some operations apply to an entire group:

- Suppress
- Delete
- Layers
- Patterning

User Defined Features (UDFs) are groups of features that are stored in a file. When a UDF is placed in a new model the created features are automatically assigned to a group. A local group is a set of features that are specifically assigned to a group to make modifications and patterning easier.

Note

All methods in this section can be used for UDFs and local groups.

Read Access to Groups and User Defined Features

Methods and Properties Introduced:

- **`pfcFeatureGroup.UDFName`**
- **`pfcFeatureGroup.UDFInstanceName`**
- **`pfcFeatureGroup.ListUDFDimensions()`**
- **`pfcUDFDimension.UDFDimensionName`**

User defined features (UDFs) are groups of features that can be stored in a file and added to a new model. A local group is similar to a UDF except it is available only in the model in which is created.

The property `pfcFeatureGroup.UDFName` provides the name of the group for the specified group instance. A particular group definition can be used more than once in a particular model.

If the group is a family table instance, the property `pfcFeatureGroup.UDFInstanceName` supplies the instance name.

The method `pfcFeatureGroup.ListUDFDimensions()` traverses the dimensions that belong to the UDF. These dimensions correspond to the dimensions specified as variables when the UDF was created. Dimensions of the original features that were not variables in the UDF are not included unless the UDF was placed using the Independent option.

The property `pfcUDFDimension.UDFDimensionName` provides access to the dimension name specified when the UDF was created, and not the name of the dimension in the current model. This name is required to place the UDF programmatically using the method `pfcSolid.CreateUDFGroup()`.

Creating Features from UDFs

Method Introduced:

- **`pfcSolid.CreateUDFGroup()`**

The method `pfcSolid.CreateUDFGroup()` is used to create new features by retrieving and applying the contents of an existing UDF file. It is equivalent to the Creo Parametric command `Feature, Create, User Defined`.

To understand the following explanation of this method, you must have a good knowledge and understanding of the use of UDFs in Creo Parametric. PTC recommends that you read about UDFs in the Creo Parametric help, and practice defining and using UDFs in Creo Parametric before you attempt to use this method.

When you create a UDF interactively, Creo Parametric prompts you for the information it needs to fix the properties of the resulting features. When you create a UDF from Creo.JS, you can provide some or all of this information programmatically by filling several compact data classes that are inputs to the method `pfcSolid.CreateUDFGroup()`.

During the call to `pfcSolid.CreateUDFGroup()`, Creo Parametric prompts you for the following:

- Information required by the UDF that was not provided in the input data structures.
- Correct information to replace erroneous information

Such prompts are a useful way of diagnosing errors when you develop your application. This also means that, in addition to creating UDFs programmatically to provide automatic synthesis of model geometry, you can also use

`pfcSolid.CreateUDFGroup()` to create UDFs semiinteractively. This can simplify the interactions needed to place a complex UDF making it easier for the user and less prone to error.

Creating UDFs

Creating a UDF requires the following information:

- **Name**—The name of the UDF you are creating and the instance name if applicable.
- **Dependency**—Specify if the UDF is independent of the UDF definition or is modified by the changers made to it.
- **Scale**—How to scale the UDF relative to the placement model.
- **Variable Dimension**—The new values of the variables dimensions and pattern parameters, those whose values can be modified each time the UDF is created.
- **Dimension Display**—Whether to show or blank nonvariable dimensions created within the UDF group.
- **References**—The geometrical elements that the UDF needs in order to relate the features it contains to the existing models features. The elements correspond to the selections that Creo Parametric prompts you for when you create a UDF interactively using the prompts defined when the UDF was created. You cannot select an embedded datum as the UDF reference.
- **Parts Intersection**—When a UDF that is being created in an assembly contains features that modify the existing geometry you must define which parts are affected or intersected. You also need to know at what level in an assembly each intersection is going to be visible.
- **Orientations**—When a UDF contains a feature with a direction that is defined in respect to a datum plane Creo Parametric must know what direction the new feature points to. When you create such a UDF interactively Creo Parametric prompts you for this information with a flip arrow.
- **Quadrants**—When a UDF contains a linearly placed feature that references two datum planes to define its location in the new model Creo Parametric prompts you to select the location of the new feature. This is determined by which side of each datum plane the feature must lie. This selection is referred to as the quadrant because there are four possible combinations for each linearly place feature.

To pass all the above values to Creo Parametric, Creo.JS uses a special class that prepares and sets all the options and passes them to Creo Parametric.

Creating Interactively Defined UDFs

Method Introduced:

- **pfcUDFPromptCreateInstructions.Create()**

This static method is used to create an instructions object that can be used to prompt a user for the required values that creates a UDF interactively.

Creating a Custom UDF

Method Introduced:

- **pfcUDFCustomCreateInstructions.Create()**

This method creates a `pfcUDFCustomCreateInstructions` object with a specified name. To set the UDF creation parameters programmatically you must modify this object as described below. The members of this class relate closely to the prompts Creo Parametric gives you when you create a UDF interactively. PTC recommends that you experiment with creating the UDF interactively using Creo Parametric before you write the Creo.JS code to fill the structure.

Setting the Family Table Instance Name

Property Introduced:

- **pfcUDFCustomCreateInstructions.InstanceName**

If the UDF contains a family table, this field can be used to select the instance in the table. If the UDF does not contain a family table, or if the generic instance is to be selected, the do not set the string.

Setting Dependency Type

Property Introduced:

- **pfcUDFCustomCreateInstructions.DependencyType**

The `pfcUDFDependencyType` object represents the dependency type of the UDF. The choices correspond to the choices available when you create a UDF interactively. This enumerated type takes the following values:

- `UDFDEP_INDEPENDENT`
- `UDFDEP_DRIVEN`



Note

`UDFDEP_INDEPENDENT` is the default value, if this option is not set.

Setting Scale and Scale Type

Properties Introduced:

- **pfcUDFCustomCreateInstructions.ScaleType**
- **pfcUDFCustomCreateInstructions.Scale**

The property `pfcUDFCustomCreateInstructions.ScaleType` specifies the length units of the UDF in the form of the `pfcUDFScaleType` object. This enumerated type takes the following values:

- `UDFSCALE_SAME_SIZE`
- `UDFSCALE_SAME_DIMS`
- `UDFSCALE_CUSTOM`
- `UDFSCALE_nil`



Note

The default value is `UDFSCALE_SAME_SIZE` if this option is not set.

The property `pfcUDFCustomCreateInstructions.Scale` specifies the scale factor. If the value of the filed *ScaleType* is set to `UDFSCALE_CUSTOM`, the property `pfcUDFCustomCreateInstructions.Scale` assigns the user defined scale factor. Otherwise, this attribute is ignored.

Setting the Appearance of the Non UDF Dimensions

Property Introduced:

- **pfcUDFCustomCreateInstructions.DimDisplayType**

The `pfcUDFDimensionDisplayType` object sets the options in Creo Parametric for determining the appearance in the model of UDF dimensions and pattern parameters that were not variable in the UDF, and therefore cannot be modified in the model. This enumerated type takes the following values:

- `UDFDISPLAY_NORMAL`
- `UDFDISPLAY_READ_ONLY`
- `UDFDISPLAY_BLANK`



Note

The default value is `UDFDISPLAY_NORMAL` if this option is not set.

The values defined by the enumerated data type `pfcUDFDimensionDisplayType` correspond to the options in Creo Parametric for determining the appearance of the UDF dimensions and pattern parameters that were not variable in the UDF, and therefore cannot be modified in the model.

Setting the Variable Dimensions and Parameters

Methods and Properties Introduced:

- **`pfcUDFCustomCreateInstructions.VariantValues`**
- **`pfcUDFVariantDimension.Create()`**
- **`pfcUDFVariantDimension.DimensionValue`**
- **`pfcUDFVariantPatternParam.Create()`**
- **`pfcUDFVariantPatternParam.PatternParam`**

`pfcUDFVariantValues` class represents an array of variable dimensions and pattern parameters.

`pfcUDFVariantDimension.Create()` is a static method creating a `pfcUDFVariantDimension`. It accepts the following parameters:

- *Name*—The symbol that the dimension had when the UDF was originally defined, not the prompt that the UDF uses when it is created interactively. To make this name easy to remember, before you define the UDF that you plan to create with the Creo.JS, you should modify the symbols of all the dimensions that you want to select to be variable. If you get the name wrong, `pfcSolid.CreateUDFGroup()` will not recognize the dimension and prompts the user for the value in the usual way and does not modify the value.
- *DimensionValue*—The new value.

Use the property `pfcUDFVariantDimension.DimensionValue` to specify the dimension value.

If you do not remember the name, you can find it by creating the UDF interactively in a test model, then using the

`pfcFeatureGroup.ListUDFDimensions()` and `pfcUDFDimension.UDFDimensionName` to find out the name.

`pfcUDFVariantPatternParam.Create()` is a static method which creates a `pfcUDFVariantPatternParam`. It accepts the following parameters:

- *Name*—The string name that the pattern parameter had when the UDF was originally defined
- *PatternParam*—The new value.

Use the property `pfcUDFVariantPatternParam.PatternParam` to specify the parameter value.

After the `pfcUDFVariantValues` object has been compiled, use `pfcUDFCustomCreateInstructions.VariantValues` to add the variable dimensions and parameters to the instructions.

Setting the User Defined References

Methods and Properties Introduced:

- **`pfcUDFReference.Create()`**
- **`pfcUDFReference.IsExternal`**
- **`pfcUDFReference.ReferenceItem`**
- **`pfcUDFCustomCreateInstructions.References`**

`pfcUDFReference.Create()` is a static method and it creates a `pfcUDFReference` object. It accepts the following parameters:

- *PromptForReference*—The prompt defined for this reference when the UDF was originally set up. It indicates which reference this structure is providing. If you get the prompt wrong, `pfcSolid.CreateUDFGroup()` will not recognize it and prompts the user for the reference in the usual way.
- *ReferenceItem*—Specifies the `pfcSelection` object representing the referenced element. You can set `pfcSelection` programmatically or prompt the user for a selection separately. You cannot set an embedded datum as the UDF reference.

There are two types of reference:

- *Internal*—The referenced element belongs directly to the model that will contain the UDF. For an assembly, this means that the element belongs to the top level.
- *External*—The referenced element belongs to an assembly member other than the placement member.

To set the reference type, use the property `pfcUDFReference.IsExternal`.

To set the item to be used for reference, use the property `pfcUDFReference.ReferenceItem`.

After the `pfcUDFReference` object has been set, use `pfcUDFCustomCreateInstructions.References` to add the program-defined references.

Setting the Assembly Intersections

Methods and Properties Introduced:

- **pfcUDFAssemblyIntersection.Create()**
- **pfcUDFAssemblyIntersection.ComponentPath**
- **pfcUDFAssemblyIntersection.VisibilityLevel**
- **pfcUDFAssemblyIntersection.InstanceNames**
- **pfcUDFCustomCreateInstructions.Intersections**

`pfcUDFAssemblyIntersection.Create()` is a static method. Use this method to create a `pfcUDFReference` object. It accepts the following parameters:

- *ComponentPath*—Is an `intpath` type object representing the component path of the part to be intersected.
- *VisibilityLevel*—The number that corresponds to the visibility level of the intersected part in the assembly. If the number is equal to the length of the component path the feature is visible in the part that it intersects. If *VisibilityLevel* is 0, the feature is visible at the level of the assembly containing the UDF.

The property `pfcUDFAssemblyIntersection.ComponentPath` sets the member identifier table that defines the location of the intersected part in the assembly that contains the UDF.

The property `pfcUDFAssemblyIntersection.VisibilityLevel` sets the length of the `pfcComponentPath` object that corresponds to the visibility level of the intersected part in the assembly.

`pfcUDFAssemblyIntersection.InstanceNames` sets an array of names for the new instances of parts created to represent the intersection geometry.

After the `pfcUDFAssemblyIntersections` object is set, use `pfcUDFCustomCreateInstructions.Intersections` to add the assembly intersections.

Setting Orientations

Property Introduced:

- **pfcUDFCustomCreateInstructions.Orientations**

`pfcUDFOrientations` class represents an array of orientations that provide the answers to Creo Parametric prompts that use a flip arrow. Each term is a `pfcUDFOrientation` object that takes the following values:

- `UDFORIENT_INTERACTIVE`—Prompt for the orientation using a flip arrow.
- `UDFORIENT_NO_FLIP`—Accept the default flip orientation.
- `UDFORIENT_FLIP`—Invert the orientation from the default orientation.

The order of orientations should correspond to the order in which Creo Parametric prompts for them when the UDF is created interactively. If you do not provide an orientation that Creo Parametric needs, it uses the default value `NO_FLIP`.

After the `pfcUDFOrientations` object is set, use `pfcUDFOrientation` to add the orientations.

Setting Quadrants

Property Introduced:

- **`pfcUDFCustomCreateInstructions.Quadrants`**

The property `pfcUDFCustomCreateInstructions.Quadrants` sets an array of points, which provide the X, Y, and Z coordinates that correspond to the prompt in Creo Parametric for selections related to feature positions. The order of quadrants should correspond to the order in which Creo Parametric prompts for them when the UDF is created interactively.

Setting the External References

Property Introduced:

- **`pfcUDFCustomCreateInstructions.ExtReferences`**

The property `pfcUDFCustomCreateInstructions.ExtReferences` sets an external reference assembly to be used when placing the UDF. This is required when placing the UDF in the component using references outside of that component. References can be to the top-level assembly of another component.

Datum Features

Datum Features

This section describes the Creo.JS methods and properties that provide read access to the properties of datum features.

Datum Plane Features

The properties of the Datum Plane feature are defined in the `pfcDatumPlaneFeat` data object.

Methods and Properties Introduced:

- **`pfcDatumPlaneFeat.Flip`**
- **`pfcDatumPlaneFeat.Constraints`**
- **`pfcDatumPlaneConstraint.ConstraintType`**
- **`pfcDatumPlaneThroughConstraint.ThroughRef`**

- **pfcDatumPlaneNormalConstraint.NormalRef**
- **pfcDatumPlaneParallelConstraint.ParallelRef**
- **pfcDatumPlaneTangentConstraint.TangentRef**
- **pfcDatumPlaneOffsetConstraint.OffsetRef**
- **pfcDatumPlaneOffsetConstraint.OffsetValue**
- **pfcDatumPlaneOffsetCoordSysConstraint.CsysAxis**
- **pfcDatumPlaneAngleConstraint.AngleRef**
- **pfcDatumPlaneAngleConstraint.AngleValue**
- **pfcDatumPlaneSectionConstraint.SectionRef**
- **pfcDatumPlaneSectionConstraint.SectionIndex**

The properties of the `pfcDatumPlaneFeat` object are described as follows:

- **Flip**—Specifies whether the datum plane was flipped during creation. Use the property `pfcDatumPlaneFeat.Flip` to determine if the datum plane was flipped during creation.
- **Constraints**—Collection of constraints given by the `pfcDatumPlaneConstraint` object. The property `pfcDatumPlaneFeat.Constraints` obtains the collection of constraints defined for the datum plane.

Use the property `pfcDatumPlaneConstraint.ConstraintType` to obtain the type of constraint. The type of constraint is given by the `pfcDatumPlaneConstraintType` enumerated data type. The available types are as follows:

- **DTMPLN_THRU**—Through constraint. The `pfcDatumPlaneThroughConstraint` object specifies this constraint. Use the property `pfcDatumPlaneThroughConstraint.ThroughRef` to get the reference selection handle for the Through constraint.
- **DTMPLN_NORM**—Normal constraint. The `pfcDatumPlaneNormalConstraint` object specifies this constraint. Use the property `pfcDatumPlaneNormalConstraint.NormalRef` to get the reference selection handle for the Normal constraint.
- **DTMPLN_PRL**—Specifies the Parallel constraint. The `pfcDatumPlaneParallelConstraint` object specifies this constraint. Use the property `pfcDatumPlaneParallelConstraint.ParallelRef` to get the reference selection handle for the Parallel constraint.
- **pfcDTMPLN_TANG**—Tangent constraint. The `pfcDatumPlaneTangentConstraint` object specifies this constraint.

Use the property

`pfcDatumPlaneTangentConstraint.TangentRef` (to get the reference selection handle for the Tangent constraint.

- **DTMPLN_OFFS**—Offset constraint. The `pfcDatumPlaneOffsetConstraint` object specifies this constraint. Use the property `pfcDatumPlaneOffsetConstraint.OffsetRef` to get the reference selection handle for the Offset constraint. Use the property `pfcDatumPlaneOffsetConstraint.OffsetValue` to get the offset value.

An Offset constraint where the offset reference is a coordinate system is given by the `pfcDatumPlaneOffsetCoordSysConstraint` object. Use the property `pfcDatumPlaneOffsetCoordSysConstraint.CsysAxis` to get the reference coordinate axis.

- **DTMPLN_ANG**—Angle constraint. The object specifies this constraint. Use the property `pfcDatumPlaneAngleConstraint.AngleRef` to get the reference selection handle for the Angle constraint. Use the property `pfcDatumPlaneAngleConstraint.AngleValue` to get the angle value.
- **DTMPLN_SEC**—Section constraint. The `pfcDatumPlaneSectionConstraint` object specifies this constraint. Use the property `pfcDatumPlaneSectionConstraint.SectionRef` to get the reference selection for the Section constraint. Use the property `pfcDatumPlaneSectionConstraint.SectionIndex` to get the section index.

Datum Plane Features

The properties of the Datum Plane feature are defined in the `pfcDatumPlaneFeat` data object.

Methods and Properties Introduced:

- **`pfcDatumPlaneFeat.Flip`**
- **`pfcDatumPlaneFeat.Constraints`**
- **`pfcDatumPlaneConstraint.ConstraintType`**
- **`pfcDatumPlaneThroughConstraint.ThroughRef`**
- **`pfcDatumPlaneNormalConstraint.NormalRef`**
- **`pfcDatumPlaneParallelConstraint.ParallelRef`**
- **`pfcDatumPlaneTangentConstraint.TangentRef`**
- **`pfcDatumPlaneOffsetConstraint.OffsetRef`**

- **pfcDatumPlaneOffsetConstraint.OffsetValue**
- **pfcDatumPlaneOffsetCoordSysConstraint.CsysAxis**
- **pfcDatumPlaneAngleConstraint.AngleRef**
- **pfcDatumPlaneAngleConstraint.AngleValue**
- **pfcDatumPlaneSectionConstraint.SectionRef**
- **pfcDatumPlaneSectionConstraint.SectionIndex**

The properties of the `pfcDatumPlaneFeat` object are described as follows:

- **Flip**—Specifies whether the datum plane was flipped during creation. Use the property `pfcDatumPlaneFeat.Flip` to determine if the datum plane was flipped during creation.
- **Constraints**—Collection of constraints given by the `pfcDatumPlaneConstraint` object. The property `pfcDatumPlaneFeat.Constraints` obtains the collection of constraints defined for the datum plane.

Use the property `pfcDatumPlaneConstraint.ConstraintType` to obtain the type of constraint. The type of constraint is given by the `pfcDatumPlaneConstraintType` enumerated data type. The available types are as follows:

- **DTMPLN_THRU**—Through constraint. The `pfcDatumPlaneThroughConstraint` object specifies this constraint. Use the property `pfcDatumPlaneThroughConstraint.ThroughRef` to get the reference selection handle for the Through constraint.
- **DTMPLN_NORM**—Normal constraint. The `pfcDatumPlaneNormalConstraint` object specifies this constraint. Use the property `pfcDatumPlaneNormalConstraint.NormalRef` to get the reference selection handle for the Normal constraint.
- **DTMPLN_PRL**—Specifies the Parallel constraint. The `pfcDatumPlaneParallelConstraint` object specifies this constraint. Use the property `pfcDatumPlaneParallelConstraint.ParallelRef` to get the reference selection handle for the Parallel constraint.
- **pfcDTMPLN_TANG**—Tangent constraint. The `pfcDatumPlaneTangentConstraint` object specifies this constraint. Use the property `pfcDatumPlaneTangentConstraint.TangentRef` to get the reference selection handle for the Tangent constraint.
- **DTMPLN_OFFS**—Offset constraint. The `pfcDatumPlaneOffsetConstraint` object specifies this constraint.

Use the property `pfcDatumPlaneOffsetConstraint.OffsetRef` to get the reference selection handle for the Offset constraint. Use the property `pfcDatumPlaneOffsetConstraint.OffsetValue` to get the offset value.

An Offset constraint where the offset reference is a coordinate system is given by the `pfcDatumPlaneOffsetCoordSysConstraint` object. Use the property `pfcDatumPlaneOffsetCoordSysConstraint.CsysAxis` to get the reference coordinate axis.

- **DTMPLN_ANG**—Angle constraint. The object specifies this constraint. Use the property `pfcDatumPlaneAngleConstraint.AngleRef` to get the reference selection handle for the Angle constraint. Use the property `pfcDatumPlaneAngleConstraint.AngleValue` to get the angle value.
- **DTMPLN_SEC**—Section constraint. The `pfcDatumPlaneSectionConstraint` object specifies this constraint. Use the property `pfcDatumPlaneSectionConstraint.SectionRef` to get the reference selection for the Section constraint. Use the property `pfcDatumPlaneSectionConstraint.SectionIndex` to get the section index.

Datum Axis Features

The properties of the Datum Axis feature are defined in the `pfcDatumAxisFeat` data object.

Properties Introduced:

- **`pfcDatumAxisFeat.Constraints`**
- **`pfcDatumAxisConstraint.ConstraintType`**
- **`pfcDatumAxisConstraint.ConstraintRef`**
- **`pfcDatumAxisFeat.DimConstraints`**
- **`pfcDatumAxisDimensionConstraint.DimOffset`**
- **`pfcDatumAxisDimensionConstraint.DimRef`**

The properties of the `pfcDatumAxisFeat` object are described as follows:

- **Constraints**—Specifies a collection of constraints given by the `pfcDatumAxisConstraint` object. The property `pfcDatumAxisFeat.Constraints` obtains the collection of constraints applied to the Datum Axis feature.

This object contains the following attributes:

- **ConstraintType**—Specifies the type of constraint in terms of the `pfcDatumAxisConstraintType` enumerated type. The constraint type determines the type of datum axis. The constraint types are:
 - ◆ `DTMAXIS_NORMAL`—Normal datum constraint.
 - ◆ `DTMAXIS_THRU`—Through datum constraint.
 - ◆ `DTMAXIS_TANGENT`—Tangent datum constraint.
 - ◆ `DTMAXIS_CENTER`—Center datum constraint.

Use the property `pfcDatumAxisConstraint.ConstraintType` to get the constraint type.

- **ConstraintRef**—Reference selection for the constraint. Use the property `pfcDatumAxisConstraint.ConstraintRef` to get the reference selection handle.
- **DimConstraints**—Collection of dimension constraints given by the `pfcDatumAxisDimensionConstraint` object. The property `pfcDatumAxisFeat.DimConstraints` obtains the collection of dimension constraints applied to the Datum Axis feature.

This `pfcDatumAxisDimensionConstraint` object contains the following attributes:

- **DimOffset**—Offset value for the dimension constraint. Use the property `pfcDatumAxisDimensionConstraint.DimOffset` to get the offset value.
- **DimRef**—Reference selection for the dimension constraint. Use the property `pfcDatumAxisDimensionConstraint.DimRef` to get the reference selection handle.

General Datum Point Features

The properties of the general Datum Point feature are defined in the data object.

Methods and Properties Introduced:

- **`pfcDatumPointFeat.FeatName`**
- **`pfcDatumPointFeat.GetPoints()`**
- **`pfcGeneralDatumPoint.Name`**
- **`pfcGeneralDatumPoint.PlaceConstraints`**
- **`pfcGeneralDatumPoint.DimConstraints`**
- **`pfcDatumPointConstraint.ConstraintRef`**
- **`pfcDatumPointConstraint.ConstraintType`**

- **pfcDatumPointConstraint.Value**

The properties of the `pfcDatumPointFeat` object are described as follows:

- **FeatName**—Name of the General Datum Point feature. Use the property `pfcDatumPointFeat.FeatName` to get the name.
- **GeneralDatumPoints**—Collection of general datum points given by the `pfcGeneralDatumPoint` object. Use the method `pfcDatumPointFeat.GetPoints()` to obtain the collection of general datum points. The `pfcGeneralDatumPoint` object consists of the following attributes:
 - **Name**—Name of the general datum point. Use the property `pfcGeneralDatumPoint.Name` to get the name.
 - **PlaceConstraints**—Collection of placement constraints given by the `pfcDatumPointPlacementConstraint` object. Use the property `pfcGeneralDatumPoint.PlaceConstraints` to obtain the collection of placement constraints.
 - **DimConstraints**—Collection of dimension constraints given by the `pfcDatumPointDimensionConstraint` object. Use the property `pfcGeneralDatumPoint.DimConstraints` to obtain the collection of dimension constraints.

The constraints for a datum point are given by the `pfcDatumPointConstraint` object. This object contains the following attributes:

- **ConstraintRef**—Reference selection for the datum point constraint. Use the property `pfcDatumPointConstraint.ConstraintRef` to get the reference selection handle.
- **ConstraintType**—Type of datum point constraint in terms of the `pfcDatumPointConstraintType` enumerated type. Use the property `pfcDatumPointConstraint.ConstraintType` to get the constraint type.
- **Value**—Constraint reference value with respect to the datum point. Use the property `pfcDatumPointConstraint.Value` to get the value of the constraint reference with respect to the datum point.

The `pfcDatumPointPlacementConstraint` and `pfcDatumPointDimensionConstraint` objects inherit from the `pfcDatumPointConstraint` object. Use the properties of the `pfcDatumPointConstraint` object for the inherited objects.

Datum Coordinate System Features

The properties of the Datum Coordinate System feature are defined in the `pfcCoordSysFeat` object.

Properties Introduced:

- **`pfcCoordSysFeat.OriginConstraints`**
- **`pfcDatumCsysOriginConstraint.OriginRef`**
- **`pfcCoordSysFeat.DimensionConstraints`**
- **`pfcDatumCsysDimensionConstraint.DimRef`**
- **`pfcDatumCsysDimensionConstraint.DimValue`**
- **`pfcDatumCsysDimensionConstraint.DimConstraintType`**
- **`pfcCoordSysFeat.OrientationConstraints`**
- **`pfcDatumCsysOrientMoveConstraint.OrientMoveConstraintType`**
- **`pfcDatumCsysOrientMoveConstraint.OrientMoveValue`**
- **`pfcCoordSysFeat.IsNormalToScreen`**
- **`pfcCoordSysFeat.OffsetType`**
- **`pfcCoordSysFeat.OnSurfaceType`**
- **`pfcCoordSysFeat.OrientByMethod`**

The properties of the `pfcCoordSysFeat` object are described as follows:

- **OriginConstraints**—Collection of origin constraints given by the `pfcDatumCsysOriginConstraint` object. Use the property `pfcCoordSysFeat.OriginConstraints` to obtain the collection of origin constraints for the coordinate system. This object contains the following attribute:
 - **OriginRef**—Selection reference for the origin. Use the property `pfcDatumCsysOriginConstraint.OriginRef` to get the selection reference handle.
- **DimensionConstraints**—Collection of dimension constraints given by the `pfcDatumCsysDimensionConstraint` object. Use the property `pfcCoordSysFeat.DimensionConstraints` to obtain the collection of dimension constraints for the coordinate system. This object contains the following attributes:
 - **DimRef**—Reference selection for the dimension constraint. Use the property `pfcDatumCsysDimensionConstraint.DimRef` to get the reference selection handle.

-
- DimValue—Value of the reference. Use the property `pfcDatumCsysDimensionConstraint.DimValue` to get the value.
 - DimConstraintType—Specifies the type of dimension constraint in terms of the `pfcDatumCsysDimConstraintType` enumerated type. Use the property `pfcDatumCsysDimensionConstraint.DimConstraintType` to get the constraint type. The constraint types are:
 - ◆ DTMCSYS_DIM_OFFSET—Offset type constraint.
 - ◆ DTMCSYS_DIM_ALIGN—Align type constraint.
 - OrientationConstraints—A collection of orientation constraints given by the `pfcDatumCsysOrientMoveConstraint` object. Use the property `pfcCoordSysFeat.OrientationConstraints` to obtain the collection of orientation constraints for the coordinate system. This object contains the following attributes:
 - OrientMoveConstraintType—Type of orientation for the constraint. The orientation type is given by the `pfcDatumCsysOrientMoveConstraintType` enumerated type. Use the property `pfcDatumCsysOrientMoveConstraint.OrientMoveConstraintType` to get the orientation type.
 - OrientMoveValue—Reference value for the constraint. Use the property `pfcDatumCsysOrientMoveConstraint.OrientMoveValue` to get the reference value.
 - IsNormalToScreen—Coordinate system is normal to screen. Use the property `pfcCoordSysFeat.IsNormalToScreen` to determine if the coordinate system is normal to screen.
 - OffsetType—Offset type of the coordinate system in terms of the `pfcDatumCsysOffsetType` enumerated type. Use the property `pfcCoordSysFeat.OffsetType` to get the offset type. The offset types are:
 - DTMCSYS_OFFSET_CARTESIAN—A cartesian coordinate system that has been defined by setting the values for the `DTMCSYS_MOVE_TRAN_X`, `DTMCSYS_MOVE_TRAN_Y`, and `DTMCSYS_MOVE_TRAN_Z` or `DTMCSYS_MOVE_ROT_X`, `DTMCSYS_MOVE_ROT_Y`, and `DTMCSYS_MOVE_ROT_Z` orientation constants.
 - DTMCSYS_OFFSET_CYLINDRICAL—A cylindrical coordinate system that has been defined by setting the values for the `DTMCSYS_MOVE_RAD`,

-
- DTMCSYS_MOVE_THETA, and DTMCSYS_MOVE_TRAN_ZI orientation constants.
 - DTMCSYS_OFFSET_SPHERICAL—A spherical coordinate system that has been defined by setting the values for the DTMCSYS_MOVE_RAD, DTMCSYS_MOVE_THETA, and DTMCSYS_MOVE_TRAN_PHI orientation constants.
 - OnSurfaceType—Specifies the on-surface type for the coordinate system in terms of the pfcDatumCsysOffsetType enumerated type. Use the property pfcCoordSysFeat.OnSurfaceType to get the on surface type property of the coordinate system. The on-surface types follow:
 - DTMCSYS_ONSURF_LINEAR—Coordinate system placed on the selected surface by using two linear dimensions.
 - DTMCSYS_ONSURF_RADIAL—Coordinate system placed on the selected surface by using a linear dimension and an angular dimension. The radius value is used to specify the linear dimension.
 - DTMCSYS_ONSURF_DIAMETER—This type is similar to the DTMCSYS_ONSURF_RADIAL type, except that the diameter value is used to specify the linear dimension. It is available only when planar surfaces are used as the reference.
 - OrientByMethod—Orientation method in terms of the pfcDatumCsysOrientByMethod enumerated type. Use the property pfcCoordSysFeat.OrientByMethod to get the orientation method. The available orientation types follow:
 - DTMCSYS_ORIENT_BY_SEL_REFS—Orientation by selected references.
 - DTMCSYS_ORIENT_BY_SEL_CSYS_AXES—Orientation by coordinate system axes.

Geometry Evaluation

Geometry Evaluation

This section describes geometry representation and discusses how to evaluate geometry using Creo.JS.

Geometry Traversal

- A simple rectangular face has one contour and four edges.

- A contour traverses a boundary so that the part face is always on the right-hand side (RHS). For an external contour the direction of traversal is clockwise. For an internal contour the direction of traversal is counterclockwise.
- If a part is extruded from a sketch that has a U-shaped cross section there will be separate surfaces at each leg of the U-channel.
- If a part is extruded from a sketch that has a square-shaped cross section, and a slot feature is then cut into the part to make it look like a U-channel, there will be one surface across the legs of the U-channel. The original surface of the part is represented as one surface with a cut through it.

Geometry Terms

Following are definitions for some geometric terms:

- **Surface**—An ideal geometric representation, that is, an infinite plane.
- **Face**—A trimmed surface. A face has one or more contours.
- **Contour**—A closed loop on a face. A contour consists of multiple edges. A contour can belong to one face only.
- **Edge**—The boundary of a trimmed surface.

An edge of a solid is the intersection of two surfaces. The edge belongs to those two surfaces and to two contours. An edge of a datum surface can be either the intersection of two datum surfaces or the external boundary of the surface.

If the edge is the intersection of two datum surfaces it belongs to those two surfaces and to two contours. If the edge is the external boundary of the datum surface it belongs to that surface alone and to a single contour.

Traversing the Geometry of a Solid Block

Methods Introduced:

- **pfcModelItemOwner.ListItems()**
- **pfcSurface.ListContours()**
- **pfcContour.ListElements()**

To traverse the geometry, follow these steps:

1. Starting at the top-level model, use `pfcModelItemOwner.ListItems()` with an argument of `ITEM_SURFACE` using the enumerated data type `pfcModelItemType`.
2. Use `pfcSurface.ListContours()` to list the contours contained in a specified surface.

3. Use `pfcContour.ListElements()` to list the edges contained in the contour.

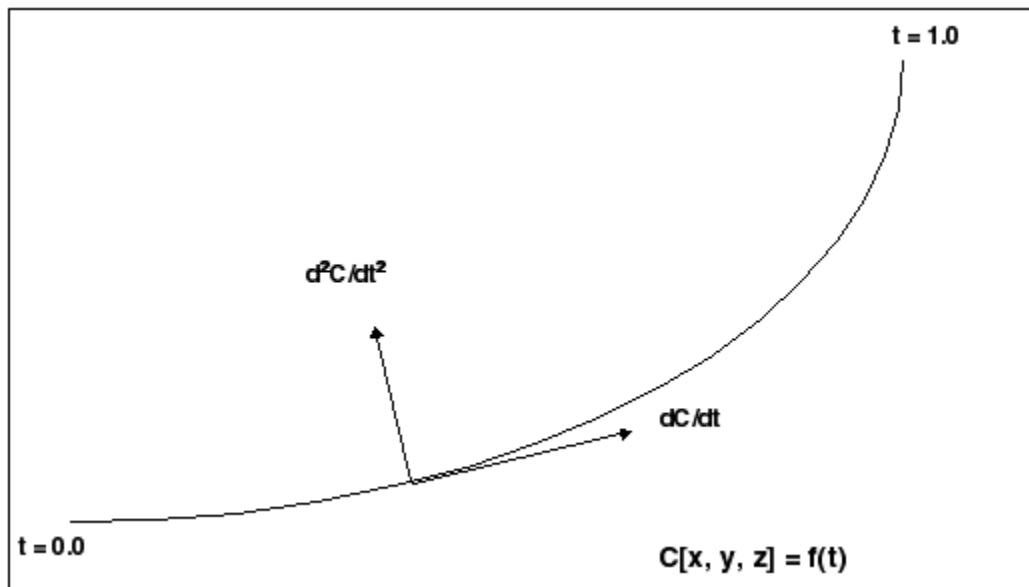
Curves and Edges

Datum curves, surface edges, and solid edges are represented in the same way in Creo.JS. You can get edges through geometry traversal or get a list of edges using the methods presented in the section [ModelItem on page 145](#).

The t Parameter

The geometry of each edge or curve is represented as a set of three parametric equations that represent the values of x , y , and z as functions of an independent parameter, t . The t parameter varies from 0.0 at the start of the curve to 1.0 at the end of it.

The following figure illustrates curve and edge parameterization.



Curve and Edge Types

Solid edges and datum curves can be any of the following types:

- LINE—A straight line represented by the class `pfcLine`.
- ARC—A circular curve represented by the class `pfcArc`.
- SPLINE—A nonuniform cubic spline, represented by the class `pfcSpline`.
- B-SPLINE—A nonuniform rational B-spline curve or edge, represented by the class `pfcBSpline`.

-
- **COMPOSITE CURVE**—A combination of two or more curves, represented by the class `pfcCompositeCurve`. This is used for datum curves only.

See the section [Geometry Representations on page 297](#) for the parameterization of each curve type. To determine what type of curve a `pfcEdge` or `pfcCurve` object represents, use the Java `instanceof` operator.

Because each curve class inherits from `pfcGeomCurve`, you can use all the evaluation methods in `pfcGeomCurve` on any edge or curve.

The following curve types are not used in solid geometry and are reserved for future expansion:

- **CIRCLE** (`pfcCircle`)
- **ELLIPSE** (`pfcEllipse`)
- **POLYGON** (`pfcPolygon`)
- **ARROW** (`pfcArrow`)
- **TEXT** (`pfcText`)

Evaluation of Curves and Edges

Methods Introduced:

- **`pfcGeomCurve.Eval3DData()`**
- **`pfcGeomCurve.EvalFromLength()`**
- **`pfcGeomCurve.EvalParameter()`**
- **`pfcGeomCurve.EvalLength()`**
- **`pfcGeomCurve.EvalLengthBetween()`**

The methods in `pfcGeomCurve` provide information about any curve or edge.

The method `pfcGeomCurve.Eval3DData()` returns a `pfcCurveXYZData` object with information on the point represented by the input parameter `t`. The method `pfcGeomCurve.EvalFromLength()` returns a similar object with information on the point that is a specified distance from the starting point.

The method `pfcGeomCurve.EvalParameter()` returns the `t` parameter that represents the input `pfcPoint3D` object.

Both `pfcGeomCurve.EvalLength()` and `pfcGeomCurve.EvalLengthBetween()` return numerical values for the length of the curve or edge.

Solid Edge Geometry

Methods and Properties Introduced:

- **`pfcEdge.Surface1`**

-
- **pfcEdge.Surface2**
 - **pfcEdge.Edge1**
 - **pfcEdge.Edge2**
 - **pfcEdge.EvalUV()**
 - **pfcEdge.GetDirection()**

 Note

The properties in the interface `pfcEdge` provide information only for solid or surface edges.

The properties `pfcEdge.Surface1` and `pfcEdge.Surface2` return the surfaces bounded by this edge. The properties `pfcEdge.Edge1` and `pfcEdge.Edge2` return the next edges in the two contours that contain this edge.

The method `pfcEdge.EvalUV()` evaluates geometry information based on the UV parameters of one of the bounding surfaces.

The method `pfcEdge.GetDirection()` returns a positive 1 if the edge is parameterized in the same direction as the containing contour, and -1 if the edge is parameterized opposite to the containing contour.

Curve Descriptors

A curve descriptor is a data object that describes the geometry of a curve or edge. A curve descriptor describes the geometry of a curve without being a part of a specific model.

Methods Introduced:

- **pfcGeomCurve.GetCurveDescriptor()**
- **pfcGeomCurve.GetNURBSRepresentation()**

 Note

To get geometric information for an edge, access the `pfcCurveDescriptor` object for one edge using `pfcGeomCurve.GetCurveDescriptor()`.

The method `pfcGeomCurve.GetCurveDescriptor()` returns the geometry for a curve as a data object.

The method `pfcGeomCurve.GetNURBSRepresentation()` returns a Non-Uniform Rational B-Spline Representation of a curve.

Contours

Methods Introduced:

- `pfcSurface.ListContours()`
- `pfcContour.InternalTraversal()`
- `pfcContour.FindContainingContour()`
- `pfcContour.EvalArea()`
- `pfcContour.EvalOutline()`
- `pfcContour.VerifyUV()`

Contours are a series of edges that completely bound a surface. A contour is not a `pfcModelItem`. You cannot get contours using the methods that get different types of `pfcModelItem`. Use the method `pfcSurface.ListContours()` to get contours from their containing surfaces.

The method `pfcContour.InternalTraversal()` returns a `pfcContourTraversal` enumerated data type that identifies whether a given contour is on the outside or inside of a containing surface.

Use the method `pfcContour.FindContainingContour()` to find the contour that entirely encloses the specified contour.

The method `pfcContour.EvalArea()` provides the area enclosed by the contour.

The method `pfcContour.EvalOutline()` returns the points that make up the bounding rectangle of the contour.

Use the method `pfcContour.VerifyUV()` to determine whether the given `pfcUVParams` argument lies inside the contour, on the boundary, or outside the contour.

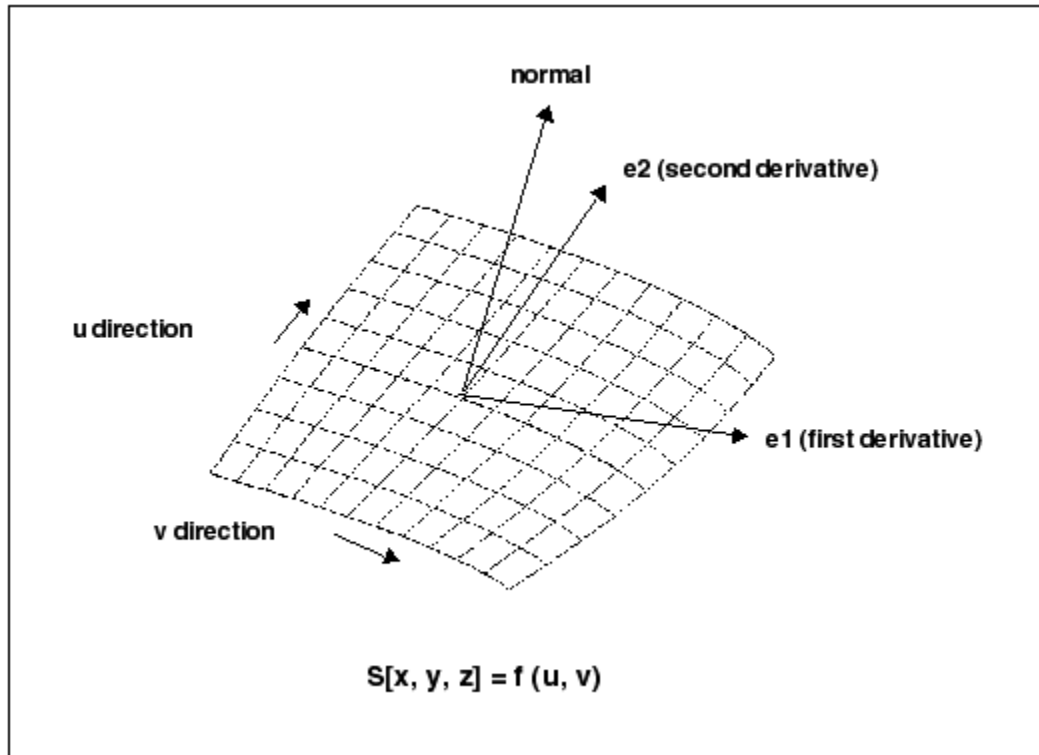
Surfaces

Using Creo.JS you access datum and solid surfaces in the same way.

UV Parameterization

A surface in Creo Parametric is described as a series of parametric equations where two parameters, *u* and *v*, determine the *x*, *y*, and *z* coordinates. Unlike the edge parameter, *t*, these parameters need not start at 0.0, nor are they limited to 1.0.

The figure on the following page illustrates surface parameterization.



Surface Types

Surfaces within Creo Parametric can be any of the following types:

- **PLANE**—A planar surface represented by the class `pfcPlane`.
- **CYLINDER**—A cylindrical surface represented by the class `pfcCylinder`.
- **CONE**—A conic surface region represented by the class `pfcCone`.
- **TORUS**—A toroidal surface region represented by the class `pfcTorus`.
- **REVOLVED SURFACE**—Generated by revolving a curve about an axis. This is represented by the class `pfcRevSurface`.
- **RULED SURFACE**—Generated by interpolating linearly between two curve entities. This is represented by the class `pfcRuledSurface`.
- **TABULATED CYLINDER**—Generated by extruding a curve linearly. This is represented by the class `pfcTabulatedCylinder`.
- **COONS PATCH**—A coons patch is used to blend surfaces together. It is represented by the class `pfcCoonsPatch`.
- **FILLET SURFACE**—A filleted surface is found where a round or fillet is placed on a curved edge or an edge with a nonconstant arc radii. On a straight edge a cylinder is used to represent a fillet. This is represented by the class `pfcFilletedSurface`.

- **SPLINE SURFACE**— A nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. This is represented by the class `pfcSplineSurface`.
- **NURBS SURFACE**—A NURBS surface is defined by basic functions (in u and v), expandable arrays of knots, weights, and control points. This is represented by the class `pfcNURBSSurface`.
- **CYLINDRICAL SPLINE SURFACE**— A cylindrical spline surface is a nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. This is represented by the class `pfcCylindricalSplineSurface`.

To determine which type of surface a `pfcSurface` object represents, access the surface type using `pfcSurface.GetSurfaceType()`.

Surface Information

Methods Introduced:

- **`pfcSurface.GetSurfaceType()`**
- **`pfcSurface.GetXYZExtents()`**
- **`pfcSurface.GetUVExtents()`**
- **`pfcSurface.GetOrientation()`**

The method `pfcSurface.GetSurfaceType()` returns the type of the surface using the enumerated data type `pfcSurfaceType` and the valid values are:

- `SURFACE_PLANE`
- `SURFACE_CYLINDER`
- `SURFACE_CONE`
- `SURFACE_TORUS`
- `SURFACE_RULED`
- `SURFACE_REVOLVED`
- `SURFACE_TABULATED_CYLINDER`
- `SURFACE_FILLET`
- `SURFACE_COONS_PATCH`
- `SURFACE_SPLINE`
- `SURFACE_NURBS`
- `SURFACE_CYLINDRICAL_SPLINE`
- `SURFACE_FOREIGN`
- `SURFACE_SPL2DER`

The method `pfcSurface.GetXYZExtents()` returns the XYZ points at the corners of the surface.

The method `pfcSurface.GetUVExtents()` returns the UV parameters at the corners of the surface.

The method `pfcSurface.GetOrientation()` returns the orientation of the surface using the enumerated data type `pfcSurfaceOrientation` and the valid values are:

- `SURFACEORIENT_NONE`—Surface that does not need orientation. For example, a solid surface needs orientation and therefore cannot be specified.
- `SURFACEORIENT_OUTWARD`—Surface that has oriented outward away from the solid model. $\frac{d\mathbf{X}}{dv}$ points outward.
- `SURFACEORIENT_INWARD`—Surface that has oriented inward toward the solid model. $\frac{d\mathbf{X}}{dv}$ points inward.

Evaluation of Surfaces

Surface methods allow you to use multiple surface information to calculate, evaluate, determine, and examine surface functions and problems.

Methods and Properties Introduced:

- `pfcSurface.OwnerQuilt()`
- `pfcSurface.EvalClosestPoint()`
- `pfcSurface.EvalClosestPointOnSurface()`
- `pfcSurfXYZData.FirstDerivative`
- `pfcSurfXYZData.SecondDerivative`
- `pfcSurface.Eval3DData()`
- `pfcSurface.EvalParameters()`
- `pfcSurface.EvalArea()`
- `pfcSurface.EvalDiameter()`
- `pfcSurface.EvalPrincipalCurv()`
- `pfcSurface.VerifyUV()`
- `pfcSurface.EvalMaximum()`
- `pfcSurface.EvalMinimum()`
- `pfcSurface.ListSameSurfaces()`

The property `pfcSurface.OwnerQuilt` returns the `Quilt` object that contains the datum surface.

The method `pfcSurface.EvalClosestPoint()` projects a three-dimensional point onto the surface. Use the method `pfcSurface.EvalClosestPointOnSurface()` to determine whether the specified three-dimensional point is on the surface, within the accuracy of the part. If it is, the method returns the point that is exactly on the surface. Otherwise the method returns null.

The method `pfcSurface.Eval3DData()` returns a `pfcSurfXYZData` object that contains information about the surface at the specified u and v parameters. The method `pfcSurface.EvalParameters()` returns the u and v parameters that correspond to the specified three-dimensional point.

The property `pfcSurfXYZData.FirstDerivative` returns the first partial derivatives of X, Y, and Z coordinates, with respect to the u and v parameters.

The property `pfcSurfXYZData.SecondDerivative` returns the second partial derivatives of X, Y, and Z coordinates, with respect to the u and v parameters.

The method `pfcSurface.EvalArea()` returns the area of the surface, whereas `pfcSurface.EvalDiameter()` returns the diameter of the surface. If the diameter varies the optional `pfcUVParams` argument identifies where the diameter should be evaluated.

The method `pfcSurface.EvalPrincipalCurv()` returns a `pfcCurvatureData` object with information regarding the curvature of the surface at the specified u and v parameters.

Use the method `pfcSurface.VerifyUV()` to determine whether the `pfcUVParams` are actually within the boundary of the surface.

The methods `pfcSurface.EvalMaximum()` and `pfcSurface.EvalMinimum()` return the three-dimensional point on the surface that is the furthest in the direction of or away from the specified vector.

The method `pfcSurface.ListSameSurfaces()` identifies other surfaces that are tangent and connect to the given surface.

Surface Descriptors

A surface descriptor is a data object that describes the shape and geometry of a specified surface. Use a surface descriptor to describe a surface in 3D without an owner ID.

Methods Introduced:

- **`pfcSurface.GetSurfaceDescriptor()`**
- **`pfcSurface.GetNURBSRepresentation()`**

The method `pfcSurface.GetSurfaceDescriptor()` returns a surfaces geometry as a data object.

The method `pfcSurface.GetNURBSRepresentation()` returns a Non-Uniform Rational B-Spline Representation of a surface.

Axes, Coordinate Systems, and Points

Coordinate axes, datum points, and coordinate systems are all model items. Use the methods that return `pfcModelItems` to get one of these geometry objects. Refer to the section [ModelItem on page 145](#) for additional information.

Evaluation of ModelItems

Properties Introduced:

- **`pfcAxis.Surf`**
- **`pfcCoordSystem.CoordSys`**
- **`pfcPoint.Point`**

The property `pfcAxis.Surf` returns the revolved surface that uses the axis.

The property `pfcCoordSystem.CoordSys` returns the `pfcTransform3D` object which includes the origin and x-, y-, and z- axes that defines the coordinate system.

The property `pfcPoint.Point` returns the xyz coordinates of the datum point.

Interference

Creo Parametric assemblies can contain interferences between components when constrained by certain rules defined by the user. The `pfcInterference` module allows the user to detect and analyze any interferences within the assembly. The analysis of this functionality should be looked at a global and selection-based analysis.

Methods and Properties Introduced:

- **`pfcCreateGlobalEvaluator()`**
- **`pfcGlobalEvaluator.ComputeGlobalInterference()`**
- **`pfcGlobalEvaluator.Assem`**
- **`pfcGlobalInterference.Volume()`**
- **`pfcGlobalInterference.SelParts()`**

To compute all the interferences within an Assembly, call `pfcCreateGlobalEvaluator()` with a `pfcAssembly` object as an argument. This call returns a `pfcGlobalEvaluator` object.

The property `pfcGlobalEvaluator.Assem` accesses the assembly to be evaluated.

The method `pfcGlobalEvaluator.ComputeGlobalInterference()` determines the set of all the interferences within the assembly.

This method returns an array of `pfcGlobalInterference` objects or null if there are no interfering parts. Each object contains a pair of intersecting parts and an object representing the interference volume, which can be extracted by using `pfcGlobalInterference.SelParts()` and `pfcGlobalInterference.Volume()` respectively.

Analyzing Interference Information

Methods and Properties Introduced:

- **`pfcSelectionPair.Create()`**
- **`pfcCreateSelectionEvaluator()`**
- **`pfcSelectionEvaluator.Selections()`**
- **`pfcSelectionEvaluator.ComputeInterference()`**
- **`pfcSelectionEvaluator.ComputeClearance()`**
- **`pfcSelectionEvaluator.ComputeNearestCriticalDistance()`**

The method `pfcSelectionPair.Create()` creates a `pfcSelection` objects as arguments.

A return from this method serves as an argument to `pfcCreateSelectionEvaluator()`, which provides a way to determine the interference data between the two selections.

`pfcSelectionEvaluator.Selections()` extracts the object to be evaluated.

`pfcSelectionEvaluator.ComputeInterference()` determines the interfering information about the provided selections. This method returns the `pfcInterferenceVolume` object or null if the selections do not interfere.

`pfcSelectionEvaluator.ComputeClearance()` computes the clearance data for the two selection. This method returns a `pfcClearanceData` object, which can be used to obtain and set clearance distance, nearest points between selections, and a Boolean `IsInterferencing` variable.

The method `pfcSelectionEvaluator.ComputeNearestCriticalDistance()` finds a critical point of the distance function between two selections.

This method returns a `pfcCriticalDistanceData` object, which is used to determine and set critical points, surface parameters, and critical distance between points.

Analyzing Interference Volume

Methods and Properties Introduced:

- **pfcInterferenceVolume.ComputeVolume()**
- **pfcInterferenceVolume.Highlight()**
- **pfcInterferenceVolume.Boundaries**

The method `pfcInterferenceVolume.ComputeVolume()` will calculate a value for interfering volume.

The method `pfcInterferenceVolume.Highlight()` will highlight the interfering volume with the color provided in the argument to the function.

The property `pfcInterferenceVolume.Boundaries` will return a set of boundary surface descriptors for the interference volume.

Dimensions and Parameters

Dimensions and Parameters

This section describes the Creo.JS methods and classes that affect dimensions and parameters.

Overview

Dimensions and parameters in Creo Parametric have similar characteristics but also have significant differences. In Creo.JS, the similarities between dimensions and parameters are contained in the `pfcBaseParameter` class. This class provides access to the parameter or dimension value and to information regarding designation and modification for a parameter. The differences between parameters and dimensions are recognizable because `pfcDimension` inherits from the class `pfcModelItem`, and can be assigned tolerances, whereas parameters are not `pfcModelItem` and cannot have tolerances.

The ParamValue Object

Both parameters and dimension objects contain an object of type `pfcParamValue`. This object contains the integer, real, string, or Boolean value of the parameter or dimension. Because of the different possible value types that can be associated with a `pfcParamValue` object there are different methods used to access each value type and some methods are not applicable for some `pfcParamValue` objects. If you try to use an incorrect method an exception will be thrown.

Accessing a ParamValue Object

Methods and Properties Introduced:

- **pfcCreateIntParamValue()**
- **pfcCreateDoubleParamValue()**
- **pfcCreateStringParamValue()**
- **pfcCreateBoolParamValue()**
- **pfcCreateNoteParamValue()**
- **pfcBaseParameter.Value**

The `pfcModelItem` utility class contains methods for creating each type of `pfcParamValue` object. After you establish the value type in the object, you can change it. The property `pfcBaseParameter.Value` returns the `pfcParamValue` associated with a particular parameter or dimension.

A `pfcNoteParamValue` is an integer value that refers to the ID of a specified note. To create a parameter of this type the identified note must already exist in the model.

Use the method `pfcCreateIntParamValue()` to create a `pfcParamValue` object with the specified integer value.

Use the method `pfcCreateDoubleParamValue()` to create a `pfcParamValue` object with the specified real value.

Use the method `pfcCreateStringParamValue()` to create a `pfcParamValue` object with the specified string value.

Use the method `pfcCreateBoolParamValue()` to create a `pfcParamValue` object with the specified Boolean value.

Use the method `pfcCreateNoteParamValue()` to create a note parameter.

Accessing the ParamValue Value

Properties Introduced:

- **pfcParamValue.discr**
- **pfcParamValue.IntValue()**
- **pfcParamValue.DoubleValue()**
- **pfcParamValue.StringValue()**
- **pfcParamValue.BoolValue()**
- **pfcParamValue.NoteId()**

The property `pfcParamValue.dscr` returns an enumeration object that identifies the type of value contained in the `pfcParamValue` object using the enumerated data type `pfcParamValueType`. The valid values and what each value specifies that the parameter has, follows:

- *PARAM_STRING*— Specifies the parameter has a string value.
- *PARAM_INTEGER*— Specifies the parameter has a integer value.
- *PARAM_BOOLEAN*— Specifies the parameter has a boolean value.
- *PARAM_DOUBLE*— Specifies the parameter has a real value.
- *PARAM_NOTE*— Specifies the parameter has a note identifier as its value.
- *PARAM_VOID*
- *PARAM_NOT_SET*

If you use an incorrect property an exception of type `pfcXBadGetParamValue` is thrown.

Parameter Objects

The following sections describe the Creo.JS methods that access parameters. The topics are as follows:

- [Creating and Accessing Parameters on page 187](#)
- [Parameter Selection Options on page 188](#)
- [Parameter Information on page 190](#)
- [Parameter Restrictions on page 192](#)

Creating and Accessing Parameters

Methods and Property Introduced:

- **`pfcParameterOwner.CreateParam()`**
- **`pfcParameterOwner.CreateParamWithUnits()`**
- **`pfcParameterOwner.GetParam()`**
- **`pfcParameterOwner.ListParams()`**
- **`pfcParameterOwner.SelectParam()`**
- **`pfcParameterOwner.SelectParameters()`**
- **`pfcFamColParam.RefParam`**

In Creo.JS models, features, surfaces, and edges inherit from the `pfcParameterOwner` class, because each of the objects can be assigned parameters in Creo Parametric.

The method `pfcParameterOwner.GetParam()` gets a parameter specified by its name.

The method `pfcParameterOwner.ListParams()` returns an array of all parameters assigned to the object.

To create a new parameter with a name and a specific value, call the method `pfcParameterOwner.CreateParam()`.

To create a new parameter with a name, a specific value, and units, call the method `pfcParameterOwner.CreateParamWithUnits()`.

The method `pfcParameterOwner.SelectParam()` allows you to select a parameter from the Creo Parametric user interface. The top model from which the parameters are selected must be displayed in the current window.

The method `pfcParameterOwner.SelectParameters()` allows you to interactively select parameters from the Creo Parametric **Parameter** dialog box based on the parameter selection options specified by the `pfcParameterSelectionOptions` object. The top model from which the parameters are selected must be displayed in the current window. Refer to the section [Parameter Selection Options on page 188](#) for more information.

The property `pfcFamColParam.RefParam` returns the reference parameter from the parameter column in a family table.

Parameter Selection Options

Parameter selection options in Creo.JS are represented by the `pfcParameterSelectionOptions` class.

Methods and Properties Introduced:

- **`pfcParameterSelectionOptions.Create()`**
- **`pfcParameterSelectionOptions.AllowContextSelection`**
- **`pfcParameterSelectionOptions.Contexts()`**
- **`pfcParameterSelectionOptions.AllowMultipleSelections`**
- **`pfcParameterSelectionOptions.SelectButtonLabel`**

The method `pfcParameterSelectionOptions.Create()` creates a new instance of the `pfcParameterSelectionOptions` object that is used by the method `pfcParameterOwner.SelectParameters()`.

The parameter selection options are as follows:

- **AllowContextSelection**—This Boolean attribute indicates whether to allow parameter selection from multiple contexts, or from the invoking parameter owner. By default, it is false and allows selection only from the invoking parameter owner. If it is true and if specific selection contexts are not yet assigned, then you can select the parameters from any context.

Use the property

`pfcParameterSelectionOptions.AllowContextSelection` to modify the value of this attribute.

- **Contexts**—The permitted parameter selection contexts in the form of the `pfcParameterSelectionContexts` object. Use the property `pfcParameterSelection.Context` to assign the parameter selection context. By default, you can select parameters from any context. This property can be set using the enumerated data type `pfcParameterSelectionContext`. The values for `pfcParameterSelectionContext` and a description of what each value specifies for selection, follow:
 - `PARAMSELECT_MODEL`—Specifies that the top level model parameters can be selected.
 - `PARAMSELECT_PART`—Specifies that parameters of any part(at any level of the top model) can be selected.
 - `PARAMSELECT_ASM`—Specifies that parameters of any assembly (at any level of the top model) can be selected.
 - `PARAMSELECT_FEATURE`—Specifies that parameters of any feature can be selected.
 - `PARAMSELECT_EDGE`—Specifies that parameters of any edge can be selected.
 - `PARAMSELECT_SURFACE`—Specifies that parameters of any surface can be selected.
 - `PARAMSELECT_QUILT`—Specifies that parameters of any quilt can be selected.
 - `PARAMSELECT_CURVE`—Specifies that parameters of any curve can be selected.
 - `PARAMSELECT_COMPOSITE_CURVE`—Specifies that parameters of any composite curve can be selected.
 - `PARAMSELECT_INHERITED`—Specifies that parameters of any inheritance feature can be selected.
 - `PARAMSELECT_SKELETON`—Specifies that parameters of any skeleton can be selected.
 - `PARAMSELECT_COMPONENT`—Specifies that parameters of any component can be selected.
 - `PARAMSELECT_ALLOW_SUBITEM_SELECTION`
- **AllowMultipleSelections**—This Boolean attribute indicates whether or not to allow the selection of multiple parameters from the dialog box, or

only a single parameter. By default, it is true and allows for the selection of multiple parameters.

Use the property

`pfcParameterSelectionOptions.AllowMultipleSelections` to modify this attribute.

- **SelectButtonLabel**—The visible label for the select button in the dialog box.

Use the property

`pfcParameterSelectionOptions.SelectButtonLabel` to set the label. If not set, the default label in the language of the active Creo Parametric session is displayed.

Parameter Information

Methods and Properties Introduced:

- **pfcBaseParameter.Value**
- **pfcBaseParameter.IsDesignated**
- **pfcBaseParameter.IsModified**
- **pfcBaseParameter.ResetFromBackup()**
- **pfcBaseParameter.IsRelationDriven**
- **pfcParameter.GetScaledValue()**
- **pfcParameter.SetScaledValue()**
- **pfcParameter.Units()**
- **pfcParameter.Description**
- **pfcParameter.GetRestriction**
- **pfcParameter.GetDriverType**
- **pfcParameter.Reorder()**
- **pfcParameter.Delete()**
- **pfcNamedModelItem.Name()**

Parameters inherit methods from the `pfcBaseParameter`, `pfcParameter` and `pfcNamedModelItem` classes.

You can access the designation status of the parameter using the property `pfcBaseParameter.IsDesignated`.

The property `pfcBaseParameter.IsModified` enables you to identify a modified parameter or dimension, and reset it to the last stored value. A parameter is said to be "modified" when the value changes but the owner of the parameter has not yet been regenerated.

The method `pfcBaseParameter.ResetFromBackup()` resets the parameter to the value it had before it was last set in the current Creo Parametric.

The property `pfcBaseParameter.IsRelationDriven` identifies whether the part or assembly relations control a dimension.

The method `pfcParameter.GetScaledValue()` returns the parameter value in the units of the parameter, instead of the units of the owner model.

The method `pfcParameter.SetScaledValue()` assigns the parameter value in the units provided, instead of using the units of the owner model.

The method `pfcParameter.Units()` returns the units assigned to the parameter.

The property `pfcParameter.Description` returns the parameter description, or null, if no description is assigned.

The property `pfcParameter.GetRestriction` identifies if the value of the parameter is restricted to a certain range or enumeration. It returns the `pfcParameterRestriction` object. Refer to the section [Parameter Restrictions on page 192](#) for more information.

The property `pfcParameter.GetDriverType` returns the driver type for a material parameter using the enumerated data type `pfcParameterDriverType`. A list of driver types including a description of each parameter value ,follows:

- `PARAMDRIVER_PARAM`—Specifies that the parameter value is driven by another parameter.
- `PARAMDRIVER_FUNCTION`—Specifies that the parameter value is driven by a function.
- `PARAMDRIVER_RELATION`—Specifies that the parameter value is driven by a relation. This is equivalent to the value obtained using `pfcBaseParameter.IsRelationDriven` for a parameter object type.

The method `pfcParameter.Reorder()` reorders the given parameter to come immediately after the indicated parameter in the **Parameter** dialog box and information files generated by Creo Parametric.

The method `pfcParameter.Delete()` permanently removes a specified parameter.

The property `pfcNamedModelItem.Name` accesses the name of the specified parameter.

Parameter Restrictions

In Creo Parametric users can assign specified limitations to the value for a given parameter wherever the parameter appears in the model. You can only read the details of the permitted restrictions from Creo.JS, but not modify the permitted values or range of values. Parameter restrictions in Creo.JS are represented by the class `pfcParameterRestriction`.

Method Introduced:

- **`pfcParameterRestriction.Type()`**

The method `pfcParameterRestriction.Type()` returns the the types of parameter restrictions using the enumerated data type `pfcRestrictionType`. The parameter restrictions are of the following types:

- `PARAMSELECT_ENUMERATION`—Specifies that the parameter is restricted to a list of permitted values.
- `PARAMSELECT_RANGE`—Specifies that the parameter is limited to a specified range of numeric values.

Enumeration Restriction

The `PARAMSELECT_ENUMERATION` type of parameter restriction is represented by the class `pfcParameterEnumeration`. It is a child of the `pfcParameterRestriction` class.

Property Introduced:

- **`pfcParameterEnumeration.PermittedValues`**

The property `pfcParameterEnumeration.PermittedValues` returns a list of permitted parameter values for this restriction in the form of a sequence of the `pfcParamValues` object.

Range Restriction

The `PARAMSELECT_RANGE` type of parameter restriction is represented by the interface `pfcParameterRange`. It is a child of the `pfcParameterRestriction` interface.

Properties Introduced:

- **`pfcParameterRange.Maximum`**
- **`pfcParameterRange.Minimum`**
- **`pfcParameterLimit.Type`**
- **`pfcParameterLimit.Value`**

The property `pfcParameterRange.Maximum` returns the maximum value limit for the parameter in the form of the `pfcParameterLimit` object.

The property `pfcParameterRange.Minimum` returns the minimum value limit for the parameter in the form of the `pfcParameterLimit` object.

The property `pfcParameterLimit.Type` returns the the types of parameter limits using the enumerated data type `pfcParameterLimitType`. The parameter limits are of the following types:

- `PARAMLIMIT_LESS_THAN`—Specifies that the parameter must be less than the indicated value.
- `PARAMLIMIT_LESS_THAN_OR_EQUAL`—Specifies that the parameter must be less than or equal to the indicated value.
- `PARAMLIMIT_GREATER_THAN`—Specifies that the parameter must be greater than the indicated value.
- `PARAMLIMIT_GREATER_THAN_OR_EQUAL`—Specifies that the parameter must be greater than or equal to the indicated value.

The property `pfcParameterLimit.Value` returns the boundary value of the parameter limit in the form of the `pfcParamValue` object.

Dimension Objects

Dimension objects include standard Creo Parametric dimensions and reference dimensions. With dimension objects you can access dimension tolerances to set the value for the dimension. You cannot do either of these actions with reference dimensions.

Getting Dimensions

Dimensions and reference dimensions are Creo Parametric model items. See the section [Getting ModelItem Objects on page 145](#) for properties that can return `pfcDimension` object.

Dimension Information

Properties Introduced:

- `pfcBaseParameter.Value`
- `pfcBaseDimension.DimValue`
- `pfcBaseDimension.DimType`
- `pfcBaseDimension.Symbol`
- `pfcBaseDimension.Texts`

See the section [Parameter Objects on page 187](#) for brief descriptions.

Note

You cannot set the value or designation status of reference dimension objects.

The property `pfcBaseParameter.Value` returns the value of the parameter or dimension.

The property `pfcBaseDimension.DimValue` accesses the dimension value as a double. This property provides a shortcut for accessing the values for the dimensions without using a `pfcParamValue` object.

The property `pfcBaseDimension.DimType` returns the type of the dimension such as linear, radial, angular, or diametrical, using the enumerated data type `pfcDimensionType`. The valid values and a description of the dimension that each value specifies follow:::

- `DIM_LINEAR`—Specifies if the dimension is linear.
- `DIM_RADIAL`—Specifies if the dimension is radial.
- `DIM_DIAMETER`—Specifies if the dimension is diametrical.
- `DIM_ANGULAR`—Specifies if the dimension is angular.

The property `pfcBaseDimension.Symbol` returns the dimension or reference dimension symbol (that is, “d#” or “rd#”).

The property `pfcBaseDimension.Texts` allows access to the text strings that precede or follow the dimension value.

Dimension Tolerances

Methods and Properties Introduced:

- **`pfcDimension.Tolerance`**
- **`pfcDimTolPlusMinus.Create()`**
- **`pfcDimTolSymmetric.Create()`**
- **`pfcDimTolLimits.Create()`**
- **`pfcDimTolSymSuperscript.Create()`**
- **`pfcDimTolISODIN.Create()`**

Only true dimension objects can have geometric tolerances.

The property `pfcDimension.Tolerance` enables you to access the dimension tolerance using the enumerated data type `pfcDimToleranceType` and the valid values are:

- `DIMTOL_LIMITS`—Displays dimension tolerances as upper and lower limits.

 Note

This format is not available when only the tolerance value for a dimension is displayed.

- `DIMTOL_PLUS_MINUS`—Displays dimensions as nominal with plus-minus tolerances. The positive and negative values are independent.
- `DIMTOL_SYMMETRIC`—Displays dimensions as nominal with a single value for both the positive and the negative tolerance.
- `DIMTOL_SYMMETRIC_SUPERSCRIPT`—Displays dimensions as nominal with a single value for positive and negative tolerance. The text of the tolerance is displayed in a superscript format with respect to the dimension text.
- `DIMTOL_ISODIN`—Displays the tolerance table type, table column, and table name, if the dimension tolerance is set to a hole or shaft table (DIN/ISO standard).
- —

A null value is similar to the nominal option in Creo Parametric.

To determine whether a given tolerance is plus/minus, symmetric, limits, or superscript use TBD.

Use the method `pfcDimTolPlusMinus.Create()` a new `DimTolPlusMinus` object. The input parameters are as follows:

- *inPlus*—Specifies the tolerance amount above the nominal value
- *inMinus*—Specifies the tolerance amount below the nominal value

Use the method `pfcDimTolSymmetric.Create()` to create a symmetric tolerance. The input parameter *inValue* specifies the tolerance value.

Use the method `pfcDimTolLimits.Create()` to create an object containing a limits-type tolerance for a dimension. The input parameters are as follows:

- *inUpperLimit*—Specifies the upper limit for the tolerance.
- *inLowerLimit*—Specifies the lower limit for the tolerance.

Use the method `pfcDimTolSymSuperscript.Create()` creates a symmetric tolerance in superscript format. The input parameter *inValue* is the tolerance value.

Use the method `pfcDimTolISODIN.Create()` creates a `pfcDimTolISODIN` object. The input parameters are as follows:

- *inTolTableType*—Specifies the tolerance table type using the enumerated data type `pfcToleranceTableType` and the valid values are:
 - `TOLTABLE_GENERAL`—Specifies the dimension tolerance type as General.
 - `TOLTABLE_BROKEN_EDGE`—Specifies the dimension tolerance type as Broken Edge
 - `TOLTABLE_SHAFTS`—Specifies the dimension tolerance type as Shafts
 - `TOLTABLE_HOLES`—Specifies the dimension tolerance type as Holes
- *inTableName*—Specifies the tolerance table name.
- *inTableColumn*—Specifies the tolerance table column.

Relations

Relations

This section describes how to access relations on all models and model items in Creo Parametric using the methods provided in Creo.JS.

Accessing Relations

In Creo.JS, the set of relations on any model or model item is represented by the `pfcRelationOwner` class. Models, features, surfaces, and edges inherit from this interface, because each object can be assigned relations in Creo Parametric.

Methods and Properties Introduced:

- **`pfcRelationOwner.RegenerateRelations()`**
- **`pfcRelationOwner.DeleteRelations()`**
- **`pfcRelationOwner.Relations`**
- **`pfcRelationOwner.EvaluateExpression()`**

The method `pfcRelationOwner.RegenerateRelations()` regenerates the relations assigned to the owner item. It also determines whether the specified relation set is valid.

The method `pfcRelationOwner.DeleteRelations()` deletes all the relations assigned to the owner item.

The property `pfcRelationOwner.Relations` returns the list of initial relations assigned to the owner item as an array of strings.

The method `pfcRelationOwner.EvaluateExpression()` evaluates the given relations-based expression, and returns the resulting value in the form of the `pfcParamValue` object. Refer to the topic [The ParamValue Object on page 185](#) in the section [Dimensions and Parameters on page 185](#) for more information on this object.

Accessing Post Regeneration Relations

Method and Property Introduced:

- **`pfcModel.PostRegenerationRelations`**
- **`pfcModel.RegeneratePostRegenerationRelations()`**
- **`pfcModel.DeletePostRegenerationRelations()`**

The property `pfcModel.PostRegenerationRelations` lists the postregeneration relations assigned to the model. It can be `NULL`, if not set.

Note

To work with postregeneration relations, use the postregeneration relations attribute in the methods `pfcRelationOwner.RegenerateRelations()` and `pfcRelationOwner.DeleteRelations()`.

You can regenerate the relation sets post-regeneration in a model using the method `pfcModel.RegeneratePostRegenerationRelations()`.

To delete all the post-regeneration relations in the specified model, call the method `pfcModel.DeletePostRegenerationRelations()`.

Assemblies and Components

Assemblies and Components

This section describes the Creo.JS functions that access the functions of a Creo Parametric assembly. You must be familiar with the following before you read this section:

- The Selection Object
- Coordinate Systems
- The Geometry section

Structure of Assemblies and Assembly Objects

The object `pfcAssembly` is an instance of `pfcSolid`. The `pfcAssembly` object can therefore be used as input to any of the `pfcSolid` and `pfcModel` property applicable to assemblies. However assemblies do not contain solid geometry items. The only geometry in the assembly is datums points, planes, axes, coordinate systems, curves, and surfaces. Therefore solid assembly features such as holes and slots will not contain active surfaces or edges in the assembly model.

The solid geometry of an assembly is contained in its components. A component is a feature of type `pfcComponentFeat`, which is a reference to a part or another assembly, and a set of parametric constraints for determining its geometrical location within the parent assembly.

Assembly features that are solid, such as holes and slots, and therefore affect the solid geometry of parts in the assembly hierarchy, do not themselves contain the geometry items that describe those modifications. These items are always contained in the parts whose geometry is modified, within local features created for that purpose.

The important Creo.JS functions for assemblies are those that operate on the components of an assembly. The object `pfcComponentFeat`, which is an instance of `pfcFeature` is defined for that purpose. Each assembly component is treated as a variety of feature, and the integer identifier of the component is also the feature identifier.

An assembly can contain a hierarchy of assemblies and parts at many levels, in which some assemblies and parts may appear more than once. To identify the role of any database item in the context of the root assembly, it is not sufficient to have the integer identifier of the item and the handle to its owning part or assembly, as would be provided by its `pfcFeature` description.

It is also necessary to give the full path of the assembly-component references down from the root assembly to the part or assembly that owns the database item. This is the purpose of the object `pfcComponentPath`, which is used as the input to Creo.JS assembly functions.

The following figure shows an assembly hierarchy with two examples of the contents of a `pfcComponentPath` object.

-
- **pfcComponentFeat.IsPlaced**
 - **pfcComponentFeat.IsPackaged**
 - **pfcComponentFeat.IsUnderconstrained**
 - **pfcComponentFeat.IsFrozen**
 - **pfcComponentFeat.Position**
 - **pfcComponentFeat.CopyTemplateContents**
 - **pfcComponentFeat.CreateReplaceOp()**

The property `pfcComponentFeat.IsBulkitem` identifies whether an assembly component is a bulk item. A bulk item is a non-geometric assembly feature that should appear in an assembly bill of materials.

The property `pfcComponentFeat.IsSubstitute` returns a true value if the component is substituted, or else it returns a false. When you substitute a component in a simplified representation, you temporarily exclude the substituted component and superimpose the substituting component in its place.

The property `pfcComponentFeat.CompType` returns the type of the assembly component.

The property `pfcComponentFeat.ModelDescr` returns the model descriptor of the component part or subassembly.

Note

The property `pfcComponentFeat.ModelDescr` throws an exception `pfcXtoolkitCantOpen` if called on an assembly component whose immediate generic is not in session. Handle this exception and typecast the assembly component as `pfcSolid`, which in turn can be typecast as `pfcFamilyMember`, and use the method `pfcFamilyMember.GetImmediateGenericInfo()` to get the model descriptor of the immediate generic model. If you want to switch this off and continue to run legacy applications set the configuration option `retrieve_instance_dependencies` to `instance_and_generic_deps`.

The property `pfcComponentFeat.IsPlaced` determines whether the component is placed.

Note

After a component is constrained or packaged, it cannot be made unplaced again.

A component of an assembly that is either partially constrained or unconstrained is known as a packaged component. Use the property `pfcComponentFeat.IsPackaged` to determine if the specified component is packaged.

The property `pfcComponentFeat.IsUnderconstrained` determines if the specified component is underconstrained, that is, it possesses some constraints but is not fully constrained.

The property `pfcComponentFeat.IsFrozen` determines if the specified component is frozen. The frozen component behaves similar to the packaged component and does not follow the constraints that you specify.

The property `pfcComponentFeat.Position` retrieves the initial position of the component before constraints and movements are applied. If the component is packaged this position is the same as the actual position of the constraint. This property modifies the assembly component data but does not regenerate the assembly component. To regenerate the component, use the method `pfcComponentFeat.Regenerate()`.

The method `pfcComponentFeat.CopyTemplateContents()` copies the template model into the model of the specified component.

The method `pfcComponentFeat.CreateReplaceOp()` creates a replacement operation used to swap a component automatically with a related component. The replacement operation can be used as an argument to `pfcSolid.ExecuteFeatureOps()`.

Regenerating an Assembly Component

Method Introduced:

- **`pfcComponentFeat.Regenerate()`**

The method `pfcComponentFeat.Regenerate()` regenerates an assembly component. The method regenerates the assembly component just as in an interactive Creo Parametric session.

Creating a Component Path

Method Introduced

- **`pfcCreateComponentPath()`**

The method `pfcCreateComponentPath()` returns a component path object, given the Assembly model and the integer id path to the desired component.

Component Path Information

Methods and Properties Introduced:

- **`pfcComponentPath.Root`**

-
- **pfcComponentPath.ComponentIds**
 - **pfcComponentPath.Leaf**
 - **pfcComponentPath.GetTransform**
 - **pfcComponentPath.SetTransform**
 - **pfcComponentPath.GetIsVisible()**

The property `pfcComponentPath.Root` returns the assembly at the head of the component path object.

The property `pfcComponentPath.ComponentIds` returns the sequence of ids which is the path to the particular component.

The property `pfcComponentPath.Leaf` returns the solid model at the end of the component path.

The method `pfcComponentPath.GetTransform` returns the coordinate system transformation between the assembly and the particular component. It has an option to provide the transformation from bottom to top, or from top to bottom. This method describes the current position and the orientation of the assembly component in the root assembly.

The method `pfcComponentPath.SetTransform` applies a temporary transformation to the assembly component, similar to the transformation that takes place in an exploded state. The transformation is only applied if the assembly is using `DynamicPositioning`.

The method `pfcComponentPath.GetIsVisible()` identifies if a particular component is visible in any simplified representation.

Assembling Components

Methods Introduced:

- **pfcAssembly.AssembleComponent()**
- **pfcAssembly.AssembleByCopy()**
- **pfcComponentFeat.GetConstraints()**
- **pfcComponentFeat.SetConstraints()**
- **pfcComponentFeat.GetConstraintsWithCompPath()**

The method `pfcAssembly.AssembleComponent()` adds a specified component model to the assembly at the specified initial position. The position is specified in the format defined by the class `pfcTransform3D`. Specify the orientation of the three axes and the position of the origin of the component coordinate system, with respect to the target assembly coordinate system.

 Note

If the transform matrix passed as the initial position of the component is incorrect and non-orthonormal, the method `pfcAssembly.AssembleComponent()` throws the exception `pfcXToolkitBadInputs`. In such scenario, you can use the method `pfcMakeMatrixOrthonormal()` to convert this non-orthonormal matrix to an orthonormal matrix.

The method `pfcAssembly.AssembleByCopy()` creates a new component in the specified assembly by copying from the specified component. If no model is specified, then the new component is created empty. The input parameters for this method follow:

- *LeaveUnplaced*—If true the component is unplaced. If false the component is placed at a default location in the assembly. Unplaced components belong to an assembly without being assembled or packaged. These components appear in the Model Tree, but not in the graphic window. Unplaced components can be constrained or packaged by selecting them from the Model Tree for redefinition. When its parent assembly is retrieved into memory, an unplaced component is also retrieved.
- *ModelToCopy*—Specify the model to be copied into the assembly
- *NewModelName*—Specify a name for the copied model

The method `pfcComponentFeat.GetConstraints()` retrieves the constraints for a given assembly component.

The method `pfcComponentFeat.SetConstraints()` allows you to set the constraints for a specified assembly component. The input parameters for this method follow:

- *Constraints*—Constraints for the assembly component. These constraints are explained in detail in the later sections.
- *ReferenceAssembly*—The path to the owner assembly, if the constraints have external references to other members of the top level assembly. If the constraints are applied only to the assembly component then the value of this parameter should be null.

This method modifies the component feature data and regenerates the assembly component.

The method `pfcComponentFeat.GetConstraintsWithCompPath()` retrieves the constraints for a given assembly component using the input argument *CompPath* which is the path to the owner assembly. Pass this input argument *CompPath*, if the constraints have references to other members of the top level assembly. Pass it as `Null`, if the constraints have references only to the owner assembly.

Constraint Attributes

Method and Properties Introduced:

- **`pfcConstraintAttributes.Create()`**
- **`pfcConstraintAttributes.Force`**
- **`pfcConstraintAttributes.Ignore`**

The method `pfcConstraintAttributes.Create()` returns the constraint attributes object based on the values of the following input parameters:

- *Ignore*—Constraint is ignored during regeneration. Use this capability to store extra constraints on the component, to quickly toggle between different constraints.
- *Force*—Constraint is forced for line and point alignment.

Use the property `pfcConstraintAttributes.Force` to force execution of a constraint for line and point alignment.

Use the property `pfcConstraintAttributes.Ignore` to ignore a constraint during regeneration.

Assembling a Component Parametrically

You can position a component relative to its neighbors (components or assembly features) so that its position is updated as its neighbors move or change. This is called parametric assembly. In Creo Parametric you can specify constraints to determine how and where the component relates to the assembly. You can add as many constraints as you need to make sure that the assembly meets the design intent.

Properties Introduced:

- **`pfcComponentConstraint.Create()`**
- **`pfcComponentConstraint.Type`**
- **`pfcComponentConstraint.AssemblyReference`**
- **`pfcComponentConstraint.AssemblyDatumSide`**

- **pfcComponentConstraint.ComponentReference**
- **pfcComponentConstraint.ComponentDatumSide**
- **pfcComponentConstraint.Offset**
- **pfcComponentConstraint.Attributes**
- **pfcComponentConstraint.UserDefinedData**

The method `pfcComponentConstraint.Create()` returns the component constraint object. Specify the input parameter *Type* using the enumerated data type `pfcComponentConstraintType` which has the following valid values:

- `ASM_CONSTRAINT_MATE`—Make two surfaces touch one another, that is coincident and facing each other.
- `ASM_CONSTRAINT_MATE_OFF`—Make two planar surfaces parallel and facing each other.
- `ASM_CONSTRAINT_ALIGN`—Make two planes coplanar, two axes coaxial and two points coincident. You can also align revolved surfaces or edges.
- `ASM_CONSTRAINT_ALIGN_OFF`—Align two planar surfaces at an offset.
- `ASM_CONSTRAINT_INSERT`—Insert a "male" revolved surface into a "female" revolved surface, making their respective axes coaxial.
- `ASM_CONSTRAINT_ORIENT`—Make two planar surfaces to be parallel in the same direction.
- `ASM_CONSTRAINT_CSYS`—Place a component in an assembly by aligning the coordinate system of the component with the coordinate system of the assembly.
- `ASM_CONSTRAINT_TANGENT`—Control the contact of two surfaces at their tangents.
- `ASM_CONSTRAINT_PNT_ON_SRF`—Control the contact of a surface with a point.
- `ASM_CONSTRAINT_EDGE_ON_SRF`—Control the contact of a surface with a straight edge.
- `ASM_CONSTRAINT_DEF_PLACEMENT`—Align the default coordinate system of the component to the default coordinate system of the assembly.
- `ASM_CONSTRAINT_SUBSTITUTE`—Use in simplified representations when a component has been substituted with some other model
- `ASM_CONSTRAINT_PNT_ON_LINE`—Control the contact of a line with a point.
- `ASM_CONSTRAINT_FIX`—Force the component to remain in its current packaged position.

-
- `ASM_CONSTRAINT_AUTO`—Use this option in the user interface for an automatic choice of constraint type based on the references.
 - `ASM_CONSTRAINT_ALIGN_ANG_OFF`
 - `ASM_CONSTRAINT_MATE_ANG_OFF`
 - `ASM_CONSTRAINT_CSYS_PNT`
 - `ASM_CONSTRAINT_LINE_NORMAL`
 - `ASM_CONSTRAINT_LINE_COPLANAR`
 - `ASM_CONSTRAINT_LINE_PARL`
 - `ASM_CONSTRAINT_LINE_DIST`
 - `ASM_CONSTRAINT_PNT_DIST`
 - `ASM_CONSTRAINT_INSERT_NORM`
 - `ASM_CONSTRAINT_INSERT_PARL`
 - `ASM_CONSTRAINT_PNT_ON_LINE_DIST`
 - `ASM_CONSTRAINT_PNT_ON_SRF_DIST`
 - `ASM_CONSTRAINT_EDGE_ON_SRF_DIST`
 - `ASM_CONSTRAINT_EDGE_ON_SRF_ANG`
 - `ASM_CONSTRAINT_EDGE_ON_SRF_NORMAL`
 - `ASM_CONSTRAINT_ALIGN_NODEP_ANGLE`
 - `ASM_CONSTRAINT_MATE_NODEP_ANGLE`
 - `ASM_CONSTRAINT_LINE_ANGLE`
 - `ASM_CONSTRAINT_EDGE_ON_SRF_PARL`
 - `ASM_CONSTRAINT_SRF_NORMAL`
 - `ASM_CONSTRAINT_EXPLICIT`

The properties `pfcComponentConstraint.AssemblyReference` and `pfcComponentConstraint.ComponentReference` return the geometric reference from the assembly and the placed component, respectively. This reference must be created using an assembly component path.

Use `null` for constraint types that do not require a component reference.

The properties `pfcComponentConstraint.AssemblyDatumSide` and `pfcComponentConstraint.ComponentDatumSide` return the orientation of the assembly and assembly component, respectively.

The enumerated data type `pfcDatumSide` represents the datum plane side to use when calculating assembly component constraints. The valid values are:

- `DATUM_SIDE_NONE`—Specifies if the constraint does not involve a datum plane.

-
- `DATUM_SIDE_RED`—Specifies the Secondary (formerly red) side of the datum plane.
 - `DATUM_SIDE_YELLOW`—Specifies the Primary (formerly yellow) side of the datum plane.

The property `pfcComponentConstraint.Offset` returns the mate or align offset value from the reference.

The property `pfcComponentConstraint.Attributes` returns the constraint attributes for a given constraint

The property `pfcComponentConstraint.UserDefinedData` returns a string that specifies user data for the given constraint.

Redefining and Rerouting Assembly Components

Use these functions to reroute previously assembled components, just as you would in an interactive Creo Parametric session.

Methods Introduced:

- **`pfcComponentFeat.RedefineThroughUI()`**
- **`pfcComponentFeat.MoveThroughUI()`**

The method `pfcComponentFeat.RedefineThroughUI()` must be used in interactive Creo.JS applications. This method displays the Creo Parametric **Constraint** dialog box. This enables the end user to redefine the constraints interactively. The control returns to Creo.JS application when the user selects **OK** or **Cancel** and the dialog box is closed.

The method `pfcComponentFeat.MoveThroughUI()` invokes a dialog box that prompts the user to interactively reposition the components. This interface enables the user to specify the translation and rotation values. The control returns to Creo.JS application when the user selects **OK** or **Cancel** and the dialog box is closed.

Exploded Assemblies

Use these methods and properties to determine and change the explode status of the assembly object.

Methods and Properties Introduced:

- **`pfcAssembly.IsExploded`**
- **`pfcAssembly.Explode()`**
- **`pfcAssembly.UnExplode()`**
- **`pfcAssembly.GetActiveExplodedState()`**
- **`pfcAssembly.GetDefaultExplodedState()`**

- **pfcExplodedState.Activate()**

The methods `pfcAssembly.Explode()` and `pfcAssembly.UnExplode()` enable you to determine and change the explode status of the assembly object.

The property `pfcAssembly.IsExploded` reports whether the specified assembly is currently exploded. Use this property in the assembly mode only. The exploded status of an assembly depends on the mode. If an assembly is opened in the drawing mode, the state of the assembly in the drawing view is displayed. The drawing view does not represent the actual exploded state of the assembly.

The method `pfcAssembly.GetActiveExplodedState()` returns the current active explode state.

The method `pfcAssembly.GetDefaultExplodedState()` returns the default explode state.

The method `pfcExplodedState.Activate()` activates the specified explode state representation.

Skeleton Models

Skeleton models are a 3-dimensional layout of the assembly. These models are holders or distributors of critical design information, and can represent space requirements, important mounting locations, and motion.

Methods Introduced:

- **pfcAssembly.AssembleSkeleton()**
- **pfcAssembly.AssembleSkeletonByCopy()**
- **pfcAssembly.GetSkeleton()**
- **pfcAssembly.DeleteSkeleton()**
- **pfcSolid.IsSkeleton**

The method `pfcAssembly.AssembleSkeleton()` adds an existing skeleton model to the specified assembly.

The method `pfcAssembly.GetSkeleton()` returns the skeleton model of the specified assembly.

The method `pfcAssembly.DeleteSkeleton()` deletes a skeleton model component from the specified assembly.

The method `pfcAssembly.AssembleSkeletonByCopy()` adds a specified skeleton model to the assembly. The input parameters are:

- *SkeletonToCopy*—Specify the skeleton model to be copied into the assembly
- *NewSkeletonName*—Specify a name for the copied skeleton model

The property `pfcSolid.IsSkeleton` determines if the specified part model is a skeleton model or a concept model. It returns a true if the model is a skeleton or else it returns a false.

Family Tables

Family Tables

This section describes how to use `Creo.JSCreo.JS` classes and methods to access and manipulate family table information.

Working with Family Tables

Creo.JS provides several methods for accessing family table information. Because every model inherits from the class `pfcFamilyMember`, every model can have a family table with which it is associated.

Accessing Instances

Methods and Properties Introduced:

- `pfcFamilyMember.Parent`
- `pfcFamilyMember.GetImmediateGenericInfo`
- `pfcFamilyMember.GetTopGenericInfo()`
- `pfcFamilyTableRow.CreateInstance()`
- `pfcFamilyMember.ListRows()`
- `pfcFamilyMember.GetRow()`
- `pfcFamilyMember.RemoveRow()`
- `pfcFamilyTableRow.InstanceName`
- `pfcFamilyTableRow.IsLocked`
- `pfcFamilyTableRow.IsExtLocked`
- `pfcFamilyTableRow.IsVerified`

To get the generic model for an instance, call the property `pfcFamilyMember.Parent`.

The property `pfcFamilyMember.Parent` has changed as a result of performance improvement in family table retrieval mechanism. Call the property `pfcFamilyMember.Parent`, it throws an exception `pfcXToolkitCantOpen`, if the immediate generic of a model instance in a nested family table is currently not in session. Handle this exception and use the

property `pfcFamilyMember.GetImmediateGenericInfo` to get the model descriptor of the immediate generic model. You can use this information to retrieve the immediate generic model.

To get the model descriptor of the top generic model, call the method `pfcFamilyMember.GetTopGenericInfo()`.

Similarly, the method `pfcFamilyMember.GetTopGenericInfo()` returns an instance model created from the information stored in the `pfcFamilyTableRow` object.

The method `pfcFamilyMember.ListRows()` returns an array of all rows in the family table, whereas `pfcFamilyMember.GetRow()` gets the row object with the name you specify.

Use the method `pfcFamilyMember.RemoveRow()` to permanently delete the row from the family table.

The property `pfcFamilyTableRow.InstanceName` returns the name that corresponds to the invoking row object.

To control whether the instance can be changed or removed, call the property `pfcFamilyTableRow.IsLocked`.

The property `pfcFamilyTableRow.IsExtLocked` specifies if the instance is locked by external application.

The property `pfcFamilyTableRow.IsVerified` specifies the verification status of the instance.

Accessing Columns

Methods and Properties Introduced:

- **`pfcFamilyMember.ListColumns()`**
- **`pfcFamilyMember.GetColumn()`**
- **`pfcFamilyMember.RemoveColumn()`**
- **`pfcFamilyTableColumn.Symbol`**
- **`pfcFamilyTableColumn.Type`**
- **`pfcFamColModelItem.RefItem`**
- **`pfcFamColParam.RefParam()`**

The method `pfcFamilyMember.ListColumns()` returns an array of all columns in the family table.

The method `pfcFamilyMember.GetColumn()` returns a family table column, given its symbolic name.

To permanently delete the column from the family table and all changed values in all instances, call the method `pfcFamilyMember.RemoveColumn()`.

The property `pfcFamilyTableColumn.Symbol` returns the string symbol at the top of the column, such as D4 or F5.

The property `pfcFamilyTableColumn.Type` returns the type of the column using the enumerated data type `pfcFamilyColumnType`. The values indicate the type of parameter governed by the column in the family table.

The property `pfcFamColModelItem.RefItem` returns the `pfcModelItem` (`pfcFeature` or `pfcDimension`) controlled by the column, whereas `pfcFamColParam.RefParam()` returns the `Parameter` controlled by the column.

Accessing Cell Information

Properties Introduced:

- **`pfcFamilyMember.GetCell()`**
- **`pfcFamilyMember.GetCellIsDefault`**
- **`pfcFamilyMember.SetCell()`**
- **`pfcParamValue.StringValue`**
- **`pfcParamValue.IntValue`**
- **`pfcParamValue.DoubleValue`**
- **`pfcParamValue.BoolValue`**

The method `pfcFamilyMember.GetCell()` returns a string `pfcParamValue` that corresponds to the cell at the intersection of the row and column arguments. Use the property `pfcFamilyMember.GetCellIsDefault` to check if the value of the specified cell is the default value, which is the value of the specified cell in the generic model.

The method `pfcFamilyMember.SetCell()` assigns a value to a column in a particular family table instance.

The properties `pfcParamValue.StringValue`, `pfcParamValue.IntValue`, `pfcParamValue.DoubleValue`, and `pfcParamValue.BoolValue` are used to get the different types of parameter values.

Creating Family Table Instances

Methods Introduced:

- **`pfcFamilyMember.AddRow()`**
- **`pfcCreateStringParamValue()`**
- **`pfcCreateIntParamValue()`**

-
- **pfcCreateDoubleParamValue()**
 - **pfcCreateBoolParamValue()**

Use the method `pfcFamilyMember.AddRow()` to create a new instance with the specified name, and, optionally, the specified values for each column. If you do not pass in a set of values, the value `*` is assigned to each column. This value indicates that the instance uses the generic value.

Creating Family Table Columns

Methods Introduced:

- **pfcFamilyMember.CreateDimensionColumn()**
- **pfcFamilyMember.CreateParamColumn()**
- **pfcFamilyMember.CreateFeatureColumn()**
- **pfcFamilyMember.CreateComponentColumn()**
- **pfcFamilyMember.CreateCompModelColumn()**
- **pfcFamilyMember.CreateGroupColumn()**
- **pfcFamilyMember.CreateMergePartColumn()**
- **pfcFamilyMember.CreateColumn()**
- **pfcFamilyMember.AddColumn()**
- **pfcCreateStringParamValue()**

The above methods initialize a column based on the input argument. These methods assign the proper symbol to the column header.

The method `pfcFamilyMember.CreateColumn()` creates a new column given a properly defined symbol and column type. The results of this call should be passed to the method `pfcFamilyMember.AddColumn()` to add the column to the family table for the model.

The method `pfcFamilyMember.AddColumn()` adds the column to the family table. You can specify the values; if you pass nothing for the values, the method assigns the value `*` to each instance to accept the default value in the column.

Action Listeners

Action Listeners

This section describes the Creo.JS methods that enable you to use action listeners.

The Creo.JS Action Listeners

An `ActionListener` is a class that is assigned to respond to certain events. In Creo.JS, you can assign action listeners to respond to events involving the following tasks:

- Changing windows
- Changing working directories
- Model operations
- Regenerating
- Creating, deleting, and redefining features
- Checking for regeneration failures

All action listeners in Creo.JS are defined by these classes:

- Interface—Named `<Object>ActionListener`. This interface defines the methods that can respond to various events.
- Default class—Named `Default<Object>ActionListener`. This class has every available method overridden by an empty implementation. You create your own action listeners by extending the default class and overriding the methods for events that interest you.



Note

When notifications are set in Creo.JS applications, every time an event is triggered, notification messages are added to the trail files. From Creo Parametric 2.0 M210 onward, a new environment variable `PROTK_LOG_DISABLE` enables you to disable this behavior. When set to `true`, the notifications messages are not added to the trail files.

Action Sources

Methods introduced:

- **`pfcActionSource.AddActionListener`**
- **`pfcActionSource.AddActionListenerWithType`**
- **`pfcActionSource.RemoveActionListener`**

Many Creo.JS classes inherit the `pfcActionSource` interface, but only the following classes currently make calls to the methods of registered `pfcActionListeners`:

- `pfcSession`

-
- Session Action Listener
 - Model Action Listener
 - Solid Action Listener
 - Model Event Action Listener
 - Feature Action Listener
 - `pfcUICommand`
 - UI Action Listener
 - `pfcModel` (and its subclasses)
 - Model Action Listener
 - Parameter Action Listener
 - `pfcSolid` (and its subclasses)
 - Solid Action Listener
 - Feature Action Listener
 - `pfcFeature` (and its subclasses)
 - Feature Action Listener

 Note

- Assigning an action listener to a source not related to it will not cause an error but the listener method will never be called.
 - The method `pfcActionSource.AddActionListener` adds action listeners to notify you of all the events.
 - The method `pfcActionSource.AddActionListenerWithType` adds action listeners to notify you about events that are specified in the input argument *ATypes*.
-

Types of Action Listeners

The following sections describe the different kinds of action listeners: session, UI command, solid, and feature.

Session Level Action Listeners

Methods introduced:

- **`pfcSessionActionListener.OnAfterDirectoryChange()`**

-
- **pfcSessionActionListener.OnAfterWindowChange()**
 - **pfcSessionActionListener.OnAfterModelDisplay()**
 - **pfcSessionActionListener.OnBeforeModelErase()**
 - **pfcSessionActionListener.OnBeforeModelDelete()**
 - **pfcSessionActionListener.OnBeforeModelRename()**
 - **pfcSessionActionListener.OnBeforeModelSave()**
 - **pfcSessionActionListener.OnBeforeModelPurge()**
 - **pfcSessionActionListener.OnBeforeModelCopy()**
 - **pfcSessionActionListener.OnAfterModelPurge()**

The `pfcSessionActionListener.OnAfterDirectoryChange()` method activates after the user changes the working directory. This method takes the new directory path as an argument.

The `pfcSessionActionListener.OnAfterWindowChange()` method activates when the user activates a window other than the current one. Pass the new window to the method as an argument.

The `pfcSessionActionListener.OnAfterModelDisplay()` method activates every time a model is displayed in a window.

 Note

Model display events happen when windows are moved, opened and closed, repainted, or the model is regenerated. The event can occur more than once in succession.

The methods `pfcSessionActionListener.OnBeforeModelErase()`, `pfcSessionActionListener.OnBeforeModelRename`, `pfcSessionActionListener.OnBeforeModelSave`, and `pfcSessionActionListener.OnBeforeModelCopy` take special arguments. They are designed to allow you to fill in the arguments and pass this data back to Creo Parametric. The model names placed in the descriptors will be used by Creo Parametric as the default names in the user interface.

UI Command Action Listeners

Methods introduced:

- **pfcSession.UICreateCommand()**
- **pfcUICommandActionListener.OnCommand()**

The `pfcSession.UICreateCommand()` method takes a `pfcUICommandActionListener` argument and returns a `pfcUICommand` action source with that action listener already registered. This `pfcUICommand` object is subsequently passed as an argument to the `pfcSession.UIAddButton()` method that adds a command button to a Creo Parametric menu. The `pfcUICommandActionListener.OnCommand` method of the registered `pfcUICommandActionListener` is called whenever the command button is clicked.

Model Level Action listeners

Methods introduced:

- **`pfcModelActionListener.OnAfterModelSave()`**
- **`pfcModelEventActionListener.OnAfterModelCopy()`**
- **`pfcModelEventActionListener.OnAfterModelRename()`**
- **`pfcModelEventActionListener.OnAfterModelErase()`**
- **`pfcModelEventActionListener.OnAfterModelDelete()`**
- **`pfcModelActionListener.OnAfterModelRetrieve()`**
- **`pfcModelActionListener.OnBeforeModelDisplay()`**
- **`pfcModelActionListener.OnAfterModelCreate()`**
- **`pfcModelActionListener.OnAfterModelSaveAll()`**
- **`pfcModelEventActionListener.OnAfterModelCopyAll()`**
- **`pfcModelActionListener.OnAfterModelEraseAll()`**
- **`pfcModelActionListener.OnAfterModelDeleteAll()`**
- **`pfcModelActionListener.OnAfterModelRetrieveAll()`**

Methods ending in `All` are called after any event of the specified type. The call is made even if the user did not explicitly request that the action take place. Methods that do not end in `All` are only called when the user specifically requests that the event occurs.

The method `pfcModelActionListener.OnAfterModelSave()` is called after successfully saving a model.

The method `pfcModelEventActionListener.OnAfterModelCopy()` is called after successfully copying a model.

The method `pfcModelEventActionListener.OnAfterModelRename()` is called after successfully renaming a model.

The method `pfcModelEventActionListener.OnAfterModelErase()` is called after successfully erasing a model.

The method

`pfcModelEventListener.OnAfterModelDelete()` is called after successfully deleting a model.

The method `pfcModelEventListener.OnAfterModelRetrieve()` is called after successfully retrieving a model.

The method `pfcModelEventListener.OnBeforeModelDisplay()` is called before displaying a model.



Note

The method

`pfcModelEventListener.OnBeforeModelDisplay()` is not supported in asynchronous mode.

The method `pfcModelEventListener.OnAfterModelCreate()` is called after the successful creation of a model.

Solid Level Action Listeners

Methods introduced:

- **`pfcSolidActionListener.OnBeforeRegen()`**
- **`pfcSolidActionListener.OnAfterRegen()`**
- **`pfcSolidActionListener.OnBeforeUnitConvert()`**
- **`pfcSolidActionListener.OnAfterUnitConvert()`**
- **`pfcSolidActionListener.OnBeforeFeatureCreate()`**
- **`pfcSolidActionListener.OnAfterFeatureCreate()`**
- **`pfcSolidActionListener.OnAfterFeatureDelete()`**

The `pfcSolidActionListener.OnBeforeRegen()` and `pfcSolidActionListener.OnAfterRegen()` methods occur when the user regenerates a solid object within the `pfcActionSource` to which the listener is assigned. These methods take the first feature to be regenerated and a handle to the `pfcSolid` object as arguments. In addition, the method `pfcSolidActionListener.OnAfterRegenerate()` includes a Boolean argument that indicates whether regeneration was successful.

Note

- It is not recommended to modify geometry or dimensions using the `pfcSolidActionListener.OnBeforeRegenerate()` method call.
- A regeneration that did not take place because nothing was modified is identified as a regeneration failure.

The methods `pfcSolidActionListener.OnBeforeUnitConvert()` and `pfcSolidActionListener.OnAfterUnitConvert()` activate when a user modifies the unit scheme (by selecting the Creo Parametric command **Set Up, Units**). The methods receive the `Solid` object to be converted and a Boolean flag that identifies whether the conversion changed the dimension values to keep the object the same size.

Note

`pfcSolidActionListeners` can be registered with the session object so that its methods are called when these events occur for any solid model that is in session.

The `pfcSolidActionListener.OnBeforeFeatureCreate()` method activates when the user starts to create a feature that requires the **Feature Creation** dialog box. Because this event occurs only after the dialog box is displayed, it will not occur at all for datums and other features that do not use this dialog box. This method takes two arguments: the solid model that will contain the feature and the `pfcModelItem` identifier.

The `pfcSolidActionListener.OnAfterFeatureCreate()` method activates after any feature, including datums, has been created. This method takes the new `pfcFeature` object as an argument.

The `pfcSolidActionListener.OnAfterFeatureDelete()` method activates after any feature has been deleted. The method receives the solid that contained the feature and the (now defunct) `pfcModelItem` identifier.

Selection Level Action Listeners

Method introduced:

- **`pfcSelectionBufferListener.OnAfterSelBufferChange`**

The method **`pfcSelectionBufferListener.OnAfterSelBufferChange`** is called after the selection buffer is changed.

Feature Level Action Listeners

Methods introduced:

- **`pfcFeatureActionListener.OnBeforeDelete()`**
- **`pfcFeatureActionListener.OnBeforeSuppress()`**
- **`pfcFeatureActionListener.OnAfterSuppress()`**
- **`pfcFeatureActionListener.OnBeforeRegen()`**
- **`pfcFeatureActionListener.OnAfterRegen()`**
- **`pfcFeatureActionListener.OnRegenFailure()`**
- **`pfcFeatureActionListener.OnBeforeRedefine()`**
- **`pfcFeatureActionListener.OnAfterCopy()`**
- **`pfcFeatureActionListener.OnBeforeParameterDelete()`**

Each method in `pfcFeatureActionListener` takes as an argument the feature that triggered the event.

`pfcFeatureActionListeners` can be registered with the object so that the action listener's methods are called whenever these events occur for any feature that is in session or with a solid model to react to changes only in that model.

The method `pfcFeatureActionListener.OnBeforeDelete()` is called before a feature is deleted.

The method `pfcFeatureActionListener.OnBeforeSuppress()` is called before a feature is suppressed.

The method `pfcFeatureActionListener.OnAfterSuppressv` is called after a successful feature suppression.

The method `pfcFeatureActionListener.OnBeforeRegen()` is called before a feature is regenerated.

The method `pfcFeatureActionListener.OnAfterRegenv` is called after a successful feature regeneration.

The method `pfcFeatureActionListener.OnRegenFailure()` is called when a feature fails regeneration.

The method `pfcFeatureActionListener.OnBeforeRedefine()` is called before a feature is redefined.

The method `pfcFeatureActionListener.OnAfterCopy()` is called after a feature has been successfully copied.

The method `pfcFeatureActionListener.OnBeforeParameterDelete()` is called before a feature parameter is deleted.

Cancelling an ActionListener Operation

Creo.JS allows you to cancel certain notification events, registered by the action listeners.

Methods Introduced:

- **CCpfcXCancelProEAction.Throw()**

The static method `CCpfcXCancelProEAction.Throw()` must be called from the body of an action listener to cancel the impending Creo Parametric operation. This method will throw a Creo.JS exception signalling to Creo Parametric to cancel the listener event.

Note: Your application should not catch the Creo.JS exception, or should rethrow it if caught, so that Creo Parametric is forced to handle it.

The following events can be cancelled using this technique:

- `pfcSessionActionListener.OnBeforeModelErase()`
- `pfcSessionActionListener.OnBeforeModelDelete()`
- `pfcSessionActionListener.OnBeforeModelRename()`
- `pfcSessionActionListener.OnBeforeModelSave()`
- `pfcSessionActionListener.OnBeforeModelPurge()`
- `pfcSessionActionListener.OnBeforeModelCopy()`
- `pfcModelActionListener.OnBeforeParameterCreate()`
- `pfcModelActionListener.OnBeforeParameterDelete()`
- `pfcModelActionListener.OnBeforeParameterModify()`
- `pfcFeatureActionListener.OnBeforeDelete()`
- `pfcFeatureActionListener.OnBeforeSuppress()`
- `pfcFeatureActionListener.OnBeforeParameterDelete()`
- `pfcFeatureActionListener.OnBeforeParameterCreate()`
- `pfcFeatureActionListener.OnBeforeRedefine()`

Interface

Interface

This section describes various methods of importing and exporting files in Creo.JS.

Exporting Files and 2D Models

Method Introduced:

- **pfcModel.Export()**

The method `pfcModel.Export()` exports model data to a file. The exported files are placed in the current Creo Parametric working directory. The input parameters are:

- *FileName*—Output file name including extensions
- *ExportData*—The `pfcExportInstructions` object that controls the export operation. The type of data that is exported is given by the `pfcExportType` object.

There are four general categories of files to which you can export models:

- File types whose instructions inherit from `pfcGeomExportInstructions`.
These instructions export files that contain precise geometric information used by other CAD systems.
- File types whose instructions inherit from `pfcCoordSysExportInstructions`.
These instructions export files that contain coordinate information describing faceted, solid models (without datums and surfaces).
- File types whose instructions inherit from `pfcFeatIdExportInstructions`.
These instructions export information about a specific feature.
- General file types that inherit only from `pfcExportInstructions`.
These instructions provide conversions to file types such as BOM (bill of materials).

For information on exporting to a specific format, see the Creo.JS API Wizard and online help for the Creo Parametric interface.

Export Instructions

Methods Introduced:

- **pfcRelationExportInstructions.Create()**
- **pfcModelInfoExportInstructions.Create()**
- **pfcProgramExportInstructions.Create()**
- **pfcIGESFileExportInstructions.Create()**
- **pfcDXFExportInstructions.Create()**
- **pfcRenderExportInstructions.Create()**
- **pfcSTLASCIIExportInstructions.Create()**

- **pfcSTLBinaryExportInstructions.Create()**
- **pfcBOMExportInstructions.Create()**
- **pfcDWGSetupExportInstructions.Create()**
- **pfcFeatInfoExportInstructions.Create()**
- **pfcMFGFeatCExportInstructions.Create()**
- **pfcMFGOperCExportInstructions.Create()**
- **pfcMaterialExportInstructions.Create()**
- **pfcCGMFILEExportInstructions.Create()**
- **pfcInventorExportInstructions.Create()**
- **pfcFIATExportInstructions.Create()**
- **pfcConnectorParamExportInstructions.Create()**
- **pfcCableParamsFileInstructions.Create()**
- **pfcCATIAFacetsExportInstructions.Create()**
- **pfcVRMLModelExportInstructions.Create()**
- **pfcSTEP2DExportInstructions.Create()**
- **pfcMedusaExportInstructions.Create()**
- **pfcCADDSEExportInstructions.Create()**
- **pfcSliceExportData.Create()**
- **pfcNEUTRALFileExportInstructions.Create()**
- **pfcProductViewExportInstructions.Create()**
- **pfcBaseSession.ExportDirectVRML()**

Export Instructions Table

Class	Used to Export
pfcRelationExportInstructions	List of the relations and parameters in a part or assembly
pfcModelInfoExportInstructions	Information about a model, including units information, features, and children
pfcProgramExportInstructions	A program file for a part or assembly that can be edited to change the model
pfcIGESEExportInstructions	A drawing in IGES format
pfcDXFExportInstructions	A drawing in DXF format
pfcRenderExportInstructions	A part or assembly in RENDER format
pfcSTLASCIIExportInstructions	A part or assembly to an ASCII STL file
pfcSTLBinaryExportInstructions	A part or assembly in a binary STL file
pfcBOMExportInstructions	A BOM for an assembly
pfcDWGSetupExportInstructions	A drawing setup file

Class	Used to Export
<code>pfcFeatInfoExportInstructions</code>	Information about one feature in a part or assembly
<code>pfcMfgFeatCLExportInstructions</code>	A cutter location (CL) file for one NC sequence in a manufacturing assembly
<code>pfcMfgOperClExportInstructions</code>	A cutter location (CL) file for all the NC sequences in a manufacturing assembly
<code>pfcMaterialExportInstructions</code>	A material from a part
<code>pfcCGMFILEExportInstructions</code>	A drawing in CGM format
<code>pfcInventorExportInstructions</code>	A part or assembly in Inventor format
<code>pfcFIATExportInstructions</code>	A part or assembly in FIAT format
<code>pfcConnectorParamExportInstructions</code>	The parameters of a connector to a text file
<code>pfcCableParamsFileInstructions</code>	Cable parameters from an assembly
<code>pfcCATIAFacetsExportInstructions</code>	A part or assembly in CATIA format (as a faceted model)
<code>pfcVRMLModelExportInstructions</code>	A part or assembly in VRML format
<code>pfcSTEP2DExportInstructions</code>	A two-dimensional STEP format file
<code>pfcMedusaExportInstructions</code>	A drawing in MEDUSA file
<code>pfcCADDSExportInstructions</code>	A CADD5 solid model
<code>pfcNEUTRALFileExportInstructions</code>	A Creo Parametric part to neutral format
<code>pfcProductViewExportInstructions</code>	A part, assembly, or drawing in Creo View format
<code>pfcSliceExportData</code>	A slice export format

Exporting Drawing Sheets

The options required to export multiple sheets of a drawing are given by the `pfcExport2DOption` object.

Methods and Properties Introduced:

- **`pfcExport2DOption.Create()`**
- **`pfcExport2DOption.ExportSheetOption`**
- **`pfcExport2DOption.ModelSpaceSheet`**
- **`pfcExport2DOption.Sheets`**

The method `pfcExport2DOption.Create()` creates a new instance of the `pfcExport2DOption` object. This object contains the following options:

- *ExportSheetOption*—Option for exporting multiple drawing sheets. Use the property `pfcExport2DOption.ExportSheetOption` to set the option for exporting multiple drawing sheets. The options are given by the `pfcExport2DSheetOption` class. A list of types and a description of what each type exports, follow:
 - `EXPORT_CURRENT_TO_MODEL_SPACE`—Only the current sheet for the drawing, as model space to a single file. This is the default type.

-
- `EXPORT_CURRENT_TO_PAPER_SPACE`—Only the current sheet for the drawing, as paper space to a single file. This type is the same as `EXPORT_CURRENT_TO_MODEL_SPACE` for formats that do not support the concept of model space and paper space.
 - `EXPORT_ALL`—All the sheets in a drawing to a single file as paper space, if applicable for the format type.
 - `EXPORT_SELECTED`—Selected sheets in a drawing as paper space and one sheet as model space.
 - *ModelSpaceSheet*—Sheet number that needs be exported as model space. This option is applicable only if the export formats support the concept of model space and paper space and if *ExportSheetOption* is set to `EXPORT_SELECTED`. Use the property `pfcExport2DOption.ModelSpaceSheet` to set this option.
 - *Sheets*—Sheet numbers that need to be exported as paper space. This option is applicable only if *ExportSheetOption* is set to `EXPORT_SELECTED`. Use the property `pfcExport2DOption.Sheets` to set this option.

Exporting to Faceted Formats

The methods described in this section support the export of Creo Parametric drawings and solid models to faceted formats like CATIA CGR.

Methods and Properties Introduced:

- **`pfcTriangulationInstructions.AngleControl`**
- **`pfcTriangulationInstructions.ChordHeight`**
- **`pfcTriangulationInstructions.StepSize()`**
- **`pfcTriangulationInstructions.FacetControlOptions`**

The property `pfcTriangulationInstructions.AngleControl` gets the angle control for the exported facet drawings and models. You can set the value between 0.0 to 1.0.

Use the property `pfcTriangulationInstructions.ChordHeight` to get the chord height for the exported facet drawings and models.

Use the method `pfcTriangulationInstructions.StepSize()` to control the step size for the exported files. The default value is 0.0.

Note

You must pass the value of Step Size value as `NULL`, if you specify the Quality value.

The property

`pfcTriangulationInstructions.FacetControlOptions` controls the facet export options using bit flags. You can set the bit flags using the `pfcFacetControlFlag` object. It has the following values:

- `FACET_STEP_SIZE_ADJUST`—Adjusts the step size according to the component size.
- `FACET_CHORD_HEIGHT_ADJUST`—Adjusts the chord height according to the component size.
- `FACET_USE_CONFIG`—If this flag is set, values of the flags `FACET_STEP_SIZE_OFF`, `FACET_STEP_SIZE_ADJUST`, and `FACET_CHORD_HEIGHT_ADJUST` are ignored and the configuration settings from the Creo Parametric user interface are used during the export operation.
- `FACET_CHORD_HEIGHT_DEFAULT`—Uses the default value set in the Creo Parametric user interface for the chord height.
- `FACET_ANGLE_CONTROL_DEFAULT`—Uses the default value set in the Creo Parametric user interface for the angle control.
- `FACET_STEP_SIZE_DEFAULT`—Uses the default value set in the Creo Parametric user interface for the step size.
- `FACET_STEP_SIZE_OFF`—Switches off the step size control.
- `FACET_FORCE_INTO_RANGE`—Forces the out-of-range parameters into range. If any of the `FACET_*_DEFAULT` option is set, then the option `pfcFACET_FORCE_INTO_RANGE` is not applied on that parameter.
- `FACET_STEP_SIZE_FACET_INCLUDE_QUILTS`—Includes quilts in the export of Creo Parametric model to the specified format.
- `EXPORT_INCLUDE_ANNOTATIONS`—Includes annotations in the export of Creo Parametric model to the specified format.

 Note

To include annotations, during the export of Creo Parametric model, you must call the method `pfcModel.Display()` before calling `pfcModel.Export()`.

- `FACET_VISIBLE_MODELS`—Exports models which have their visibility status set to **Show** in Creo Parametric. Models that are hidden are not exported.

Exporting Using Coordinate System

The methods described in this section support the export of files with information about the faceted solid models without datums and surfaces. The files are exported in reference to the coordinate-system feature in the model being exported.

Methods and Properties Introduced:

- **pfcCoordSysExportInstructions.CsysName**
- **pfcCoordSysExportInstructions.Quality**
- **pfcCoordSysExportInstructions.MaxChordHeight**
- **pfcCoordSysExportInstructions.AngleControl**
- **pfcCoordSysExportInstructions.GetSliceExportData()**
- **pfcCoordSysExportInstructions.SetSliceExportData()**
- **pfcCoordSysExportInstructions.StepSize**
- **pfcCoordSysExportInstructions.FacetControlOptions()**
- **pfcSelection.SetIntf3DCsys()**

The property `pfcCoordSysExportInstructions.CsysName` returns the name of the name of a coordinate system feature in the model being exported. It is recommended to use the coordinate system that places the part or assembly in its upper-right quadrant, so that all position and distance values of the exported assembly or part are positive.

The property `pfcCoordSysExportInstructions.Quality` can be used instead of

`pfcModel.CoordSysExportInstructions.MaxChordHeight()` and `pfcCoordSysExportInstructions.AngleControl`. You can set the value between 1 and 10. The higher the value you pass, the lower is the Maximum Chord Height setting and higher is the Angle Control setting the method uses. The default Quality value is 1.0.



Note

You must pass the value of Quality as `NULL`, if you use Maximum Chord Height and Angle Control values. If Quality, Maximum Chord Height, and Angle Control are all `NULL`, then the Quality setting of 3 is used.

Use the property

`pfcCoordSysExportInstructions.MaxChordHeight` to work with the maximum chord height for the exported files. The default value is 0.1.



Note

You must pass the value of Maximum Chord Height as NULL, if you specify the Quality value.

Use the property `pfcCoordSysExportInstructions.AngleControl` to work with the angle control setting for the exported files. The default value is 0.1.



Note

You must pass the value of Angle Control value as NULL, if you specify the Quality value.

The methods

`pfcCoordSysExportInstructions.GetSliceExportData()` and `pfcCoordSysExportInstructions.SetSliceExportData()` get and set the `pfcSliceExportData` data object that specifies data for the slice export. The options in this object are described as follows:

- *CompIds*—Array of integers that identify the components that form the path from the root assembly down to the component part or assembly being referred to. Use the property `pfcSliceExportData.CompIds` to work with the component IDs.

The property `pfcCoordSysExportInstructions.StepSize` controls the step size for the exported files. The default value is 0.0.



Note

You must pass the value of Step Size value as NULL, if you specify the Quality value.

The property

`pfcCoordSysExportInstructions.FacetControlOptions` controls the facet export options using bit flags. You can set the bit flags using the `pfcFacetControlFlag` object. For more information on the bit flag values, please refer to the section [Exporting to Faceted Formats on page 224](#).

The function `pfcSelection.SetIntf3DCsys()` sets the reference coordinate system for the export. The input argument *ReferenceCsys* is the reference coordinate system selection. Call this method without any argument to set default coordinate system. Reference coordinate system is not supported for CADDs and NEUTRAL file types.

Exporting to PDF and U3D

The methods described in this section support the export of Creo Parametric drawings and solid models to Portable Document Format (PDF) and U3D format. You can export a drawing or a 2D model as a 2D raster image embedded in a PDF file. You can export Creo Parametric solid models in the following ways:

- As a U3D model embedded in a one-page PDF file
- As 2D raster images embedded in the pages of a PDF file representing saved views
- As a standalone U3D file

While exporting multiple sheets of a Creo Parametric drawing to a PDF file, you can export all sheets, the current sheet, or selected sheets.

With these methods you can also insert a variety of non-geometric information to improve document content, navigation, and search.

Methods Introduced:

- **`pfcPDFExportInstructions.Create()`**
- **`pfcPDFExportInstructions.FilePath`**
- **`pfcPDFExportInstructions.Options`**
- **`pfcPDFExportInstructions.ProfilePath`**
- **`pfcPDFOption.Create()`**
- **`pfcPDFOption.OptionType`**
- **`pfcPDFOption.OptionValue`**

The method `pfcPDFExportInstructions.Create()` creates a new instance of the `pfcPDFExportInstructions` data object that describes how to export Creo Parametric drawings or solid models to the PDF and U3D formats. The options and a description of what each option specifies, follow:

- *FilePath*—Name of the output file. Use the property `pfcPDFExportInstructions.FilePath` to set the name of the output file.
- *Options*—Specifies a collection of PDF export options of the type `pfcPDFOption`. Create a new instance of this object using the method `pfcPDFOption.Create()`. This object contains the following attributes:

-
- *OptionType*—Type of option in terms of the `pfcPDFOptionType` enumerated data type. Set this option using the property `pfcPDFOption.OptionType`.
 - *OptionValue*—Value of the option in terms of the `pfcArgValue` object. Set this option using the property `pfcPDFOption.OptionValue`.

Use the property `pfcPDFExportInstructions.Options` to set the collection of PDF export options.

- *ProfilePath*—Export profile path. Use the property `pfcPDFExportInstructions.ProfilePath` to set the profile path. When you set the profile path, the PDF export options set in the data object `pfcPDFExportInstructions` are ignored when the method `pfcModel.Export()` is called. You can set the profile path as `NULL`.



Note

You can specify the profile path only for drawings.

The types of options given by the `pfcPDFOptionType` enumerated data type) available for export to PDF and U3D formats follow:

- `PDFOPT_FONT_STROKE`—Switch between using TrueType fonts or “stroking” text in the resulting document. This option is given by the `pfcPDFFontStrokeMode` enumerated data type and takes the following values:
 - `PDF_USE_TRUE_TYPE_FONTS`—Specifies TrueType fonts. This is the default type.
 - `PDF_STROKE_ALL_FONTS`—Specifies the option to stroke all fonts.
- `PDFOPT_COLOR_DEPTH`—Choose between color, grayscale, or monochrome output. This option is given by the `pfcPDFColorDepth` enumerated data type and takes the following values:
 - `PDF_CD_COLOR`—Specifies color output. This is the default value.
 - `PDF_CD_GRAY`—Specifies grayscale output.
 - `PDF_CD_MONO`—Specifies monochrome output.
- `PDFOPT_HIDDENLINE_MODE`—Set the style for hidden lines in the resulting PDF document. This option is given by the `pfcPDFHiddenLineMode` enumerated data type and takes the following values:
 - `PDF_HLM_SOLID`—Specifies solid hidden lines.

-
- `PDF_HLM_DASHED`—Specifies dashed hidden lines. This is the default type.
 - `PDFOPT_SEARCHABLE_TEXT`—If true, stroked text is searchable. The default value is true.
 - `PDFOPT_RASTER_DPI`—Set the resolution for the output of any shaded views in DPI. It can take a value between 100 and 600. The default value is 300.
 - `PDFOPT_LAUNCH_VIEWER`—If true, launches the Adobe Acrobat Reader. The default value is true.
 - `PDFOPT_LAYER_MODE`—Set the availability of layers in the document. It is given by the `pfcPDFLayerMode` enumerated data type and takes the following values:
 - `PDF_LAYERS_ALL`—Exports the visible layers and entities. This is the default.
 - `PDF_LAYERS_VISIBLE`—Exports only visible layers in a drawing.
 - `PDF_LAYERS_NONE`—Exports only the visible entities in the drawing, but not the layers on which they are placed.
 - `PDFOPT_PARAM_MODE`—Enables you to set the availability of model parameters as searchable metadata in the PDF document. It is given by the `pfcPDFParameterMode` enumerated data type and takes the following values:
 - `PDF_PARAMS_ALL`—Exports the drawing and the model parameters to PDF. This is the default.
 - `PDF_PARAMS_DESIGNATED`—Exports only the specified model parameters in the PDF metadata.
 - `PDF_PARAMS_NONE`—Exports the drawing to PDF without the model parameters.
 - `PDFOPT_HYPERLINKS`—Sets hyperlinks to be exported as label text only or sets the underlying hyperlink URLs as active. The default value is true, specifying that the hyperlinks are active.
 - `PDFOPT_BOOKMARK_ZONES`—If true, adds bookmarks to the PDF showing zoomed in regions or zones in the drawing sheet. The zone on an A4-size drawing sheet is ignored.
 - `PDFOPT_BOOKMARK_VIEWS`—If true, adds bookmarks to the PDF document showing zoomed in views on the drawing.
 - `PDFOPT_BOOKMARK_SHEETS`—If true, adds bookmarks to the PDF document showing each of the drawing sheets.

-
- `PDFOPT_BOOKMARK_FLAG_NOTES`—If true, adds bookmarks to the PDF document showing the text of the flag note.
 - `PDFOPT_TITLE`—Specifies a title for the PDF document.
 - `PDFOPT_AUTHOR`—Specifies the name of the person generating the PDF document.
 - `PDFOPT_SUBJECT`—Specifies the subject of the PDF document.
 - `PDFOPT_KEYWORDS`—Specifies relevant keywords in the PDF document.
 - `PDFOPT_PASSWORD_TO_OPEN`—Sets a password to open the PDF document. By default, this option is `NULL`, which means anyone can open the PDF document without a password.
 - `PDFOPT_MASTER_PASSWORD`—Sets a password to restrict or limit the operations that the viewer can perform on the opened PDF document. By default, this option is `NULL`, which means you can make any changes to the PDF document regardless of the settings of the modification flags `PDFOPT_ALLOW_*`.
 - `PDFOPT_RESTRICT_OPERATIONS`—If true, restricts or limits operations in the PDF document. False is the default.
 - `PDFOPT_ALLOW_MODE`—Set the security settings for the PDF document. This option must be set if `PDFOPT_RESTRICT_OPERATIONS` is set to true. It is given by the `pfcPDFRestrictOperationsMode` enumerated data type and takes the following values:
 - `PDF_RESTRICT_NONE`—User can perform any of the permitted viewer operations on the PDF document. This is the default value.
 - `PDF_RESTRICT_FORMS_SIGNING`—Restricts the user from adding digital signatures to the PDF document.
 - `PDF_RESTRICT_INSERT_DELETE_ROTATE`—Restricts the user from inserting, deleting, or rotating the pages in the PDF document.
 - `PDF_RESTRICT_COMMENT_FORM_SIGNING`—Restricts the user from adding or editing comments in the PDF document.
 - `PDF_RESTRICT_EXTRACTING`—Restricts the user from extracting pages from the PDF document.
 - `PDFOPT_ALLOW_PRINTING`—If true, you can print the PDF document. By default, it is true.
 - `PDFOPT_ALLOW_PRINTING_MODE`—Set the print resolution. It is given by the `pfcPDFPrintingMode` enumerated data type and takes the following values:
 - `PDF_PRINTING_LOW_RES`—Low resolution for printing.

-
- `PDF_PRINTING_HIGH_RES`—High resolution for printing. This is the default value.
 - `PDFOPT_ALLOW_COPYING`—If true, you can copy content from the PDF document. The default is true.
 - `PDFOPT_ALLOW_ACCESSIBILITY`—If true, screen reader devices for the visually-impaired, extract data independent of the value given by the `pfcPDFRestrictOperationsMode` enumerated data type. The default is true.
 - `PDFOPT_PENTABLE`—If true, uses the standard Creo Parametric pentable to control the line weight, line style, and line color of the exported geometry. The default value is false.
 - `PDFOPT_LINECAP`—Control the treatment of the ends of the geometry lines exported to PDF. It is given by the `pfcPDFLinecap` enumerated data type and takes the following values:
 - `PDF_LINECAP_BUTT`—Specifies the butt cap square end. This is the default value.
 - `PDF_LINECAP_ROUND`—Specifies the round cap end.
 - `PDF_LINECAP_PROJECTING_SQUARE`—Specifies the projecting square cap end.
 - `PDFOPT_LINEJOIN`—Control the treatment of the joined corners of connected lines exported to PDF. It is given by the `pfcPDFLinejoin` enumerated data type and takes the following values:
 - `PDF_LINEJOIN_MITER`—Miter join. This is the default.
 - `PDF_LINEJOIN_ROUND`—Round join.
 - `PDF_LINEJOIN_BEVEL`—Bevel join.
 - `PDFOPT_SHEETS`—Specify the sheets from a Creo Parametric drawing that are to be exported to PDF. It is given by the `pfcPrintSheets` enumerated data type and takes the following values:
 - `PRINT_CURRENT_SHEET`—Only the current sheet is exported to PDF
 - `PRINT_ALL_SHEETS`—All the sheets are exported to PDF. This is the default value.
 - `PRINT_SELECTED_SHEETS`—Sheets of a specified range are exported to PDF. If this value is assigned, then the value of the option `PDFOPT_SHEET_RANGE` must also be known.
 - `PDFOPT_SHEET_RANGE`—Range of sheets in a drawing that are to be exported to PDF. If this option is set, then the option `PDFOPT_SHEETS` must be set to the value `PRINT_SELECTED_SHEETS`.

-
- `PDFOPT_EXPORT_MODE`—Select the object to be exported to PDF and the export format. It is given by the `pfcPDFExportMode` enumerated data type and takes the following values:
 - `PDF_2D_DRAWING`—Only drawings are exported to PDF. This is the default value.
 - `PDF_3D_AS_NAMED_VIEWS`—3D models are exported as 2D raster images embedded in PDF files.
 - `PDF_3D_AS_U3D_PDF`—3D models are exported as U3D models embedded in one-page PDF files.
 - `PDF_3D_AS_U3D`—A 3D model is exported as a U3D (.u3d) file. This value ignores the options set for the `pfcPDFOptionType` enumerated data type.
 - `PDFOPT_LIGHT_DEFAULT`—Set the default lighting style used while exporting 3D models in the U3D format to a one-page PDF file, that is when the option `PDFOPT_EXPORT_MODE` is set to `PDF_3D_AS_U3D`. The values for this option are given by the `pfcPDFU3DLightingMode` enumerated data type.
 - `PDFOPT_RENDER_STYLE_DEFAULT`—Set the default rendering style used while exporting Creo Parametric models in the U3D format to a one-page PDF file, that is when the option `PDFOPT_EXPORT_MODE` is set to `PDF_3D_AS_U3D`. The values for this option are given by the `pfcPDFU3DRenderMode` enumerated data type.
 - `PDFOPT_SIZE`—Specify the page size of the exported PDF file. The values for this option are given by the `pfcPlotPaperSize` enumerated data type. If the value is set to `VARIABLESIZEPLOT`, you also need to set the options `PDFOPT_HEIGHT` and `PDFOPT_WIDTH`.
 - `PDFOPT_HEIGHT`—Set the height for a user-defined page size of the exported PDF file. The default value is 0.0.
 - `PDFOPT_WIDTH`—Set the width for a user-defined page size of the exported PDF file. The default value is 0.0.
 - `PDFOPT_ORIENTATION`—Specify the orientation of the pages in the exported PDF file. It is given by the `pfcSheetOrientation` enumerated data type.
 - `ORIENT_PORTRAIT`—Exports the pages in portrait orientation. This is the default value.
 - `ORIENT_LANDSCAPE`—Exports the pages in landscape orientation.
 - `PDFOPT_TOP_MARGIN`—Specify the top margin of the view port. The default value is 0.0.

-
- `PDFOPT_LEFT_MARGIN`—Specify the left margin of the view port. The default value is 0.0.
 - `PDFOPT_BACKGROUND_COLOR_RED`—Specifies the default red background color that appears behind the U3D model. You can set any value within the range of 0.0 to 1.0. The default value is 1.0.
 - `PDFOPT_BACKGROUND_COLOR_GREEN`—Specifies the default green background color that appears behind the U3D model. You can set any value within the range of 0.0 to 1.0. The default value is 1.0.
 - `PDFOPT_BACKGROUND_COLOR_BLUE`—Specifies the default blue background color that appears behind the U3D model. You can set any value within the range of 0.0 to 1.0. The default value is 1.0.
 - `PDFOPT_ADD_VIEWS`—If true, you can add view definitions to the U3D model from a file. The default is true.
 - `PDFOPT_VIEW_TO_EXPORT`—Specifies the view or views to be exported to the PDF file. It is given by the `pfcPDFSelectedViewMode` enumerated data type and takes the following values:
 - `PDF_VIEW_SELECT_CURRENT`—Exports the current graphical area to a one-page PDF file.
 - `PDF_VIEW_SELECT_ALL`—Exports all the views to a multi-page PDF file. Each page contains one view with the view name displayed at the bottom center of the view port.
 - `PDF_VIEW_SELECT_BY_NAME`—Exports the selected view to a one-page PDF file with the view name printed at the bottom center of the view port. If this value is assigned, then the option `PDFOPT_SELECTED_VIEW` must also be set.
 - `PDFOPT_SELECTED_VIEW`—Sets the option `PDFOPT_VIEW_TO_EXPORT` to the value `PDF_VIEW_SELECT_BY_NAME`, if the corresponding view is successfully found.
 - `PDFOPT_PDF_SAVE`—Specifies the PDF save options. It is given by the `pfcPDFSaveMode` enumerated data type and takes the following values:
 - `PDF_ARCHIVE_1`—Applicable only for the value `PDF_2D_DRAWING`. Saves the drawings as PDF with the following conditions:
 - ◆ The value of `pfcPDFLayerMode` is set to `PDF_LAYERS_NONE`.
 - ◆ The value of `PDFOPT_HYPERLINKS` is set to `FALSE`.
 - ◆ The shaded views in the drawings will not have transparency and may overlap other data in the PDF.
 - ◆ The value of `PDFOPT_PASSWORD_TO_OPEN` is set to `NULL`.

- ◆ The value of `PDFOPT_MASTER_PASSWORD` is set to `NULL`.
- `PDF_FULL`—Saves the PDF with the values set by you. This is the default value.

Exporting 3D Geometry

Creo.JS allows you to export three dimensional geometry to various formats. Pass the instructions object containing information about the desired export file to the method `pfcModel.Export()`.

Export Instructions

Methods and Properties Introduced:

- `pfcModel.ExportIntf3D()`
- `pfcBaseSession.ExportProfileLoad()`
- `pfcExport3DInstructions.Configuration()`
- `pfcExport3DInstructions.ReferenceSystem()`
- `pfcExport3DInstructions.Geometry()`
- `pfcExport3DInstructions.IncludedEntities()`
- `pfcExport3DInstructions.LayerOptions()`
- `pfcGeometryFlags.Create()`
- `pfcInclusionFlags.Create()`
- `pfcLayerExportOptions.Create()`
- `pfcSTEP3DExportInstructions.Create()`
- `pfcVDA3DExportInstructions.Create()`
- `pfcIGES3DNewExportInstructions.Create()`
- `pfcCATIAModel3DExportInstructions.Create()`
- `pfcACIS3DExportInstructions.Create()`
- `pfcCatiaPart3DExportInstructions.Create()`
- `pfcCatiaProduct3DExportInstructions.Create()`
- `pfcCatiaCGR3DExportInstructions.Create()`
- `pfcDXF3DExportInstructions.Create()`
- `pfcDWG3DExportInstructions.Create()`
- `pfcJT3DExportInstructions.Create()`
- `pfcParaSolid3DExportInstructions.Create()`
- `pfcUG3DExportInstructions.Create()`

- **pfcTriangulationInstructions.Create()**

Use the method `pfcModel.ExportIntf3D()` to export Creo Parametric models to other file formats. All the options that can be set with these interfaces and methods, can also be set using the export profile option in Creo Parametric. Refer to the Creo Parametric Data Exchange Online Help for more information.

- `pfcSTEP3DExportInstructions`
- `pfcVDA3DExportInstructions`
- `pfcIGES3DNewExportInstructions`
- `pfcCATIAModel3DExportInstructions`
- `pfcACIS3DExportInstructions`
- `pfcCatiaPart3DExportInstructions`
- `pfcCatiaProduct3DExportInstructions`
- `pfcCatiaCGR3DExportInstructions`
- `pfcJT3DExportInstructions`
- `pfcParaSolid3DExportInstructions`
- `pfcUG3DExportInstructions`
- `pfcDWG3DExportInstructions`
- `pfcDXF3DExportInstructions`
- `pfcTriangulationInstructions`
- `pfcVDA3DExportInstructions`
- `pfcSWPart3DExportInstructions`
- `pfcSWAsm3DExportInstructions`

The method `pfcModel.ExportIntf3D()` exports a Creo Parametric model to the specified output format using the default export profile. The export options must be set using the export profile option in Creo Parametric.

The class `pfcExport3DInstructions` contains data to export a part or an assembly to a specified 3D format. The fields of this class are:

- **Configuration**—While exporting an assembly you can specify the structure and contents of the output files using the enumerated data type `pfcAssemblyConfiguration`. The options and a description of the export that results, follow:
 - `EXPORT_ASM_FLAT_FILE`—Exports all the geometry of the assembly to a single file as if it were a part.
 - `EXPORT_ASM_SINGLE_FILE`—Exports an assembly structure to a file with external references to component files. This file contains only top-level geometry.

-
- `EXPORT_ASM_MULTI_FILE`—Exports an assembly structure to a single file and the components to component files. It creates component parts and subassemblies with their respective geometry and external references. This option supports all levels of hierarchy.
 - `EXPORT_ASM_ASSEMBLY_PARTS`—Exports an assembly as multiple parts containing geometry information of its components and assembly features.
 - `ReferenceSystem`—The reference coordinate system used for export. If this value is null, use the default coordinate system.
 - `Geometry`—The object describing the type of geometry to export. The `pfcGeometryFlags.Create()` returns this instruction object. The types of geometry supported by the export operation with a description of the export that results, follow:
 - `Wireframe`—Export edges only.
 - `Solid`—Export surfaces along with topology.
 - `Surfaces`—Export all model surfaces.
 - `Quilts`—Export as quilt.
 - `IncludedEntities`—The object returned by the method `pfcInclusionFlags.Create()` that determines whether to include certain entities. The entities follow:
 - `Datums`—Determines whether datum curves are included when exporting files. If `true` the datum curve information is included during export. The default is `false`.
 - `Blanked`—Determines whether entities on blanked layers are exported. If `true` entities on blanked layers are exported. The default is `false`.
 - `Facets`—Determines whether facets are included when exporting files. If `true` the facet information is included during export. The default is `false`.
 - `LayerOptions`—The instructions object returned by the method `pfcLayerExportOptions.Create()` that describes how to export layers. To export layers you can specify the following:
 - *UseAutoId*—Enables you to set or remove an interface layer ID. A layer is recognized with this ID when exporting the file to a specified output format. If `true`, automatically assigns interface IDs to layers not assigned IDs and exports them. The default `false`.
 - *LayerSetupFile*—Specifies the name and complete path of the layer setup file. This file contains the layer assignment information which includes the name of the layer, its display status, the interface ID and number of sub layers.

The method `pfcBaseSession.ExportProfileLoad()` loads the specified profile for export. You can use this function when you want to use the export profile of your choice instead of the default export profile in a particular Creo Parametric session. The input argument *ProfileFile* is the full path to the profile along with the profile name and extension.



Note

Once the export profile file is loaded in a Creo Parametric session, it will be active in the interactive mode as well.

The method `pfcTriangulationInstructions.Create()` creates a object that will be used to define the parameters for faceted exports.

Export 3D Instructions Table

Class	Used to Export
<code>pfcSTEP3DExportInstructions</code>	Part or assembly in STEP format
<code>pfcVDA3DExportInstructions</code>	Part or assembly in VDA format
<code>pfcIGES3DNewExportInstructions</code>	Part or assembly in IGES format
<code>pfcCATIAModel3DExportInstructions</code>	Part or assembly in CATIA MODEL format
<code>pfcACIS3DExportInstructions</code>	Part or assembly in ACIS format
<code>pfcCatiaPart3DExportInstructions</code>	Part or assembly in CATIA PART format
<code>pfcCatiaProduct3DExportInstructions</code>	Part or assembly in CATIA PRODUCT format
<code>pfcCatiaCGR3DExportInstructions</code>	A part or assembly in CATIA CGR format
<code>pfcJT3DExportInstructions</code>	Part or assembly in JT format
<code>pfcParaSolid3DExportInstructions</code>	Part or assembly in PARASOLID format
<code>pfcUG3DExportInstructions</code>	Part or assembly in UG format
<code>pfcDWG3DExportInstructions</code>	Part or assembly in DWG format
<code>pfcDXF3DExportInstructions</code>	Part or assembly in DXF format
<code>pfcTriangulationInstructions</code>	Part or assembly in faceted format

Export Utilities

Methods Introduced:

- **`pfcBaseSession.IsConfigurationSupported()`**
- **`pfcBaseSession.IsGeometryRepSupported()`**

The method `pfcBaseSession.IsConfigurationSupported()` checks whether the specified assembly configuration is valid for a particular model and the specified export format. The input parameters for this method are:

- *Configuration*—Structure and content of the output files.

- *Type*—Output file type to create.

The method returns a true value if the configuration is supported for the specified export type.

The method `pfcBaseSession.IsGeometryRepSupported()` checks whether the specified geometric representation is valid for a particular export format. The input parameters are :

- *Flags*—The type of geometry supported by the export operation.
- *Type*—The output file type to create.

The method returns a true value if the geometry combination is valid for the specified model and export type.

The methods `pfcBaseSession.IsConfigurationSupported()` and `pfcBaseSession.IsGeometryRepSupported()` must be called before exporting an assembly to the specified export formats except for the CADDs and STEP2D formats. The return values of both the methods must be true for the export operation to be successful.

Use the method `pfcModel.Export()` to export the assembly to the specified output format.

Shrinkwrap Export

To improve performance in a large assembly design, you can export lightweight representations of models referred to as shrinkwrap models. A shrinkwrap model is based on the external surfaces of the source part or assembly model and captures the outer shape of the source model.

You can create the following types of nonassociative exported shrinkwrap models:

- **Surface Subset**—This type consists of a subset of the original model surfaces.
- **Faceted Solid**—This type is a faceted solid representing the original solid.
- **Merged Solid**—The external components from the reference assembly model are merged into a single part representing the solid geometry in all collected components.

Methods Introduced:

- **`pfcSolid.ExportShrinkwrap()`**

You can export the specified solid model as a shrinkwrap model using the method `pfcSolid.ExportShrinkwrap()`. This method takes the `pfcShrinkwrapExportInstructions` object as an argument.

Use the appropriate class given in the following table to create the required type of shrinkwrap. All the classes have their own static method to create an object of the specified type. The object created by these interfaces can be used as an object of type `pfcShrinkwrapExportInstructions` or `pfcShrinkwrapModelExportInstructions`.

Type of Shrinkwrap Model	Class to Use
Surface Subset	<code>pfcShrinkwrapSurfaceSubsetInstructions</code>
Faceted Part	<code>pfcShrinkwrapFacetedPartInstructions</code>
Faceted VRML	<code>pfcShrinkwrapFacetedVRMLInstructions</code>
Faceted STL	<code>pfcShrinkwrapFacetedSTLInstructions</code>
Merged Solid	<code>ShrinkwrapMergedSolidInstructions</code>

Setting Shrinkwrap Options

The class `pfcShrinkwrapModelExportInstructions` contains the general methods available for all the types of shrinkwrap models. The object created by any of the interfaces specified in the preceding table can be used with these methods.

Properties Introduced:

- **`pfcShrinkwrapModelExportInstructions.Method`**
- **`pfcShrinkwrapModelExportInstructions.Quality`**
- **`pfcShrinkwrapModelExportInstructions.AutoHoleFilling`**
- **`pfcShrinkwrapModelExportInstructions.IgnoreSkeleton`**
- **`pfcShrinkwrapModelExportInstructions.IgnoreQuilts`**
- **`pfcShrinkwrapModelExportInstructions.AssignMassProperties`**
- **`pfcShrinkwrapModelExportInstructions.IgnoreSmallSurfaces`**
- **`pfcShrinkwrapModelExportInstructions.SmallSurfPercentage`**
- **`pfcShrinkwrapModelExportInstructions.DatumReferences`**

The property `pfcShrinkwrapModelExportInstructions.Method` returns the method used to create the shrinkwrap using the enumerated data type `pfcShrinkwrapMethod` and the types of shrinkwrap methods follow:

- `SWCREATE_SURF_SUBSET`—Surface Subset
- `SWCREATE_FACETED_SOLID`—Faceted Solid
- `SWCREATE_MERGED_SOLID`—Merged Solid

The property `pfcShrinkwrapModelExportInstructions.Quality` specifies the quality level for the system to use when identifying surfaces or components that contribute to the shrinkwrap model. Quality ranges from 1 which produces the coarsest representation of the model in the fastest time, to 10 which produces the most exact representation. The default value is 1.

The property `pfcShrinkwrapModelExportInstructions.AutoHoleFilling` sets a flag that forces Creo Parametric to identify all holes and surfaces that intersect a single surface and fills those holes during shrinkwrap. The default value is true.

The property `pfcShrinkwrapModelExportInstructions.IgnoreSkeleton` determines whether the skeleton model geometry must be included in the shrinkwrap model.

The property `pfcShrinkwrapModelExportInstructions.IgnoreQuilts` determines whether external quilts must be included in the shrinkwrap model.

The property `pfcShrinkwrapModelExportInstructions.AssignMassProperties` assigns mass properties to the shrinkwrap model. The default value is false and the mass properties of the original model is assigned to the shrinkwrap model. If the value is set to true, the user must assign a value for the mass properties.

The property `pfcShrinkwrapModelExportInstructions.IgnoreSmallSurfaces` sets a flag that forces Creo Parametric to skip surfaces smaller than a certain size. The default value is false. The size of the surface is specified as a percentage of the model size. This size can be modified using the property `pfcShrinkwrapModelExportInstructions.SmallSurfPercentage`.

The property `pfcShrinkwrapModelExportInstructions.DatumReferences` specifies and selects the datum planes, points, curves, axes, and coordinate system references to be included in the shrinkwrap model.

Surface Subset Options

Methods and Properties Introduced:

- **`pfcShrinkwrapSurfaceSubsetInstructions.Create()`**
- **`pfcShrinkwrapSurfaceSubsetInstructions.AdditionalSurfaces`**
- **`pfcShrinkwrapSurfaceSubsetInstructions.OutputModel`**

The static method

`pfcShrinkwrapSurfaceSubsetInstructions.Create()` returns an object used to create a shrinkwrap model of surface subset type. Specify the name of the output model in which the shrinkwrap is to be created as an input to this method.

The property

`pfcShrinkwrapSurfaceSubsetInstructions.AdditionalSurfaces` selects individual surfaces to be included in the shrinkwrap model.

The property

`pfcShrinkwrapSurfaceSubsetInstructions.OutputModel` sets the template model.

Faceted Solid Options

The `pfcShrinkwrapFacetedFormatInstructions` class consists of the following types:

- `SWFACETED_PART`—Creo Parametric part with normal geometry. This is the default format type.
- `SWFACETED_STL`—An STL file.
- `SWFACETED_VRML`—A VRML file.

Use the `Create` method to create the object of the specified type. Upcast the object to use the general methods available in this class.

Properties Introduced:

- **`pfcShrinkwrapFacetedFormatInstructions.Format`**
- **`pfcShrinkwrapFacetedFormatInstructions.FrameFile`**

The property `pfcShrinkwrapFacetedFormatInstructions.Format` returns the output file format of the shrinkwrap model.

Use the property

`pfcShrinkwrapFacetedFormatInstructions.FrameFile` to select a frame file to create a faceted solid motion envelope model that represents the full motion of the mechanism captured in the frame file. Specify the name and complete path of the frame file.

Faceted Part Options

Methods and Properties Introduced:

- **`pfcShrinkwrapFacetedPartInstructions.Create()`**
- **`pfcShrinkwrapFacetedPartInstructions.Lightweight`**

The static method

`pfcShrinkwrapFacetedPartInstructions.Create()` returns an object used to create a shrinkwrap model of shrinkwrap faceted type. The input parameters of this method follow:

- *OutputModel*—Output model where the shrinkwrap must be created.
- *Lightweight*—Specify this value as `True` if the shrinkwrap model is a Lightweight Creo Parametric part.

The property

`pfcShrinkwrapFacetedPartInstructions.Lightweight` specifies if the Creo Parametric part is exported as a light weight faceted geometry.

VRML Export Options

Methods and Properties Introduced:

- **`pfcShrinkwrapVRMLInstructions.Create()`**
- **`pfcShrinkwrapVRMLInstructions.OutputFile`**

The static method `pfcShrinkwrapVRMLInstructions.Create()` returns an object used to create a shrinkwrap model of shrinkwrap VRML format. Specify the name of the output model as an input to this method.

The property `pfcShrinkwrapVRMLInstructions.OutputFile` specifies the name of the output file to be created.

STL Export Options

Methods and Properties Introduced:

- **`pfcShrinkwrapSTLInstructions.Create()`**
- **`pfcShrinkwrapSTLInstructions.OutputFile`**

The static method `pfcShrinkwrapSTLInstructions.Create()` returns an object used to create a shrinkwrap model of shrinkwrap STL format. Specify the name of the output model as an input to this method.

The property `pfcShrinkwrapSTLInstructions.OutputFile` specifies the name of the output file to be created.

Merged Solid Options

Methods and Properties Introduced:

- **`pfcShrinkwrapMergedSolidInstructions.Create()`**
- **`pfcShrinkwrapMergedSolidInstructions.AdditionalComponents`**

The static method

`pfcShrinkwrapMergedSolidInstructions.Create()` returns an object used to create a shrinkwrap model of merged solids format. Specify the name of the output model as an input to this method.

The property

`pfcShrinkwrapMergedSolidInstructions.AdditionalComponents` specifies individual components of the assembly to be merged into the shrinkwrap model.

Importing Files

Method Introduced:

- **`pfcModel.Import()`**

The method `pfcModel.Import()` reads a file into Creo Parametric. The format must be the same as it would be if these files were created by Creo Parametric. The parameters are:

- *FilePath*—Absolute path of the file to be imported along with its extension.
- *ImportData*—The `pfcImportInstructions` object that controls the import operation.

Import Instructions

Methods Introduced:

- **`pfcRelationImportInstructions.Create()`**
- **`pfcIGESSectionImportInstructions.Create()`**
- **`pfcProgramImportInstructions.Create()`**
- **`pfcConfigImportInstructions.Create()`**
- **`pfcDWGSetupImportInstructions.Create()`**
- **`pfcSpoolImportInstructions.Create()`**
- **`pfcConnectorParamsImportInstructions.Create()`**
- **`pfcASSEMBTreeCFGImportInstructions.Create()`**
- **`pfcWireListImportInstructions.Create()`**
- **`pfcCableParamsImportInstructions.Create()`**
- **`pfcSTEPImport2DInstructions.Create()`**
- **`pfcIGESImport2DInstructions.Create()`**
- **`pfcDXFImport2DInstructions.Create()`**
- **`pfcDWGImport2DInstructions.Create()`**

The methods described in this section create an instructions data object to import a file of a specified type into Creo Parametric. The details are as shown in the table below:

Class	Used to Import
<code>pfcRelationImportInstructions</code>	List of relations and parameters in a part or assembly.
<code>pfcIGESSectionImportInstructions</code>	Section model in IGES format.
<code>ProgramImportInstructions</code>	Program file for a part or assembly that can be edited to change the model.
<code>pfcConfigImportInstructions</code>	Configuration instructions.
<code>pfcDWGSetupImportInstructions</code>	Drawing s/u file.
<code>pfcSpoolImportInstructions</code>	Spool instructions.
<code>pfcConnectorParamsImportInstructions</code>	Connector parameter instructions.
<code>pfcASSEMBTreeCFGImportInstructions</code>	Assembly tree CFG instructions.
<code>pfcWireListImportInstructions</code>	Wirelist instructions.
<code>pfcCableParamsImportInstructions</code>	Cable parameters from an assembly.
<code>pfcSTEPImport2DInstructions</code>	Part or assembly in STEP format.
<code>pfcIGESImport2DInstructions</code>	Part or assembly in IGES format.
<code>pfcDXFImport2DInstructions</code>	Drawing in DXF format.
<code>pfcDWGImport2DInstructions</code>	Drawing in DWG format.

Note

- The method `pfcModel.Import()` does not support importing of CADAM type of files.
- If a model or the file type STEP, IGES, DWX, or SET already exists, the imported model is appended to the current model. For more information on methods that return models of the types STEP, IGES, DWX, and SET, refer to [Getting a Model Object on page 59](#).

Importing 2D Models

Method Introduced:

- **`pfcBaseSession.Import2DModel()`**

The method `pfcBaseSession.Import2DModel()` imports a two dimensional model based on the following parameters:

- *NewModelName*—Specifies the name of the new model.
- *Type*—Specifies the type of the model. The type can be one of the following:
 - STEP

-
- IGES
 - DXF
 - DWG
 - SET
 - *FilePath*—Location of the file to be imported along with the file extension
 - *Instructions*—Specifies the `pfcImport2DInstructions` object that controls the import operation.

The class `pfcImport2DInstructions` contains the following attributes:

- *Import2DViews*—Defines whether to import 2D drawing views.
- *ScaleToFit*—If the current model has a different sheet size than that specified by the imported file, set the parameter to true to retain the current sheet size. Set the parameter to false to retain the sheet size of the imported file.
- *FitToLeftCorner*—If this parameter is set to true, the bottom left corner of the imported file is adjusted to the bottom left corner of the current model. If it is set to false, the size of imported file is retained.

 Note

The method `pfcBaseSession.Import2DModel()` does not support importing of CADAM type of files.

Importing 3D Geometry

Methods Introduced:

- **`pfcBaseSession.GetImportSourceType()`**
- **`pfcBaseSession.ImportNewModel()`**

For some input formats, the method `pfcBaseSession.GetImportSourceType()` returns the type of model that can be imported using a designated file. The input parameters of this method are:

- *FileToImport*—Path to the file along with its name and extension. which is
- *NewModelType*—Type of model to be imported. The types of models that can be imported defined by the enumerated data type `pfcNewModelImportType` and the valid values are:
 - `IMPORT_NEW_IGES`

-
- `IMPORT_NEW_VDA`
 - `IMPORT_NEW_NEUTRAL`
 - `IMPORT_NEW_CADDS`
 - `IMPORT_NEW_STEP`
 - `IMPORT_NEW_STL`
 - `IMPORT_NEW_VRML`
 - `IMPORT_NEW_POLTXT`
 - `IMPORT_NEW_CATIA_SESSION`
 - `IMPORT_NEW_CATIA_MODEL`
 - `IMPORT_NEW_DXF`
 - `IMPORT_NEW_ACIS`
 - `IMPORT_NEW_PARASOLID`
 - `IMPORT_NEW_ICEM`
 - `IMPORT_NEW_DESKTOP`
 - `IMPORT_NEW_CATIA_PART`
 - `IMPORT_NEW_CATIA_PRODUCT`
 - `IMPORT_NEW_UG`
 - `IMPORT_NEW_PRODUCTVIEW`
 - `IMPORT_NEW_CATIA_CGR`
 - `IMPORT_NEW_JT`
 - `IMPORT_NEW_SW_PART`
 - `IMPORT_NEW_SW_ASSEM`
 - `pfcIMPORT_NEW_INVENTOR_PART`
 - `pfcIMPORT_NEW_INVENTOR_ASSEM`
 - `IMPORT_NEW_CC`
 - `IMPORT_NEW_SEEDGE_PART`
 - `IMPORT_NEW_SEEDGE_ASSEMBLY`
 - `IMPORT_NEW_SEEDGE_SHEETMETAL_PART`
 - *Type*—Type of the model. It can be a part, assembly or drawing. The type is defined by the enumerated data type `pfcModelType`.
 - *NewModelName*—Name for the imported model.
 - *Filter*—Layer filter which is defined by the `classpfcLayerImportFilter`. This parameter is optional. —
 - *FileToImport*—Path of the file along with its name and extension.

-
- *NewModelImportType*—Type of model to be imported.

The method `pfcBaseSession.ImportNewModel()` is used to import an external 3D format file and creates a new model or set of models of type `pfcModel`. The input parameters of this method are:

- *FileToImport*—Path to the file along with its name and extension. which is
- *NewModelType*—Type of model to be imported. The types of models that can be imported defined by the enumerated data type `pfcNewModelImportType` and the valid values are:

- `IMPORT_NEW_IGES`
- `IMPORT_NEW_VDA`
- `IMPORT_NEW_NEUTRAL`
- `IMPORT_NEW_CADD`
- `IMPORT_NEW_STEP`
- `IMPORT_NEW_STL`
- `IMPORT_NEW_VRML`
- `IMPORT_NEW_POLTXT`
- `IMPORT_NEW_CATIA_SESSION`
- `IMPORT_NEW_CATIA_MODEL`
- `IMPORT_NEW_DXF`
- `IMPORT_NEW_ACIS`
- `IMPORT_NEW_PARASOLID`
- `IMPORT_NEW_ICEM`
- `IMPORT_NEW_DESKTOP`
- `IMPORT_NEW_CATIA_PART`
- `IMPORT_NEW_CATIA_PRODUCT`
- `IMPORT_NEW_UG`
- `IMPORT_NEW_PRODUCTVIEW`
- `IMPORT_NEW_CATIA_CGR`
- `IMPORT_NEW_JT`
- `IMPORT_NEW_SW_PART`
- `IMPORT_NEW_SW_ASSEM`
- `pfcIMPORT_NEW_INVENTOR_PART`
- `pfcIMPORT_NEW_INVENTOR_ASSEM`
- `IMPORT_NEW_CC`

-
- `IMPORT_NEW_SEDGE_PART`
 - `IMPORT_NEW_SEDGE_ASSEMBLY`
 - `IMPORT_NEW_SEDGE_SHEETMETAL_PART`
 - *Type*—Type of the model. It can be a part, assembly or drawing. The type is defined by the enumerated data type `pfcModelType`.
 - *NewModelName*—Name for the imported model.
 - *Filter*—Layer filter which is defined by the `classpfcLayerImportFilter`. This parameter is optional. —

Printing Files

The printer instructions for printing a file are defined in `pfcPrinterInstructions` data object.

Methods Introduced:

- **`pfcPrinterInstructions.Create()`**
- **`pfcPrinterInstructions.PrinterOption`**
- **`pfcPrinterInstructions.PlacementOption`**
- **`pfcPrinterInstructions.ModelOption`**
- **`pfcPrinterInstructions.WindowId`**

The method `pfcPrinterInstructions.Create()` creates a new instance of the `pfcPrinterInstructions` object. The object contains the following instruction attributes:

- *PrinterOption*—Printer settings for printing a file in terms of the `pfcPrintPrinterOption` object. Set this attribute using the property `pfcPrinterInstructions.PrinterOption`.
- *PlacementOption*—Placement options for printing purpose in terms of the `pfcPrintMdlOption` object. Set this attribute using the property `pfcPrinterInstructions.PlacementOption`.
- *ModelOption*—Model options for printing purpose in terms of the `pfcPrintPlacementOption` object. Set this attribute using the property `pfcPrinterInstructions.ModelOption`.
- *WindowId*—Current window identifier. Set this attribute using the property `pfcPrinterInstructions.WindowId`.

Printer Options

The printer settings for printing a file are defined in the `pfcPrintPrinterOption` object.

Methods and Properties Introduced:

- **pfcPrintPrinterOption.Create()**
- **pfcBaseSession.GetPrintPrinterOptions()**
- **pfcPrintPrinterOption.DeleteAfter**
- **pfcPrintPrinterOption.FileName**
- **pfcPrintPrinterOption.PaperSize**
- **pfcPrintSize.Create()**
- **pfcPrintSize.Height**
- **pfcPrintSize.Width**
- **pfcPrintSize.PaperSize**
- **pfcPrintPrinterOption.PenTable**
- **pfcPrintPrinterOption.PrintCommand**
- **pfcPrintPrinterOption.PrinterType**
- **pfcPrintPrinterOption.Quantity**
- **pfcPrintPrinterOption.RollMedia**
- **pfcPrintPrinterOption.RotatePlot**
- **pfcPrintPrinterOption.SaveMethod**
- **pfcPrintPrinterOption.SaveToFile**
- **pfcPrintPrinterOption.SendToPrinter**
- **pfcPrintPrinterOption.Slew**
- **pfcPrintPrinterOption.SwHandshake**
- **pfcPrintPrinterOption.UseTtf**

The method `pfcPrintPrinterOption.Create()` creates a new instance of the `pfcPrintPrinterOption` object.

The method `pfcBaseSession.GetPrintPrinterOptions()` retrieves the printer settings.

The `pfcPrintPrinterOption` object contains the following options:

- *DeleteAfter*—Determines if the file is deleted after printing. Set it to true to delete the file after printing. Use the property `pfcPrintPrinterOption.DeleteAfter` to assign this option.
- *FileName*—Specifies the name of the file to be printed. Use the property `pfcPrintPrinterOption.FileName` to set the name.

Note

If the method `pfcModel.Export()` is called for `pfcExportType` object, then the argument *FileName* is ignored, and can be passed as `NULL`. You must use the method `pfcModel.Export()` to set the *FileName*.

- *PaperSize*—Parameters of the paper to be printed in terms of the `pfcPrintSize` object. The property `pfcPrintPrinterOption.PaperSize` assigns the *PaperSize* option. Use the method `pfcPrintSize.Create()` to create a new instance of the `pfcPrintSize` object. This object contains the following options:
 - *Height*—Height of paper. Use the property `pfcPrintSize.Height` to set the paper height.
 - *Width*—Width of paper. Use the property `pfcPrintSize.Width` to set the paper width.
 - *PaperSize*—Size of the paper used for the plot in terms of the `pfcPlotPaperSize` object. Use the property `pfcPrintSize.PaperSize` to set the paper size.

Note

If you want to plot a layout without adding a border on the paper, use the following paper sizes defined in the enumerated data type `pfcPlotPaperSize`:

- ◆ `CEEMPTYPLOT`—The paper size is 22.5 x 36 in
 - ◆ `CEEMPTYPLOT_MM`—The paper size is 625 x 1000 mm
-

- *PenTable*—File containing the pen table. Use the property `pfcPrintPrinterOption.PenTable` to set this option.
- *PrintCommand*—Command to be used for printing. Use the property `pfcPrintPrinterOption.PrintCommand` to set the command.
- *PrinterType*—Printer type. Use the property `pfcPrintPrinterOption.PrinterType` to assign the type.
- *Quantity*—Number of copies to be printed. Use the property `pfcPrintPrinterOption.Quantity` to assign the quantity.

- *RollMedia*—Determines if roll media is to be used for printing. Set it to true to use roll media. Use the property `pfcPrintPrinterOption.RollMedia` to assign this option.
- *RotatePlot*—Determines if the plot is rotated by 90 degrees. Set it to true to rotate the plot. Use the property `pfcPrintPrinterOption.RotatePlot` to set this option.
- *SaveMethod*—Save method in terms of the `pfcPrintSaveMethod` enumerated data type. Use the property `pfcPrintPrinterOption.SaveMethod` to specify the save method. The available methods are as follows:
 - `PRINT_SAVE_SINGLE_FILE`—Plot is saved to a single file.
 - `PRINT_SAVE_MULTIPLE_FILE`—Plot is saved to multiple files.
 - `PRINT_SAVE_APPEND_TO_FILE`—Plot is appended to a file.
- *SaveToFile*—Determines if the file is saved after printing. Set it to true to save the file after printing. Use the property `pfcPrintPrinterOption.SaveToFile` to assign this option.
- *SendToPrinter*—Determines if the plot is directly sent to the printer. Set it to true to send the plot to the printer. Use the property `pfcPrintPrinterOption.SendToPrinter` to set this option.
- *Slew*—Speed of the pen in centimeters per second in X and Y direction. Use the property `pfcPrintPrinterOption.Slew` to set this option.
- *SwHandshake*—Determines if the software handshake method is to be used for printing. Set it to true to use the software handshake method. Use the property `pfcPrintPrinterOption.SwHandshake` to set this option.
- *UseTtf*—Whether TrueType fonts or stroked text is used for printing. Set this option to true to use TrueType fonts and to false to stroke all text. Use the property `pfcPrintPrinterOption.UseTtf` to set this option.

Placement Options

The placement options for printing are defined in the `pfcPrintPlacementOption` object.

Methods and Properties Introduced:

- **`pfcPrintPlacementOption.Create()`**
- **`pfcBaseSession.GetPrintPlacementOptions()`**
- **`pfcPrintPlacementOption.BottomOffset`**
- **`pfcPrintPlacementOption.ClipPlot`**
- **`pfcPrintPlacementOption.KeepPanzoom`**

-
- **pfcPrintPlacementOption.LabelHeight**
 - **pfcPrintPlacementOption.PlaceLabel**
 - **pfcPrintPlacementOption.Scale**
 - **pfcPrintPlacementOption.ShiftAllCorner**
 - **pfcPrintPlacementOption.SideOffset**
 - **pfcPrintPlacementOption.X1ClipPosition**
 - **pfcPrintPlacementOption.X2ClipPosition**
 - **pfcPrintPlacementOption.Y1ClipPosition**
 - **pfcPrintPlacementOption.Y2ClipPosition**

The method `pfcPrintPlacementOption.Create()` creates a new instance of the `pfcPrintPlacementOption` object.

The method `pfcBaseSession.GetPrintPlacementOptions()` retrieves the placement options.

The `pfcPrintPlacementOption` object contains the following options:

- *BottomOffset*—Offset from the lower-left corner of the plot. Use the property `pfcPrintPlacementOption.BottomOffset` to set this option.
- *ClipPlot*—Whether the plot is clipped. Set this option to true to clip the plot or to false to avoid clipping of plot. Use the property `pfcPrintPlacementOption.ClipPlot` to set this option.
- *KeepPanzoom*—Determines whether pan and zoom values of the window are used. Set this option to true use pan and zoom and false to skip them. Use the property `pfcPrintPlacementOption.KeepPanzoom` to set this option.
- *LabelHeight*—Height of the label in inches. Use the property `pfcPrintPlacementOption.LabelHeight` to set this option.
- *PlaceLabel*—Whether you want to place the label on the plot. Use the property `pfcExport.PrintPlacementOption.PlaceLabel` to set this option.
- *Scale*—Scale used for the plot. Use the property `pfcPrintPlacementOption.Scale` to set this option.
- *ShiftAllCorner*—Determines whether all corners are shifted. Set this option to true to shift all corners or to false to skip shifting of corners. Use the property `pfcPrintPlacementOption.ShiftAllCorner` to set this option.
- *SideOffset*—Offset from the sides. Use the property `pfcPrintPlacementOption.SideOffset` to set this option.

-
- *X1ClipPosition*—First X parameter for defining the clip position. Use the property `pfcPrintPlacementOption.X1ClipPosition` to set this option.
 - *X2ClipPosition*—Second X parameter for defining the clip position. Use the property `pfcPrintPlacementOption.X2ClipPosition` to set this option.
 - *Y1ClipPosition*—First Y parameter for defining the clip position. Use the property `pfcPrintPlacementOption.Y1ClipPosition` to set this option.
 - *Y2ClipPosition*—Second Y parameter for defining the clip position. Use the property `pfcPrintPlacementOption.Y2ClipPosition` to set this option.

Model Options

The model options for printing purpose are defined in the `pfcPrintMdlOption` object.

Methods and Properties Introduced:

- **`pfcPrintMdlOption.Create()`**
- **`pfcBaseSession.GetPrintMdlOptions()`**
- **`pfcPrintMdlOption.DrawFormat`**
- **`pfcPrintMdlOption.FirstPage`**
- **`pfcPrintMdlOption.LastPage`**
- **`pfcPrintMdlOption.LayerName`**
- **`pfcPrintMdlOption.LayerOnly`**
- **`pfcPrintMdlOption.Mdl`**
- **`pfcPrintMdlOption.Quality`**
- **`pfcPrintMdlOption.Segmented`**
- **`pfcPrintMdlOption.Sheets`**
- **`pfcPrintMdlOption.UseDrawingSize`**
- **`pfcPrintMdlOption.UseSolidScale`**

The method `pfcPrintMdlOption.Create()` creates a new instance of the `pfcExport.PrintMdlOption` object.

The method `pfcBaseSession.GetPrintMdlOptions()` retrieves the model options.

The `pfcPrintMdlOption` object contains the following options:

-
- *DrawFormat*—Displays the drawing format used for printing. Use the property `pfcPrintMdlOption.DrawFormat` to set this option.
 - *FirstPage*—First page number. Use the property `pfcPrintMdlOption.FirstPage` to set this option.
 - *LastPage*—Last page number. Use the property `pfcPrintMdlOption.LastPage` to set this option.
 - *LayerName*—Name of the layer. Use the property `pfcPrintMdlOption.LayerName` to set the name.
 - *LayerOnly*—Prints the specified layer only. Set this option to `true` to print the specified layer. Use the property `pfcPrintMdlOption.LayerOnly` to set this option.
 - *Mdl*—Specifies the model to be printed. Use the property `pfcPrintMdlOption.Mdl` to set this option.
 - *Quality*—Determines the quality of the model to be printed. It checks for no line, no overlap, simple overlap, and complex overlap. Use the property `pfcPrintMdlOption.Quality` to set this option.
 - *Segmented*—If set to `true`, the printer prints the drawing in full size, but in segments that are compatible with the selected paper size. This option is available only if you are plotting a single page. Use the property `pfcPrintMdlOption.Segmented` to set this option.
 - *Sheets*—Sheets that need to be printed in terms of the `pfcPrintSheets` enumerated data type. Use the property `pfcPrintMdlOption.Sheets` to specify the sheets. The sheets can be of the following types:
 - `PRINT_CURRENT_SHEET`—Only the current sheet is printed.
 - `PRINT_ALL_SHEETS`—All the sheets are printed.
 - `PRINT_SELECTED_SHEETS`—Sheets of a specified range are printed.
 - *UseDrawingSize*—Overrides the paper size specified in the printer options with the drawing size. Set this option to `true` to use the drawing size. Use the property `pfcPrintMdlOption.UseDrawingSize` to set this option.
 - *UseSolidScale*—Prints with the scale used in the solid model. Set this option to `true` to use solid scale. Use the property `pfcPrintMdlOption.UseSolidScale` to set this option.

Plotter Configuration File (PCF) Options

The printing options for PCF file are defined in the `pfcPrinterPCFOptions` object.

Methods Introduced:

- **pfcPrinterPCFOptions.Create()**
- **pfcPrinterPCFOptions.PrinterOption**
- **pfcPrinterPCFOptions.PlacementOption**
- **pfcPrinterPCFOptions.ModelOption**

The method `pfcPrinterPCFOptions.Create()` creates a new instance of the `pfcPrinterPCFOptions` object.

The `pfcPrinterPCFOptions` object contains the following options:

- *PrinterOption*—Printer settings for printing a file in terms of the `pfcPrintPrinterOption` object. Set this attribute using the property `pfcPrinterPCFOptions.PrinterOption`.
- *PlacementOption*—Placement options for printing purpose in terms of the `pfcPrintMdlOption` object. Set this attribute using the property `pfcPrinterPCFOptions.PlacementOption`.
- *ModelOption*—Model options for printing purpose in terms of the `pfcPrintPlacementOption` object. Set this attribute using the property `pfcPrinterPCFOptions.ModelOption`.

Solid Operations

Method Introduced:

- **pfcSolid.CreateImportFeat()**

The method `pfcSolid.CreateImportFeat()` creates a new import feature in the solid and takes the following input arguments:

- *IntfData*—The source of data from which to create the import feature. It is given by the `pfcIntfDataSource` object. The type of source data that can be imported is given by the `pfcIntfType` class and can be of the following types:
 - `INTF_NEUTRAL`
 - `INTF_NEUTRAL_FILE`
 - `INTF_IGES`
 - `INTF_STEP`
 - `INTF_VDA`
 - `INTF_ICEM`
 - `INTF_ACIS`
 - `INTF_DXF`
 - `INTF_CDRS`

- INTF_STL
- INTF_VRML
- INTF_PARASOLID
- INTF_AI
- INTF_CATIA_PART
- INTF_UG
- INTF_PRODUCTVIEW
- INTF_CATIA_CGR
- INTF_JT
- *CoordSys*—The pointer to a reference coordinate system. If this is NULL, the function uses the default coordinate system.
- *FeatAttr*—The attributes for creation of the new import feature given by the `pfcImportFeatAttr` object. If this pointer is NULL, the function uses the default attributes.

Window Operations

Method Introduced:

- **`pfcWindow.ExportRasterImage()`**

The method `pfcWindow.ExportRasterImage()` outputs a standard Creo Parametric raster output file.

Simplified Representations

Simplified Representations

Creo.JS gives programmatic access to all the simplified representation functionality of Creo Parametric. Create simplified representations either permanently or on the fly and save, retrieve, or modify them by adding or deleting items.

Overview

Using Creo.JS, you can create and manipulate assembly simplified representations just as you can using Creo Parametric interactively.



Note

Creo.JS supports simplified representation of assemblies only, not parts.

Simplified representations are identified by the `pfcSimRep` class. This class is a child of `pfcModelItem`, so you can use the methods dealing with `pfcModelItems` to collect, inspect, and modify simplified representations.

The information required to create and modify a simplified representation is stored in a class called `pfcSimRepInstructions` which contains several data objects and fields, including:

- `String`—The name of the simplified representation
- `pfcSimRepAction`—The rule that controls the default treatment of items in the simplified representation.
- `pfcSimRepItem`—An array of assembly components and the actions applied to them in the simplified representation.

A `pfcSimRepItem` is identified by the assembly component path to that item. Each `pfcSimRepItem` has its own `pfcSimRepAction` assigned to it. `pfcSimRepAction` is a visible data object that includes a field of type `pfcSimRepActionType`. You can use the property `pfcSimRepItem.Action` to set the actions. To delete an existing item, you must set the action as `NULL`.

`pfcSimRepActionType` is an enumerated type that specifies the possible treatment of items in a simplified representation. The possible values are as follows:

Values	Action
<code>SIMPREP_NONE</code>	No action is specified.
<code>SIMPREP_REVERSE</code>	Reverse the default rule for this component (for example, include it if the default rule is exclude).
<code>SIMPREP_INCLUDE</code>	Include this component in the simplified representation.
<code>SIMPREP_EXCLUDE</code>	Exclude this component from the simplified representation.
<code>SIMPREP_SUBSTITUTE</code>	Substitute the component in the simplified representation.
<code>SIMPREP_GEOM</code>	Use only the geometrical representation of the component.
<code>SIMPREP_GRAPHICS</code>	Use only the graphics representation of the component.
<code>SIMPREP_SYMB</code>	Use the symbolic representation of the component.
<code>SIMPREP_BOUNDBOX</code>	Use the boundary box representation of the component.
<code>SIMPREP_DEFENV</code>	Use the default envelope representation of the

Values	Action
	component.
SIMPREP_LIGHT_GRAPH	Use the light weight graphics representation of the component.
SIMPREP_AUTO	Use the automatic representation of the component.

Retrieving Simplified Representations

Methods Introduced:

- **pfcBaseSession.RetrieveAssemSimpRep()**
- **pfcBaseSession.RetrieveGeomSimpRep()**
- **pfcBaseSession.RetrieveGraphicsSimpRep()**
- **pfcBaseSession.RetrieveSymbolicSimpRep()**
- **pfcRetrieveExistingSimpRepInstructions.Create**

You can retrieve a named simplified representation from a model using the method `pfcBaseSession.RetrieveAssemSimpRep()`, which is analogous to the Assembly mode option **Retrieve Rep** in the **SIMPLFD REP** menu. This method retrieves the object of an existing simplified representation from an assembly without fetching the generic representation into memory. The method takes two arguments, the name of the assembly and the simplified representation data.

To retrieve an existing simplified representation, pass an instance of `pfcRetrieveExistingSimpRepInstructions.Create()` and specify its name as the second argument to this method. Creo Parametric retrieves that representation and any active submodels and returns the object to the simplified representation as a `pfcAssembly` object.

You can retrieve geometry, graphics, and symbolic simplified representations into session using the methods `pfcBaseSession.RetrieveGeomSimpRep()`, `pfcBaseSession.RetrieveGraphicsSimpRep()`, and `pfcBaseSession.RetrieveSymbolicSimpRep()` respectively. Like `pfcBaseSession.RetrieveAssemSimpRep()`, these methods retrieve the simplified representation without bringing the master representation into memory. Supply the name of the assembly whose simplified representation is to be retrieved as the input parameter for these methods. The methods output the assembly. They do not display the simplified representation.

Creating and Deleting Simplified Representations

Methods Introduced:

- **pfcCreateNewSimpRepInstructions.Create()**
- **pfcSolid.CreateSimpRep()**

- **pfcSolid.DeleteSimpRep()**

To create a simplified representation, you must allocate and fill a `pfcSimpRepInstructions` object by calling the method `pfcCreateNewSimpRepInstructions.Create()`. Specify the name of the new simplified representation as an input to this method. You should also set the default action type and add `SimpRepItems` to the object.

To generate the new simplified representation, call `pfcSolid.CreateSimpRep()`. This method returns the `pfcSimpRep` object for the new representation.

The method `pfcSolid.DeleteSimpRep()` deletes a simplified representation from its model owner. The method requires only the `pfcSimpRep` object as input.

Extracting Information About Simplified Representations

Methods and Properties Introduced:

- **pfcSimpRep.GetInstructions()**
- **pfcSimpRepInstructions.DefaultAction**
- **pfcCreateNewSimpRepInstructions.NewSimpName**
- **pfcSimpRepInstructions.IsTemporary**

Given the object to a simplified representation, `pfcSimpRep.GetInstructions()` fills out the `pfcSimpRepInstructions` object.

The `pfcSimpRepInstructions.DefaultAction`, `pfcCreateNewSimpRepInstructions.NewSimpName`, and `pfcSimpRepInstructions.IsTemporary` properties return the associated values contained in the `pfcSimpRepInstructions` object.

Modifying Simplified Representations

Methods and Properties Introduced:

- **pfcSimpRepInstructions.DefaultAction**
- **pfcCreateNewSimpRepInstructions.NewSimpName**
- **pfcSimpRepInstructions.IsTemporary**

Using *Creo*.JS, you can modify the attributes of existing simplified representations. After you create or retrieve a simplified representation, you can make calls to the methods listed in this section to designate new values for the fields in the `pfcSimpRepInstructions` object.

To modify an existing simplified representation retrieve it and then get the `pfcSimpRepInstructions` object by calling `pfcSimpRep.GetInstructions()`. If you created the representation programmatically within the same application, the `pfcSimpRepInstructions` object is already available. After you modify the data object, reassign it to the corresponding simplified representation by calling the method `pfcSimpRep.SetInstructions()`.

Adding Items to and Deleting Items from a Simplified Representation

Methods and Properties Introduced:

- **`pfcSimpRepReverse.Create()`**
- **`pfcSimpRepInclude.Create()`**
- **`pfcSimpRepExclude.Create()`**
- **`pfcSimpRepSubstitute.Create()`**
- **`pfcSimpRepGeom.Create()`**
- **`pfcSimpRepGraphics.Create()`**
- **`pfcSimpRepDefaultEnvelope.Create()`**
- **`pfcSimpRepBoundingBox.Create()`**
- **`pfcSimpRepLightWeightGraphics.Create()`**
- **`pfcSimpRepItem.Create()`**
- **`pfcSimpRep.SetInstructions()`**
- **`pfcSimpRepInstructions.Items`**

The method `pfcSimpRepReverse.Create()` creates an object that reverses the default rule for the assembly component in the simplified representation.

The method `pfcSimpRepInclude.Create()` creates an object that includes the component in the simplified representation.

The method `pfcSimpRepExclude.Create()` creates an object that excludes the component from the simplified representation.

The method `pfcSimpRepSubstitute.Create()` creates an object that substitutes the component with a component from the `pfcSubstitution` class in the simplified representation.

The method `pfcSimpRepGeom.Create()` creates an object that represents the component that contains complete geometric information in the simplified representation.

The method `pfcSimpRepGraphics.Create()` creates an object that represents the component that contains only display information in the simplified representation. These graphic representations cannot be modified or referenced, they can only be browsed through.

 Note

As compared to graphics representations, geometry representations take longer to retrieve and require more memory.

The method `pfcSimpRepDefaultEnvelope.Create()` creates an object that represents the component in default envelope type of simplified representation.

The method `pfcSimpRepBoundingBox.Create()` creates an object that represents the component in boundary box type of simplified representation.

The method `pfcSimpRepLightWeightGraphics.Create()` creates an object that represents the component in lightweight type of simplified representation. Light graphics representations of assemblies contain assembly information and 3D thumbnail graphics representations of assembly components.

The method `pfcSimpRepItem.Create()` creates an object that defines the status of the component or feature of the simplified representation.

The method `pfcSimpRep.SetInstructions()` sets or changes the instructions that specify the internal data required to define the simplified representation.

The method `pfcSimpRepInstructions.Items` applies an array of actions to the components or features in the simplified representation.

You can add and delete items from the list of components in a simplified representation using Creo.JS. If you created a simplified representation using the option **Exclude** as the default rule, you would generate a list containing the items you want to include. Similarly, if the default rule for a simplified representation is **Include**, you can add the items that you want to be excluded from the simplified representation to the list, setting the value of the `pfcSimpRepActionType` to `SIMPREP_EXCLUDE`.

How to Add Items

1. Get the `pfcSimpRepInstructions` object, as described in the previous section.
2. Specify the action to be applied to the item with a call to one of the following methods.

-
3. Initialize a `pfcSimpRepItem` object for the item by calling the method `pfcSimpRepItem.Create()`.
 4. Add the item to the `pfcSimpRepItem` sequence. Put the new `pfcSimpRepInstructions` using `pfcSimpRepInstructions.Items()`.
 5. Reassign the `pfcSimpRepInstructions` object to the corresponding `pfcSimpRep` object by calling `pfcSimpRep.SetInstructions()`

How to Remove Items

Follow the procedure above, except remove the unwanted `pfcSimpRepItem` from the sequence.

Simplified Representation Utilities

Methods Introduced:

- `pfcModelItemOwner.ListItems()`
- `pfcModelItemOwner.GetItemById()`
- `pfcSolid.GetSimpRep()`
- `pfcSolid.SelectSimpRep()`
- `pfcSolid.ActivateSimpRep()`
- `pfcSolid.GetActiveSimpRep()`

This section describes the utility methods that relate to simplified representations.

The method `pfcModelItemOwner.ListItems()` can list all of the simplified representations in a Solid.

The method `pfcModelItemOwner.GetItemById()` initializes a `pfcSimpRep` object. It takes an integer `id`.



Note

Creo.JS supports simplified representation of Assemblies only, not Parts.

The method `pfcSolid.GetSimpRep()` initializes a `pfcSimpRep` object. The method takes the following arguments:

- *SimpRepname*— The name of the simplified representation in the solid. If you specify this argument, the method ignores the *rep_id*.

The method `pfcSolid.SelectSimpRep()` creates a Creo Parametric menu to enable interactive selection. The method takes the owning solid as input, and outputs the object to the selected simplified representation. If you choose the **Quit** menu button, the method throws an exception `pfcXToolkitUserAbort`

Use the methods `pfcSolid.GetActiveSimpRep()` and `pfcSolid.ActivateSimpRep()` to find and get the currently active simplified representation, respectively. Given an assembly object, `pfcSolid.GetActiveSimpRep()` returns the object to the currently active simplified representation. If the current representation is the master representation, the return is null.

The method `pfcSolid.ActivateSimpRep()` activates the requested simplified representation.

To set a simplified representation to be the currently displayed model, you must also call `pfcModel.Display()`.

Task Based Application Libraries

Task Based Application Libraries

Applications created using different Creo Parametric API products are interoperable. These products use Creo Parametric as the medium of interaction, eliminating the task of writing native-platform specific interactions between different programming languages.

With application interoperability Creo.JS applications can call into Creo TOOLKIT from areas not covered in the native interface. You can an HTML front end on legacy Creo TOOLKIT applications..

Managing Application Arguments

Creo.JS passes application data to and from tasks in other applications as members of a sequence of `pfcArgument` objects. Application arguments consist of a label and a value. The value can be of any one of the following types:

- Integer
- Double
- Boolean
- ASCII string (a non-encoded string, provided for compatibility with arguments provided from C applications)
- String (a fully encoded string)
- `pfcSelection` (a selection of an item in a Creo Parametric session)

-
- `pfcTransform3D` (a coordinate system transformation matrix)

Methods and Properties Introduced:

- `pfcCreateIntArgValue()`
- `pfcCreateDoubleArgValue()`
- `pfcCreateBoolArgValue()`
- `pfcCreateASCIIStringArgValue()`
- `pfcCreateStringArgValue()`
- `pfcCreateSelectionArgValue()`
- `pfcCreateTransformArgValue()`
- `pfcArgValue.dscr`
- `pfcArgValue.IntValue`
- `pfcArgValue.DoubleValue`
- `pfcArgValue.BoolValue`
- `pfcArgValue.ASCIIStringValue`
- `pfcArgValue.StringValue`
- `pfcArgValue.SelectionValue`
- `pfcArgValue.TransformValue`

The class `pfcArgValue` contains one of the seven types of values. Creo.JS provides different properties to create each of the seven types of argument values.

Use the method `pfcCreateIntArgValue()` to create an integer argument value object.

Use the method `pfcCreateDoubleArgValue()` to create a double argument value object.

Use the method `pfcCreateBoolArgValue()` to create an Boolean argument value object.

Use the method `pfcCreateASCIIStringArgValue()` to create an ASCII string argument value object.

Use the method `pfcCreateStringArgValue()` to create a string argument value object.

Use the method `pfcCreateSelectionArgValue()` to create a new selection argument value object.

Use the method `pfcCreateTransformArgValue()` to create a new transform argument value object.

The property `pfcArgValue.dscr` returns the type of value contained in the argument value object.

Use the properties `pfcArgValue.IntValue`, `pfcArgValue.DoubleValue`, `pfcArgValue.BoolValue`, `pfcArgValue.ASCIIStringValue`, `pfcArgValue.StringValue`, `pfcArgValue.SelectionValue`, and `pfcArgValue.TransformValue` to access and modify the argument values.

Modifying Arguments

Methods and Properties Introduced:

- **`pfcArgument.Create()`**
- **`pfcArgument.Label`**
- **`pfcArgument.Value`**

The method `pfcArgument.Create()` creates a new argument. Provide a name and value as the input arguments of this method.

The property `pfcArgument.Label` returns the label of the argument.

The property `pfcArgument.Value` returns the value of the argument.

Launching a Creo Parametric TOOLKIT DLL

The methods described in this section enable a Creo.JS user to register and launch a Creo TOOLKIT DLL from a Creo.JS application. The ability to launch and control a Creo TOOLKIT application enables the following:

- Reuse of existing Creo TOOLKIT code with Creo.JS applications.
- ATB operations.

Methods and Properties Introduced:

- **`pfcBaseSession.LoadProToolkitDll()`**
- **`pfcBaseSession.LoadProToolkitLegacyDll()`**
- **`pfcBaseSession.GetProToolkitDll()`**
- **`pfcDll.ExecuteFunction()`**
- **`pfcDll.Id`**
- **`pfcDll.IsActive()`**
- **`pfcDll.Unload()`**

Use the method `pfcBaseSession.LoadProToolkitDll()` to register and start a Creo TOOLKIT DLL. The input parameters of this method are similar to the fields of a registry file and are as follows:

- *ApplicationName*—Name of the application to initialize.
- *DllPath*—Full path to the DLL binary file.

- *TextPath*—Path to the message and user interface files for the application.
- *UserDisplay*—Set this parameter to `true` to register the application in the Creo Parametric user interface and to see error messages if the application fails. If this parameter is `false`, the application is invisible to the user.

The `user_initialize()` function for the application is called when the application is started. The method returns a handle to the loaded Creo TOOLKIT DLL.

To register and start a legacy Pro/TOOLKIT DLL that is not Unicode-compliant, use the method `pfcBaseSession.LoadProToolkitLegacyDll()`. It takes the same input parameters as the earlier method `pfcBaseSession.LoadProToolkitDll()`.

Use the method `pfcBaseSession.GetProToolkitDll()` to obtain a Creo TOOLKIT DLL handle. Specify the *Application_Id*, that is, the identifier string for the DLL as the input parameter of this method. The method returns the DLL object or null if the DLL was not in session. The *Application_Id* can be determined as follows:

- Use the function `ProToolkitDllIdGet()` within the DLL application to get a string representation of the DLL application. Pass `NULL` to the first argument of `ProToolkitDllIdGet()` to get the string identifier for the calling application.
- Use the `Get` method for the `Id` attribute in the DLL interface. The property `pfcDll.Id` returns the DLL identifier string.

Use the method `pfcDll.ExecuteFunction()` to call a properly designated function in the Creo TOOLKIT DLL library. The input parameters of this method are:

- *FunctionName*—Name of the function in the Creo TOOLKIT DLL application.
- *InputArguments*—Input arguments to be passed to the library function.

The method returns an object of class `pfcFunctionReturn`. This interface contains data returned by a Creo TOOLKIT function call. The object contains the return value, as integer, of the executed function and the output arguments passed back from the function call.

The method `pfcDll.IsActive()` determines whether a Creo TOOLKIT DLL previously loaded by the method `pfcBaseSession.LoadProToolkitDll()` is still active.

The method `pfcDll.Unload()` is used to shutdown a Creo TOOLKIT DLL previously loaded by the method `pfcBaseSession.LoadProToolkitDll()` and the `user_terminate()` function for the application is called.

Launching Tasks from J-Link Task Libraries

Use the methods described in this section to launch tasks from a predefined J-Link task library.

Methods Introduced:

- **pfcBaseSession.StartJLinkApplication()**
- **pfcJLinkApplication.ExecuteTask()**
- **pfcJLinkApplication.IsActive()**
- **pfcJLinkApplication.Stop()**

Use the method `pfcBaseSession.StartJLinkApplication()` to start a J-Link application. The input parameters of this method are similar to the fields of a registry file. A list of input parameters and what each parameter specifies, follows:

- *ApplicationName*—A unique name that is assigned to this J-Link application.
- *ClassName*—Name of the Java class that contains the start and stop method of the J-Link application. . This should be a fully qualified Java package and class name.
- *StartMethod*—Start method of the J-Link application.
- *StopMethod*—Stop method of the J-Link application.
- *AdditionalClassPath*—Locations of packages and classes that must be loaded when starting this J-Link application. If this parameter is specified as null, the default classpath locations are used.
- *TextPath*—Application text path for menus and messages. If this parameter is specified as null, the default text locations are used.
- *UserDisplay*—Whether to display the application in the **Auxiliary Applications** dialog box in Creo Parametric.

When starting the application, the static `start()` method is invoked. The method returns a `pfcJLinkApplication` referring to the J-Link application.

The method `pfcJLinkApplication.ExecuteTask()` calls a registered task method in a J-Link application. The input parameters of this method follow:

- Name of the task to be executed.
- A sequence of name value pair arguments contained by the interface `pfcArguments`.

The method outputs an array of output arguments.

The method `pfcJLinkApplication.IsActive()` returns a `True` value if the application specified by the `pfcJLinkApplication` object is active.

The method `pfcJLinkApplication.Stop()` stops the application specified by the `pfcJLinkApplication` object. This method activates the static `Stop()` method for the application.

Graphics

Graphics

This section covers Creo.JS Graphics including displaying lists, displaying text and using the mouse.

Overview

Use the methods described in this section allow you to draw temporary graphics in a display window. Methods that are identified as 2D are used to draw entities such as arcs, polygons, and text, in screen coordinates. Other entities may be drawn using the coordinate system for the for the current model's or the lines, circles, and polylines for the screen coordinate. Methods are also included for manipulating text properties and accessing mouse input.

Getting Mouse Input

The following methods are used to read the mouse position in screen coordinates when the mouse button is pressed. Each method outputs the position and an enumerated type description of which mouse button was pressed when the mouse was at that position. These values are contained in the class `pfcMouseStatus`.

The enumerated values are defined in `pfcMouseButton` and are as follows:

- `MOUSE_BTN_LEFT`
- `MOUSE_BTN_RIGHT`
- `MOUSE_BTN_MIDDLE`
- `MOUSE_BTN_LEFT_DOUBLECLICK`

Methods Introduced:

- **`pfcSession.UIGetNextMousePick()`**
- **`pfcSession.UIGetCurrentMouseStatus()`**

The method `pfcSession.UIGetNextMousePick()` returns the mouse position when you press a mouse button. The input argument is the mouse button that you expect the user to select.

The method `pfcSession.UIGetCurrentMouseStatus()` returns a value whenever the mouse is moved or a button is pressed. With this method a button does not have to be pressed for a value to be returned. You can use an input argument to flag whether or not the returned positions are snapped to the window grid.

Drawing a Mouse Box

Use this method to draw a mouse box.

Method Introduced:

- **`pfcSession.UIPickMouseBox()`**

The method `pfcSession.UIPickMouseBox()` draws a dynamic rectangle from a specified point in screen coordinates to the current mouse position until the user presses the left mouse button. The return value for this method is of the type `pfcOutline3D`.

You can supply the first corner location programmatically or the user can select both corners of the box.

Displaying Graphics

All the methods in this section draw graphics in the Creo Parametric current window and use the color and linestyle set by calling `pfcBaseSession.SetStdColorFromRGB()` and `pfcBaseSession.SetLineStyle()`. The methods draw the graphics in the Creo Parametric graphics color. The default graphics color is white.

The methods in this section are called using the class `pfcDisplay`. This class is extended by the `pfcBaseSession` class. With this architecture you can call all these methods on any `pfcSession` object.

Methods Introduced:

- **`pfcDisplay.SetPenPosition()`**
- **`pfcDisplay.DrawLine()`**
- **`pfcDisplay.DrawPolyline()`**
- **`pfcDisplay.DrawCircle()`**
- **`pfcDisplay.DrawArc2D()`**
- **`pfcDisplay.DrawPolygon2D()`**

The method `pfcDisplay.SetPenPosition()` sets the point at which you want to start drawing a line. The method `pfcDisplay.DrawLine()` draws a line to the given point from the position given in the last call to either of the two functions. Call `pfcDisplay.SetPenPosition()` for the start of the

polyline, and `pfcDisplay.DrawLine()` for each vertex. If you use these methods in two-dimensional modes, use screen coordinates instead of solid coordinates.

The method `pfcDisplay.DrawCircle()` uses solid coordinates for the center of the circle and the radius value. The circle will be placed to the XY plane of the model.

The method `pfcDisplay.DrawPolyline()` also draws polylines, using an array to define the polyline.

In two-dimensional models the Display Graphics methods draw graphics at the specified screen coordinates.

The method `pfcDisplay.DrawPolygon2D()` draws a polygon in screen coordinates. The method `pfcDisplay.DrawArc2D()` draws an arc in screen coordinates.

Controlling Graphics Display

Properties Introduced:

- **`pfcDisplay.CurrentGraphicsColor`**
- **`pfcDisplay.CurrentGraphicsMode`**

The property `pfcDisplay.CurrentGraphicsColor` returns the Creo Parametric standard color used to display graphics. The Creo Parametric default is `COLOR_DRAWING` (white).

The property `pfcDisplay.CurrentGraphicsMode` returns the mode used to draw graphics using the enumerated data type `pfcGraphicsMode` and the valid values are:

- `DRAW_GRAPHICS_NORMAL`—Creo Parametric draws graphics in the required color in each invocation.
- `DRAW_GRAPHICS_COMPLEMENT`—Creo Parametric draws graphics normally, but will erase graphics drawn a second time in the same location. This allows you to create rubber band lines.

Displaying Text in the Graphics Window

Method Introduced:

- **`pfcDisplay.DrawText2D()`**

The method `pfcDisplay.DrawText2D()` places text at a position specified in screen coordinates. If you want to add text to a particular position on the solid, you must transform the solid coordinates into screen coordinates by using the view matrix.

Controlling Text Attributes

Properties Introduced:

- **pfcDisplay.TextHeight**
- **pfcDisplay.WidthFactor**
- **pfcDisplay.RotationAngle**
- **pfcDisplay.SlantAngle**

These properties control the attributes of text added by calling the method `pfcDisplay.DrawText2D()`.

You can access the following information:

- Text height (in screen coordinates)
- Width ratio of each character, including the gap, as a proportion of the height
- Rotation angle of the whole text, in counterclockwise degrees
- Slant angle of the text, in clockwise degrees

Controlling Text Fonts

Properties Introduced:

- **pfcDisplay.DefaultFont**
- **pfcDisplay.CurrentFont**
- **pfcDisplay.GetFontById()**
- **pfcDisplay.GetFontByName()**

The properties `pfcDisplay.DefaultFont` and `pfcDisplay.CurrentFont` return the default Creo Parametric text font and the current font used for text graphics in Creo Parametric, respectively. The text fonts are identified in Creo Parametric by names and by integer identifiers. To find a specific font, use the methods `pfcDisplay.GetFontById()` or `pfcDisplay.GetFontByName()`.

External Data

External Data

This section explains using External Data in Creo.JS.

External Data

This section describes how to store and retrieve external data. External data enables a Creo.JS application to store its own data in a Creo Parametric database in such a way that it is invisible to the Creo Parametric user. This method is different from other means of storage accessible through the Creo Parametric user interface.

Introduction to External Data

External data provides a way for the Creo Parametric application to store its own private information about a Creo Parametric model within the model file. The data is built and interrogated by the application as a workspace data structure. It is saved to the model file when the model is saved, and retrieved when the model is retrieved. The external data is otherwise ignored by Creo Parametric; the application has complete control over form and content.

The external data for a specific Creo Parametric model is broken down into classes and slots. A class is a named “bin” for your data, and identifies it as yours so no other Creo Parametric API application or other classes in your own application, use it by mistake. An application usually needs only one class. The class name should be unique for each application and describe the role of the data in your application.

Each class contains a set of data slots. Each slot is identified by an identifier and optionally, a name. A slot contains a single data item of one of the following types:

Creo.JS Type	Data
pfcExternalDataTypeEXTDATA_INTEGER	integer
pfcExternalDataTypeEXTDATA_DOUBLE	double
pfcExternalDataTypeEXTDATA_STRING	string

The Creo.JS interface used to access external data in Creo Parametric is described in the following table:

Creo.JS Type	Data Type
pfcExternalDataAccess	Top level object and is created when attempting to access external data.
pfcExternalDataClass	Class of external data and is identified by a unique name.
pfcExternalDataSlot	Container for one item of data. Each slot is stored in a class.
pfcExternalData	Compact data structure that contains either an integer, double or string value.

Compatibility with Creo Parametric TOOLKIT

Creo.JS and Creo TOOLKIT share external data in the same manner. Creo.JS external data is accessible by Creo TOOLKIT and the reverse is also true. However, an error results if Creo.JS attempts to access external data previously stored by Creo TOOLKIT as a stream.

Accessing External Data

Methods Introduced:

- **pfcModel.AccessExternalData()**
- **pfcModel.TerminateExternalData()**
- **pfcExternalDataAccess.IsValid()**

The method `pfcModel.AccessExternalData()` prepares Creo Parametric to read external data from the model file. It returns the `pfcExternalDataAccess` object that is used to read and write data. This method should be called only once for any given model in session.

The method `pfcModel.TerminateExternalData()` stops Creo Parametric from accessing external data in a model. When you use this method all external data in the model is removed. Permanent removal occurs when the model is saved.

Note

If you need to preserve the external data created in session, you must save the model before calling this function. Otherwise, your data is lost.

The method `pfcExternalDataAccess.IsValid()` determines if the `pfcExternalDataAccess` object can be used to read and write data.

Storing External Data

Method and Properties Introduced:

- **pfcExternalDataAccess.CreateClass()**
- **pfcExternalDataClass.CreateSlot()**
- **pfcExternalDataSlot.Value**

The first step in storing external data in a new class and slot is to set up a class using the method `pfcExternalDataAccess.CreateClass()`, which provides the class name. The method outputs `pfcExternalDataClass`, used by the application to reference the class.

The next step is to use `pfcExternalDataClass.CreateSlot()` to create an empty data slot and input a slot name. The method outputs a `pfcExternalDataSlot` object to identify the new slot.



Note

Slot names cannot begin with a number.

The property `pfcExternalDataSlot.Value` specifies the data type of a slot and writes an item of that type to the slot. The input is a `pfcExternalData` object that you can create by calling any one of the methods in the next section.

Initializing Data Objects

Methods Introduced:

- **`pfcCreateIntExternalData()`**
- **`pfcCreateDoubleExternalData()`**
- **`pfcCreateStringExternalData()`**

These methods initialize a `pfcExternalData` object with the appropriate data inputs.

Retrieving External Data

Methods and Properties Introduced:

- **`pfcExternalDataAccess.LoadAll()`**
- **`pfcExternalDataAccess.ListClasses()`**
- **`pfcExternalDataClass.ListSlots()`**
- **`pfcExternalData.dscr`**
- **`pfcExternalData.IntegerValue`**
- **`pfcExternalData.DoubleValue`**
- **`pfcExternalData.StringValue`**

For improved performance, external data is not loaded automatically into memory with the model. When the model is in session, call the method `pfcExternalDataAccess.LoadAll()` to retrieve all the external data for the specified model from the Creo Parametric model file and put it in the workspace. The method needs to be called only once to retrieve all the data.

The method `pfcExternalDataAccess.ListClasses()` returns an array of `pfcExternalDataClasses` registered in the model. The method `pfcExternalDataClass.ListSlots()` provide an array of `pfcExternalDataSlots` existing for each class.

To find out a data type of a `pfcExternalData`, call `pfcExternalData.dscr` and then call one of these properties to get the data, depending on the data type:

- `pfcExternalData.IntegerValue`
- `pfcExternalData.DoubleValue`
- `pfcExternalData.StringValue`

Exceptions

Most exceptions thrown by external data methods in Creo.JS extend `pfcXExternalDataError`, which is a subclass of `pfcXToolkitError`.

An additional exception thrown by external data methods is `pfcXBadExternalData`. This exception signals an error accessing data. For example, external data access might have been terminated or the model might contain stream data from Creo TOOLKIT.

The following table lists these exceptions.

Exception	Cause
<code>pfcXExternalDataInvalidObject</code>	Generated when a model or class is invalid.
<code>pfcXExternalDataClassOrSlotExists</code>	Generated when creating a class or slot and the proposed class or slot already exists.
<code>pfcXExternalDataNamesTooLong</code>	Generated when a class or slot name is too long.
<code>pfcXExternalDataSlotNotFound</code>	Generated when a specified class or slot does not exist.
<code>pfcXExternalDataEmptySlot</code>	Generated when the slot you are attempting to read is empty.
<code>pfcXExternalDataInvalidSlotName</code>	Generated when a specified slot name is invalid.
<code>pfcXBadGetExternalData</code>	Generated when you try to access an incorrect data type in a <code>pfcExternalData</code> object.

Windchill Connectivity APIs

Windchill Connectivity APIs

Creo Parametric has the capability to be directly connected to Windchill solutions, including Windchill ProjectLink and PDMLink servers. This access allows users to manage and control the product data seamlessly from within Creo Parametric.

This section lists Creo.JS APIs that support Windchill servers and server operations in a connected Creo Parametric session.

Introduction

The methods introduced in this section provide support for the basic Windchill server operations from within Creo Parametric. With these methods, operations such as registering a Windchill server, managing workspaces, and check in or check out of objects will be possible via Creo.JS. The capabilities of these APIs are similar to the operations available from within the Creo Parametric client, with some restrictions.

Some of these APIs are supported from a non-interactive, that is, batch mode application or asynchronous application.

Accessing a Windchill Server from a Creo Parametric Session

Creo Parametric allows you to register Windchill servers as a connection between the Windchill database and Creo Parametric. Although the represented Windchill database can be from Windchill ProjectLink or Windchill PDMLink all types of databases are represented in the same way.

You can use the following identifiers when referring to Windchill servers in Creo.JS:

- **Codebase URL**—This is the root portion of the URL that is used to connect to a Windchill server. For example `http://wcserver.company.com/Windchill`.
- **Server Alias**—A server alias is used to refer to the server after it has been registered. The alias is also used to construct paths to files in the server workspaces and commonspace. The server alias is chosen by the user or application and it need not have any direct relationship to the codebase URL. An alias can be any normal name, such as `my_alias`.

Accessing Information Before Registering a Server

To start working with a Windchill server, you must establish a connection by registering the server in Creo Parametric. The methods described in this section allow you to connect to a Windchill server and access information related to the server.

Methods and Properties Introduced:

- **`pfcBaseSession.AuthenticateBrowser()`**
- **`pfcBaseSession.GetServerLocation()`**
- **`pfcServerLocation.Class`**
- **`pfcServerLocation.Location`**
- **`pfcServerLocation.Version`**
- **`pfcServerLocation.ListContexts()`**
- **`pfcServerLocation.CollectWorkspaces()`**

Use the method `pfcBaseSession.AuthenticateBrowser()` to set the authentication context using a valid username and password. A successful call to this method allows the Creo Parametric session to register with any server that accepts the username and password combination. A successful call to this method also ensures that an authentication dialog box does not appear during the registration process. You can call this method any number of times to set the authentication context for any number of Windchill servers, provided that you register the appropriate servers or servers immediately after setting the context.

The property `pfcServerLocation.Location` specifies a `pfcServerLocation` object representing the codebase URL for a possible server. The server may not have been registered yet, but you can use this object and the methods it contains to gather information about the server prior to registration.

The property `pfcServerLocation.Class` specifies the class of the server or server location. The values are:

- `Windchill`—Denotes a Windchill PDMLink server.
- `ProjectLink`—Denotes Windchill ProjectLink type of servers.

The property `pfcServerLocation.Version` specifies the version of Windchill that is configured on the server or server location, for example, 9.0 or 10.0. This method accepts the server codebase URL as the input.

 Note

`pfcServerLocation.Version` works only for Windchill servers and throws the `pfcXToolkitUnsupported` exception, if the server is not a Windchill server.

The method `pfcServerLocation.ListContexts()` gives a list of all the available contexts for a specified server. A context is used to associate a workspace with a product, project, or library.

The method `pfcServerLocation.CollectWorkspaces()` returns the list of available workspaces for the specified server. The workspace objects returned contain the name of each workspace and its context.

Registering and Activating a Server

The Creo.JS methods call the same underlying API as Creo Parametric to register and unregister servers. Hence, registering the servers using Creo.JS methods is similar to registering the servers using the Creo Parametric user interface. Therefore, the servers registered by Creo.JS are available in the Creo Parametric Server Registry. The servers are also available in other locations in the Creo Parametric user interface such as, the **Folder Navigator** and the embedded browser.

Methods Introduced:

- **`pfcBaseSession.RegisterServer()`**
- **`pfcServer.Activate()`**
- **`pfcServer.Unregister()`**

The method `pfcBaseSession.RegisterServer()` registers the specified server with the codebase URL. You can automate the registration of servers in interactive mode. To preregister the servers use the standard `config.fld` setup. If you do not want the servers to be preregistered in batch mode, set the environment variable `PTC_WF_ROOT` to an empty directory before starting Creo Parametric.

A successful call to `pfcBaseSession.AuthenticateBrowser()` with a valid username and password is essential for `pfcBaseSession.RegisterServer()` to register the server without launching the authentication dialog box. Registration of the server establishes the server alias. You must designate an existing workspace to use when registering the server. After the server has been registered, you may create a new workspace.

The method `pfcServer.Activate()` sets the specified server as the active server in the Creo Parametric session.

The method `pfcServer.Unregister()` unregisters the specified server. This is similar to **Server Registry ▶ Delete** through the user interface.

Accessing Information From a Registered Server

Properties Introduced:

- `pfcServer.IsActive`
- `pfcServer.Alias`
- `pfcServer.Context`

The property `pfcServer.IsActive` specifies if the server is active.

The property `pfcServer.Alias` returns the alias of a server if you specify the codebase URL.

The property `pfcServer.Context` returns the active context of the active server.

Information on Servers in Session

Methods Introduced:

- `pfcBaseSession.GetActiveServer()`
- `pfcBaseSession.GetServerByAlias()`
- `pfcBaseSession.GetServerByUrl()`
- `pfcBaseSession.ListServers()`

The method `pfcBaseSession.GetActiveServer()` returns the active server handle.

The method `pfcBaseSession.GetServerByAlias()` returns the handle to the server matching the given server alias, if it exists in session.

The method `pfcBaseSession.GetServerByUrl()` returns the handle to the server matching the given server URL and workspace name, if it exists in session.

The method `pfcBaseSession.ListServers()` returns a list of servers registered in this session.

Accessing Workspaces

For every workspace, a new distinct storage location is maintained in the user's personal folder on the server (server-side workspace) and on the client (client-side workspace cache). Together, the server-side workspace and the client-side workspace cache make up the workspace.

Methods and Properties Introduced:

-
- **pfcWorkspaceDefinition.Create()**
 - **pfcWorkspaceDefinition.WorkspaceName**
 - **pfcWorkspaceDefinition.WorkspaceContext**

The class `pfcWorkspaceDefinition` contains the name and context of the workspace. The method `pfcServerLocation.CollectWorkspaces()` returns an array of workspace data. Workspace data is also required for the method `pfcServer.CreateWorkspace()` to create a workspace with a given name and a specific context.

The method `pfcWorkspaceDefinition.Create()` creates a new workspace definition object suitable for use when creating a new workspace on the server.

The property `pfcWorkspaceDefinition.WorkspaceName` retrieves the name of the workspace.

The property `pfcWorkspaceDefinition.WorkspaceContext` retrieves the context of the workspace.

Creating and Modifying the Workspace

Methods and Properties Introduced:

- **pfcServer.CreateWorkspace()**
- **pfcServer.ActiveWorkspace**
- **pfcServerLocation.DeleteWorkspace()**

The method `pfcServer.CreateWorkspace()` creates and activates a new workspace.

The property `pfcServer.ActiveWorkspace` retrieves the name of the active workspace.

The method `pfcServerLocation.DeleteWorkspace()` deletes the specified workspace. The method deletes the workspace only if the following conditions are met:

- The workspace is not the active workspace.
- The workspace does not contain any checked out objects.

Use one of the following techniques to delete an active workspace:

- Make the required workspace inactive using `pfcServer.ActiveWorkspace()` with the name of some other workspace and then call `pfcServerLocation.DeleteWorkspace()`.
- Unregister the server using `pfcServer.Unregister()` and delete the workspace.

Workflow to Register a Server

To Register a Server with an Existing Workspace

Perform the following steps to register a Windchill server with an existing workspace:

1. Set the appropriate authentication context using the method `pfcBaseSession.AuthenticateBrowser()` with a valid username and password.
2. Look up the list of workspaces using the method `pfcServerLocation.CollectWorkspaces()`. If you already know the name of the workspace on the server, then ignore this step.
3. Register the workspace using the method `pfcBaseSession.RegisterServer()` with an existing workspace name on the server.
4. Activate the server using the method `pfcServer.Activate()`.

To Register a Server with a New Workspace

Perform the following steps to register a Windchill server with a new workspace:

1. Perform steps 1 to 4 in the preceding section to register the Windchill server with an existing workspace.
2. Use the method `pfcServerLocation.ListContexts()` to choose the required context for the server.
3. Create a new workspace with the required context using the method `pfcServer.CreateWorkspace()`. This method automatically makes the created workspace active.

Note

You can create a workspace only after the server is registered.

Aliased URL

An aliased URL serves as a handle to the server objects. You can access the server objects in the commonspace (shared folders) and the workspace using an aliased URL. An aliased URL is a unique identifier for the server object and its format is as follows:

- Object in workspace has a prefix `wtws`
`wtws://<server_alias>/<workspace_name>/<object_server_name>`

where <object_server_name> includes <object_name>.<object_extension>

For example,

wtws://my_server/my_workspace/abcd.prt,
wtws://my_server/my_workspace/intf_file.igs

where

<server_alias> is my_server

<workspace_name> is my_workspace

- Object in commonspace has a prefix wtpub
wtpub://<server_alias>/<folder_location>/<object_server_name>

For example,

wtpub://my_server/path/to/cs_folder/abcd.prt

where

<server_alias> is my_server

<folder_location> is path/to/cs_folder



Note

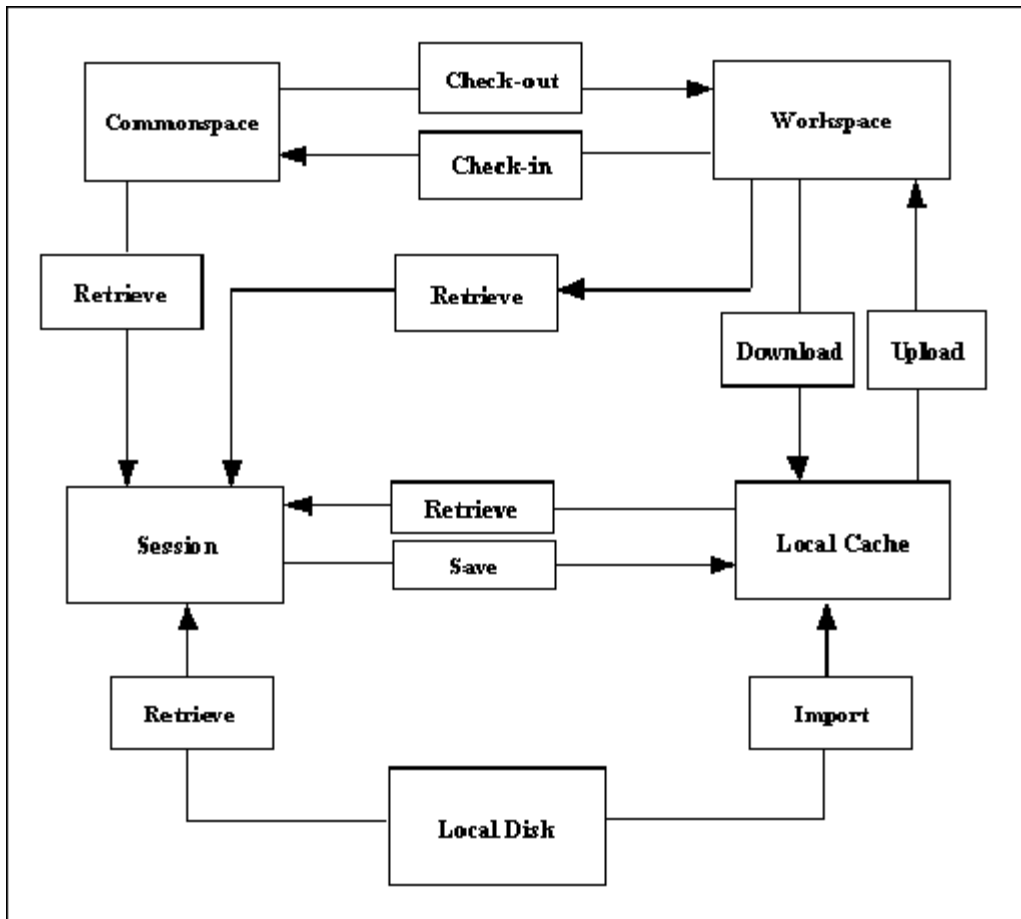
- object_server_name must be in lowercase.
 - The APIs are case-sensitive to the aliased URL.
 - <object_extension> should not contain Creo Parametric versions, for example, .1 or .2, and so on.
-

Server Operations

After registering the Windchill server with Creo Parametric, you can start accessing the data on the Windchill servers. The Creo Parametric interaction with Windchill servers leverages the following locations:

- Commonsplace (Shared folders)
- Workspace (Server-side workspace)
- Workspace local cache (Client-side workspace)
- Creo Parametric session
- Local disk

The methods described in this section enable you to perform the basic server operations. The following illustration shows how data is transferred among these locations.



Save

Method Introduced:

- **pfcModel.Save()**

The method `pfcModel.Save()` stores the object from the session in the local workspace cache, when a server is active.

Upload

An upload transfers Creo Parametric files and any other dependencies from the local workspace cache to the server-side workspace.

Methods Introduced:

- **pfcServer.UploadObjects()**
- **pfcServer.UploadObjectsWithOptions()**
- **pfcUploadOptions.Create()**

The method `pfcServer.UploadObjects()` uploads the object to the workspace. The object to be uploaded must be present in the current Creo Parametric session. You must save the object to the workspace using `pfcModel.Save()` or `pfcBaseSession.ImportToCurrentWS()` before attempting to upload it.

The method `pfcServer.UploadObjectsWithOptions()` uploads objects to the workspace using the options specified in the `pfcUploadOptions` class. These options allow you to upload the entire workspace, auto-resolve missing references, and indicate the target folder location for the new content during the upload. You must save the object to the workspace using `pfcModel.Save()`, or import it to the workspace using `pfcServer.UploadObjectsWithOptions()` before attempting to upload it.

Create the `pfcUploadOptions` object using the method `pfcUploadOptions.Create()`.

The methods available for setting the upload options are described in the following section.

Checkin

After you have finished working on objects in your workspace, you can share the design changes with other users. The checkin operation copies the information and files associated with all changed objects from the workspace to the Windchill database.

Methods and Properties Introduced:

- **`pfcServer.CheckinObjects()`**
- **`pfcCheckinOptions.Create()`**
- **`pfcUploadBaseOptions.DefaultFolder`**
- **`pfcUploadBaseOptions.NonDefaultFolderAssignments`**
- **`pfcUploadBaseOptions.AutoresolveOption`**
- **`pfcCheckinOptions.BaselineName`**
- **`pfcCheckinOptions.BaselineNumber`**
- **`pfcCheckinOptions.BaselineLocation`**
- **`pfcCheckinOptions.BaselineLifecycle`**
- **`pfcCheckinOptions.KeepCheckedout`**

The method `pfcServer.CheckinObjects()` checks in an object into the database. The object to be checked in must be present in the current Creo Parametric session. Changes made to the object are not included unless you save the object to the workspace using the method `pfcModel.Save()` before you check it in.

If you pass `NULL` as the value of the *Options* parameter, the checkin operation is similar to the **Auto Check-In** option in Creo Parametric. For more details on **Auto Check-In**, refer to the online help for Creo Parametric.

Use the method `pfcCheckinOptions.Create()` to create a new `pfcCheckinOptions` object.

By using an appropriately constructed *Options* argument, you can control the checkin operation.

Use the APIs listed above to access and modify the checkin options.

The method `pfcUploadBaseOptions.DefaultFolder` specifies the default folder location on the server for the automatic checkin operation.

The method

`pfcUploadBaseOptions.NonDefaultFolderAssignment` specifies the folder location on the server to which the objects will be checked in.

The method `pfcUploadBaseOptions.AutoresolveOption` specifies the option used for auto-resolving missing references. These options are defined in the `pfcServerAutoresolveOption` enumerated data type, and are as follows:

- *AutoresolveOption*—Specifies the option used for auto-resolving missing references. These options are defined in the `pfcServerAutoresolveOption`
 - `SERVER_DONT_AUTORESOLVE`—Model references missing from the workspace are not automatically resolved. This may result in a conflict upon checkin. This option is used by default.
 - `SERVER_AUTORESOLVE_IGNORE`—Missing references are automatically resolved by ignoring them.
 - `SERVER_AUTORESOLVE_UPDATE_IGNORE`—Missing references are automatically resolved by updating them in the database and ignoring them if not found.

The object `pfcCheckinOptions` specifies the baseline information for the objects upon checkin.

The method `pfcCheckinOptions.BaselineName` specifies the name of the baseline.

The method `pfcCheckinOptions.BaselineNumber` specifies the number of the baseline.

The default format for the baseline name and baseline number is `Username + time (GMT) in milliseconds`.

The method `pfcCheckinOptions.BaselineLocation` specifies the location of the baseline.

The method `pfcCheckinOptions.BaselineLifecycle` specifies the name of the lifecycle.

The method `pfcCheckinOptions.KeepCheckedout` specifies the checked out status of the object. If the value specified is `true`, then the contents of the selected object are checked into the Windchill server and automatically checked out again for further modification.

Retrieval

Standard Creo.JS provides several methods that are capable of retrieving models. When using these methods with Windchill servers, remember that these methods do not check out the object to allow modifications.

Methods Introduced:

- **`pfcBaseSession.RetrieveModel()`**
- **`pfcBaseSession.RetrieveModelWithOpts()`**
- **`pfcBaseSession.OpenFile()`**

The methods `pfcBaseSession.RetrieveModel()`, `pfcBaseSession.RetrieveModelWithOpts()`, and `pfcBaseSession.OpenFile()` load an object into a session given its name and type. The methods search for the object in the active workspace, the local directory, and any other paths specified by the `search_path` configuration option.

The method `pfcBaseSession.RetrieveModel()` retrieves the specified model into the Creo Parametric session from standard directory.

The method `pfcBaseSession.RetrieveModelWithOpts()` retrieves the specified model into the Creo Parametric session based on the path specified in the model descriptor.

Note

Both `pfcBaseSession.RetrieveModel()` and `pfcBaseSession.RetrieveModelWithOpts()` do not create a window for the model or display it anywhere.

The method `pfcBaseSession.OpenFile()` retrieves the specified model and displays it in a window.

Checkout and Download

To modify an object from the commonspace, you must check out the object. The process of checking out communicates your intention to modify a design to the Windchill server. The object in the database is locked, so that other users can obtain read-only copies of the object, and are prevented from modifying the object while you have checked it out.

Checkout is often accompanied by a download action, where the objects are brought from the server-side workspace to the local workspace cache. In Creo.JS, both operations are covered by the same set of methods.

Methods and Properties Introduced:

- **pfcServer.CheckoutObjects()**
- **pfcServer.CheckoutMultipleObjects()**
- **pfcCheckoutOptions.Create()**
- **pfcCheckoutOptions.Dependency**
- **pfcCheckoutOptions.SelectedIncludes**
- **pfcCheckoutOptions.IncludeInstances**
- **pfcCheckoutOptions.Version**
- **pfcCheckoutOptions.Download**
- **pfcCheckoutOptions.Readonly**

The method `pfcServer.CheckoutObjects()` checks out and optionally downloads the object to the workspace based on the configuration specifications of the workspace. The input arguments of this method are as follows:

- *Mdl*—Specifies the object to be checked out. This is applicable if the model has already been retrieved without checking it out.
- *File*—Specifies the top-level object to be checked out.
- *Checkout*—The checkout flag. If you specify the value of this argument as `true`, the selected object is checked out. Otherwise, the object is downloaded without being checked out. The download action enables you to bring read-only copies of objects into your workspace. This allows you to examine the object without locking it.
- *Options*—Specifies the checkout options object. If you pass `NULL` as the value of this argument, then the default Creo Parametric checkout rules apply. Use the method `pfcCheckoutOptions.Create()` to create a new `pfcCheckoutOptions` object.

Use the method `pfcServer.CheckoutMultipleObjects()` to check out and download multiple objects to the workspace based on the configuration specifications of the workspace. This method takes the same input arguments as listed above, except for *Mdl* and *File*. Instead it takes the argument *Files* that specifies the sequence of the objects to check out or download.

 Note

Creo.JS methods do not support `AS_STORED` configuration.

By using an appropriately constructed *Options* argument in the above functions, you can control the checkout operation. Use the APIs listed above to modify the checkout options.

The property `pfcCheckoutOptions.Dependency` specifies the dependency rule used while checking out dependents of the object selected for checkout. The types of dependencies given by the `pfcServerDependency` enumerated data type are as follows:

- `SERVER_DEPENDENCY_ALL`—All the objects that are dependent on the selected object are downloaded, that is, they are added to the workspace.
- `SERVER_DEPENDENCY_REQUIRED`—All the objects that are required to successfully retrieve the selected object in the CAD application are downloaded, that is, they are added to workspace.
- `SERVER_DEPENDENCY_NONE`—None of the dependent objects from the selected object are downloaded, that is, they are not added to workspace.

The property `pfcCheckoutOptions.SelectedIncludes` specifies the sequence of URLs to the selected instances, if *IncludeInstances* is of type `SERVER_INCLUDE_SELECTED`.

The property `pfcCheckoutOptions.IncludeInstances` specifies the rule for including instances from the family table during checkout. The type of instances given by the `pfcServerIncludeInstances` enumerated type are as follows:

- `SERVER_INCLUDE_ALL`—All the instances of the selected object are checked out.
- `SERVER_INCLUDE_SELECTED`—The application can select the family table instance members to be included during checkout.
- `SERVER_INCLUDE_NONE`—No additional instances from the family table are added to the object list.

The property `pfcCheckoutOptions.Version` specifies the version of the checked out object. If this value is set to `NULL`, the object is checked out according to the current workspace configuration.

The property `pfcCheckoutOptions.Download` specifies the checkout type as `download` or `link`. The value `download` specifies that the object content is downloaded and checked out, while `link` specifies that only the metadata is downloaded and checked out.

The property `pfcCheckoutOptions.Readonly` specifies the checkout type as a read-only checkout. This option is applicable only if the checkout type is `link`.

The following truth table explains the dependencies of the different control factors in the method `pfcServer.CheckoutObjects()` and the effect of different combinations on the end result.

Argument <i>checkout</i> in <code>pfcServer.CheckoutObjects</code>	<code>pfcCheckoutOptions.Download</code>	<code>pfcCheckoutOptions.Readonly</code>	Result
true	true	NA	Object is checked out and its content is downloaded.
true	false	NA	Object is checked out but content is not downloaded.
false	NA	true	Object is downloaded without checkout and as read-only.
false	NA	false	Not supported

Undo Checkout

Method Introduced:

- **`pfcServer.UndoCheckout()`**

Use the method `pfcServer.UndoCheckout()` to undo a checkout of the specified object. When you undo a checkout, the changes that you have made to the content and metadata of the object are discarded and the content, as stored in the server, is downloaded to the workspace. This method is applicable only for the model in the active Creo Parametric session.

Import and Export

Creo.JS provides you with the capability of transferring specified objects to and from a workspace. Import and export operations must take place in a session with no models. An import operation transfers a file from the local disk to the workspace.

Methods and Properties Introduced:

- **`pfcBaseSession.ExportFromCurrentWS()`**
- **`pfcBaseSession.ImportToCurrentWS()`**

- **pfcWSImportExportMessage.Description**
- **pfcWSImportExportMessage.FileName**
- **pfcWSImportExportMessage.MessageType**
- **pfcWSImportExportMessage.Resolution**
- **pfcWSImportExportMessage.Succeeded**
- **pfcBaseSession.SetWSExportOptions()**
- **pfcWSExportOptions.Create()**
- **pfcWSExportOptions.IncludeSecondaryContent()**

The method `pfcBaseSession.ExportFromCurrentWS()` exports the specified objects from the current workspace to a disk in a linked session of Creo Parametric.

The method `pfcBaseSession.ImportToCurrentWS()` imports the specified objects from a disk to the current workspace in a linked session of Creo Parametric.

Both `pfcBaseSession.ExportFromCurrentWS()` and `pfcBaseSession.ImportToCurrentWS()` allow you to specify a dependency criterion to process the following items:

- All external dependencies
- Only required dependencies
- No external dependencies

Both `pfcBaseSession.ExportFromCurrentWS()` and `pfcBaseSession.ImportToCurrentWS()` return the messages generated during the export or import operation in the form of the `pfcWSImportExportMessages` object. Use the APIs listed above to access the contents of a message. The message specified by the `pfcWSImportExportMessage` object contains the following items:

The property `pfcWSImportExportMessage.Description`—specifies the description of the problem or the message information.

The property `pfcWSImportExportMessage.FileName`—specifies the object name or the name of the object path.

The property `pfcWSImportExportMessage.MessageType`—specifies the severity of the message in the form of the `pfcWSImportExportMessageType` enumerated data type. The severity is one of the following types:

- `WSIMPEX_MSG_INFO`—Specifies an informational type of message.
- `WSIMPEX_MSG_WARNING`—Specifies a low severity problem that can be resolved according to the configured rules.
- `WSIMPEX_MSG_CONFLICT`—Specifies a conflict that can be overridden.

- `WSIMPEX_MSG_ERROR`—Specifies a conflict that cannot be overridden or a serious problem that prevents processing of an object.

The property `pfcWSImportExportMessage.Resolution`—specifies the resolution applied to resolve a conflict that can be overridden. This is applicable when the message is of the type `WSIMPEX_MSG_CONFLICT`.

The property `pfcWSImportExportMessage.Succeeded`—determines whether the resolution succeeded or not. This is applicable when the message is of the type `WSIMPEX_MSG_CONFLICT`.

The method `pfcBaseSession.SetWSExportOptions()` sets the export options used while exporting the objects from a workspace in the form of the `pfcWSExportOptions` object. Create this object using the method `pfcWSExportOptions.Create()`.

`pfcWSExportOptions.IncludeSecondaryContent()` indicates whether or not to include secondary content while exporting the primary Creo Parametric model files.

File Copy

Creo.JS provides you with the capability of copying a file from the workspace or target folder to a location on the disk and vice-versa.

Methods Introduced:

- **`pfcBaseSession.CopyFileToWS()`**
- **`pfcBaseSession.CopyFileFromWS()`**

Use the method `pfcBaseSession.CopyFileToWS()` to copy a file from the disk to the workspace. The file can optionally be added as secondary content to a given workspace file. If the viewable file is added as secondary content, a dependency is created between the Creo Parametric model and the viewable file.

Use the method `pfcBaseSession.CopyFileFromWS()` to copy a file from the workspace to a location on disk.

When importing or exporting Creo Parametric models, PTC recommends that you use methods `pfcBaseSession.ImportToCurrentWS()` and `pfcBaseSession.ExportFromCurrentWS()`, respectively, to perform the import or export operation. Methods that copy individual files do not traverse Creo Parametric model dependencies, and therefore do not copy a fully retrievable set of models at the same time.

Additionally, only the methods `pfcBaseSession.ImportToCurrentWS()` and `pfcBaseSession.ExportFromCurrentWS()` provide full metadata exchange and support. That means `pfcBaseSession.ImportToCurrentWS()` can communicate all the Creo Parametric designated parameters, dependencies, and family table information to a PDM system while `pfcBaseSession.ExportFromCurrentWS()` can

update exported Creo Parametric data with PDM system changes to designated and system parameters, dependencies, and family table information. Hence PTC recommends the use of `pfcBaseSession.CopyFileToWS()` and `pfcBaseSession.CopyFileFromWS()` to process only non-Creo Parametric files.

Server Object Status

Methods Introduced:

- **`pfcServer.IsObjectCheckedOut()`**
- **`pfcServer.IsObjectModified()`**

The methods described in this section verify the current status of the object in the workspace. The method `pfcServer.IsObjectCheckedOut()` specifies whether the object is checked out for modification. The value `true` indicates that the specified object is checked out to the active workspace.

The value `false` indicates one of the following statuses:

- The specified object is not checked out
- The specified object is only uploaded to the workspace, but was never checked in
- The specified object is only saved to the local workspace cache, but was never uploaded

The method `pfcServer.IsObjectModified()` specifies whether the object has been modified in the workspace. This method returns `true` if the object was modified locally.

Delete Objects

Method Introduced:

- **`pfcServer.RemoveObjects()`**

The method `pfcServer.RemoveObjects()` deletes the array of objects from the workspace. When passed with the *ModelNames* array as `NULL`, this method removes all the objects in the active workspace.

Conflicts During Server Operations

An exception is provided to capture the error condition while performing the following server operations using the specified APIs:

Operation	API
Checkin an object or workspace	<code>pfcServer.CheckinObjects()</code>
Checkout an object	<code>pfcServer.CheckoutObjects()</code>
Undo checkout of an object	<code>fcServer.UndoCheckout()</code>

Operation	API
Upload object	<code>pfcServer.UploadObjects()</code>
Download object	<code>pfcServer.CheckoutObjects()</code> (with <code>download as true</code>)
Delete workspace	<code>pfcServerLocation.DeleteWorkspace()</code>
Remove object	<code>pfcServer.RemoveObjects()</code>

These APIs throw a common exception `pfcXToolkitCheckoutConflict` if an error is encountered during server operations. The exception description will include the details of the error condition. This description is similar to the description displayed by the Creo Parametric HTML user interface in the conflict report.

Utility APIs

The methods specified in this section enable you to obtain the handle to the server objects to access them. The handle may be the aliased URL or the model name of the http URL. These utilities enable the conversion of one type of handle to another.

Methods Introduced:

- **`pfcServer.GetAliasedUrl()`**
- **`pfcBaseSession.GetModelNameFromAliasedUrl()`**
- **`pfcBaseSession.GetAliasFromAliasedUrl()`**
- **`pfcBaseSession.GetUrlFromAliasedUrl()`**

The method `pfcServer.GetAliasedUrl()` enables you to search for a server object by its name. Specify the complete filename of the object as the input, for example, `test_part.prt`. The method returns the aliased URL for a model on the server. For more information regarding the aliased URL, refer to the section [Aliased URL on page 282](#). During the search operation, the workspace takes precedence over the shared space.

You can also use this method to search for files that are not in the Creo Parametric format. For example, `my_text.txt`, `intf_file.stp`, and so on.

The method `pfcBaseSession.GetModelNameFromAliasedUrl()` returns the name of the object from the given aliased URL on the server.

The method `pfcBaseSession.GetAliasFromAliasedUrl()` converts an aliased URL to a standard URL for the objects on the server.

For example, `wtws://my_alias/Creo Parametric/abcd.prt` is converted to an appropriate URL on the server as `<server.mycompany.com>/Windchill`.

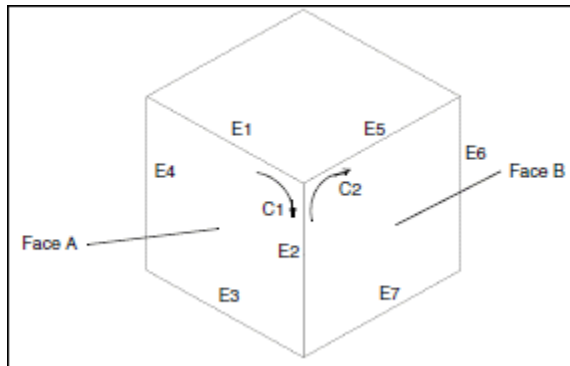
The method `pfcBaseSession.GetUrlFromAliasedUrl()` returns the server alias from aliased URL.

Geometry Traversal

Geometry Traversal

This section illustrates the relationships between faces, contours, and edges. Examples E-1 through E-5 show some sample parts and list the information about their surfaces, faces, contours, and edges.

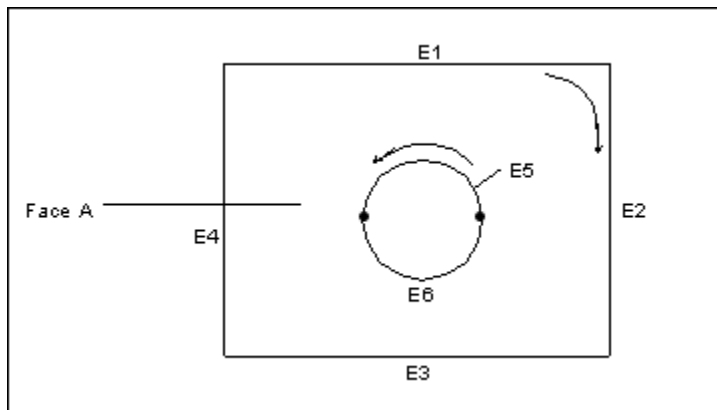
Example 1



This part has 6 faces.

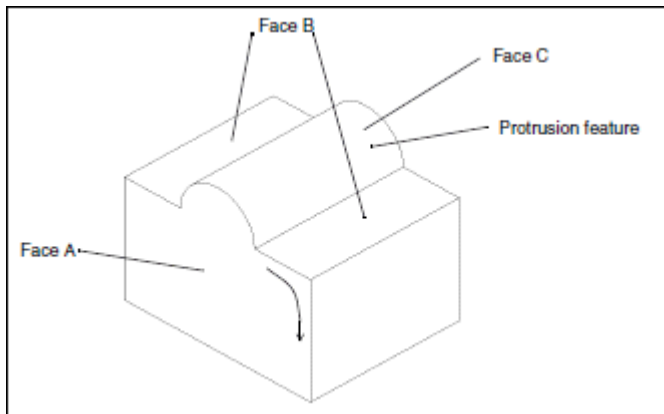
- Face A has 1 contour and 4 edges.
- Edge E2 is the intersection of faces A and B.
- Edge E2 is a component of contours C1 and C2.

Example 2



Face A has 2 contours and 6 edges.

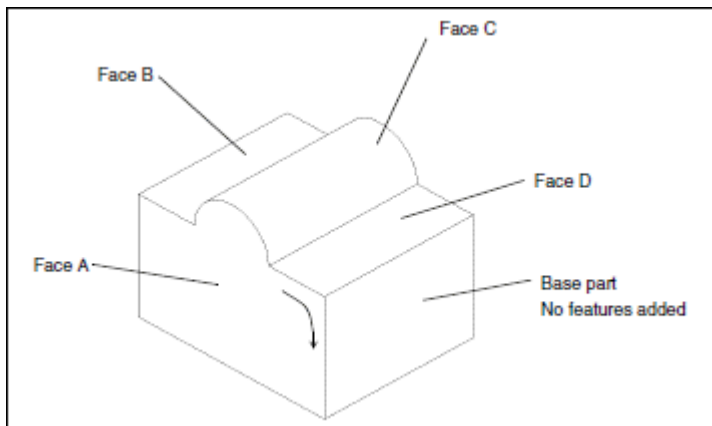
Example 3



This part was extruded from a rectangular cross section. The feature on the top was added later as an extruded protrusion in the shape of a semicircle.

- Face A has 1 contour and 6 edges.
- Face B has 2 contours and 8 edges.
- Face C has 1 contour and 4 edges.

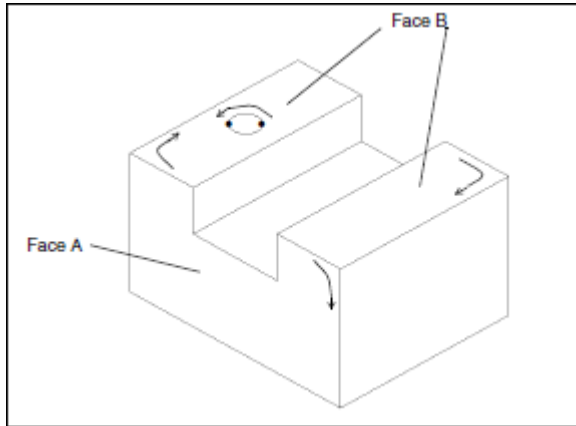
Example 4



This part was extruded from a cross section identical to Face A. In the Sketcher, the top boundary was sketched with two lines and an arc. The sketch was then extruded to form the base part, as shown.

- Face A has 1 contour and 6 edges.
- Face B has 1 contour and 4 edges.
- Face C has 1 contour and 4 edges.
- Face D has 1 contour and 4 edges.

Example 5



This part was extruded from a rectangular cross section. The slot and hole features were added later.

- Face A has 1 contour and 8 edges.
- Face B has 3 contours and 10 edges.

Geometry Representations

Geometry Representations

This section describes the geometry representations of the data used by Creo.JS.

Surface Parameterization

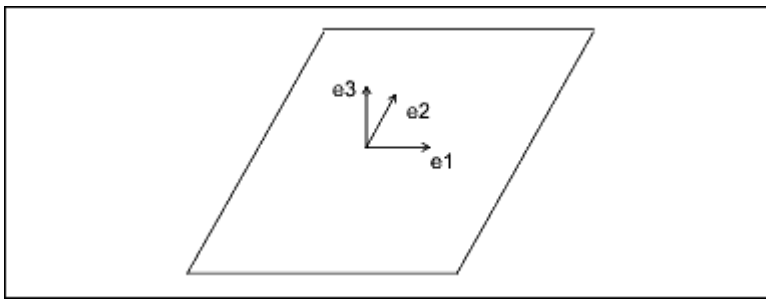
A surface in Creo Parametric contains data that describes the boundary of the surface, and a pointer to the primitive surface on which it lies. The primitive surface is a three-dimensional geometric surface parameterized by two variables u and v . The surface boundary consists of closed loops (contours) of edges. Each edge is attached to two surfaces, and each edge contains the u and v values of the portion of the boundary that it forms for both surfaces. Surface boundaries are traversed clockwise around the outside of a surface, so an edge has a direction in each surface with respect to the direction of traversal.

This section describes the surface parameterization. The surfaces are listed in order of complexity. For ease of use, the alphabetical listing of the data structures is as follows:

- [Cone on page 299](#)
- [Coons Patch on page 302](#)
- [Cylinder on page 298](#)
- [Cylindrical Spline Surface on page 305](#)

- [Fillet Surface on page 303](#)
- [General Surface of Revolution on page 300](#)
- [NURBS on page 308](#)
- [Plane on page 298](#)
- [Ruled Surface on page 301](#)
- [Spline Surface on page 303](#)
- [Tabulated Cylinder on page 302](#)
- [Torus on page 300](#)

Plane



The plane entity consists of two perpendicular unit vectors (e_1 and e_2), the normal to the plane (e_3), and the origin of the plane.

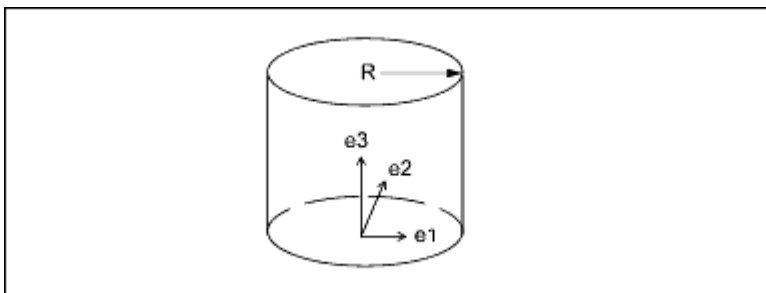
Data Format:

```
e1[3]      Unit vector, in the u direction
e2[3]      Unit vector, in the v direction
e3[3]      Normal to the plane
origin[3]   Origin of the plane
```

Parameterization:

$$(x, y, z) = u * e_1 + v * e_2 + \text{origin}$$

Cylinder



The generating curve of a cylinder is a line, parallel to the axis, at a distance R from the axis. The radial distance of a point is constant, and the height of the point is v .

Data Format:

e1[3] Unit vector, in the u direction
 e2[3] Unit vector, in the v direction
 e3[3] Normal to the plane
 origin[3] Origin of the cylinder
 radius Radius of the cylinder

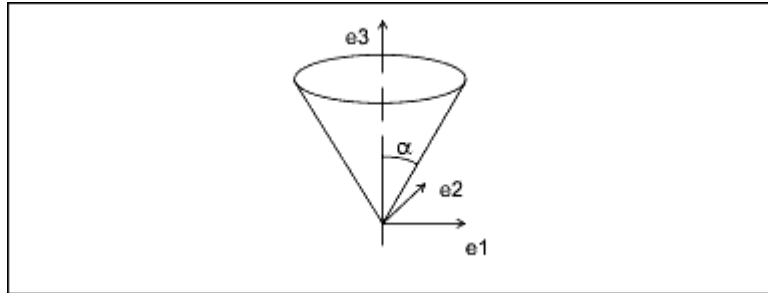
Parameterization:

$$(x, y, z) = \text{radius} * [\cos(u) * e1 + \sin(u) * e2] + v * e3 + \text{origin}$$

Engineering Notes:

For the cylinder, cone, torus, and general surface of revolution, a local coordinate system is used that consists of three orthogonal unit vectors ($e1$, $e2$, and $e3$) and an origin. The curve lies in the plane of $e1$ and $e3$, and is rotated in the direction from $e1$ to $e2$. The u surface parameter determines the angle of rotation, and the v parameter determines the position of the point on the generating curve.

Cone



The generating curve of a cone is a line at an angle α to the axis of revolution that intersects the axis at the origin. The v parameter is the height of the point along the axis, and the radial distance of the point is $v * \tan(\alpha)$.

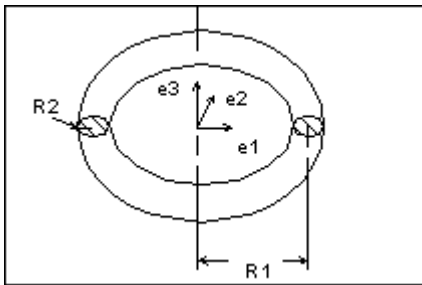
Data Format:

e1[3] Unit vector, in the u direction
 e2[3] Unit vector, in the v direction
 e3[3] Normal to the plane
 origin[3] Origin of the cone
 alpha Angle between the axis of the cone
 and the generating line

Parameterization:

$$(x, y, z) = v * \tan(\alpha) * [\cos(u) * e1 + \sin(u) * e2] + v * e3 + \text{origin}$$

Torus



The generating curve of a torus is an arc of radius $R2$ with its center at a distance $R1$ from the origin. The starting point of the generating arc is located at a distance $R1 + R2$ from the origin, in the direction of the first vector of the local coordinate system. The radial distance of a point on the torus is $R1 + R2 * \cos(v)$, and the height of the point along the axis of revolution is $R2 * \sin(v)$.

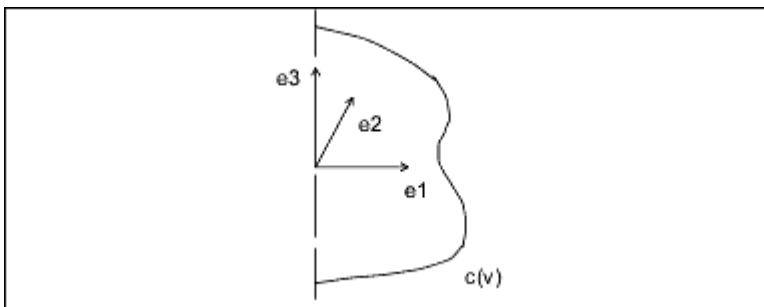
Data Format:

e1[3] Unit vector, in the u direction
e2[3] Unit vector, in the v direction
e3[3] Normal to the plane
origin[3] Origin of the torus
radius1 Distance from the center of the
 generating arc to the axis of
 revolution
radius2 Radius of the generating arc

Parameterization:

$$(x, y, z) = (R1 + R2 * \cos(v)) * [\cos(u) * e1 + \sin(u) * e2] + R2 * \sin(v) * e3 + \text{origin}$$

General Surface of Revolution



A general surface of revolution is created by rotating a curve entity, usually a spline, around an axis. The curve is evaluated at the normalized parameter v , and the resulting point is rotated around the axis through an angle u . The surface of revolution data structure consists of a local coordinate system and a curve structure.

Data Format:

```

e1[3]      Unit vector, in the u direction
e2[3]      Unit vector, in the v direction
e3[3]      Normal to the plane
origin[3]   Origin of the surface of revolution
curve      Generating curve

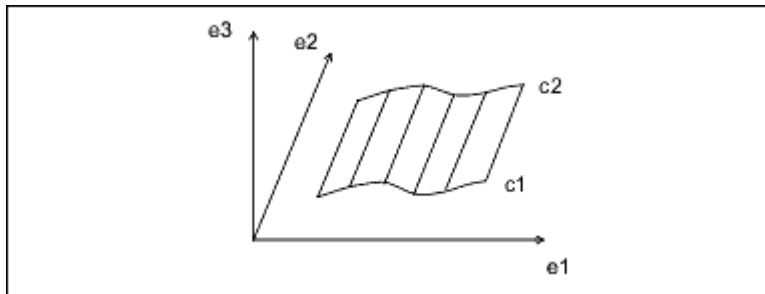
```

Parameterization:

curve(v) = (c1, c2, c3) is a point on the curve.

$$\begin{aligned}
 (x, y, z) = & [c1 * \cos(u) - c2 * \sin(u)] * e1 + \\
 & [c1 * \sin(u) + c2 * \cos(u)] * e2 + \\
 & c3 * e3 + \text{origin}
 \end{aligned}$$

Ruled Surface



A ruled surface is the surface generated by interpolating linearly between corresponding points of two curve entities. The *u* coordinate is the normalized parameter at which both curves are evaluated, and the *v* coordinate is the linear parameter between the two points. The curves are not defined in the local coordinate system of the part, so the resulting point must be transformed by the local coordinate system of the surface.

Data Format:

```

e1[3]      Unit vector, in the u direction
e2[3]      Unit vector, in the v direction
e3[3]      Normal to the plane
origin[3]   Origin of the ruled surface
curve_1     First generating curve
curve_2     Second generating curve

```

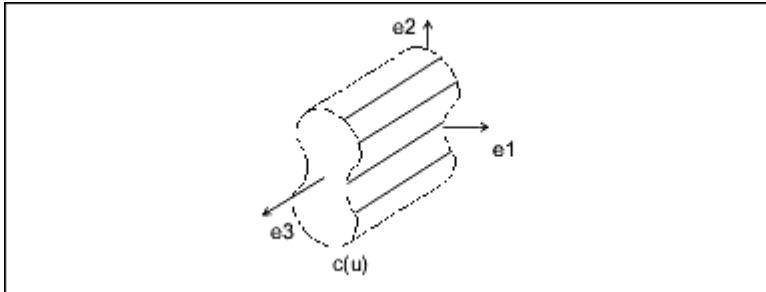
Parameterization:

(x', y', z') is the point in local coordinates.

$$(x', y', z') = (1 - v) * C1(u) + v * C2(u)$$

$$(x, y, z) = x' * e1 + y' * e2 + z' * e3 + \text{origin}$$

Tabulated Cylinder



A tabulated cylinder is calculated by projecting a curve linearly through space. The curve is evaluated at the u parameter, and the z coordinate is offset by the v parameter. The resulting point is expressed in local coordinates and must be transformed by the local coordinate system to be expressed in part coordinates.

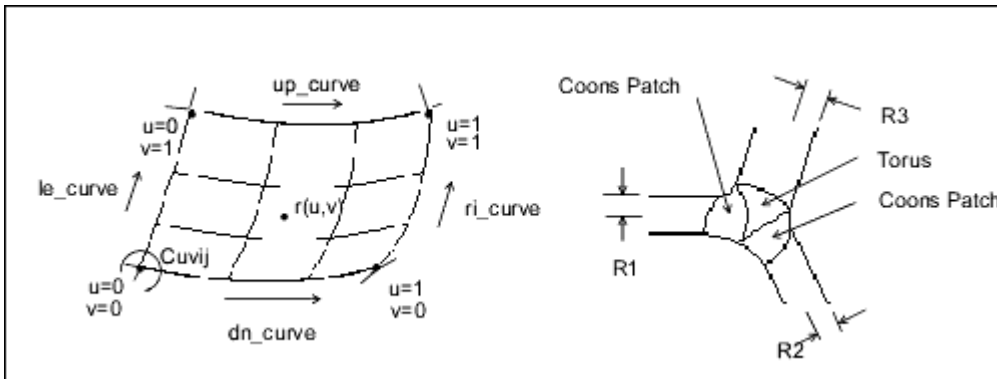
Data Format:

```
e1[3]      Unit vector, in the u direction
e2[3]      Unit vector, in the v direction
e3[3]      Normal to the plane
origin[3]   Origin of the tabulated cylinder
curve       Generating curve
```

Parameterization:

(x', y', z') is the point in local coordinates.
 $(x', y', z') = C(u) + (0, 0, v)$
 $(x, y, z) = x' * e1 + y' * e2 + z' * e3 + origin$

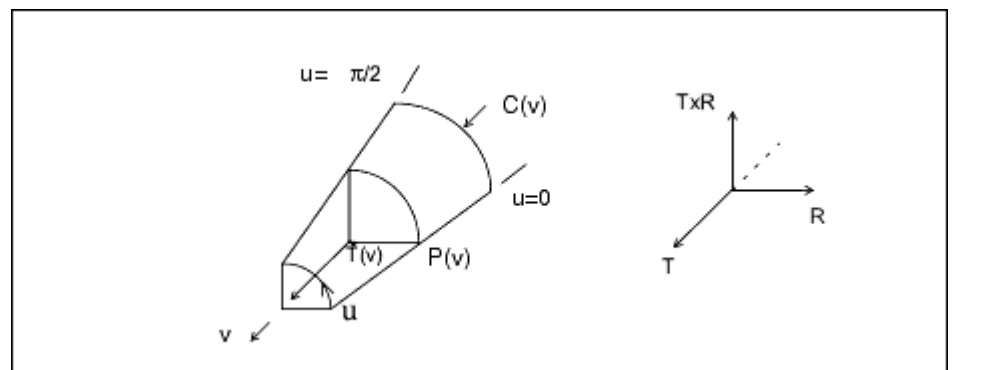
Coons Patch



A Coons patch is used to blend surfaces together. For example, you would use a Coons patch at a corner where three fillets (each of a different radius) meet.

Data Format:

```
le_curve    u = 0 boundary
ri_curve    u = 1 boundary
dn_curve    v = 0 boundary
up_curve    v = 1 boundary
```



Data Format:

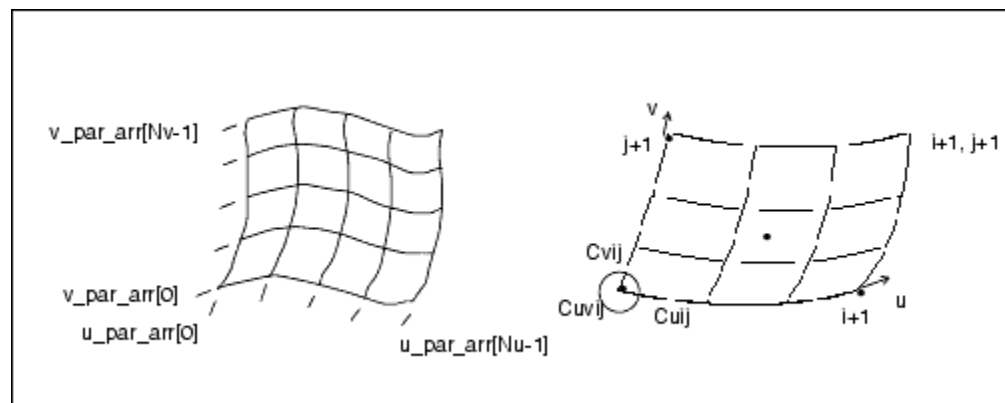
pnt_spline P(v) spline running along the $u = 0$ boundary

fillet_spline	T(v) spline running along the arc boundary
ctr_spline	C(v) spline along the centers of the fillet arcs
tan_spline	T(v) spline of unit tangents to the axis of the fillet arcs

$$R(v) = P(v) - C(v)$$

$$(x, y, z) = C(v) + R(v) * \cos(u) + T(v) \times R(v) * \sin(u)$$

Optimal surface



The parametric spline surface is a nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. The grid is curvilinear in uv space. Use this for bicubic blending between corner points.

Data Format:

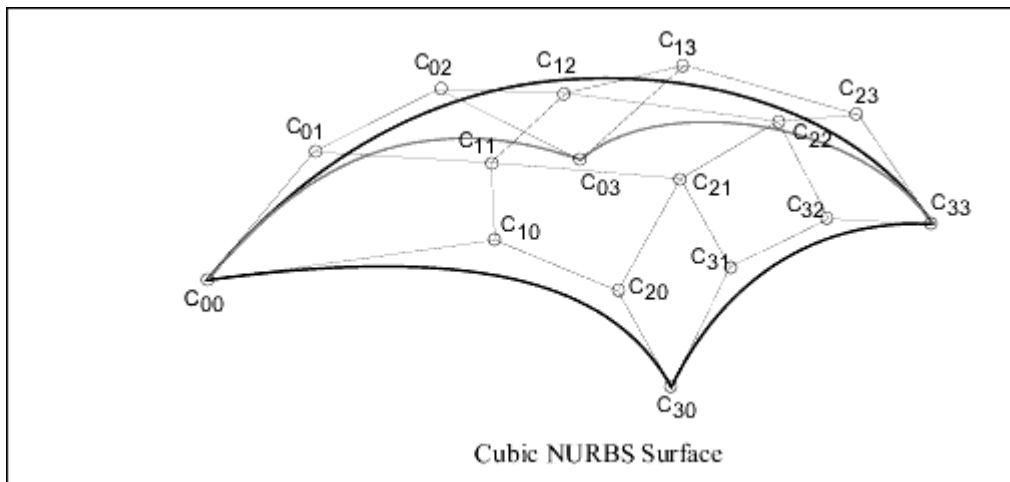
<code>u_par_arr[]</code>	Point parameters, in the u direction, of size Nu
<code>v_par_arr[]</code>	Point parameters, in the v direction, of size Nv
<code>point_arr[][3]</code>	Array of interpolant points, of size Nu x Nv
<code>u_tan_arr[][3]</code>	Array of u tangent vectors at interpolant points, of size Nu x Nv
<code>v_tan_arr[][3]</code>	Array of v tangent vectors at interpolant points, of size Nu x Nv
<code>uvder_arr[][3]</code>	Array of mixed derivatives at interpolant points, of size Nu x Nv

Engineering Notes:

- Allows for a unique 3x3 polynomial around every patch.
- There is second order continuity across patch boundaries.
- The point and tangent vectors represent the ordering of an array of `[i][j]`, where u varies with i, and v varies with j. In walking through the `point_arr[][3]`, you will find that the innermost variable representing v(j) varies first.

NURBS Surface

The NURBS surface is defined by basis functions (in u and v), expandable arrays of knots, weights, and control points.



Data Format:

deg[2]	Degree of the basis functions (in u and v)
u_par_arr[]	Array of knots on the parameter line u
v_par_arr[]	Array of knots on the parameter line v
wghts[]	Array of weights for rational NURBS, otherwise NULL
c_point_arr[][3]	Array of control points

Definition:

$$R(u, v) = \frac{\sum_{i=0}^{N1} \sum_{j=0}^{N2} C_{i,j} \times B_{i,k}(u) \times B_{j,l}(v)}{\sum_{i=0}^{N1} \sum_{j=0}^{N2} w_{i,j} \times B_{i,k}(u) \times B_{j,l}(v)}$$

k = degree in u

l = degree in v

N1 = (number of knots in u) - (degree in u) - 2

N2 = (number of knots in v) - (degree in v) - 2

$B_{i,k}$ = basis function in u

$B_{j,l}$ = basis function in v

w_{ij} = weights

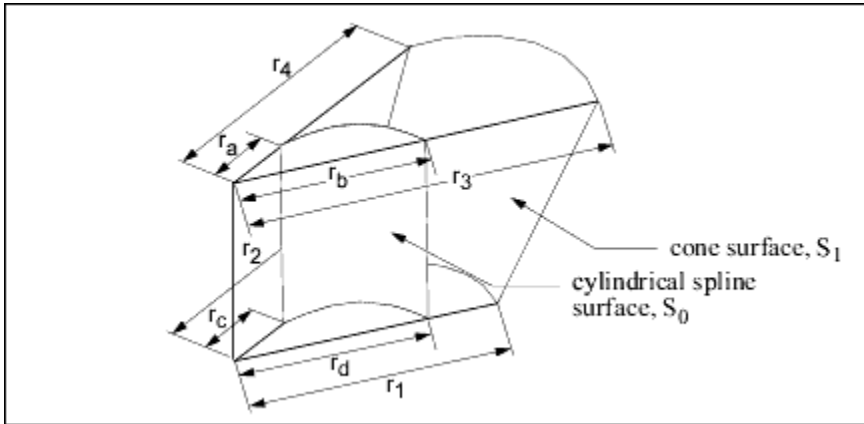
$C_{i,j}$ = control points (x,y,z) * $w_{i,j}$

Engineering Notes:

The weights and c_points_arr arrays represent matrices of size wghts[N1+1] [N2+1] and c_points_arr [N1+1] [N2+1]. Elements of the matrices are packed into arrays in row-major order.

Cylindrical Spline Surface

The cylindrical spline surface is a nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. The grid is curvilinear in modeling space.



Data Format:

e1[3] x' vector of the local coordinate system
 e2[3] y' vector of the local coordinate system
 e3[3] z' vector of the local coordinate system, which corresponds to the axis of revolution of the surface
 origin[3] Origin of the local coordinate system
 splsrf Spline surface data structure

The spline surface data structure contains the following fields:

u_par_arr[] Point parameters, in the u direction, of size Nu
 v_par_arr[] Point parameters, in the v direction, of size Nv
 point_arr[][3] Array of points, in cylindrical coordinates, of size Nu x Nv. The array components are as follows:
 point_arr[i][0] - Radius
 point_arr[i][1] - Theta
 point_arr[i][2] - Z
 u_tan_arr[][3] Array of u tangent vectors, in cylindrical coordinates, of size Nu x Nv
 v_tan_arr[][3] Array of v tangent vectors, in cylindrical coordinates, of size Nu x Nv
 uvder_arr[][3] Array of mixed derivatives, in cylindrical coordinates, of size Nu x Nv

Engineering Notes:

If the surface is represented in cylindrical coordinates (r, theta, z), the local coordinate system values (x', y', z') are interpreted as follows:

```

x' = r cos (theta)
y' = r sin (theta)
z' = z

```

A cylindrical spline surface can be obtained, for example, by creating a smooth rotational blend (shown in the figure on the previous page).

In some cases, you can replace a cylindrical spline surface with a surface such as a plane, cylinder, or cone. For example, in the figure, the cylindrical spline surface S_1 was replaced with a cone ($r_1 = r_2$, $r_3 = r_4$, and $r_1 \neq r_3$).

If a replacement cannot be done (such as for the surface S_0 in the figure ($r_a \neq r_b$ or $r_c \neq r_d$)), leave it as a cylindrical spline surface representation.

Edge and Curve Parameterization

This parameterization represents edges (line, arc, and spline) as well as the curves (line, arc, spline, and NURBS) within the surfaces.

This section describes edges and curves, arranged in order of complexity. For ease of use, the alphabetical listing is as follows:

- [Arc on page 307](#)
- [Line on page 307](#)
- [NURBS on page 308](#)
- [Spline on page 308](#)

Line

Data Format:

```

end1[3]    Starting point of the line
end2[3]    Ending point of the line

```

Parameterization:

```

(x, y, z) = (1 - t) * end1 + t * end2

```

Arc

The arc entity is defined by a plane in which the arc lies. The arc is centered at the origin, and is parameterized by the angle of rotation from the first plane unit vector in the direction of the second plane vector. The start and end angle parameters of the arc and the radius are also given. The direction of the arc is counterclockwise if the start angle is less than the end angle, otherwise it is clockwise.

Data Format:

```

vector1[3]    First vector that defines the
               plane of the arc
vector2[3]    Second vector that defines the

```

	plane of the arc
origin[3]	Origin that defines the plane of the arc
start_angle	Angular parameter of the starting point
end_angle	Angular parameter of the ending point
radius	Radius of the arc.

Parameterization:

t' (the unnormalized parameter) is
 $(1 - t) * \text{start_angle} + t * \text{end_angle}$
 $(x, y, z) = \text{radius} * [\cos(t') * \text{vector1} +$
 $\sin(t') * \text{vector2}] + \text{origin}$

Spline

The spline curve entity is a nonuniform cubic spline, defined by a series of three-dimensional points, tangent vectors at each point, and an array of unnormalized spline parameters at each point.

Data Format:

par_arr[]	Array of spline parameters (t) at each point.
pnt_arr[][3]	Array of spline interpolant points
tan_arr[][3]	Array of tangent vectors at each point

Parameterization:

x , y , and z are a series of unique cubic functions, one per segment, fully determined by the starting and ending points, and tangents of each segment.

Let p_max be the parameter of the last spline point. Then, t , the unnormalized parameter, is $t * p_max$.

Locate the i th spline segment such that:

$\text{par_arr}[i] < t' < \text{par_arr}[i+1]$

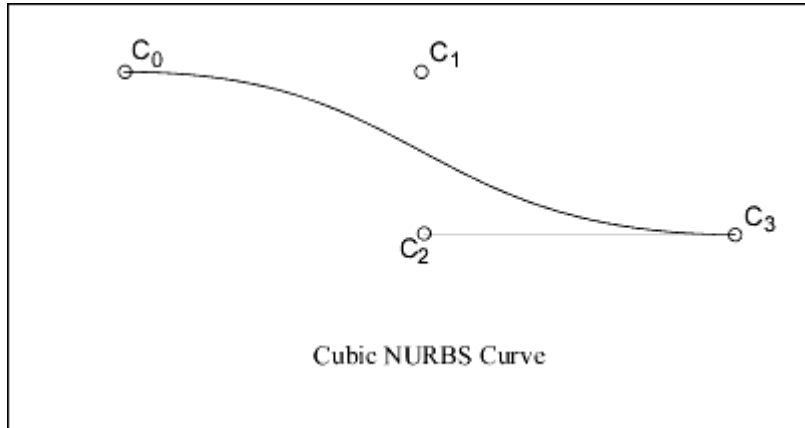
(If $t < 0$ or $t > +1$, use the first or last segment.)

$t0 = (t' - \text{par_arr}[i]) / (\text{par_arr}[i+1] - \text{par_arr}[i])$

$t1 = (\text{par_arr}[i+1] - t') / (\text{par_arr}[i+1] - \text{par_arr}[i])$

NURBS

The NURBS (nonuniform rational B-spline) curve is defined by expandable arrays of knots, weights, and control points.



Data Format:

degree Degree of the basis function
 params[] Array of knots
 weights[] Array of weights for rational
 NURBS, otherwise NULL.
 c_pnts[][3] Array of control points

Definition:

$$R(t) = \frac{\sum_{i=0}^N C_i \times B_{i,k}(t)}{\sum_{i=0}^N w_i \times B_{i,k}(t)}$$

k = degree of basis function
 N = (number of knots) - (degree) - 2
 wi = weights
 Ci = control points (x, y, z) * wi
 Bi,k = basis functions

By this equation, the number of control points equals N+1.

References:

Faux, I.D., M.J. Pratt. Computational Geometry for Design and Manufacture. Ellis Harwood Publishers, 1983.

Mortenson, M.E. Geometric Modeling. John Wiley & Sons, 1985.

Index

A

- Accuracy
 - getting and setting, 117
- ActionListener
 - feature-level, 219
 - session-level, 214
 - solid-level, 217
 - UI command, 215
- ActionSource interface, 213
- Activate
 - window, 136
- Allocate
 - simplified representations, 259
- Arcs
 - description, 175
 - representation, 307
- Area
 - surface, 181
- Arrows, 175
- Assemblies
 - coordinate systems, 141
 - hierarchy, 198
 - structure of, 198
- Axes, 183
 - evaluating, 183

B

- B-splines
 - description, 175
- best practices for working with Creo.
JS, 15

C

- Cells

- accessing, 211
- Circles, 175
- Clear
 - window, 136
- Close
 - window, 136
- Colors, 44
- Composite curves
 - description, 175
- Cones
 - class representation, 179
 - geometry representation, 299
- Configuration options, 43
- Contours
 - evaluating, 178
 - traversing, 174
- Contours, locating in a model, 178
- Coons patches
 - geometry representation, 302
- Coordinate systems, 183
 - assemblies, 141
 - datum, 141
 - drawing, 141
 - Drawing View, 141
 - evaluating, 183
 - screen, 140
 - section, 141
 - solid, 140
 - window, 140
- Coordinate transformations, 139
- Copy
 - models, 65
- Create
 - family table columns, 212
 - family table instance, 209
 - local group, 153

- material, 129
- parameters, 187
- simplified representations, 259
- UDFs, 157
- window, 135
- Create Interactively Defined UDFs, 159
- Creating UDFs, 158
- Creo Parametric
 - accessing, 44
- Creo.js
 - Chrome Dev Tools(CDT), 33
- Creo.JS application, 7
- Creo.js Debugger
 - Embedded Browser, 33
- Curves, 175
 - data structures, 307
 - determining the type, 175
 - t parameter, 175
 - types, 175
 - reserved, 175
- Cylinders, 179
 - geometry representation, 298
 - spline surfaces, 305
 - tabulated, 179
 - geometry representation, 302

D

- Debugging
 - embedded V8 inspector, 33
- Debugging Creo.js
 - Chrome Web Browser, 33
- Delete
 - feature pattern, 153
 - models, 65
 - simplified representations, 259
- Depth
 - selection, 55
- Descriptors
 - model, 60
- Detail.DetailEntityInstructions
 - interface

- description, 96
- Dimension2D.Dimension2D interface
 - description, 80
- Dimensions, 193
 - information, 193
 - tolerances, 194
- Display
 - model in window, 135
 - models, 65
 - selection, 55
- Drawing
 - transformations, 144

E

- Edges, 175
 - determining the type, 175
 - evaluating, 176
 - t parameter, 175
 - traversing, 174
 - types, 175
 - reserved, 175
- Ellipses, 175
- Erase
 - models, 65
- Evaluation
 - axes, 183
 - contour, 178
 - coordinate system, 183
 - edge, 176
 - point, 183
 - surface, 180
- Examples
 - normalizing a coordinate transformation matrix, 144
- existing Web.Link applications
 - migration, 15

F

- Faces
 - traversing, 174
- Family tables

- cells, 211
- columns
 - accessing, 210
- instances
 - accessing, 209
- symbols, 210
- Features
 - creating, 151
 - groups, 153
 - information, 150
 - operations, 151
 - patterns, 153
 - read-only, 150
 - resuming, 151
 - suppressing, 151
 - user-defined, 156
- Files
 - message, 44
 - contents, 45
 - naming restrictions, 45
- Fillet surfaces
 - geometry representation, 303

G

- General surface of revolution, 300
- Generic model
 - getting, 209
- Geometry
 - solid edge, 176
 - terms, 174
- Groups, 153, 156
- guidelines for migrating Web.Link applications to Creo.JS, 16

H

- Highlight
 - selections, 55

I

- Information

- Drawing, 71
- Install
 - Creo.JS, 8
- Installation
 - Creo.JS, 8
- Interactively Defined UDFs
 - create, 159

K

- Keywords
 - instanceof
 - using, 175

L

- Layers, 147
- Lines
 - description, 175
 - representation, 307
 - styles, 44
- Lists
 - of current windows, 135
 - of materials, 129
 - of ModelItems, 145
 - of pattern members, 153
 - of rows in a family table, 209
 - of subitems, 145
 - of views, 138
 - of windows, 135
- Local groups
 - creating, 153
- Locks, 209

M

- Macros, 43
- Mass properties, 126
- Materials, 129
- Matrix
 - code example, 144
- Matrix3D object, 144
- Message files, 44

- contents, 45
- restrictions, 45
- Message window
 - reading from, 47
 - writing to, 47
- migrating
 - guidelines for migrating Web.Link applications to Creo.JS, 16
 - Web.Link applications to Creo.JS, 15
- ModelItems
 - duplicating, 147
 - evaluating, 183
 - getting, 145
 - information, 146
- Models
 - descriptors, 60
 - Drawing
 - Obtaining, 70
 - getting, 59
 - operations, 65
 - retrieving, 61
- Modify
 - simplified representations, 260

N

- Normalize
 - matrix, 144
- NURBS
 - representation, 308
 - surface, 304

O

- Open
 - file, 61
- Operations
 - Drawing, 71
 - feature, 151
 - model, 65
 - solid, 117, 256
 - view, 139

- window, 136, 257
- Outlines
 - contour, 178
- overview, 7

P

- Parameters, 187
 - information, 190
- ParamValue objects, 185
- Parts, 129
- Pattern leaders, 153
- Patterns, 153
- pfcExceptions.
 - pfcXToolkitDrawingCreateErrors
 - description, 69
- pfcModel.Models
 - information, 62
- pfcSelect.Selection.GetSelItem method
 - getting a ModelItem, 145
- Planes, 179
 - geometry representation, 298
- Points, 183
 - evaluating, 183
- Polygons, 175
- Principal curve, 181

R

- Refresh
 - window, 136, 257
- Regenerate
 - events, 214
 - solids, 117, 256
- Rename
 - models, 65
- Repaint
 - events, 214
 - window, 136, 257
- Reset
 - view, 139
- Restrictions
 - on text message files, 45

Retrieve
 geometry of a simplified representation, 259
 graphics of a simplified representation, 259
 material, 129
 simplified representations, 259
 view, 138

Revolved surfaces, 179

Rotate
 view, 139

Ruled surfaces, 179
 geometry representation, 301

S

sample applications, 29

Save
 models, 65
 view, 139

Screen coordinate system, 140

security settings, 15

Selection
 accessing data, 55
 controlling display, 55

Sheets
 Drawing, 72

Simplified representations
 adding items, 261
 creating, 259
 deleting, 259
 items, 261
 extracting information from, 260
 modifying, 260
 retrieving
 geometry, 259
 graphics, 259
 utilities, 263

Solids
 accuracy, 117
 coordinate system, 140
 information, 117

 mass properties, 126
 operations, 117, 256

Splines
 cylindrical spline surface, 305
 description, 175
 representation, 308
 surface, 303

Status
 feature, 150

Surfaces, 178
 cylindrical spline, 305
 evaluating, 180
 area, 181
 evaluating parameters, 181
 fillet
 geometry representation, 303
 general surface of revolution, 300

NURBS
 geometry representation, 304
 revolved, 179
 ruled, 179, 301
 spline, 303
 traversing, 174
 types, 179

T

t parameter
 description, 175

Table
 creating, 87-88
 drawing cells, 87-88
 retrieving, 89
 selecting, 88

Table.Table interface
 description, 87

Tabulated cylinders, 179
 geometry representation, 302

testing web tool, 30

Text, 175
 message files, 44

Tolerance, 194

- Torii, 179
- Torus, 300
- Transformations, 139, 141
 - solid to coordinate system datum coordinates, 144
 - solid to screen coordinates, 143
 - in a drawing, 144
- Traversal
 - of a solid block, 174

U

- UDFs, 156
 - creating, 157
- Utilities
 - simplified representations, 263

V

- Values
 - ParamValue, 186
- View2D.View2D interface
 - description, 75
- Views
 - Drawing, 75
 - getting a view object, 138
 - list of, 138
 - operations, 139
 - retrieving, 138
 - saving, 139
- Visibility, 150
- Visit
 - simplified representations, 260

W

- Window coordinate system, 140
- Windows
 - activating, 136
 - clearing, 136
 - closing, 136
 - creating, 135
 - flush, 136

- operations, 136, 257
- repainting, 136
- working with Creo.JS, 8
 - best practices, 15
 - call from browser, 11
 - call to browser, 14
 - embedded scripts, 9
 - exception handling, 12
 - external scripts, 9
 - scripts, 9
 - security settings, 15
- Write
 - to the message window, 47